

Mastering Azure Security

Second Edition

Keeping your Microsoft Azure workloads safe



Mustafa Toroman | Tom Janetscheck



Mastering Azure Security

Second Edition

Keeping your Microsoft Azure workloads safe

Mustafa Toroman

Tom Janetscheck



BIRMINGHAM—MUMBAI

Mastering Azure Security

Second Edition

Copyright © 2022 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the authors, nor Packt Publishing or its dealers and distributors, will be held liable for any damages caused or alleged to have been caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

Group Product Manager: Vijin Boricha

Publishing Product Manager: Shrilekha Malpani

Senior Editor: Athikho Sapuni Rishana

Content Development Editor: Sayali Pingale

Technical Editor: Arjun Varma

Copy Editor: Safis Editing

Associate Project Manager: Neil Dmello

Proofreader: Safis Editing

Indexer: Tejal Daruwale Soni

Production Designer: Ponraj Dhandapani

Marketing Coordinator: Sanjana Gupta

First published: March 2022

Production reference: 1240322

Published by Packt Publishing Ltd.

Livery Place
35 Livery Street
Birmingham
B3 2PB, UK.

978-1-80323-855-5

www.packt.com

Contributors

About the authors

Mustafa Toroman is a solution architect focused on cloud-native applications and migrating existing systems to the cloud. He is very interested in DevOps processes and cybersecurity, and he is also an Infrastructure as Code enthusiast and DevOps Institute Ambassador. Mustafa often speaks at international conferences about cloud technologies. He has been an MVP for Microsoft Azure since 2016 and a C# Corner MVP since 2020. Mustafa has also authored several books about Microsoft Azure and cloud computing, all published by Packt.

Thanks to my patient wife, who tolerates my never-ending side projects. Without her putting up with me, I would not be able to do a fraction of what I do. I would also like to thank my family for all the support, my mother for making great sacrifices for me to be here today, my father for believing in me, my brother for being who he is. Another thank you to Bakir, my mentor who steered me in the direction I am still on. Thank you to my Infra team; although I left, you are my brothers and the best team in the world. I would like to thank Sasa and Adis for being my faithful advisors. And last but not least, thanks to Tom for being my partner in crime on this one.

Tom Janetscheck is a senior program manager at Microsoft's Cloud Security CxE team for Microsoft Defender for Cloud. Prior to this, he has been working in different internal and external IT and consulting roles for almost two decades, with a strong focus on cloud infrastructure, architecture, and security. As a well-known international conference speaker, tech blogger, and book author, and as one of the founders and organizers of the Azure Saturday community conferences, the former Microsoft MVP is actively taking his experience into international tech communities. In his spare time, Tom is an enthusiastic motorcyclist, scuba diver, guitarist, bassist, rookie drummer, and station officer at a local fire department.

I would like to thank my wife and sons for always supporting me in whatever I do and for being my bastion of calm; I would also like to thank my great friend, Mustafa Toroman, for collaborating on this project, and also thanks to my great friend and team lead, Yuri Diogenes, for always pushing and helping me to overcome limits! Thanks to our wider team's manager, Rebecca Halla, my director, Nicholas DiCola, and our entire Defender for Cloud CxE team: Bojan, Fernanda, Future, Liana, Safeena, Shay, and Stan – you folks definitely rock and it's my pleasure to work with all of you every day! Also, thank you to the entire Defender for Cloud Engineering/Dev teams for your dedication, enthusiasm, and partnership to make Defender for Cloud the great platform it is today. Last but not least, special thanks to Ben Kliger who encouraged me to move out of my comfort zone and to take the step into Defender for Cloud engineering, this move changed my life!

About the reviewer

Matt Hansen is a cloud solution architect at Microsoft who focuses on Azure infrastructure and security. He has been involved in the industry for 15 years as an engineer and architect and has been working with Azure since 2013. Matt serves as a subject matter expert, advisory committee member, and Adjunct Professor for cloud and security technologies.

Matt holds over 50 industry certifications, including CCSP, CISSP, Azure Security Engineer, Azure Network Engineer, and has held all versions of the Azure Solutions architecture certifications. Matt holds a BS in network engineering, an MS in engineering management, and an MS in information systems security.

I attribute much of my accomplishments in life, including this, to my wonderful wife, Heather, who has supported me through everything I've done; I wouldn't be where I am today without you.

I also want to thank Packt for this wonderful opportunity. The world of cybersecurity is ever-changing, and I'm excited to have been a part of this book, enabling organizations to build safe and secure environments on Microsoft Azure.

Table of Contents

Preface

Section 1: Identity and Governance

1

An Introduction to Azure Security

Exploring the shared responsibility model	4	Azure network	9
On-premises	5	Azure infrastructure availability	10
IaaS	5	Azure infrastructure integrity	12
PaaS	5	Azure infrastructure monitoring	13
SaaS	5	Understanding Azure security foundations	14
Division of security in the shared responsibility model	5	Summary	15
Physical security	7	Questions	16

2

Governance and Security

Understanding governance in Azure	20	Parameters	31
Using common sense to avoid mistakes	22	Policy assignments	34
Using management locks	23	Initiative definitions	35
Using management groups for governance	26	Initiative assignments	35
Understanding Azure Policy	29	Policy exemptions	35
Mode	31	Policy best practices	37
		Defining Azure blueprints	38
		Blueprint definitions	38
		Blueprint publishing	39

Azure Resource Graph	45	Querying Azure Resource Graph with the Azure CLI	48
Querying Azure Resource Graph with PowerShell	46	Advanced queries	49
		Summary	51
		Questions	51

3

Managing Cloud Identities

Exploring passwords and passphrases	54	Understanding role-based access control	85
Dictionary attacks and password protection	56	Creating custom RBAC roles	89
Understanding MFA	59	Protecting admin accounts with Azure AD PIM	94
How to enable MFA in Azure AD	61	Managing Azure AD roles in PIM	95
MFA activation from a user's perspective	64	Managing Azure resources with PIM	100
Introducing security defaults	67	Hybrid authentication and Single Sign-On	101
Using Conditional Access	70	Understanding passwordless authentication	105
Named locations	71	Global settings	106
Custom controls	72	Licensing considerations	108
Terms of use	73	Summary	108
Conditional Access policies	74	Questions	109
Introducing Azure AD Identity Protection	79		
Azure AD Identity Protection at a glance	79		

Section 2: Cloud Infrastructure Security

4

Azure Network Security

Understanding Azure Virtual Network	114	Creating an S2S connection	124
Connecting on-premises networks with Azure	123	Connecting a VNet to another VNet	128
		VNet service endpoints	131
		Private endpoints	134

Considering other VNet security options	134	Hub VNet	141
Azure Firewall deployment and configuration	135	Understanding Azure Application Gateway	142
Azure DDoS protection	139	Understanding Azure Front Door	145
Azure Bastion	140	Summary	145
Hub-and-spoke network topology	141	Questions	145

5

Azure Key Vault

Understanding Azure Key Vault	148	Using Azure Key Vault in deployment scenarios	157
Understanding access policies	149	Creating an Azure Key Vault and secret Azure VM deployment	157 160
Understanding service-to-service authentication	150	Summary	165
Understanding managed identities for Azure resources	152	Questions	165

6

Data Security

Technical requirements	168	Working on Azure SQL Database	177
Understanding Azure Storage	168	Summary	186
Understanding Azure virtual machine disks	174	Questions	187

Section 3: Security Management

7

Microsoft Defender for Cloud

Introducing Microsoft Defender for Cloud	192	Enabling Microsoft Defender for Cloud's enhanced security	203
Enabling Microsoft Defender for Cloud	197	Cloud Security Posture Management with Defender for Cloud	207
Using auto-provisioning to deploy extensions	200		

Working with recommendations	209	Microsoft Defender for Servers	224
How to prioritize remediation	213	Microsoft Defender for Containers	234
Working with resource exemptions	215	Threat detection summary	234
Custom policies and (regulatory) compliance	218	Automating security	235
Using the regulatory compliance dashboard	220	Continuous export	235
Working with regulatory compliance standards	222	Workflow automation	237
		REST APIs	241
		Multi-cloud capabilities in Microsoft Defender for Cloud	241
Cloud workload protection and multi-cloud capabilities	224	Summary	243
		Questions	243

8

Microsoft Sentinel

Introduction to SIEM	245	Microsoft Sentinel automation	258
Getting started with Microsoft Sentinel	247	Creating workbooks	261
Configuring data connectors and retention	249	Using threat hunting and notebooks	262
Working with Microsoft Sentinel dashboards	251	Advanced threat detection	264
Setting up rules and alerts	252	Using community resources	266
		Summary	266
		Questions	266

9

Security Best Practices

Log Analytics design considerations	269	Storage account access keys	282
Understanding Azure SQL Database security features	271	Summary	283
Security in Azure App Service	273	Questions	284

Assessments

Index

Other Books You May Enjoy

Preface

Security is integrated into every cloud, but most users take cloud security for granted. Revised to cover product updates up to early 2022, this book will help you understand Microsoft Azure's shared responsibility model that can address any challenge, cybersecurity in the cloud, and how to design secure solutions in Microsoft Azure.

Who this book is for

This book is for Azure cloud professionals, Azure architects, and security professionals looking to implement safe and secure cloud services using Azure Security Center and other Azure security features. A fundamental understanding of security concepts and prior exposure to the Azure Cloud will assist with understanding the key concepts covered in the book.

What this book covers

Chapter 1, An Introduction to Azure Security, covers how the cloud is changing the concept of IT, and security is not an exception. Cybersecurity requires a different approach in the cloud, and we need to understand what the differences are, new threats, and how to tackle them.

Chapter 2, Governance and Security, goes into how to create policies and rules in Microsoft Azure in order to create standards, enforce these policies and rules, and maintain quality levels.

Chapter 3, Managing Cloud Identities, explains why identity is one of the most important parts of security. With the cloud, identity is even more expressed than ever before. You'll learn how to keep identities secure and safe in Microsoft Azure and how to keep track of access rights and monitor any anomalies in user behavior.

Chapter 4, Azure Network Security, covers how the network is the first line of defense in any environment. Keeping resources safe and unreachable by attackers is a very important part of security. You'll learn how to achieve this in Microsoft Azure with built-in or custom tools.

Chapter 5, Azure Key Vault, explains how to manage secrets and certificates in Azure and deploy resources to Microsoft Azure with Infrastructure as Code in a secure way.

Chapter 6, Data Security, covers how to protect data in the cloud with additional encryption using Microsoft or your own encryption key.

Chapter 7, Microsoft Defender for Cloud, covers how to use Defender for Cloud to detect threats in Microsoft Azure, on-premises and in other clouds, and how to view assessments, reports, and recommendations in order to increase cloud security.

Chapter 8, Microsoft Sentinel, covers how to use Microsoft Sentinel to monitor security for your Azure and on-premise resources, including detecting threats before they happen and using artificial intelligence to analyze and investigate threats. Using Microsoft Sentinel to automate responses to security threats and stop them immediately is also covered.

Chapter 9, Security Best Practices, introduces best practices for Azure security, including how to set up a bulletproof Azure environment, finding the hidden security features that are placed all over Azure, and other tools that may help you increase security in Microsoft Azure.

To get the most out of this book

You will require the following software, which is open source and free to use, except for Microsoft Azure, which is subscription-based and billed based on usage per minute. However, even for Microsoft Azure, a trial subscription can be used.

Software/Hardware covered in the book	OS requirements
A Microsoft Azure subscription	Windows, macOS X, and Linux (any)
PowerShell	Windows, macOS X, and Linux (any)
Azure PowerShell	Windows, macOS X, and Linux (any)
Azure CLI	Windows, macOS X, and Linux (any)
Visual Studio Code	Windows, macOS X, and Linux (any)

If you are using the digital version of this book, we advise you to type the code yourself or access the code via the GitHub repository (link available in the next section). Doing so will help you avoid any potential errors related to the copy/pasting of code.

Download the color images

We also provide a PDF file that has color images of the screenshots/diagrams used in this book. You can download it here: https://static.packt-cdn.com/downloads/9781803238555_ColorImages.pdf.

Download the example code files

You can download the example code files for this book from GitHub at <https://github.com/PacktPublishing/Mastering-Azure-Security-Second-Edition>. In case there's an update to the code, it will be updated on the existing GitHub repository.

We also have other code bundles from our rich catalog of books and videos available at <https://github.com/PacktPublishing/>. Check them out!

Conventions used

There are a number of text conventions used throughout this book.

Code in text: Indicates code words in the text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and Twitter handles. Here is an example: "Behind the `parameters` section, there is a `resource` section in which the key vault reference is defined."

A block of code is set as follows:

```
# Grant your user account access rights to Azure Key Vault secrets
Set-AzKeyVaultAccessPolicy '
-VaultName $kvName '
-ResourceGroupName $rgName '
-UserPrincipalName (Get-AzContext).account.id '
-PermissionsToSecrets get, set
```

Bold: Indicates a new term, an important word, or words that you see on screen. For example, words in menus or dialog boxes appear in the text like this. Here is an example: "Click on **Review + create** and after the final validation is passed, click **Create**."

Tips or Important Notes

Appear like this.

Get in touch

Feedback from our readers is always welcome.

General feedback: If you have questions about any aspect of this book, mention the book title in the subject of your message and email us at customercare@packtpub.com.

Errata: Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you would report this to us. Please visit www.packtpub.com/support/errata, selecting your book, clicking on the Errata Submission Form link, and entering the details.

Piracy: If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at copyright@packt.com with a link to the material.

If you are interested in becoming an author: If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit authors.packtpub.com.

Share Your Thoughts

Once you've read *Mastering Azure Security*, we'd love to hear your thoughts! Please [click here](#) to go straight to the Amazon review page for this book and share your feedback.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Section 1: Identity and Governance

This section deals with cybersecurity in the cloud, as well as how to create and enforce policies in Azure and how to manage and secure identities in Azure.

This part of the book comprises the following chapters:

- *Chapter 1, An Introduction to Azure Security*
- *Chapter 2, Governance and Security*
- *Chapter 3, Managing Cloud Identities*

1

An Introduction to Azure Security

When cloud computing comes up in a conversation, security is, very often, the main topic. When data leaves local data centers, many wonder what happens to it. We are used to having complete control over everything, from physical servers, networks, and hypervisors, to applications and data. Then, all of a sudden, we are supposed to transfer much of that to someone else. It's natural to feel a little tension and distrust at the beginning, but, if we dig deep, we'll see that cloud computing can offer us more security than we could ever achieve on our own.

Microsoft Azure is a cloud computing service provided through Microsoft-managed data centers dispersed around the world. Azure data centers are built to top industry standards and comply with all the relevant certification authorities, such as ISO/IEC 27001:2013 and NIST SP 800-53, to name a couple. These standards guarantee that Microsoft Azure is built to provide security and reliability.

In this chapter, we'll learn about Azure security concepts and how security is structured in Microsoft Azure data centers, using the following topics:

- Exploring the shared responsibility model
- Physical security
- Azure network

- Azure infrastructure availability
- Azure infrastructure integrity
- Azure infrastructure monitoring
- Understanding Azure security foundations

Exploring the shared responsibility model

While Microsoft Azure is very secure, the responsibility for building a secure environment doesn't rest with Microsoft alone. Its shared responsibility model divides responsibility between Microsoft and its customers.

Before we can discuss which party looks after which aspect of security, we need to first discuss cloud service models. There are three basic models:

- **Infrastructure as a Service (IaaS)**
- **Platform as a Service (PaaS)**
- **Software as a Service (SaaS)**

These models differ in terms of what is controlled by Microsoft and the customer. A general breakdown can be seen in the following diagram:

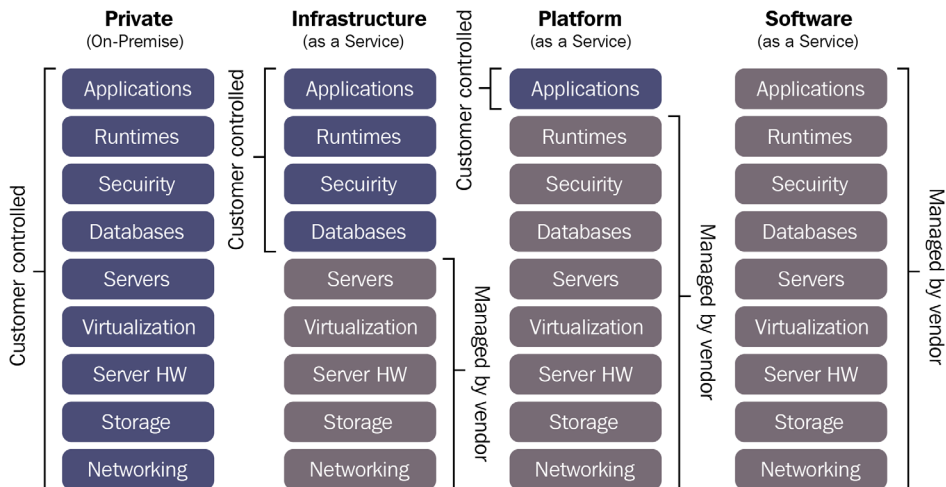


Figure 1.1 – Basic cloud service models

Let's look at these services in a little more detail.

On-premises

In an on-premises environment, we, as users, take care of everything: the network, physical servers, storage, and so on. We need to set up virtualization stacks (if used), configure and maintain servers, install and maintain software, manage databases, and so on. Most importantly, all aspects of security are our responsibility: physical security, network security, host and OS security, and application security for all application software running on our servers.

IaaS

With IaaS, Microsoft takes over some of the responsibilities. We only take care of data, runtime, applications, and some aspects of security, which we'll discuss a little later on.

An example of an IaaS product in Microsoft Azure is Azure **Virtual Machines (VM)**.

PaaS

PaaS gives Microsoft even more responsibility. We only take care of our applications. However, this still means looking after a part of the security. Some examples of PaaS in Microsoft Azure are Azure SQL Database and web apps.

SaaS

SaaS gives a large amount of control away, and we manage very little, including some aspects of security. In Microsoft's ecosystem, a popular example of SaaS is Office 365; however, we will not discuss this in this book.

Now that we have a basic understanding of shared responsibility, let's understand how responsibility for security is allocated.

Division of security in the shared responsibility model

The shared responsibility model divides security into three zones:

- Always controlled by the customer
- Always controlled by Microsoft
- Varies by service type

Irrespective of the cloud service model, customers will always retain the following security responsibilities:

- Data governance and rights management
- Endpoint protection
- Account and access management

Similarly, Microsoft always handles the following, in terms of security, for any of its cloud service models:

- Physical data center
- Physical network
- Physical hosts

Finally, there are a few security responsibilities that are allocated based on the cloud service model:

- Identity and directory infrastructure
- Applications
- Network
- Operating system

The distribution of responsibility, based on different cloud service models, is shown in the following diagram:

Responsibility Zones

Responsibility	SaaS	PaaS	IaaS	On-prem	
Data governance & rights management	Customer	Customer	Customer	Customer	Always retained by customer
Client endpoints	Customer	Customer	Customer	Customer	
Account & access management	Customer	Customer	Customer	Customer	
Identity & directory infrastructure	Microsoft	Microsoft	Customer	Customer	Varies by Service Type
Application	Microsoft	Microsoft	Customer	Customer	
Network controls	Microsoft	Microsoft	Customer	Customer	
Operating system	Microsoft	Microsoft	Customer	Customer	
Physical hosts	Microsoft	Microsoft	Microsoft	Customer	Transfers to Cloud Provider
Physical network	Microsoft	Microsoft	Microsoft	Customer	
Physical datacenter	Microsoft	Microsoft	Microsoft	Customer	

 Microsoft
  Customer

Figure 1.2 – The distribution of responsibility between the customer and service provider for different cloud service models (image courtesy of Microsoft, License: MIT)

Now that we know how security is divided, let's move on to one specific aspect of it: the physical security that Microsoft manages. This section is important as we won't discuss it in much detail in the chapters to come.

Physical security

Everything starts with physical security. No matter what we do to protect our data from attacks coming from outside of our network, it would all be in vain if someone was to walk into data centers or server rooms and take away disks from our servers. Microsoft takes physical security very seriously in order to reduce the risk of unauthorized access to data and data center resources.

Azure data centers can be accessed only through strictly defined access points. A facility's perimeter is safeguarded by tall fences made of steel and concrete. To enter Azure data centers, a person needs to go through at least two checkpoints: first to enter the facility perimeter, and second to enter the building. Both checkpoints are staffed by professional and trained security personnel. In addition to the access points, security personnel patrol the facility's perimeter. The facility and its buildings are covered by video surveillance, which is monitored by security personnel.

After entering the building, two-factor authentication with biometrics is required to gain access to the inside of the data center. If their identity is validated, a person can access only approved parts of the data center. Approval, besides defining areas that can be accessed, also defines periods that can be spent inside these areas. It also strictly defines whether a person can access these areas alone or needs to be accompanied by someone.

Before accessing each area inside the data center, a mandatory metal detector check is performed. To prevent unauthorized data leaving or entering the data center, only approved devices are allowed. Additionally, all server racks are monitored from the front and back using video surveillance. When leaving a data center area, an additional metal detector screening is required. This helps Microsoft make sure that nothing that can compromise its data's security is brought in or removed from the data center without authorization.

A review of physical security is conducted periodically for all facilities. This aims to satisfy all security requirements at all times.

After equipment reaches the end of its life, it is disposed of securely, with rigorous data and hardware disposal policies. During the disposal process, Microsoft personnel ensure that data is not available to untrusted parties. All data devices are either wiped (if possible) or physically destroyed in order to render the recovery of any information impossible.

All Microsoft Azure data centers are designed, built, and operated in a way that satisfies top industry standards, such as ISO 27001, HIPAA, FedRAMP, SOC 1, and SOC 2, to name a few. In many cases, specific region or country standards are followed as well, such as Australia IRAP, UK GCloud, and Singapore MTCS.

As an added precaution, all data inside any Microsoft Azure data center is encrypted at rest. Even if someone managed to get their hands on disks with customers' data, which is virtually impossible with all the security measures, it would take an enormous effort (both from a financial and time perspective) to decrypt any of the data.

But in the cloud era, network security is equally, if not more, important than physical security. Most services are accessed over the internet, and even isolated services depend on the network layer. So, next, we need to take a look at Azure network architecture.

Azure network

Networking in Azure can be separated into two parts: managed by Microsoft and managed by us. In this section, we will discuss the part of networking managed by Microsoft. It's important to understand the architecture, reliability, and security setup of this part to provide more context once we move to parts of network security that we need to manage.

As with Azure data centers generally, the Azure network follows industry standards with three distinct models/layers:

- Core
- Distribution
- Access

All three models use distinct hardware to completely separate all the layers. The core layer uses data center routers, the distribution layer uses access routers and L2 aggregation (this layer separates L3 routing from L2 switching), and the access layer uses L2 switches.

Azure network architecture includes two levels of L2 switches:

- **First level:** Aggregates traffic
- **Second level:** Loops to incorporate redundancy

This approach allows for more flexibility and better port scaling. Another benefit of this approach is that L2 and L3 are wholly separated, which allows for the use of distinct hardware for each layer in the network. Distinct hardware minimizes the chances of a fault in one layer affecting another one. The use of trunks allows for resource sharing for better connectivity. The network inside an Azure data center is distributed into clusters for better control, scaling, and fault tolerance.

In terms of network topology, Azure data centers contain the following elements:

- **Edge network:** An edge network represents a separation point between the Microsoft network and other networks (such as the internet or corporate networks). It is responsible for providing internet connectivity and ExpressRoute peering into Azure (covered in *Chapter 4, Azure Network Security*).
- **Wide area network:** The wide area network is Microsoft's intelligent backbone. It covers the entire globe and provides connectivity between Azure regions.

- **Regional gateways network:** A regional gateway is a point of aggregation for Azure regions and applies to all data centers within the region. It provides connectivity between data centers within the Azure region and enables connectivity with other regions.
- **Data center network:** A data center network enables connectivity between data centers and enables communication between servers within the data center. The data center network is based on a modified version of the Clos network. The Clos network uses the principle of multistage circuit-switching. The network is separated into three stages – ingress, middle, and egress. Each stage contains multiple switches and uses an r-way shuffle between stages. When a call is made, it enters the ingress switch and from there it can be routed to any available middle switch, and from the middle switch to any available egress switch. As the number of devices (switches) in use is huge, it minimizes the chance of hardware failure. All devices are situated at different locations with independent power and cooling, so an environmental failure has a minimal impact as well.

Azure networking is built upon highly redundant infrastructure in each Azure data center. Implemented redundancy is **need plus one (N+1)** or better, with full failover features within, and between, Azure data centers. Full failover tolerance ensures constant network and service availability. From the outside, Azure data centers are connected by dedicated, high-bandwidth network circuits redundantly that connect properties with over 1,200 **Internet Service Providers (ISPs)** on a global level. Edge capacity across the network is over 2,000 Gbps, which presents an enormous network potential.

Distributed Denial of Service (DDoS) is becoming a huge issue in terms of service availability. As the number of cloud services increases, DDoS attacks become more targeted and sophisticated. With the help of geographical distribution and quick detection, Microsoft can help you mitigate these DDoS attacks and minimize the impact. Let's take a look at this in more detail.

Azure infrastructure availability

Azure is designed, built, and operated to deliver highly available and reliable infrastructure. Improvements are constantly implemented to increase availability and reliability, along with efficiency and scalability. Delivery of a more secure and trusted cloud is always a priority.

Uninterruptible power supplies and vast banks of batteries ensure that the flow of electricity stays uninterrupted in case of short-term power disruptions. In the case of long-term power disruptions, emergency generators can provide backup power for days. Emergency power generators are used in cases of extended power outages or planned maintenance. In cases of natural disasters, when the external power supply is unavailable for long periods, each Azure data center has fuel reserves on-site.

Robust and high-speed, fiber optic networks connect data centers to major hubs. It's important that, along with connections through major hubs, data centers are connected directly. Everything is distributed into nodes, which host workloads closer to users to reduce latency, provide geo-redundancy, and increase resiliency.

Data in Azure can be placed in two separate regions: primary and secondary regions. A customer can choose where the primary and secondary regions will be. The secondary region is a backup site. In each region, primary and secondary, Azure keeps three healthy copies of your data at all times. This means that six copies of the data are available at any time. If any data copy becomes unavailable at any time, it's immediately declared invalid, a new copy is created, and the old one is destroyed.

Microsoft ensures high availability and reliability through constant monitoring, incident response, and service support. Each Azure data center operates 24/7/365 to ensure that everything is running, and all services are available at all times. Of course, *available at all times* is a goal that, ultimately, is impossible to reach. Many circumstances can impact uptime, and sometimes it's impossible to control all of them. Realistically, the aim is to achieve the best possible **Service Level Agreement (SLA)** so as to ensure that potential downtime is limited as far as possible. The SLA can vary based on a number of factors and is different per service and configuration. If we take into account all the factors we can control, the best SLA we can achieve would be 99.99%, also known as *four nines*.

Closely connected to infrastructure availability is infrastructure integrity. Integrity affects the availability terms of deployment, where all steps must be verified from different perspectives. New deployments must not cause any downtime or affect existing services in any way.

Azure infrastructure integrity

All software components installed in the Azure environment are custom built. This, of course, refers to software installed and managed by Microsoft as part of Azure Service Fabric. Custom software is built using Microsoft's **Security Development Lifecycle (SDL)** process, including operating system images and SQL databases. All software deployment is conducted as part of the strictly defined change management and release management process. All nodes and fabric controllers use customized versions of Windows Server 2019. The installation of any unauthorized software is not allowed.

VMs running in Azure are grouped into clusters. Each cluster contains around 1,000 VMs. All VMs are managed by the **Fabric Controller (FC)**. The FC is scaled out and redundant. Each FC is responsible for the life cycle management of applications running in its own cluster. This includes the provisioning and monitoring of hardware in that cluster. If any server fails, the FC automatically rebuilds a new instance of that server.

Each Azure software component undergoes a build process (as part of the release management process) that includes virus scans using endpoint protection anti-virus tools. As each software component undergoes this process, nothing goes to production without a clean-virus scan. During the release management process, all components go through a build process. During this process, an anti-virus scan is performed. Each virus scan creates a log in the build directory and, if any issues are detected, the process for this component is frozen. Any software components for which the issue is detected undergo inspection by Microsoft security teams in order to detect the exact issue.

Azure is a closed and locked-down environment. All nodes and guest VMs have their default Windows administrator account disabled. No user accounts are created directly on any of the nodes or guest VMs as well. Administrators from Azure support can connect to them only with proper authorization to perform maintenance tasks and emergency repairs.

With all precautions taken to provide maximum availability and security, incidents may occur from time to time. To detect these issues and mitigate them as soon as possible, Microsoft implemented monitoring and incident management.

Azure infrastructure monitoring

All hardware, software, and network devices in Azure data centers are constantly reviewed and updated. Reviews and updates are performed mandatorily at least once a year, but additional reviews and updates are performed as needed. Any changes (to hardware, software, or the network) must go through the release management process and need to be developed, tested, and approved in development and test environments prior to release to production. In this process, all changes must be reviewed and approved by the Azure security and compliance team.

All Azure data centers use integrated deployment systems for the distribution and installation of security updates for all software provided by Microsoft. If third-party software is used, the customer or software manufacturer is responsible for security updates, depending on how the software is provided and used. For example, if third-party software is installed using Azure Marketplace, the manufacturer is responsible for providing updates. If the software is manually installed, then it depends on the specific software. For Microsoft software, a special team within Microsoft, named **Microsoft Security Response Center**, is responsible for monitoring and identifying any security incident 24/7/365. Furthermore, any incident must be resolved in the shortest possible time frame.

Vulnerability scanning is performed across the Azure infrastructure (servers, databases, and network) at least once every quarter. If there is a specific issue or incident, vulnerability scanning is performed more often. Microsoft performs penetration tests, but also hires independent consultants to perform penetration tests. This ensures that nothing goes undetected. Any security issues are addressed immediately in order to increase security and stop any exploit when the issue is detected.

In case of any security issue, Microsoft has incident management in place. In the event that Microsoft is aware of a security issue, it takes the following action:

1. The customer is notified of the incident.
2. An immediate investigation is started to provide detailed information regarding the security incident.
3. Steps are taken to mitigate the effects and minimize the damage of the security incident.

Incident management is clearly defined in order to manage, escalate, and resolve all security incidents promptly.

Understanding Azure security foundations

Overall, we can see that with Microsoft Azure, the cloud can be very secure. But it's very important to understand the shared responsibility model as well. Just putting applications and data into the cloud doesn't make it secure. Microsoft provides certain parts of security and ensures that physical and network security is in place. Customers must assume part of the responsibility and ensure that the right measures are taken on their side as well.

For example, let's say we place our database and application in Microsoft Azure, but our application is vulnerable to SQL injection (still a very common data breach method). Can we blame Microsoft if our data is breached?

Let's be more extreme and say we publicly exposed the endpoint and forgot to put in place any kind of authentication. Is this Microsoft's responsibility?

If we look at the level of physical and network security that Microsoft provides in Azure data centers, not many organizations can say that they have the same level in their local data centers. More often than not, physical security is totally neglected. Server rooms are not secure, access is not controlled, and many times there is not even a dedicated server room, but just server racks in some corner or corridor. Even when a server room is under lock and key, no change of management is in place, and no one controls or reviews who is entering the server room and why. On the other hand, Microsoft implements top-level security in its data centers. Everything is under constant surveillance, and every access needs to be approved and reviewed. Even if something is missed, everything is still encrypted and additionally secured. In my experience, this is again something that most organizations don't bother with.

Similar things can be said about network security. In most organizations, almost all network security is gone after the firewall. Networks are usually unsegmented, no traffic control is in place inside the network, and so on. Routing and traffic forwarding are basic or non-existent. Microsoft Azure again addresses these problems very well and helps us have secure networks for our resources.

But even with all the components of security that Microsoft takes care of, this is only the beginning. Using Microsoft Azure, we can achieve better physical and network security than we could in local data centers, and we can concentrate on other things.

The shared responsibility model has different responsibilities for different cloud service models, and it's sometimes unclear what needs to be done. Luckily, even if it's not Microsoft's responsibility to address these parts of security, there are many security services available in Azure. Many of Azure's services have the single purpose of addressing security and helping us protect our data and resources in Azure data centers. Again, it does not stop there. Most of Azure's services have some sort of security features built-in, even when these services are not security-related. Microsoft takes security very seriously and enables us to secure our resources with many different tools.

The tools available vary from tools that help us to increase security by simply enabling a number of options, to tools that have lots of configuration options that help us design security, to tools that monitor our Azure resources and give us security recommendations that we need to implement. Some Azure tools use machine learning to help us detect security incidents in real time, or even before they happen.

This book will cover all aspects of Microsoft Azure security, from governance and identity, to network and data protection, to advanced tools. The final goal is to understand cloud security, to learn how to combine different tools to maximize security, and finally, to master Azure security!

Summary

The most important lesson in this chapter is to understand the shared responsibility model in Azure. Microsoft takes care of some parts of security, especially in terms of physical security, but we need to take care of the rest.

With Azure networking, integrity, availability, and monitoring, we don't have any influence and can't change anything (at least in the sections we discussed here). However, they are important to understand as we can apply a lot of things in the parts of security that we can manage. They will also provide more context and help us to better understand the complete security setup in Azure.

In the next chapter, we will move on to identity, which is one of the most important pillars of security. In Azure, identity is even more important, as most services are managed and accessible over the internet. Therefore, we need to take additional steps to make identity and access secure and bulletproof.

Questions

As we conclude, here is a list of questions for you to test your knowledge regarding this chapter's material. You will find the answers in the *Assessments* section of the *Appendix*:

1. Whose responsibility is security in the cloud?
 - A. User's
 - B. Cloud provider's
 - C. Responsibility is shared
2. According to the shared responsibility model, who is responsible for the security of physical hosts?
 - A. User
 - B. Cloud provider
 - C. Both
3. According to the shared responsibility model, who is responsible for the physical network?
 - A. User
 - B. Cloud provider
 - C. Depends on the service model
4. According to the shared responsibility model, who is responsible for network controls?
 - A. User
 - B. Cloud provider
 - C. Depends on the service model
5. According to the shared responsibility model, who is responsible for data governance?
 - A. User
 - B. Cloud provider
 - C. Depends on the service model

6. Which architecture is used for Azure networking?
 - A. DLA
 - B. Quantum 10 (Q10)
 - C. Both, but DLA is replacing Q10
 - D. Both, but Q10 is replacing DLA
7. In case of a security incident, what is the first step?
 - A. Immediate investigation
 - B. Mitigation
 - C. Customer is notified

2

Governance and Security

Before digging deep into the technical features that help to secure and protect your cloud environments on Azure, we first need to stop, reflect, plan, and act accordingly. Governance is essential and you need guardrails in the cloud. When talking about cloud computing, customers often exhibit a kind of *Let's get started* mentality. That's great, but in the cloud, too, rules are necessary. And, in addition to that, you surely don't want to end up with a cloud construct that does not entirely fit your company's needs, do you?

With this chapter, we want to help you design the foundation for a secure cloud strategy and act according to your plan by leading you through the following topics:

- Understanding governance in Azure
- Using common sense to avoid mistakes
- Using management locks
- Using management groups for governance
- Understanding Azure Policy
- Defining Azure blueprints
- Azure Resource Graph

Understanding governance in Azure

If we try to find out what governance actually means, we find a good definition in the online business dictionary. The first part of this definition is as follows:

Establishment of policies, and continuous monitoring of their proper implementation, by the members of the governing body of an organization.

To read the entire definition, go to the following link: <https://businessdictionary.info/definition/governance/>.

The most important words in this definition are *policies*, *monitoring*, and *implementation*. If you switch *implementation* for *deployment*, you get three aspects that are essential for security. So, you could say that governance is essential for security!

What's true in real life is true within the context of IT, too. I'm talking about rules. Rules only really work if they are enforced by *policies*. You can tell your administrators what they are not allowed to do. But if they can do what they are not allowed to, they are still able to accidentally break your rules. So, you need to define policies that help you to enforce or to monitor your corporate rules.

Monitoring is the second important part in terms of security. If you don't see what happens, you have no chance to react or make the right decisions. This is why monitoring is so important and there is no way around it. In later chapters, we will cover some monitoring best practices for Azure that will help you to get the right amount of information for your environments.

Finally, there is the aspect of *implementation*. You can use the Azure portal to manually deploy Azure resources. However, with that approach, you will definitely need a tight set of policies that enforce your rules so that manual deployments will not break them. If you want to make sure that all environments will strictly adhere to your rules, automatic deployments are your way to go. With a robust governance plan with policies **and role-based access controls (RBACs)**, and by using deployments from a DevOps pipeline, such as Azure DevOps, you can make sure that your deployments and the target environment will be consistent. If you then make sure that changes to your environment can only be done from your DevOps pipeline tool, but never manually from the Azure portal, you are on your way to protecting your environments from inadvertent changes. You can use PowerShell for *imperative* deployments, or **Azure Resource Manager (ARM)** and Terraform templates for *declarative* deployments. Whatever best fits your needs, use it!

Important Note: Different Deployment Models

If you're using imperative scripting languages such as PowerShell to deploy resources, you need to explicitly describe what should be done in order to get the infrastructure you want. You need to define the steps in the correct order. With declarative languages such as ARM templates or Terraform, you only describe what resources have to be deployed to your target environment, so it's not about the *how* but about the *what*. We will cover examples for all of these languages later in this book.

With Azure, and with cloud computing in general, we surely need new approaches for security and management, on the one hand, but, on the other, some principles are still valid, even after several decades in the industry. There is the **principle of least privilege**, or **POLP**. This principle states that programs or users will only have the least set of privileges necessary to complete a task. **Segregation of Duties**, or **SoD**, is another principle that is still valid even in the cloud era. SoD is the concept of having more than one person required to complete a task. For example, you might have a team that is responsible for creating and managing accounts in Azure Active Directory, but another team that is responsible for Office 365 management, and thus for creating user mailboxes. A third team might take care of Azure resources. In this resource team, some administrators manage databases, other people take care of networks, and then there are your VM specialists who only manage compute resources. SoD is also part of multi-step approvals. For example, when one of your administrators requests a role that they are eligible for, someone else has to approve that request.

When starting your cloud journey with Microsoft Azure, the first thing to do is to plan your Azure hierarchy (as shown in *Figure 2.1*). The idea is to ideally only have one contract for one enterprise, but to have several Azure subscriptions. Every Azure subscription is tied to exactly one Azure Active Directory tenant (of which you ideally also only have one!). What you possibly want to do is to divide your company into locations, such as Europe and North America. Or maybe you want to divide your company based on departments, such as the IT and finance departments. Maybe you want to have it all. Or you even chose another hierarchy. Whatever fits your needs, go for it and do it before actually starting.

Plan first, act second!

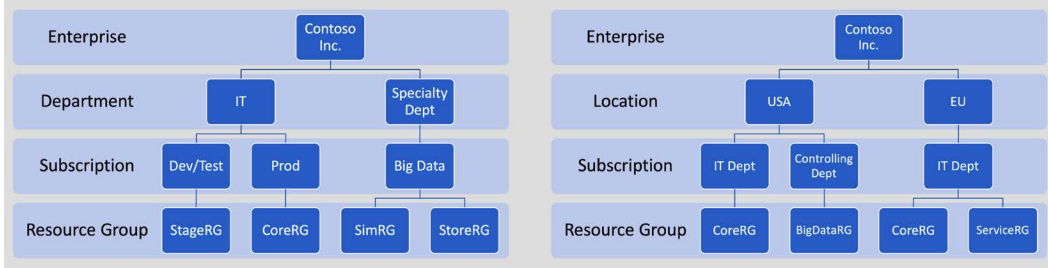


Figure 2.1 – Plan first, act second – defining a possible Azure hierarchy

Now, that you know what governance is and why it is important, let's move on and see what tools we have to make our lives easier in terms of governance and security.

Using common sense to avoid mistakes

I have often been asked how I make sure that an Azure administrator does not accidentally delete production resources in the cloud. The answer, besides the use of technical features to be discussed in the next section, is that *common sense* helps a lot! You don't accidentally walk into your on-premises data center and pull one of your servers out of its rack, just to throw it out of a window afterward, do you? You usually do not run a *nuke all* script against your Hyper-V farm just to wonder where all your VMs have gone later that day, do you? And you surely do not accidentally erase your production SAN! There's a common opinion that, on Azure, you cannot accidentally delete a resource group.

Well, you can, if you accidentally confirm the *Are you sure?* dialog and then accidentally enter the resource group's name into the prompt. Resources are even more easily removed – either by accident, or by intention. So, we need something that helps us to reduce this risk of accidents. Common sense and a good sense of caution are the first steps toward a successful governance approach in the cloud.

However, even when being careful, you can still accidentally delete stuff that you shouldn't. For example, you could accidentally delete single resources, such as a storage account, a key vault, or whatever else. I remember a summer afternoon about 12 years ago when I was running a Linux firewall distribution on a small-sized hardware mainboard, comparable with today's Raspberry Pis. I used this tiny piece of hardware as my home firewall solution, and it was doing a great job. It had three local area network connections for three different networks, there was even a Wi-Fi module onboard, and the OS was installed on a **compact flash (CF)** card. Only RAM was limited and since the CF card was exclusively reserved for the OS installation, all logs were written to a RAM disk. This is why I had to regularly delete log files from the `/var/log/*` directories. I did not want to enable log rotation, but rather export the log to another Linux server that was running in my environment. Well, long story short: what I can tell you is that running the following command while having elevated rights works pretty well, even if you have not changed your working directory to the intended one:

```
rm -rf *
```

The result was that I had one more Saturday afternoon re-installing and re-configuring the firewall. Next time, think before you act! Lesson learned.

In the next section, we will explore a technical feature to prevent administrators from accidentally deleting Azure resources.

Using management locks

Of course, there is a technical feature that prevents you from accidentally deleting anything on Azure – a feature called **management locks**. There are two different types of locks in Azure:

- **Delete** locks ensure that no one can delete resources from your Azure subscription, by accident or on purpose. Authorized users can still read and modify a resource, but they can no longer delete it.
- **ReadOnly** locks make sure that only authorized users can read a resource, but also that they cannot modify it nor delete it.

In every subscription I create, I usually use a *core resource group* to which I deploy resources that are used across several other resource groups. For example, if I have a virtual network that is used by several services across the entire subscription, or an Azure key vault in which I store administrative credentials as secrets, then these types of resources are created in one of my core resource groups. As you can imagine, the resources there are very important, and so I always use a Delete Lock in the scope of this core resource group.

When talking about scope in this chapter, we are talking about management hierarchies. Locks, policies, and access rights always apply to a particular scope and they always apply from the top down.

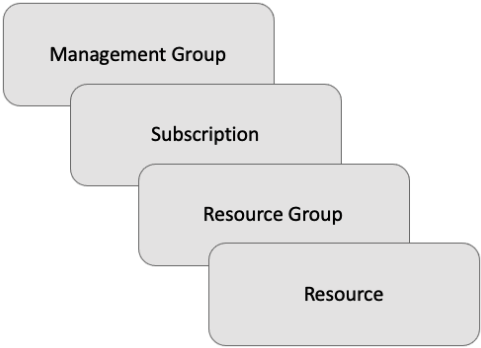


Figure 2.2 – Azure scope from the top down: management group, subscription, resource group, and single resource

If you create a lock at the subscription level, it will apply to all resource groups and to all resources within that particular subscription. If you create it at the resource group level, it will only apply to resources within that particular resource group.

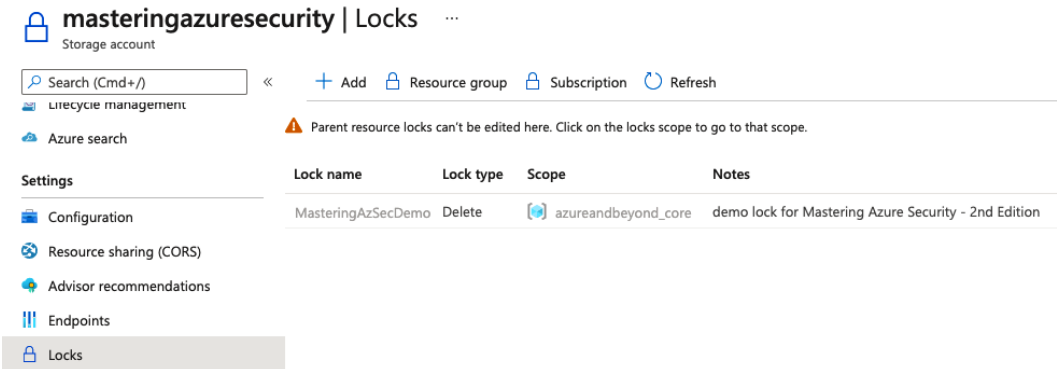


Figure 2.3 – Azure management lock on a resource group

To demonstrate, I have created a storage account, `masteringazuresecurity`, in one of my core resource groups. If you selected **Locks** in the left management pane, you would see that someone has created a **Delete** lock with the name `MasteringAzSecDemo` at the resource group level. When you now try to delete the storage account, the following message will appear:

[All services](#) > [masteringazuresecurity](#) >

Delete storage account ...

masteringazuresecurity

 'masteringazuresecurity' can't be deleted because this resource or its parent has a delete lock. Locks must be removed before this resource can be deleted. [Learn more about delete locks](#) 

Figure 2.4 – Warning message when trying to delete a locked resource

Important Note

You can only create or delete management locks when you have access to the `Microsoft.Authorization/*` or `Microsoft.Authorization/locks/*` management actions. There are several built-in roles in Azure, of which only *Owner* and *User Access Administrator* are granted those particular management actions.

Now you know how management locks are used in Azure and about the scopes that can be used to create them. In the next section, we will explore the use of management groups, one of the most important levels of scope, and how you can leverage them to define your cloud governance.

Using management groups for governance

As previously mentioned, it is essential to have a plan and stick to it in terms of governance, guardrails, rules, and policies. Organizations that use many subscriptions need an efficient way to manage access, policies, and compliance rules for all their subscriptions. With Azure *management groups*, we are given a management level of scope above the subscription level.

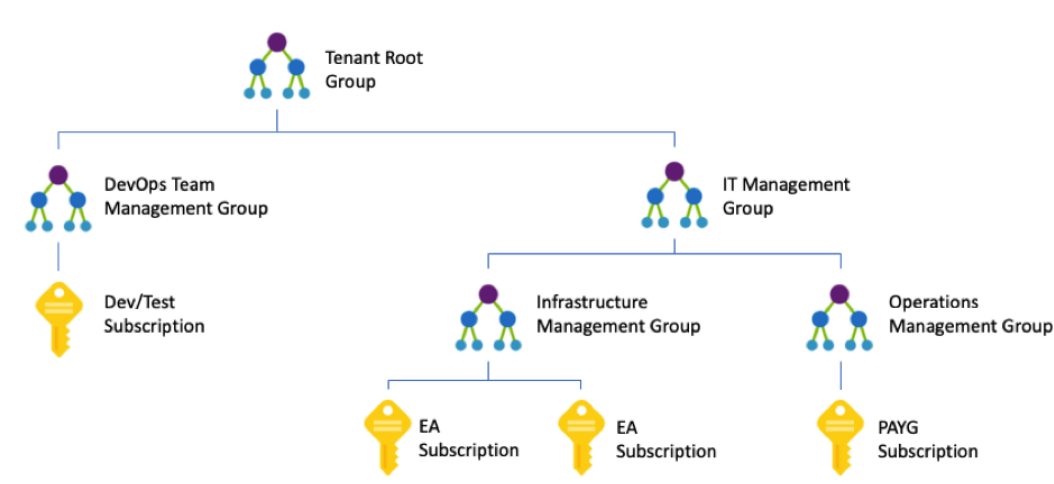


Figure 2.5 – Example of a management group hierarchy

With management groups, we can configure different management scopes that allow us to granularly manage governance settings with little effort. The preceding diagram shows a possible management group hierarchy. As you can see, a management group can contain both subscriptions and other management groups. The example shows a hierarchy in which several types of subscriptions have been created and attached to different management groups. You can have **pay-as-you-go (PAYG)** subscriptions, **enterprise agreement (EA)** subscriptions, and **dev/test** subscriptions, such as a Visual Studio subscription, in a single tenant and attach them to any of your management groups.

Maybe you want to make sure that all the resources you create in Azure for production are only stored in any of the European Azure regions – West Europe and North Europe. In this scenario, you can create a management group that contains all your production subscriptions and then apply a policy to the management group that limits resource creation to only those regions. Another scenario for management groups might be to provide user access to several Azure subscriptions at once instead of managing access for every single subscription individually.

Each Azure AD directory is given a top-level management group called a root management group. All subscriptions and management groups that are created for this Azure tenant belong to this root group. The root management group enables customers to create global policies and role assignments that are valid for all Azure subscriptions within an Azure tenant's scope.

The root group's default display name is **Tenant Root Group**. Here are some important facts that you should know regarding a tenant root group:

- Only user accounts that have been assigned Owner or Contributor roles on the tenant root group can change their display name.
- The root group cannot be deleted or moved to another management group.
- In an Azure hierarchy, all subscriptions and management groups fold up to the one tenant root group in the respective directory. There is only one tenant root group, and all other management groups, subscriptions, and resources are child objects within this tenant root group. That said, all resources within all subscriptions fold up to the tenant root group for global management. New subscriptions are automatically attached to the tenant root group when created.
- No one is given default access to the tenant root group. Only Azure AD global administrators have the right to elevate themselves to gain access and to make changes if necessary.

You may want to give your security administrators read access to all resources that are created within the scope of your Azure AD tenant as they need to see all your subscriptions and management groups. Alternatively, you may want to give application access to all your Azure subscriptions; for example, the ability to automatically deploy resources or to gather billing-related information. Therefore, you might want to change some settings on your tenant root group so that you only have to manage access rights once. As no one is able to access a tenant root group by default, you first need to elevate access for an Azure AD Global administrator.

When elevating access for an Azure AD Global administrator, the account is assigned the User Access Management role in Azure at the root scope (/). This role at the root scope allows the account to view all resources and assign access to any subscription or management group in the whole directory!

To elevate access, carry out the following steps:

1. Sign in to the Azure portal or Azure Active Directory Admin Center as a Global administrator.
2. Click **Azure Active Directory** and then **Properties** in the navigation pane.

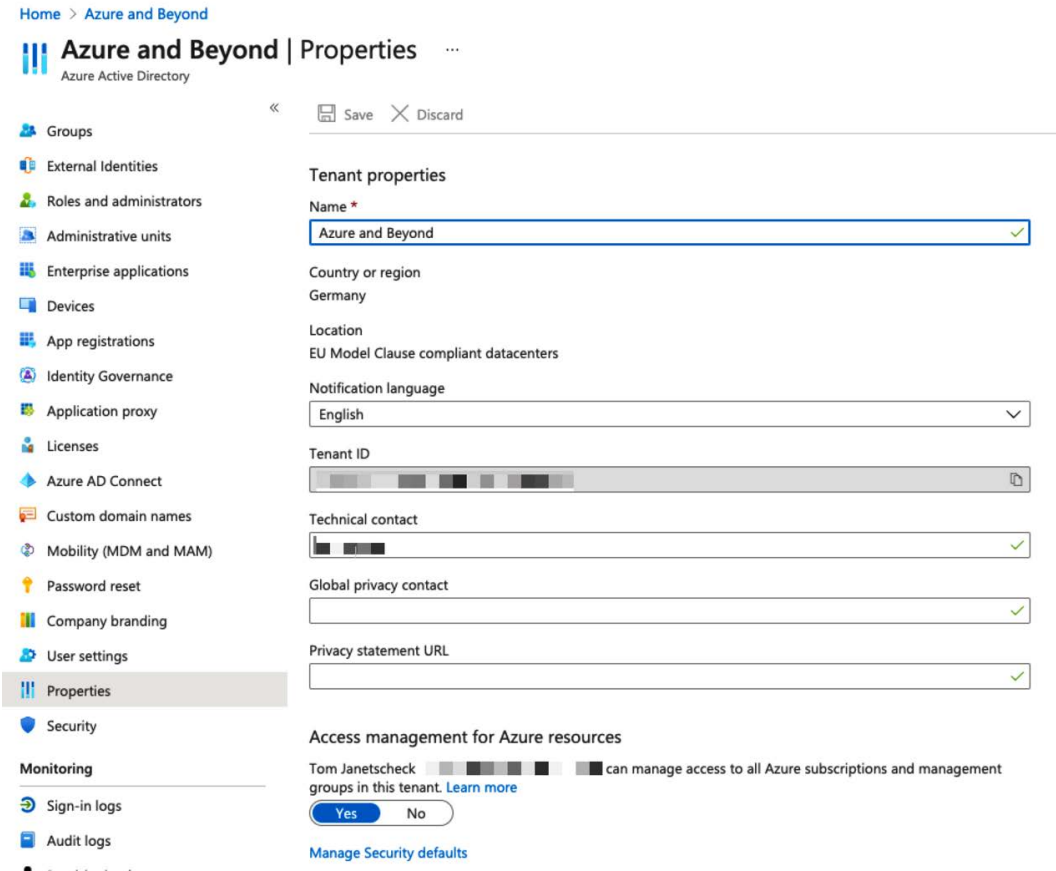


Figure 2.6 – Azure Active Directory – Properties

3. Under **Access management for Azure resources**, set the toggle switch to **Yes**, as shown in the preceding screenshot.

Tip

Access rights should always be as restrictive as possible, and they should only be granted when needed. That said, be sure to remove elevated access once you have made your changes to the tenant root group.

To remove elevated access, either set the toggle switch back to **No** or use PowerShell to remove the `User Access Administrator` role assignment from the root scope:

```
Remove-AzRoleAssignment -SignInName <username@example.com>  
-RoleDefinitionName "User Access Administrator" -Scope "/"
```

Important Note

To be able to execute this PowerShell command, you need to install the latest version of the *Az PowerShell module*. As you cannot have both, the old *AzureRM*, and the new *Az PowerShell* modules installed in parallel, you first need to uninstall the *AzureRM* module. For more information, please refer to <https://docs.microsoft.com/powershell/azure>.

You just learned how management groups are used for governance. In the next section, we will look at Azure Policy, the service that enforces your rules.

Understanding Azure Policy

Rules are important, but to be sure that they are not broken, you either need to monitor their application or you need to enforce them. With Azure Policy, you get a service that you can use to achieve both. Azure Policy allows you to create, assign, and manage policies. Policies that you define enforce different rules for resources that you create in a policy's scope.

The Azure Policy service evaluates resources for non-compliance with assigned policies and then applies a defined action. For example, you may want to only allow your Azure administrators to create Azure resources in the North Europe and West Europe Azure regions, or you may only want to have a certain VM SKU size in one of your Azure subscriptions. In these cases, you can create a policy. Once this policy is created and activated, new and existing resources are evaluated for policy compliance. New resources can be prevented from being created if they are non-compliant and existing resources can be rendered compliant if needed.

There are several different effect types that you can define for your policies and which are evaluated in a particular order:

- **Disabled** is checked first to determine whether a policy's rule should be evaluated. It is useful when testing the effect of a new policy definition or when you want to disable only one assignment instead of all assignments of a particular policy definition.

- **Append** and **Modify** are evaluated next. Append is used to add additional fields to a requested resource during resource creation or a resource update. With an append policy, you can, for example, specify allowed IP addresses for a resource. The modify effect is helpful when adding tags to a resource, for example, **resourceOwner** or **costCenter**.
- **Deny** is then evaluated and used to deny a resource request that does not match your compliance standard. The resource request will fail after evaluation. With a deny policy, you can, for example, prevent your administrators from creating resources in Azure regions that you do not allow, or prevent them from deploying VM SKU sizes you have not approved.
- **Audit** and **AuditIfNotExists** are evaluated last and used to create a warning event in the activity log when evaluating a non-compliant resource. The resource request will not be stopped, but you will be able to be informed about non-compliance.
- Similar to **AuditIfNotExists**, the **DeployIfNotExists (DINE)** effect will evaluate a resource and, if it is non-compliant, start a template deployment to auto-remediate the non-compliant resource. For example, if you create a storage account without requesting HTTPS access, you can use a DINE policy to enforce this setting on the storage account.

When using Azure Policy, you might need to create a custom policy definition or use one of the built-in policy definitions that already come with Azure, such as any of these:

- **Allowed locations:** This policy definition restricts the available locations for new resources. It is used to enforce your geo-compliance requirements.
- **Audit VMs that do not use managed disks:** This policy audits VMs that do not use managed disks. A warning event for every VM will be generated in the activity log.
- **Allowed virtual machine size SKUs:** This definition specifies a set of VM SKUs that you can deploy.

Policy definitions describe resource compliance and what effect to take when a resource is, or becomes, non-compliant. The schema for policy definitions is documented at <https://schema.management.azure.com/schemas/2018-05-01/policyDefinition.json>. Policy definitions are created in JSON.

A policy definition contains the following elements:

- Mode
- Parameters
- Display name (as it is found in the Azure portal or in the CLI)

- Description (what this policy is actually doing, when to use it, and so on)
- Policy rule (the rule definition)
- Logical evaluation (what is the condition for resource compliance)
- Effect (what happens if a resource is non-compliant)

Mode

The policy mode determines which resource types are evaluated by the policy. There are two modes supported:

- **all**: All resource groups and resource types are evaluated.
- **indexed**: Only resource types that support tags and locations are evaluated.

If you create a policy definition through the Azure portal, the mode is always set to *all*. If you use PowerShell or the Azure CLI, you can specify the mode value manually.

Important Note

If your custom policy definition does not include a mode value, it defaults to *all* in PowerShell and to *null* in the Azure CLI. The *null* value is the same as using *indexed* for backward compatibility reasons.

Parameters

Parameters in a policy definition help to reduce the number of policy definitions. You can think of them as fields in a form, such as name, surname, and street address. The parameters always stay the same; only their values change depending on who fills the form out. By including parameters in your policy definition, you can reuse the policy and just change the values accordingly.

Parameter properties

A parameter has the following properties:

- **name**: The name of your parameter.
- **type**: The parameter type can be `string`, `array`, `object`, `boolean`, `integer`, `float`, or `datetime`.
- **metadata**: The parameter's sub-properties used by the Azure portal to display user-friendly information about the parameter. These include `description`, `displayName`, `strongType`, and `assignPermissions`.

- `description`: An explanation of what the parameter is used for.
- `displayName`: A friendly name for the parameter shown in the Azure portal.
- `strongType`: (Optional) This property is used when assigning the policy through the Azure portal. A current list of supported `strongType` options is published at <https://docs.microsoft.com/azure/governance/policy/concepts/definition-structure#strongtype>.
- `assignPermission`: (Optional) If this value is set to `true`, Azure Portal will create role assignments during policy assignment. This option can be useful if you want to assign permissions outside the policy's assignment scope.
- `defaultValue`: (Optional) If no value is specified during policy assignment, this value is used. `defaultValue` is mandatory if you update an existing policy definition that is already assigned.
- `allowedValues`: (Optional) This property provides an array of values that the parameter will accept during policy assignment.

You may want to restrict Azure resource creation to just a few Azure regions. In a policy that is used to only allow resource creation in European Azure regions, you could define a parameter called `allowedLocations`. With each policy assignment, this parameter would be used and evaluated. With the `strongType` value defined, you would get an additional field in the Azure portal when assigning the policy:

```
"parameters": {  
  "allowedLocations": {  
    "type": "array",  
    "metadata": {  
      "description": "The list of allowed locations for  
resources.",  
      "displayName": "Allowed locations",  
      "strongType": "location"  
    },  
    "defaultValue": [ "westeurope" ],  
    "allowedValues": [  
      "northeurope",  
      "westeurope"  
    ]  
  }  
}
```

The preceding code shows the parameter definition section in an Azure policy definition. The `allowedLocations` parameter, in this case, is an array that contains two allowed values, `northeurope` and `westeurope`. The default value is set to `westeurope`, which means that this is the location that is taken by default if you do not set the parameter to another value. Then, in the policy rule, the parameter is referenced as follows:

```
"policyRule": {
  "if": {
    "not": {
      "field": "location",
      "in": "[parameters('allowedLocations')]"
    }
  },
  "then": {
    "effect": "deny"
  }
}
```

The whole policy definition for this purpose might look like this:

```
{
  "properties": {
    "mode": "all",
    "parameters": {
      "allowedLocations": {
        "type": "array",
        "metadata": {
          "description": "The list of locations that  
can be specified when deploying resources",
          "strongType": "location",
          "displayName": "Allowed locations"
        }
      },
      "defaultValue": [ "westeurope " ]
      "allowedValues": [
        "northeurope",
        "westeurope"
      ]
    }
  }
}
```

```
    }  
  },  
  "displayName": "Allowed locations",  
  "description": "With this policy you can restrict  
resource creation to Azure regions your organization allows.",  
  "policyRule": {  
    "if": {  
      "not": {  
        "field": "location",  
        "in": "[parameters('allowedLocations')]"  
      }  
    },  
    "then": {  
      "effect": "deny"  
    }  
  }  
}
```

Policy assignments

When you create a policy definition, you need to assign it to a specific scope for the policy to take effect. This is what Microsoft calls a policy assignment. The scope for a policy assignment can be anything from a management group over a subscription down to a resource group. Policy assignments are passed on from parent to child resources. So, policies that are assigned to a management group or a subscription are also applied to all downstream resources within that scope. However, you can also exclude a sub-scope from the policy assignment. For example, say you want to prevent your administrators from creating resources outside Europe, but you have one resource group in which you need to create IoT resources outside Europe. In this case, you could create a policy definition that only allows resource creation in one of the European Azure regions. This policy is assigned to the management group, meaning it will be applied to all subscriptions, resource groups, and resources within that scope. Then, you exclude the one resource group, except for the policy assignment, so the policy will not be applied to that group.

Initiative definitions

Policies are used to enforce exactly one rule. Initiatives are collections of several rules that belong together. For example, there is a pre-configured initiative definition that comes with Azure Security Center. In this initiative definition, you will find all the audit policies that will reflect in the recommendations section of Azure Security Center, which is covered later in this book. Initiatives are used to simplify policy assignments. With initiatives, you do not need to assign several policies. You assign one initiative and add the corresponding policies to it.

Initiative assignments

Just like with policies, initiatives need to be assigned to a specific scope in order to be applied. The scope for initiative assignments is identical to a policy assignment's scope.

Policy exemptions

Policy exemptions offer a great way to fine-tune policy assignments. As you learned before, you can exclude a scope from an assignment, which is a static setting. So, for example, in case you assign a policy or initiative definition to a management group, you can select to exclude one particular subscription from the assignment. With policy exemptions, you can exempt particular resources or scopes with additional contextual information. A policy exemption has one of two exemption categories:

- *Waiver* is selected in case you want to temporarily accept the non-compliant state of a resource.
- *Mitigated* is selected in case the policy intent is already met by a different method or process.

In addition, policy exemptions can be created with an expiration date, which is great in combination with the waiver category.

The best aspect of policy exemptions is the fact that, for compliance reasons, it's always possible to find out *who* has created *what* exemption for *what* reason. Even after the expiration date is reached, the exemption is not deleted, but preserved for record-keeping; only the exemption will no longer be effective.

Figure 2.7 shows policy exemption details for an exemption that has been created based on the Azure Security Benchmark, which is the default policy initiative used by Azure Security Center. The **Created by** field is immutable and automatically filled with the account name of the person who has created the exemption. The scope in this case is a single VM, which should be excluded from an assessment within Azure Security Center.

Edit exemption

Contoso-Linux-VM

Basics

Policies

Review + save

i Policy exemption is now available! For pricing details, see <https://aka.ms/policypricing>

Policy exemptions are used by Azure Policy to exempt a resource hierarchy or an individual resource from evaluation of initiatives or definitions.

Exemption scope ⓘ

Contoso/CONTOSODEMO/Contoso-Linux-VM

Assignment name ⓘ

Azure Security Benchmark

Exemption name * ⓘ

ASC-Management ports should be closed on your virtual machines

Exemption ID

/subscriptions/ /resourceGroups/CONTOSODEMO/providers/Microsoft.Compute...

Exemption category * ⓘ

Waiver

The exemption is granted because the non-compliance state of the resource is temporarily accepted.

Exemption expiration setting ⓘ

☒ The exemption does not expire

Exemption description ⓘ

accepting the risk for demo

Created by

Figure 2.7 – Policy exemption details

Important Note

In order to create a policy exemption, your account needs access to the read and write verb of the `Microsoft.Authorization/policyExemptions` operation group on the resource or scope the exemption is being created on. These actions are included in the built-in roles, *Resource Policy Contributor* and *Security Admin*. Exemptions, however, come with additional security measures, meaning that the creator of an exemption needs the `exempt/action` verb on the target assignment in addition.

Policy exemptions are an important capability to be used in Azure Security Center, so we will take another look at it in *Chapter 7, Microsoft Defender for Cloud*.

Policy best practices

When it comes to policy, there are some best practices that you should keep in mind:

- Define policies and initiatives at the management group level. By doing so, you can assign them to all child subscriptions and resource groups without needing to redefine them. If you define policies and initiatives at the subscription level, you can only assign them to this single subscription. So, in short, definitions should be created at the management group level, while assignments can be at the management, subscription, or resource group level.
- As always, before blocking your users from working, you should first test your new policies. You can do so by defining audit policies instead of starting with a deny policy. With the audit effect, you can get a feeling for the impact your policy definition will make. A deny policy could break your DevOps deployment chain and, with an audit effect, you will get an idea of what your policy will do later.
- It is a good idea to create initiative definitions, even if you only want to create a single policy definition. If you have an initiative, you can easily add further policies later if you need to do so.
- All policies within an initiative definition are evaluated when the initiative definition is evaluated. If you have a particular policy you do not want to be evaluated within that context, you should remove it from the definition and assign it individually.
- Instead of disabling a policy definition or creating several assignments, consider using policy exemptions in case you want to exempt a resource or scope from being evaluated. Policy exemptions are great for keeping track for compliance reasons.

In the preceding sections, you have learned about governance in general, scopes, policies, and locks. In the next section, you will learn how to bring it all together by defining Azure blueprints.

Defining Azure blueprints

Now that you know how to define your Azure hierarchy and how to work with locks and policies, you might want to create a template that is valid for all your subscriptions and for those that are yet to be created as well. With the Azure blueprints service, you get exactly what you need for this purpose. A blueprint, in this case, is a repeatable template that you define once and then use during the creation of all your Azure subscriptions in the future. The good thing here is that you can define organizational sets of rules and automatically apply them to all your subscriptions while accelerating the deployment process at the same time. With Azure Blueprints, you can declaratively deploy Azure resources and artifacts to all your subscriptions, such as the following:

- Role assignments
- Policy assignments
- ARM templates
- Resource groups
- Locks

In the section about management locks in this chapter, I explained that I usually have a core resource group in all of my subscriptions and my customers' subscriptions. These resource groups are usually locked so they are protected from accidental deletion. In larger organizations, you might want to give your developers or specialty departments options for creating their own environments without breaking your rules. No matter what it is, as soon as you want to automate complex deployments of standardized Azure environments, Azure Blueprints is your service of choice!

Blueprint definitions

Azure blueprints are defined by so-called artifacts. An artifact can currently be one of the following:

- **Resource group:** A resource group that is created by a blueprint can be used by other artifacts within the scope of the blueprint. For example, you can create a resource group in a blueprint and then reference an ARM template to this new resource group.

- **Policy assignment:** You can assign an existing policy to a subscription or resource group that a blueprint is assigned to. For example, you can assign the `only-European-Azure-locations` policy from one of the preceding chapters in this book.
- **Role assignment:** You can assign management access to a subscription that this blueprint is assigned to. For example, you can automatically assign contributor rights to new subscriptions for your infrastructure administrators.
- **ARM template:** With an ARM template, you can declaratively deploy complex Azure environments. ARM templates can be used within the scope of Azure blueprints. For example, you can automatically create a new Log Analytics workspace in a core resource group that is created within the scope of a blueprint.

Important Note

A blueprint definition can be saved either to a management group or a subscription. If you create a blueprint definition at the management group level, you can use the blueprint for assignment on all child subscriptions within the scope of this particular management group.

Blueprint publishing

Every new blueprint that is created will initially be in draft mode. After finishing all configurations, the blueprint needs to be published. When publishing a blueprint, you need to define a version string and optional change notes. When additional changes are made to this blueprint, the published version will still exist and changes are done in draft mode, again.

When changing a blueprint (and saving the changes), several versions of the same blueprint will exist to make sure that you can still assign old versions and that published versions are not touched when applying changes to the blueprint.

You can start by creating a new blank blueprint or selecting one of the blueprint examples from the Azure portal. For example, there are blueprint definitions that assign policies that are necessary for addressing specific regulatory compliance controls, including **Australian Government ISM PROTECTED** or **CIS Microsoft Azure Foundations Benchmark v.1.1.0/1.3.0**.

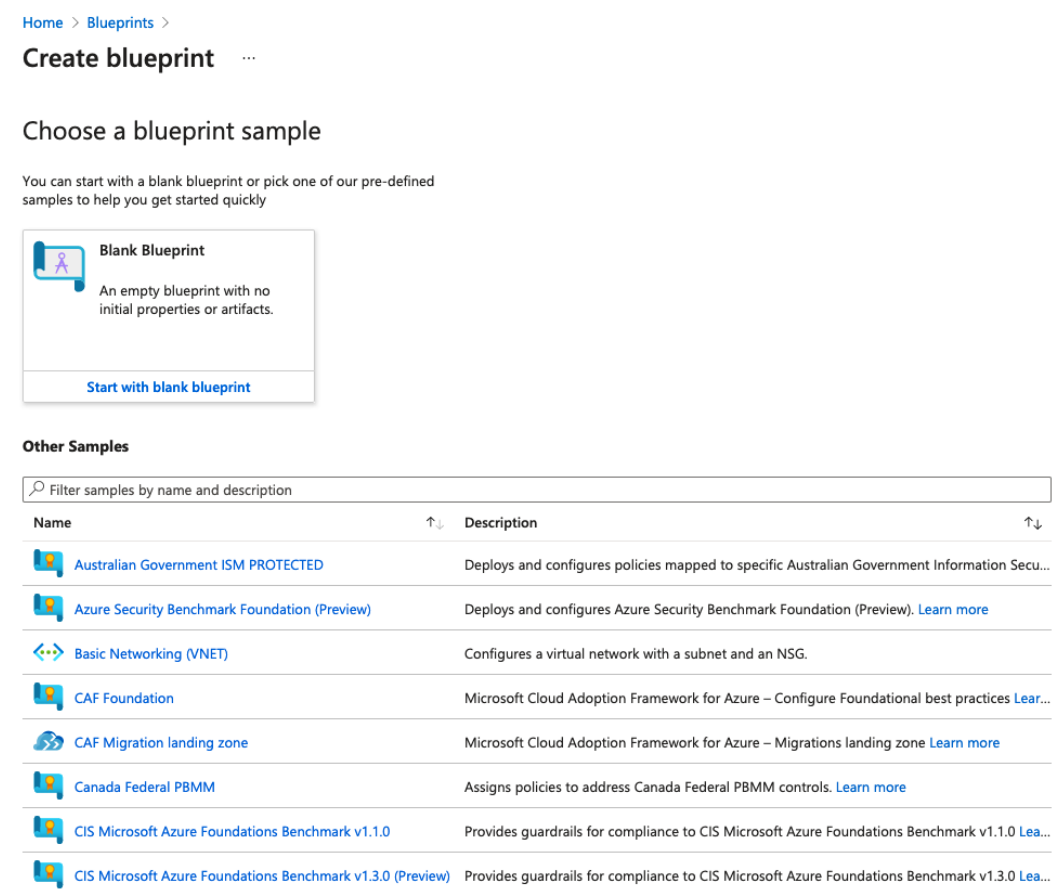


Figure 2.8 – When creating a blueprint definition, you can select one of the predefined sample blueprints

To start creating a blueprint definition, follow these steps:

1. Navigate to **All Services | Blueprints**.
2. Select **Create**.

3. When the wizard appears, you need to give your blueprint a name and then select the location to save your blueprint definition. In the following screenshot, I decided to save the blueprint in **Tenant Root Group** to be able to use the blueprint in all subscriptions that are, or may later be, attached to my Azure AD tenant:

[Home](#) > [Blueprints](#) >

Create blueprint ...

Basics Artifacts

Blueprint name * ⓘ

MasteringAzureSecurity_2ndEd ✓

Blueprint description

Assigns policies to address specific ISO 27001:2013 controls. ✓

Definition location * ⓘ

Tenant Root Group ✓ ...

The management group or subscription where the blueprint is saved. The definition location determines the scope that the blueprint may be assigned to. Learn more at aka.ms/BlueLocation.

[Save Draft](#) [Discard](#) [Next : Artifacts »](#)

Figure 2.9 – Creating a blueprint – basic information

I decided to use the **ISO 27001** blueprint sample for this demonstration. On the second tab, the **Artifacts** tab, you can add, remove, or edit artifacts according to your needs. You can save your draft at any time and come back later if you want to change, add, or remove anything.

Name	↑↓	Latest Version	↑↓	Unpublished chang...↑↓	Last modified	↑↓	Definition location	↑↓
 Contoso		1.0		No	10/30/2020		Tenant Root Group	***
 masteringAzureSecurity		0.1		No	9/16/2019		Tenant Root Group	***
 MasteringAzureSecurity_2ndEd		Draft		Yes	9/20/2021			***
				Yes	12/6/2018			***
		1.0		Yes	12/6/2018			***

View blueprint
Publish blueprint
Edit blueprint
Delete blueprint

Figure 2.10 – Context dialog after saving the blueprint in draft mode

- When you are done, you need to publish your blueprint definition to a new version using the **Publish blueprint** option from the menu shown in *Figure 2.10*. The version is basically a custom text field that you need to define. It makes sense to select version numbers so that you can easily iterate through your versions. However, you can use whatever blueprint version names fit your needs. I decided to name the blueprint version 1.0:

Name	↑↓	Latest Version	↑↓	Unpublished chang...↑↓	Last modified	↑↓	Definition location	↑↓
 Contoso		1.0		No	10/30/2020		Tenant Root Group	***
 masteringAzureSecurity		0.1		No	9/16/2019		Tenant Root Group	***
 MasteringAzureSecurity_2ndEd		1.0		No	9/20/2021			***
				Yes	12/6/2018			***
		1.0		Yes	12/6/2018			***

View blueprint
Edit blueprint
Assign blueprint
Delete blueprint

Figure 2.11 – Context dialog after publishing the new blueprint

After publishing your blueprint, the context dialog changes and gives you a new menu option, **Assign blueprint**.

- You can now assign your new blueprint to any of your existing subscriptions or choose to create a new subscription direct from the dialog.

[Home](#) > [Blueprints](#) >

Assign blueprint ...

Basics

Subscription(s) * ⓘ

Contoso ▼

Create new [Learn more about creating subscriptions.](#)

Assignment name * ⓘ

Assignment-MasteringAzureSecurity_2ndEd ✓

Location * ⓘ

West US 2 ▼

Blueprint definition version * ⓘ

1.0 ▼

Lock Assignment

☒ Don't Lock ☐ Do Not Delete ☐ Read Only

The assignment is not locked. Users, groups, and service principals with permissions can modify and delete deployed resources. [Learn more](#)

Managed Identity ⓘ

☒ System assigned☐ User assigned

i By clicking "Assign" with a system assigned identity, you agree to grant the Azure Blueprints service temporary Owner access to this subscription so that we can properly deploy all Artifacts. We will automatically remove this access when the blueprint assignment process is finished.

Blueprint parameters

Allowed locations for resources and resource groups ⓘ

0 selected ▼

Assign

Cancel

Figure 2.12 – New blueprint assignment

As soon as you assign your blueprint, the policies you defined in the blueprint definition are applied, and resource groups and resources are created. You can also assign a lock when assigning a blueprint. If you apply a lock during a blueprint assignment, you can only remove it when unassigning the blueprint.

Important Note

You can assign a management lock when assigning a blueprint to a subscription. However, if you do so, the lock cannot simply be removed by a subscription owner, but only when unassigning the blueprint from the subscription.

You need to define a location for your blueprint assignment. This is because blueprints use **Managed Identity (MI)** to deploy the artifacts you define in your blueprint definition, such as resources and resource groups. You can either use a *system-assigned MI* or decide to use a *user-assigned MI*. If you use a system-assigned MI, which is the default, this MI is temporarily granted owner rights on the subscription(s) within the scope of your blueprint assignment. Owner access is needed to make sure that all your artifacts and locks can properly be created and set by the Blueprints service. When the blueprint assignment process is finished, owner access for the system-assigned MI is automatically removed from your subscription(s).

Depending on the artifacts you have defined in your blueprint definition, you might have to define your artifact parameters during the assignment process.

Home > Blueprints >

Assign blueprint ...

Blueprint parameters

Allowed locations for resources and resource groups ⓘ

0 selected ▼

Artifact parameters

Artifact / Parameter	Parameter Value
▼ ⓘ Subscription	
▶ ⓘ [Preview]: Deploy Log Analytics Agent for Linux VM Scale Sets (V...	
▶ ⓘ [Preview]: Deploy Log Analytics Agent for Linux VMs	
▶ ⓘ [Preview]: Deploy Log Analytics Agent for Windows VM Scale Sets ...	
▼ ⓘ [Preview]: Deploy Log Analytics Agent for Windows VMs	
Log Analytics workspace for Windows VMs ⓘ	Set value(s)
Optional: List of VM images that have supported Windows OS to add to scope ⓘ	

Figure 2.13 – Artifact parameters are configured during blueprint assignments

In our example, we started with the ISO 27001 blueprint example. This blueprint definition contains several policies, such as policies for restricting resource deployment to only a few Azure regions or for deploying Azure Log Analytics agents to Windows and Linux VMs. All parameters for these policies could have been defined in the blueprint definition already. However, if we had done so, all parameters would always be the same for all subscriptions and could not be defined depending on the subscription you assign a blueprint to. This might be great for corporate restrictions that need to be enforced for every system environment, but it would restrict the blueprint/policy flexibility.

The best practice for this case is to define all parameters that will never be changed in the blueprint definition. For example, this could be useful if there are only certain Azure regions that are permitted for your company's resources. But for Log Analytics, it makes sense to deploy a separate workspace to every Azure region so as to avoid being charged for cross-region network traffic. This would ideally be defined during assignment of the blueprint.

Best Practice

Define artifact parameters that are valid for all your subscriptions in the *blueprint definition*. Define artifact parameters that only fit specific subscriptions during *blueprint assignment*.

Now that you know how to define your governance concepts and how all the features we have today work together, we will take a look at Azure Resource Graph, an engine that helps you to gather information about all the resources you have created within the scope of your Azure tenant.

Azure Resource Graph

Have you ever tried to get information about all the Azure resources that are deployed outside of Europe? Or what about listing all the Azure VMs across all your subscriptions? In the *Understanding governance in Azure* section, we mentioned that monitoring is essential for security and that you are clueless if you do not have effective monitoring practices. Now, if you tried to get information such as the information we have discussed when covering ARM, you might end up waiting for a long time due to performance limitations. You might not even have the ability to get all the information you need at once. Azure Resource Graph is a service that helps you to gather information about all your Azure resources across all your tenant's Azure subscriptions. Sounds great, doesn't it? Well, it is great.

You can think of Azure Resource Graph as a large index database for all your resources that can be queried using **Kusto Query Language (KQL)**. As soon as you create a new or update an existing resource, ARM notifies Azure Resource Graph of this change and the Azure Resource Graph database is updated. To make sure that there is no notification missed, and in case you update resources outside of ARM, Azure Resource Graph regularly runs a full scan of your resources in addition to receiving the notifications, meaning that the database is kept current.

In order to be able to use Azure Resource Graph to query your resources, you need to have at least read access to the resources you want to query. In other words, you are presented with results only for the resources that you are allowed to see. So, query results always depend on the account that is currently logged in to the Azure portal, the Azure CLI, PowerShell, or the REST API.

Querying Azure Resource Graph with PowerShell

If you want to use PowerShell to query Azure Resource Graph, follow these steps:

1. Install the latest version of the `Az.resourcegraph` PowerShell module with the following commands.

First, install the PowerShell module for Azure Resource Graph:

```
Install-Module -name Az.resourcegraph
```

Then, import the module into your current PowerShell session:

```
Import-Module -Name Az.ResourceGraph
```

2. Make sure that the module was properly installed by using the following PowerShell command:

```
Get-command -module Az.resourcegraph
```

You should see the following output:

```
PS /Users/tom> Get-command -module Az.resourcegraph
```

CommandType	Name	Version	Source
Function	Get-AzResourceGraphQuery	0.11.0	Az.ResourceGraph
Function	New-AzResourceGraphQuery	0.11.0	Az.ResourceGraph
Function	Remove-AzResourceGraphQuery	0.11.0	Az.ResourceGraph
Function	Update-AzResourceGraphQuery	0.11.0	Az.ResourceGraph
Cmdlet	Search-AzGraph	0.11.0	Az.ResourceGraph

```
PS /Users/tom> █
```

Figure 2.14 – The `Az.ResourceGraph` module was successfully installed and imported

You can now start to query Azure Resource Graph with PowerShell. Let's, for example, get a list of all Azure resources that contain the word `SecureScore` in their name. We want to limit the output to five results. Now, the interesting thing is that the `Search-AzGraph` cmdlet already gives you the database column, which is to be queried, so for the query, you only need to define your filter logic.

A KQL query usually looks like this:

```
database
|where <filter logic>
|project columnname1, columnname2
|limit n
```

Since the database name is given by `Search-AzGraph`, we only need to define the filter logic. So, for the result just described, your PowerShell command, including the query, could be as follows:

```
Search-AzGraph -Query 'project name, type | where name contains
"SecureScore" | limit 5'
```

This command will give you up to five results that contain the word `SecureScore` anywhere in the resource name.

```
PS /Users/tom> Search-AzGraph -Query 'project name, type | where name contains "SecureScore" | limit 5'

name                                     type
----                                     -
Get-SecureScoreData                     microsoft.logic/workflows
azureloganalyticsdatacollector-Get-SecureScoreData microsoft.web/connections
SecureScoreData-                        microsoft.operationalinsights/workspaces
azureloganalyticsdatacollector-Get-SecureScoreData microsoft.web/connections
Get-SecureScoreData                     microsoft.logic/workflows

PS /Users/tom> █
```

Figure 2.15 – Your first Azure Resource Graph query result with PowerShell

You've now learned how to use Powershell to query Azure Resource Graph. In the next section, you will learn how to leverage the Azure CLI for the same results.

Querying Azure Resource Graph with the Azure CLI

As you might have recognized from the preceding screenshots, I am using PowerShell Core in Visual Studio Code on macOS. This works absolutely fine, and since I am into both PowerShell and macOS, for me this is a great way to go. However, with the Azure CLI, the Python-based CLI that can run on Windows, macOS, and Linux, you can also easily query Azure Resource Graph. If you have already installed the latest version of the Azure CLI, you just need to interactively log in with the following command:

```
az login
```

You will then be informed that there is a browser window open that you can use to enter your credentials, beat the MFA challenge, and handle all the other security stuff that comes with Azure AD (and which we will address in the next chapter of this book). As soon as you have logged in, the Azure CLI will give you an overview of all the subscriptions that your account has access rights to:

```
PS /Users/tom> az login
The default web browser has been opened at https://login.microsoftonline.com/common/oauth2/authorize. Please continue the
login in the web browser. If no web browser is available or if the web browser fails to open, use device code flow with
`az login --use-device-code`.
You have logged in. Now let us find all the subscriptions to which you have access...
```

Figure 2.16 – Azure CLI – interactive login successful

If you now try to get help for `az graph` (which is the starting command to get access to Azure Resource Graph), you will be presented with a message asking you to install the graph extension into your Azure CLI environment first.

```
PS /Users/tom> az graph -h
The command requires the extension resource-graph. Do you want to install it
now? The command will continue to run after the extension is installed. (Y/n)
: █
```

Figure 2.17 – Please install the Az graph extension

You can also manually install the extension by executing the following command:

```
az extension add --name resource-graph
```

After the extension is installed, you can run a query to get the same result as described before with PowerShell. The Azure CLI command in this case will be as follows:

```
az graph query -q "project name, type | where name contains
'SecureScore' | limit 5"
```

Running this query will give you the following output:

```
PS /Users/tom> az graph query -q "project name, type | where name contains 'SecureScore' | limit 5"
{
  "count": 5,
  "data": [
    {
      "name": "Get-SecureScoreData",
      "type": "microsoft.logic/workflows"
    },
    {
      "name": "azureloganalyticsdatacollector-Get-SecureScoreData",
      "type": "microsoft.web/connections"
    },
    {
      "name": "azureloganalyticsdatacollector-Get-SecureScoreData",
      "type": "microsoft.web/connections"
    },
    {
      "name": "SecureScoreData-",
      "type": "microsoft.operationalinsights/workspaces"
    },
    {
      "name": "SecureScoreData-",
      "type": "microsoft.operationalinsights/workspaces"
    }
  ],
  "skip_token": null,
  "total_records": 5
}
PS /Users/tom> █
```

Figure 2.18 – Your first Azure Resource Graph query result with the Azure CLI

As you can see, the query itself will stay the same; it's only wrapped in an Azure CLI command.

Advanced queries

Now that you know how to run queries against Azure Resource Graph with the Azure CLI and PowerShell, you may be interested in running some advanced queries, right? So, here you go. The queries you find here can be run in both the Azure CLI and PowerShell.

First, let's try to get more information about a resource – not just the name and type, and without limiting the output to a number of results; let's filter for a particular subscription and all resources that have `SecureScore` in their name. The query might be as follows:

```
where subscriptionId == 'your subscription ID' and name
contains 'SecureScore'
```

The command for the Azure CLI is as follows:

```
az graph query -q "where subscriptionId == 'your subscription
ID' and name contains 'SecureScore'"
```

For PowerShell, enter the following command:

```
Search-AzGraph -Query "where subscriptionId == 'your  
subscription ID' and name contains 'SecureScore'"
```

You will see, depending on your access rights and the resource type, that the results of your query will now give you a lot of information.

But with Azure Resource Graph and KQL, you can create even more complex queries. The following example will provide you with information about unhealthy resources (based on Azure Security Center assessment results) in your environment. First, let's store the query in the `$query` variable:

```
$query = "securityresources  
| where type =~ 'microsoft.security/assessments'  
| extend assessmentStatusCode = tostring(properties.status.  
code)  
| extend displayname = tostring(properties.displayName)  
| where assessmentStatusCode == 'Unhealthy'  
| extend resource = tostring(properties.resourceDetails.Id) "
```

Then, you can use PowerShell or the Azure CLI to execute the query. In PowerShell, the command is as follows:

```
Search-AzGraph -Query $query
```

In the Azure CLI, the command is as follows:

```
az graph query -q $query
```

The nice thing is that even if there are lots of resources returned by your query, you will get your results in near-real time because you query the Azure Resource Graph database instead of ARM, which would have to *talk* to each and every resource first.

Summary

In this chapter, you have learned why governance is essential for security and what features Azure has to offer to help you in building a governance concept for your company. You've learned how to group and organize subscriptions and resources, how to enforce policies and all the best practices to ensure that you do so effectively and without breaking your system, and how to create consistent and repeatable environments. We know that it is not easy to find a way through this jungle of options, but when you stop, think, plan, act, and repeat, you will be on a good path to finding what fits your needs perfectly.

In the next chapter, you will not only learn how to keep track of access rights and monitor any anomalies in user behavior by learning how to manage cloud identities, but you will also learn about strategies to protect identities and how to reduce the attack surface of privileged accounts.

Questions

As we conclude, here is a list of questions for you to test your knowledge regarding this chapter's material. You will find the answers in the *Assessments* section of the *Appendix*:

1. What are the most important parts of governance?
 - A. Policies
 - B. Monitoring
 - C. Implementation
 - D. All of the above
 - E. None of the above
2. What needs to be defined in governance regarding deployments?
 - A. We need to define what can be deployed.
 - B. We need to define deployment steps.
 - C. We need to define both the *what* and the *how*.
 - D. This depends on the deployment method.

3. Which feature or capability prevents the accidental removal of resources?
 - A. Management groups
 - B. Read-only locks
 - C. Delete locks
 - D. Deny locks
4. What is the top level of a resource hierarchy?
 - A. Tenant
 - B. Resource group
 - C. Subscription
 - D. Management group
5. At which level are blueprints assigned?
 - A. Subscription
 - B. Resource group
 - C. Resource
 - D. Management group
 - E. Management group or subscription
 - F. None of the above
6. What can you define with blueprints?
 - A. Role assignments
 - B. Policy assignments
 - C. ARM deployments
 - D. All of the above
 - E. None of the above
7. What is Azure Resource Graph?
 - A. A service to control deployments
 - B. A service to gather information about deployments
 - C. Both
 - D. None of the above

3

Managing Cloud Identities

Back in the days when IT security was synonymous with blocking network traffic, life was easy. You could be sure that your physical walls constituted your perimeter, so you simply needed to protect your borders. However, today, your network endpoints are no longer enough to secure your perimeter. Today, we see an increasing number of phishing and credential theft attacks against corporate environments. We're in an age where corporate data is moving out of its secure enterprise network environment. It's the age of cloud services such as Microsoft 365 and Microsoft Azure, and so traditional security strategies aren't enough to safeguard data as the data is outside secure walls. We need a new approach to add on top of the traditional approach. This new approach is *identity is the new perimeter*. This makes identity something we need to protect more than ever before. Of course, this does not mean that we have to do away with the old method of protecting the network, and we'll explore this in *Chapter 4, Azure Network Security*.

In general, to avoid random attacks, you need to raise the bar and make an attack attempt as expensive as possible for an attacker. It's just like protecting your home: if you secure your front door with a higher **resistance class (RC)**, have grids in front of your windows, and have an automated lighting system to light up your property and your neighbor does not, then burglars looking for random targets will most probably not even try to attack your home. This is because an attack attempt against your home would probably be too time-consuming, or the probability of being detected is too high; thus, it would be too expensive from an attacker's point of view. This does not work if you are suffering a sophisticated attack that is planned and targeted against your home or identities (there is no absolute security!), but it helps against all the random attack attempts out in the wild.

In this chapter, we will not only cover strategies to protect your identities, but you will also learn how to reduce the attack surface of your privileged accounts. The topics we will cover are the following:

- Exploring passwords and passphrases
- Understanding multi-factor authentication
- Using Conditional Access
- Introducing Azure AD Identity Protection
- Understanding role-based access control
- Protecting admin accounts with Azure AD PIM
- Hybrid authentication and Single Sign-On
- Understanding passwordless authentication
- Licensing considerations

Exploring passwords and passphrases

When talking about identity security, there is one principle you always have to keep in mind: **assume breach**. The question is not *whether your accounts are attacked* but *whether you know about it* and *whether you can prevent attackers from being successful*. We can already see that there are many successful attacks against cloud identities every day. There is no single solution for protecting your identities, but you need to leverage a broader toolset to be resilient against identity attacks.

Important Note

The question is not whether your cloud identities are under attack, but whether you can ensure that attackers are not successful.

If we start with passwords, you could say that no one likes them. Well, attackers do, because passwords often give them leverage for an upcoming *identity-focused attack* attempt. Given the fact that people tend to create their passwords based on their life experiences, an attacker generally needs to be good at gathering background information about a specific user to guess their password. This, of course, means some effort on the attacker's side, and this would most probably be executed in a sophisticated attack against a high-value target. You can see that passwords are not a good solution for protecting your accounts.

Furthermore, passwords can lead to **denial of service (DoS)**. In **Active Directory Domain Services (AD DS)** on Windows Server, lots of companies employ a security policy that will lock out user accounts for a given time (or infinitely) following a given number of failed login attempts. The problem is that authenticated users—that is, users who have been able to log in against AD— can read this security policy and list user accounts from AD. An authenticated user can be any employee (internal attacker), or someone who is trying to attack a company from outside and who was able to get access to a set of login credentials (external attacker). By leveraging PowerShell, an attacker can easily do the following:

- Find out what security policy is in place
- List all user accounts
- Lock all user accounts by trying to log in with incorrect passwords
- Repeat this script every x minutes

If someone does so, no one in this directory will be able to log in again, so the company will suffer a DoS. With that being said, you should *not* implement this lockout policy but monitor login attempts instead, and then make educated decisions if something unusual happens. Keep an eye on your security event logs on Windows servers or the `syslog/auth` log domain on Linux.

Dictionary attacks and password protection

Dictionary attacks, such as brute-force and password spray attacks, still find success every day. In a dictionary attack, attackers try combinations of usernames and well-known and often-used passwords against an authentication service. The brute-force attack is more noisy and easier to recognize. With brute force, an attacker will try lots of passwords for a single user account, hoping that one of the attacks will be successful. In the backend, you will see lots of failed login attempts, so you can easily react to them. However, a password spray attack is way more sensitive since an attacker will only use a small set of passwords against lots of user accounts. If the attacks are very slow and widely distributed, it is very hard to notice an attack. To avoid successful password spray and brute-force attacks in the cloud and, to be more precise, in Azure AD, there are some easy best practices:

- Encourage your users to use passphrases instead of passwords.
- Block passwords or patterns that should not be used in your environment.

Currently, you can use passwords or passphrases with up to 256 (!) characters in Azure AD. Even if you were only to use all 26 letters from the alphabet in uppercase and lowercase, this would mean $(2 \times 26)^{256}$ possibilities, which results in a number near infinity.

With that said, and knowing that you cannot only use uppercase and lowercase letters but also numbers and other characters in a passphrase, this results in a plethora of combinations. You see that you should definitely enable and encourage your users to use passphrases with a lot of characters.

For passwords you do not want to allow in your organization, there is another option you can enable. By default, Microsoft does not allow you to use passwords or password patterns that can be found on password lists and that are known to be used in dictionary attacks. For example, a pattern that is not allowed is your username, or any part of it. In addition to that, it might make sense for your company to avoid passwords that are easily relatable to your enterprise. These could include well-known marketing phrases, corporate abbreviations, and so on. In Azure AD, there is an option to create a custom banned password list that can be used to either audit or enforce the usage of secure passwords. You can even use this custom list to enable password protection for your on-premises Windows Server AD by installing and enabling an on-premises agent.

The custom banned password list can contain up to 1,000 case-sensitive terms between 4 and 16 characters in length. One nice feature is the fact that character substitution such as *e* and 3 or *o* and 0 is considered.

The screenshot shows the Azure AD Security portal interface. The breadcrumb navigation at the top reads: Home > tjasc > Security > Authentication methods. The main heading is "Authentication methods | Password protection" with a key icon and a feedback link. Below the heading is a search bar labeled "Search (Cmd+/)".

On the left sidebar, under the "Manage" section, "Policies" is selected, and "Password protection" is highlighted. Under the "Monitoring" section, "Activity", "User registration details", "Registration and reset events", and "Bulk operation results" are listed.

The main content area is titled "Custom smart lockout". It includes two input fields: "Lockout threshold" set to 10 and "Lockout duration in seconds" set to 60. Below these is the "Custom banned passwords" section, which has a toggle switch for "Enforce custom list" set to "Yes". The "Custom banned password list" is displayed as a text area containing the following entries: "AzureSecurity", "azuresecurity", "PacktPublishing", "packtpublishing", and "securityrocks". A green checkmark is visible next to the last entry.

At the bottom, there are two more settings: "Password protection for Windows Server Active Directory" with a toggle switch for "Enable password protection on Windows Server Active Directory" set to "Yes", and a "Mode" dropdown menu currently set to "Audit" (with "Enforced" also visible).

Figure 3.1 – Configuration of a custom banned password list in Azure AD

In order to configure a custom banned password list, you need to navigate to **Azure Active Directory | Security | Authentication methods | Password protection** in the Azure portal, as shown in *Figure 3.1*. On that screen, you'll also find configuration options for the **Custom smart lockout** feature. As mentioned before, you might find an on-premises AD security policy forcing accounts to be locked after a given number of failed login attempts. This is kind of an on/off decision, and you can easily increment the lockout counter with the same password to force account lockouts. Remember that this is why you should disable such policies in your on-premises AD and monitor login attempts instead.

However, in Azure AD, smart lockout is the feature that combines the best of two worlds: letting users work without blocking them unnecessarily and preventing attackers from guessing your passwords at the same time. To achieve that goal, smart lockout comes with some interesting features:

- Users are not meant to be blocked from working. Smart lockout recognizes attack attempts and login attempts from valid users. Attack attempts are treated differently, so legitimate users are still able to work while attackers are blocked.
- Smart lockout stores the last three bad password hashes. Doing so, you cannot simply use the lockout counter to enforce DoS by using the same bad password several times.
- The default lockout threshold is 10 bad attempts with a lockout duration of 60 seconds. Smart lockout is always on for Azure AD; however, it does not guarantee that legitimate user accounts are never locked out. The service uses familiar versus unfamiliar locations to try to figure out whether a login attempt comes from a bad actor or a genuine user. However, there is still a probability that users will be blocked from logging in. And one important point is that the service does not necessarily protect you from highly sensitive password spray attacks. This is not because the service is bad but because those kinds of attacks can become really difficult to discover.

Imagine an attacker has a list of nine passwords and they use a widely spread network of bots to attack thousands of user accounts in one custom domain over a time frame of weeks. In that situation, there are some facts to realize:

- The number of login attempts per user account will not exceed the default threshold of 10.
- By using several widely spread host systems to run the attack, it is hard to discover whether a login attempt is valid or malicious. However, Microsoft will also take into account whether a login attempt comes from a known bad IP address or an unfamiliar location.

So, in this scenario, the service will probably not block an attacker. Of course, the nine passwords in the list need to be quite sophisticated to be accepted. However, there is still a probability of an attacker being successful at guessing your passwords.

Now that you know why passwords are not enough to protect your accounts, let's move one step further and see how **multi-factor authentication (MFA)** can give an additional layer of security to your accounts.

Understanding MFA

There are few technical features that protect your accounts more than using MFA. With MFA, it is not enough to know a username and a password; you are also challenged to prove who you are using another authentication factor. With MFA, you generally need to be able to log in with the following:

- Something you are, such as your user account name or a biometric attribute
- Something you know, such as a password
- Something you have, such as an additional authentication factor (smartcard, smartphone app, or security key)

Given the fact that an MFA challenge is only triggered following a successful login attempt, it is still reliant on passphrases that are not easy to guess. In other words, if an MFA challenge is triggered, the respective username/password combination has already been successfully validated (refer to the following screenshot for reference):

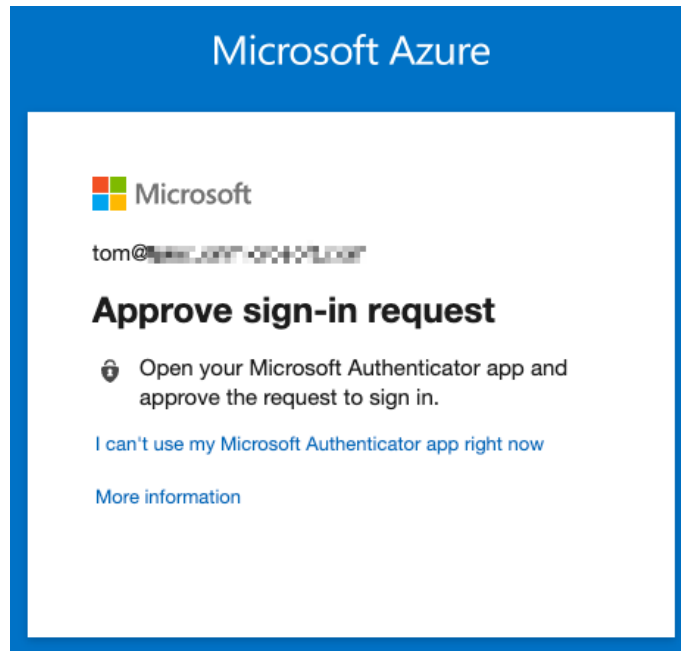


Figure 3.2 – An MFA challenge is triggered after the user's credentials have successfully been validated

Today, there are several options for using MFA in Azure AD:

- A push message from the Microsoft Authenticator smartphone app
- A **one-time password (OTP)** from the Microsoft Authenticator smartphone app
- A text message with an OTP sent to your mobile device
- A phone call to an authentication phone
- A security key or token

If you have set up MFA the right way, you can react to all situations with a combination of these options. If you do not have access to mobile data or Wi-Fi, you can use the OTP code from a text message or from your smartphone app. If you leave your smartphone at home, you can get a call to your office phone (or another authentication phone you defined during the configuration process).

Important Note

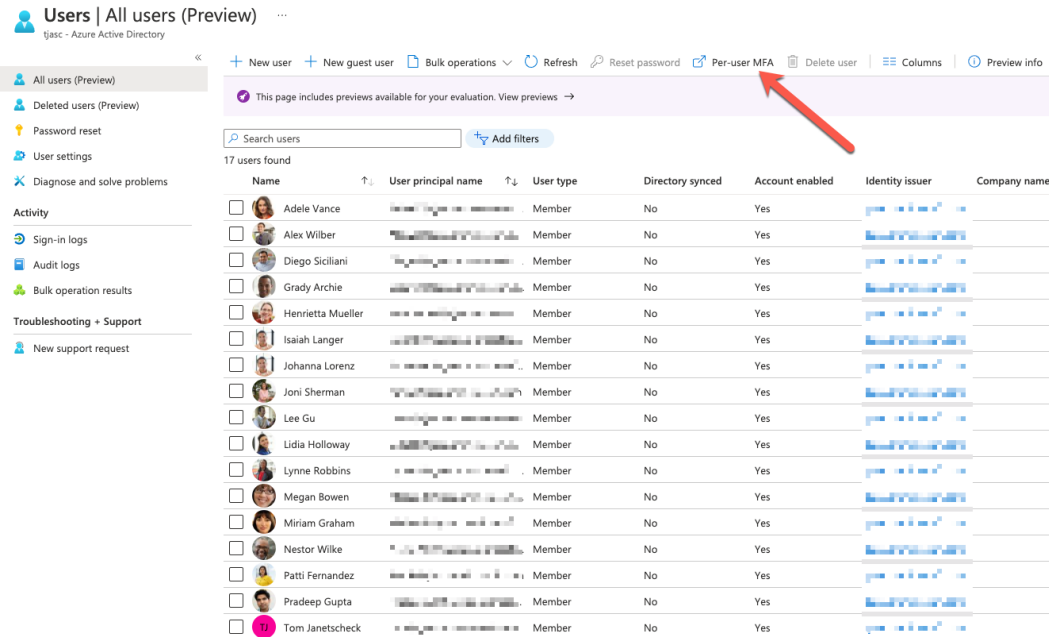
It's important to understand that you should not use your mobile phone number as your authentication phone for obvious reasons. If you lose your phone or leave it at home, few options will remain open to you.

In *Chapter 2, Governance and Security*, you learned about the segregation of duties and the principle of least privilege. Accounts only get the access rights they really need to fulfill a task. However, it is important to prevent being accidentally locked out from your Azure AD environment. So, there is still a need for accounts with higher privileges, such as subscription owner or Global Administrator access. This is why you need to create at least two administrative emergency access accounts. Some of these emergency access or break glass accounts need to be protected with MFA, too. Now, a single smartphone does not seem to be a good option for these kinds of accounts because, according to Murphy's law, something will go wrong; the smartphone's owner is always on holiday, ill, or not available for other reasons when you need them. A good thing to know is that you can have up to five authentication factors connected to a single account. For example, you could associate three security keys, one smartphone app, and a phone number with one account to make sure that there will always be at least one method you can use to fulfill an MFA challenge.

How to enable MFA in Azure AD

From an administrator's perspective, MFA activation in Azure AD is straightforward:

1. In the Azure portal, navigate to **Azure Active Directory** and select the **Users** navigation pane. In the upper navigation pane, select **Per-user MFA**, as shown in *Figure 3.3*:



The screenshot shows the 'Users | All users (Preview)' page in the Azure portal. The left-hand navigation pane includes sections for 'Users' (All users, Deleted users, Password reset, User settings, Diagnose and solve problems), 'Activity' (Sign-in logs, Audit logs, Bulk operation results), and 'Troubleshooting + Support' (New support request). The main content area has a top navigation bar with links: '+ New user', '+ New guest user', 'Bulk operations', 'Refresh', 'Reset password', 'Per-user MFA' (highlighted with a red arrow), 'Delete user', 'Columns', and 'Preview info'. Below this bar is a search bar and a table of 17 users. The table columns are: Name, User principal name, User type, Directory synced, Account enabled, Identity issuer, and Company name. All users listed are 'Member' type and have 'Account enabled' set to 'Yes'.

Name	User principal name	User type	Directory synced	Account enabled	Identity issuer	Company name
☐ Adele Vance		Member	No	Yes		
☐ Alex Wilber		Member	No	Yes		
☐ Diego Siciliani		Member	No	Yes		
☐ Grady Archie		Member	No	Yes		
☐ Henrietta Mueller		Member	No	Yes		
☐ Isaiah Langer		Member	No	Yes		
☐ Johanna Lorenz		Member	No	Yes		
☐ Joni Sherman		Member	No	Yes		
☐ Lee Gu		Member	No	Yes		
☐ Lidia Holloway		Member	No	Yes		
☐ Lynne Robbins		Member	No	Yes		
☐ Megan Bowen		Member	No	Yes		
☐ Miriam Graham		Member	No	Yes		
☐ Nestor Wilke		Member	No	Yes		
☐ Patti Fernandez		Member	No	Yes		
☐ Pradeep Gupta		Member	No	Yes		
☐ Tom Janetscheck		Member	No	Yes		

Figure 3.3 – MFA activation in the Azure portal

2. You will be redirected to a retro-style portal at the `windowsazure.com` domain, in which you can enable or disable MFA for single users or bulk-update them by selecting several accounts or using a CSV file in the **users** tab.

3. Before you enable MFA for all user accounts, first switch from the **users** tab to the **service settings** tab to configure some custom settings for your environment, as shown in *Figure 3.4*:

Microsoft

multi-factor authentication

users **service settings**

app passwords [\(learn more\)](#)

☒ Allow users to create app passwords to sign in to non-browser apps
☐ Do not allow users to create app passwords to sign in to non-browser apps

trusted ips [\(learn more\)](#)

☐ Skip multi-factor authentication for requests from federated users on my intranet

Skip multi-factor authentication for requests from following range of IP address subnets

192.168.1.0/27

192.168.1.0/27

192.168.1.0/27

verification options [\(learn more\)](#)

Methods available to users:

☐ Call to phone
☒ Text message to phone
☒ Notification through mobile app
☒ Verification code from mobile app or hardware token

remember multi-factor authentication on trusted device [\(learn more\)](#)

☐ Allow users to remember multi-factor authentication on devices they trust (between one to 365 days)

Number of days users can trust devices for

NOTE: For the optimal user experience, we recommend using Conditional Access sign-in frequency to extend session lifetimes on trusted devices, locations, or low-risk sessions as an alternative to 'Remember MFA on a trusted device' settings. If using 'Remember MFA on a trusted device,' be sure to extend the duration to 90 or more days. [Learn more about reauthentication prompts.](#)

save

Manage advanced settings and view reports [Go to the portal](#)

©2021 Microsoft Legal | Privacy

Figure 3.4 – Service settings for MFA

The first option is to allow or disallow users to create app passwords for legacy non-browser apps that do not support MFA. Outlook used to be such an application back in the days; however, nowadays, most applications should be able to authenticate with MFA.

You can also define trusted IPs for which you can skip MFA. So, for example, if your users are authenticating from one of your corporate offices and you have defined your external IP range(s) here, you can define that MFA challenges are only triggered when someone wants to authenticate from outside your corporate buildings. You can also enable and disable single verification options if they do not fit your company's needs. The last setting is to allow users to remember MFA for devices they (but not you as a company!) trust. If you enable that option, users are not prompted with an MFA challenge again for a given period of time (between 1 and 365 days) on a particular device.

- Now, back to enabling user accounts for MFA on the **users** tab. When you click the **Enable** link on the right side after selecting one or several user accounts (see *Figure 3.5*), you are prompted to read the online deployment guide and then click on **enable multifactor auth**. That's all from an administrator's point of view. Pretty easy? Indeed.

multi-factor authentication

users service settings

Before you begin, take a look at the [multi-factor auth deployment guide](#).

View: Sign-in allowed users Multi-Factor Auth status: Any bulk update

☐	DISPLAY NAME ^	USER NAME	MULTI-FACTOR AUTH STATUS
☐	Adele Vance	[redacted]	Disabled
☐	Alex Wilber	[redacted]	Disabled
☐	Diego Siciliani	[redacted]	Disabled
☐	Grady Archie	[redacted]	Disabled
☐	Henrietta Mueller	[redacted]	Disabled
☐	Isalah Langer	[redacted]	Disabled
☐	Johanna Lorenz	[redacted]	Disabled
☒	John Doe	[redacted]	Disabled
☐	Joni Sherman	[redacted]	Disabled
☐	Lee Gu	[redacted]	Disabled
☐	Lidia Holloway	[redacted]	Disabled
☐	Lynne Robbins	[redacted]	Disabled
☐	Megan Bowen	[redacted]	Disabled

John Doe

quick steps

[Enable](#)

[Manage user settings](#)



Figure 3.5 – Enabling MFA for one or several user accounts

The MFA-activated user has to configure all individual settings after the next login attempt. Telephone numbers for the account are taken from the information that is already stored in Azure AD. However, the user can update these numbers in the wizard that appears after the next logon.

MFA activation from a user's perspective

After enforcing MFA for John, a new window will appear after his next successful login, telling him that his organization needs more information to protect his account, as shown in *Figure 3.6*:

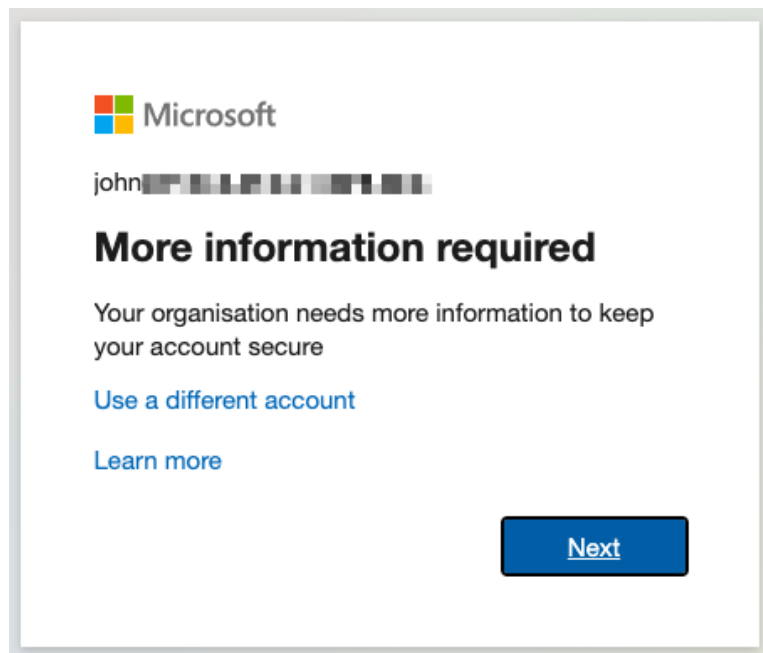


Figure 3.6 – New message at the first login after enforcing MFA

John now only has two options: **Use a different account** to log in or click **Next** to proceed through the MFA activation process. With that being said, you should inform your users before activating MFA to let them know what's going to happen and to reduce the number of support tickets required!

Let's look at what happens if John decides to complete the process:

1. John decides to proceed, so he clicks the **Next** button.
2. On the following screen, John is asked to download the **Microsoft Authenticator** app to his smartphone. Once finished, he clicks **Next**.

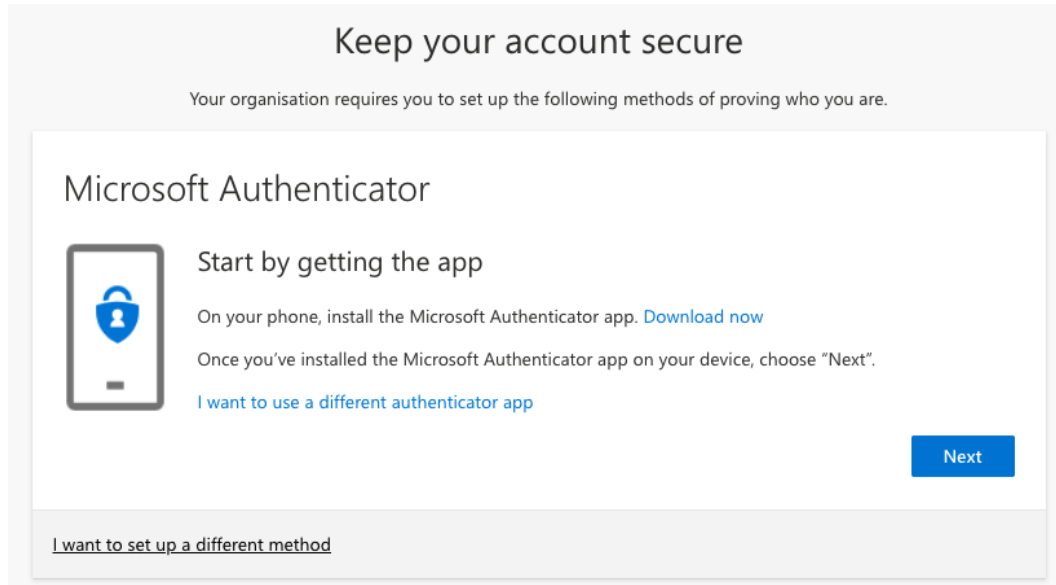


Figure 3.7 – Downloading the Microsoft Authenticator app

3. The next screen gives John a short intro into what's coming, and he presses **Next** again.

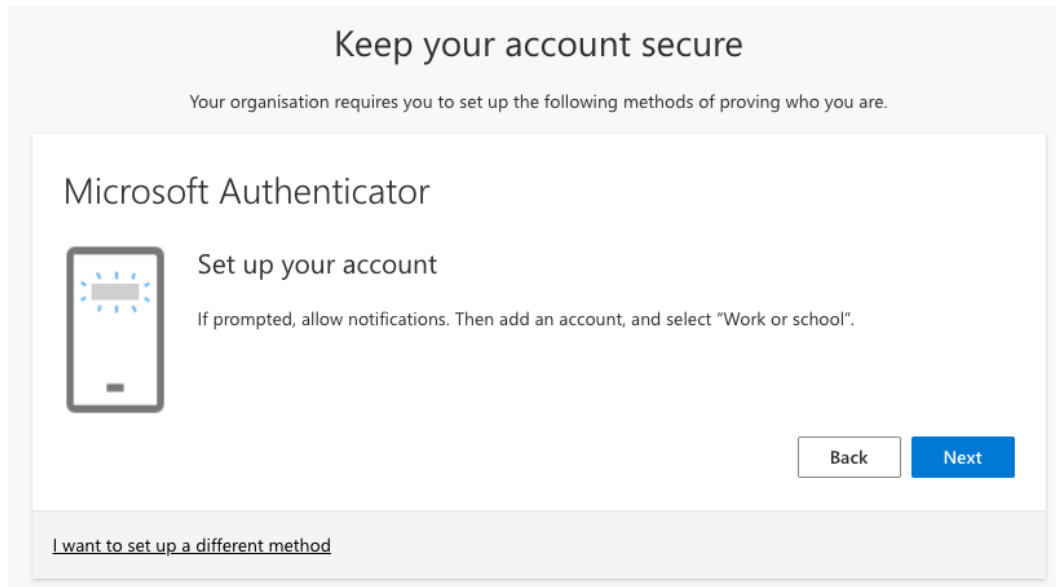


Figure 3.8 – Keep your account secure

4. John is asked to use his authenticator app to scan the QR code. This will set up his account in the Microsoft Authenticator app on his phone.

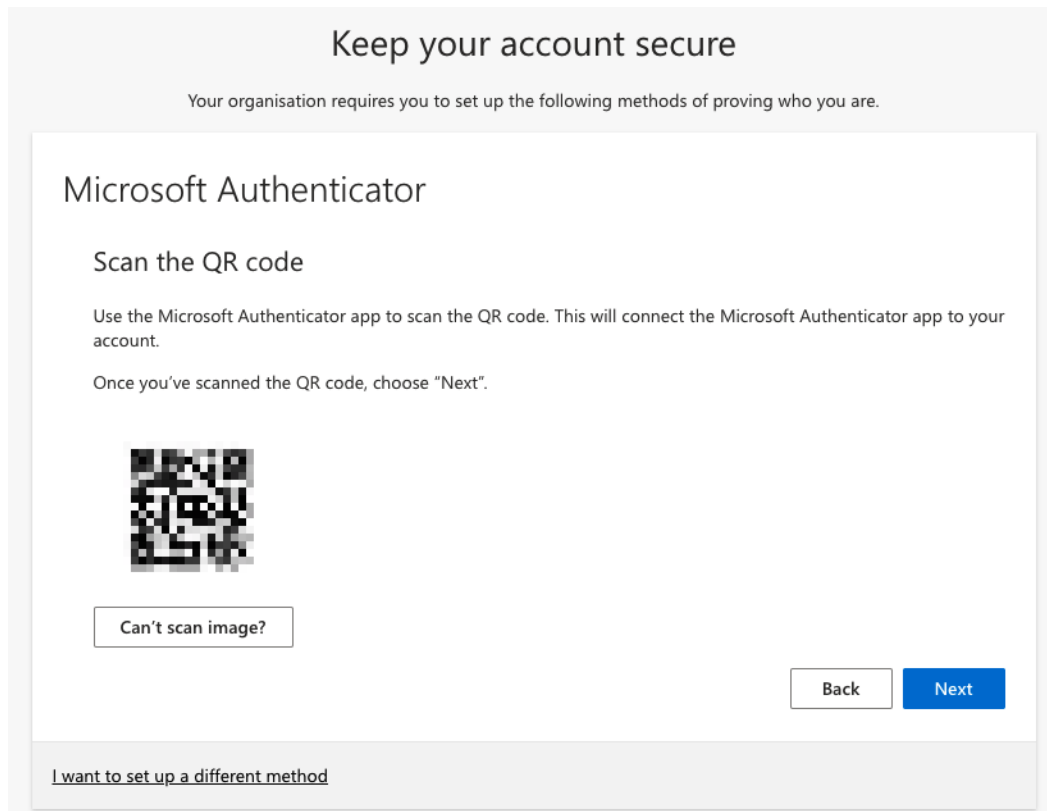


Figure 3.9 – Scanning the QR code to set up the account in the Microsoft Authenticator app

5. After the account has been set up in the app, clicking on the **Next** button will trigger an MFA challenge in the app. John approves the login attempt and the account has been set up successfully.

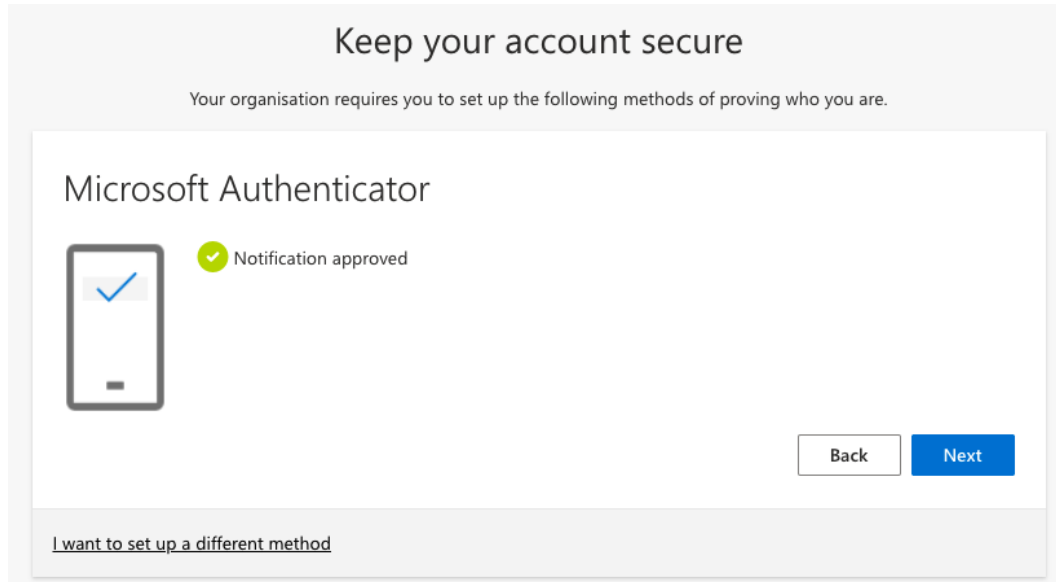


Figure 3.10 – Notification has successfully been approved

Now that you know how to configure MFA from an administrator's and a user's perspective, let's move one step further and take a look at security defaults in Azure AD.

Introducing security defaults

Security defaults are a rather new capability that will enforce basic identity security mechanisms across an Azure AD. These capabilities will ensure that user and administrator accounts are better protected from common identity-related attacks, such as brute force, or password spray.

Security defaults are enabled by default on new Azure AD enrollments but might need to be manually enabled on existing ones. To manage security defaults, navigate to **Azure Active Directory** and click the **Properties** option in the left navigation pane. Then, click the **Manage Security defaults** link and switch the **Enable Security defaults** setting to **Yes**, as shown in *Figure 3.11*:

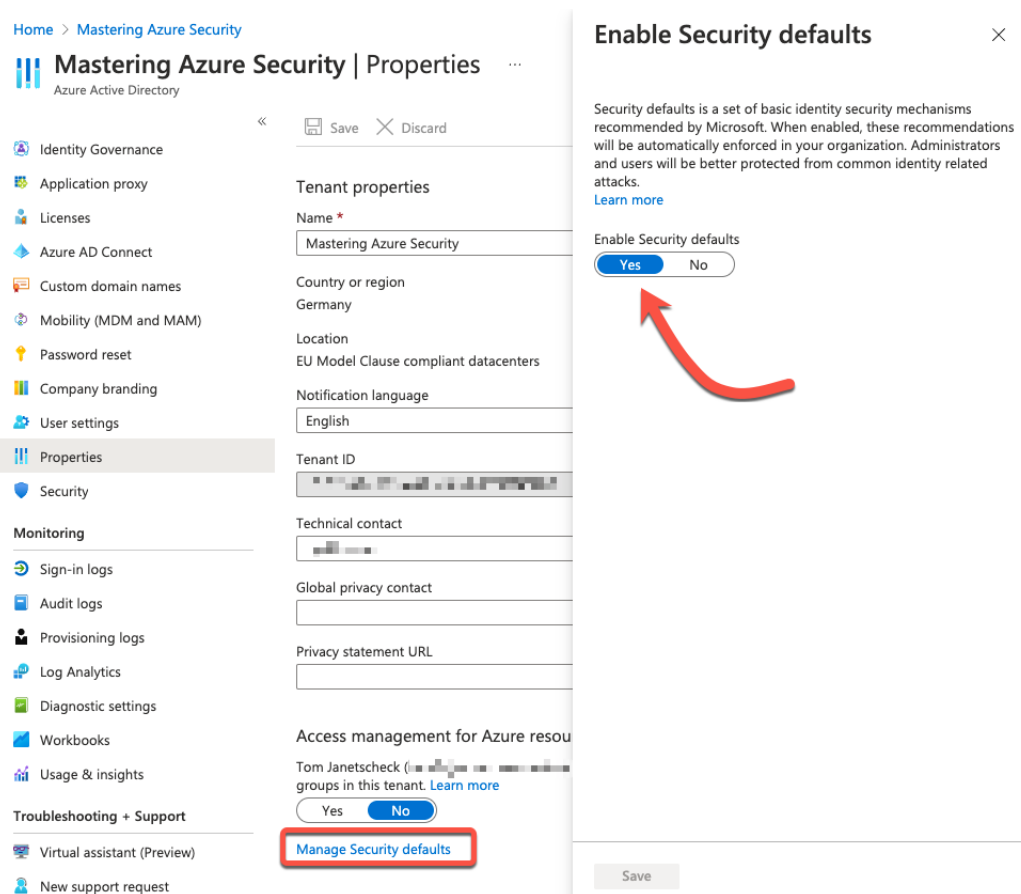


Figure 3.11 – Enable Security defaults

Security defaults will require all users and administrators to use MFA and block legacy authentication protocols. Once security defaults have been enabled, users will be asked to proceed through the MFA procedure you already know from the previous section. However, the first screen that users are presented with will look slightly different, as shown in *Figure 3.12*:

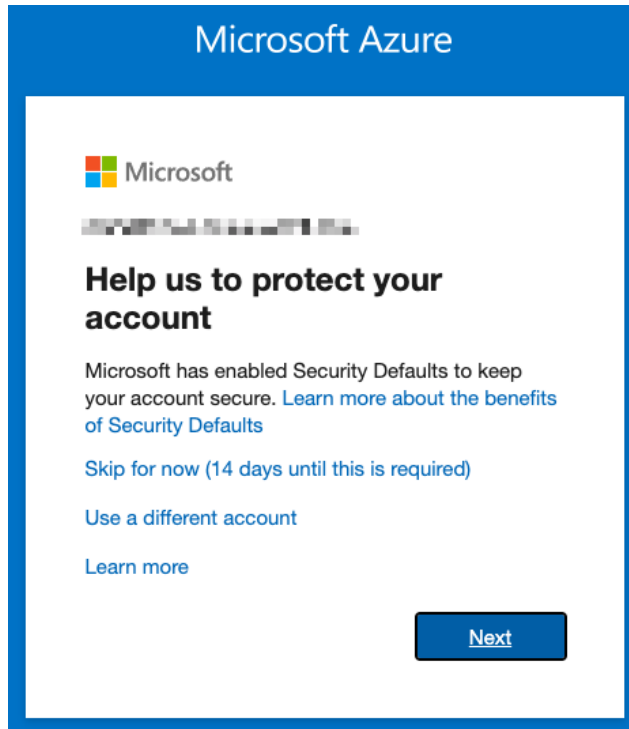


Figure 3.12 – Security defaults have been enabled

Besides the **Use a different account** option, or to proceed by clicking **Next**, there is a third option that will let users *skip* the MFA configuration for now. However, 14 days after the first sign-in event, every user is required to configure MFA, so this option will no longer be available as of then. The rest of the configuration is the same as you learned about in the *MFA activation from a user's perspective* section.

Security defaults are a great and easy way to protect all accounts with the same setting across your Azure AD directory. Now that you know how to enable MFA, and how to configure the settings from a user's perspective, let's move one step further and learn how Conditional Access can be used to fine-tune the MFA process according to your company's business needs.

Using Conditional Access

MFA, the feature we discussed in the previous section, is the perfect option for better protecting your cloud identities. But it is still kind of an on/off decision. Either you activate a user account for MFA, or you don't. Wouldn't it be great to dynamically react to authentication attempts and then decide whether an MFA challenge is needed? With Conditional Access, there is a feature that enables us to define authentication conditions that require more or fewer challenges in the authentication process.

Conditional Access gives customers a broad variety of options to include or exclude in a policy. For example, you could enforce the usage of MFA for specified directory roles, such as Azure AD Global Administrators, and in addition to that, require that the login is performed on a corporate-owned device, or you could exclude your *normal* Office 365 workers from being challenged by MFA, but only if they are working from a corporate office. As soon as one of them tries to log in from a train station, mobile device, home office, and so on, an MFA challenge is triggered. You see, there are lots of options, and we'll cover them in this section. The following screenshot is a reference of Conditional Access in Azure Active Directory:

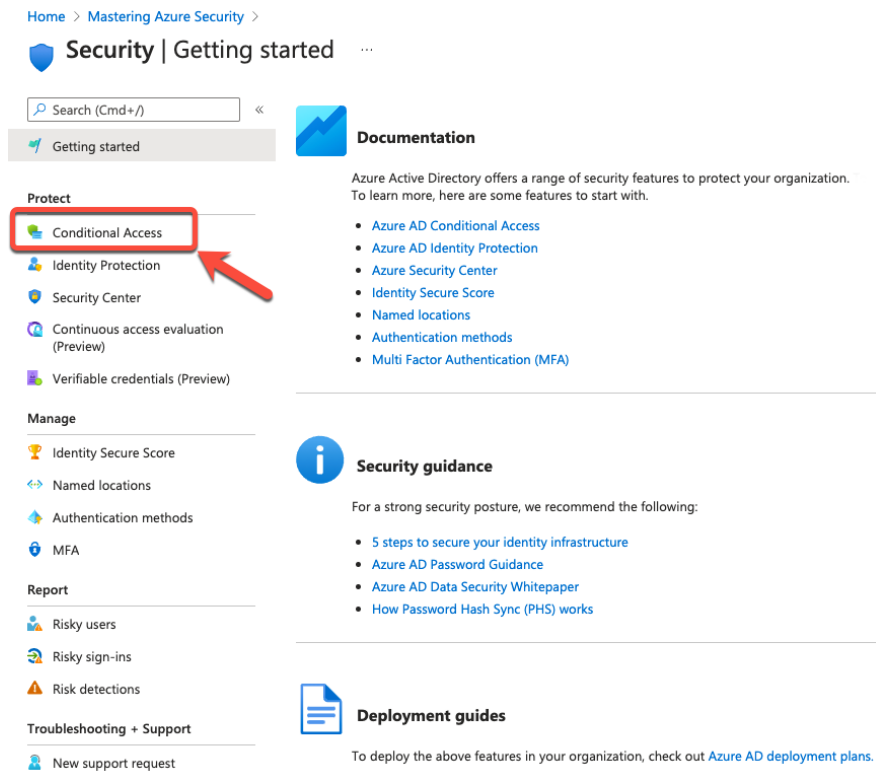


Figure 3.13 – Conditional Access in Azure AD's security blade

You can find all Conditional Access-related settings in the Azure portal under **Azure Active Directory | Security | Conditional Access**.

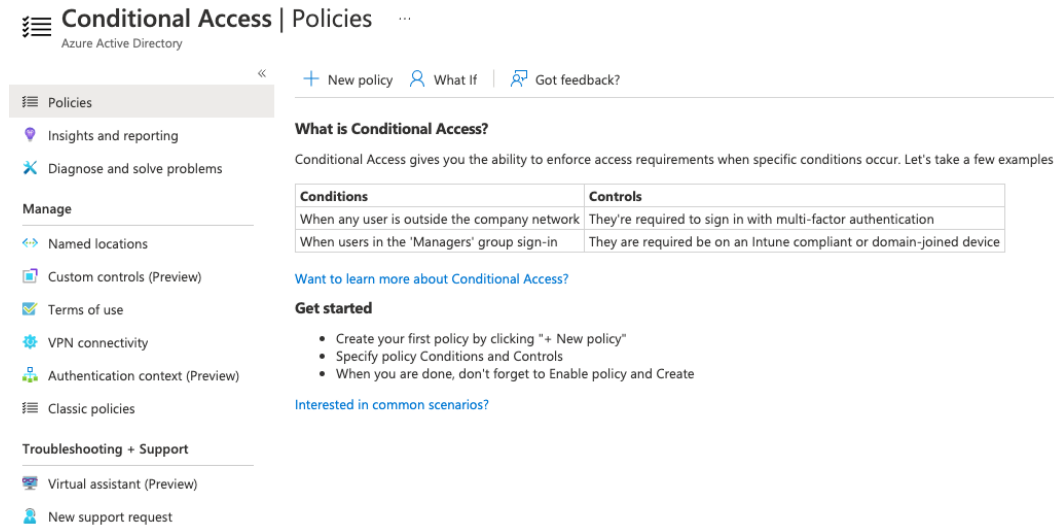


Figure 3.14 – Conditional Access overview page

The main section that appears now is the **Policies** section. Conditional Access policies define and enforce the complexity of authentication based on several conditions. Policies can depend on locations from which authentications come, user and sign-in risk, devices, applications, or a combination of all of these. You can use Conditional Access policies to decide when and for whom access to your cloud environment will be allowed or blocked. So, by using Conditional Access policies, you can always make sure to apply the right amount of security enforcements when they are needed to keep your environment secure, and not to challenge your users unnecessarily when not needed.

Before we take a deeper look at how to define all those conditions in a Conditional Access policy, let's first have a look at named locations that can be used within these policies.

Named locations

There is a management area for defining custom settings that you can use in your Conditional Access policies. The first area to mention is **Named locations**. For named locations, there are three different options to click (as shown in *Figure 3.15*).

The first option is + **Countries location**, an option to create a location filter based on IP addresses or GPS coordinates that are mapped to a particular country.

The second option is + **IP ranges location**, with which you can define IP ranges to use in custom Conditional Access policies. If you select **Mark as trusted location** for an IP range, a user's sign-in risk is lowered when an authentication request comes from there. Sign-in risks are important for Azure AD Identity Protection, the feature we will look into in the *Introducing Azure AD Identity Protection* section.

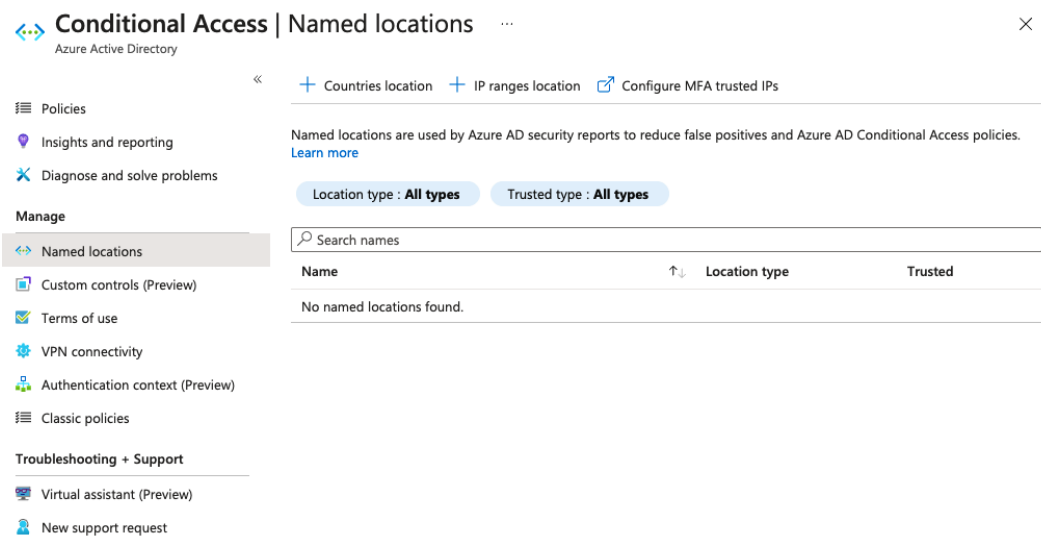


Figure 3.15 – Conditional Access – Named locations

The third option for named locations is **Configure MFA trusted IPs**, which will redirect you to the **Services** settings in the legacy portal you already know from the previous section. You can use this option to define IP address ranges that should always be excluded from MFA challenges.

Custom controls

With custom controls, you can redirect your users during the authentication process to an external compatible service to satisfy further authentication requirements outside of Azure AD. During the authentication process, the user's browser window is redirected to a third-party service, where the user needs to perform further authentication requirements before being granted access to their cloud resources. Custom controls depend on the third-party application. For example, you could integrate a third-party MFA provider into your authentication workflow instead of relying on the Azure AD MFA service.

Terms of use

You can define custom terms of use that need to be accepted before users are allowed to access certain cloud applications. For example, you could allow users to use their own devices to access your environment, but only if they accept your terms of use.

New terms of use ...

Terms of use

Create and upload documents

Name *	MyToU	✓
Display name *	Mastering Azure Security Terms of Use	✓
Terms of use document	Upload required PDF English ▼	
	Upload required PDF Bosnian ▼	✕
	Upload required PDF German ▼	✕
	+ Add language	
Require users to expand the terms of use	On Off	
Require users to consent on every device	On Off	
Expire consents	On Off	
Duration before re-acceptance required (days)	Example: '90'	

Conditional access

Enforce with conditional access policy templates *	Create conditional access policy later	✓
--	---	---

 This terms of use will appear in the grant control list when creating a conditional access policy. 

Figure 3.16 – Terms of use

Conditional Access policies

As mentioned before, Conditional Access policies help you to always apply the right amount of security for every authentication situation, without unnecessarily blocking or challenging your users when not needed. Conditional Access policies consist of *assignments* and *access controls*. Assignments define who will be impacted by this Conditional Access policy and in which situation. Access controls define whether access is blocked or granted, and if it's granted, on which conditions.

Assignments

In the **Assignments** section, you can select users and groups, cloud apps or user actions, and conditions to include or exclude in a Conditional Access policy, as shown in *Figure 3.17*:

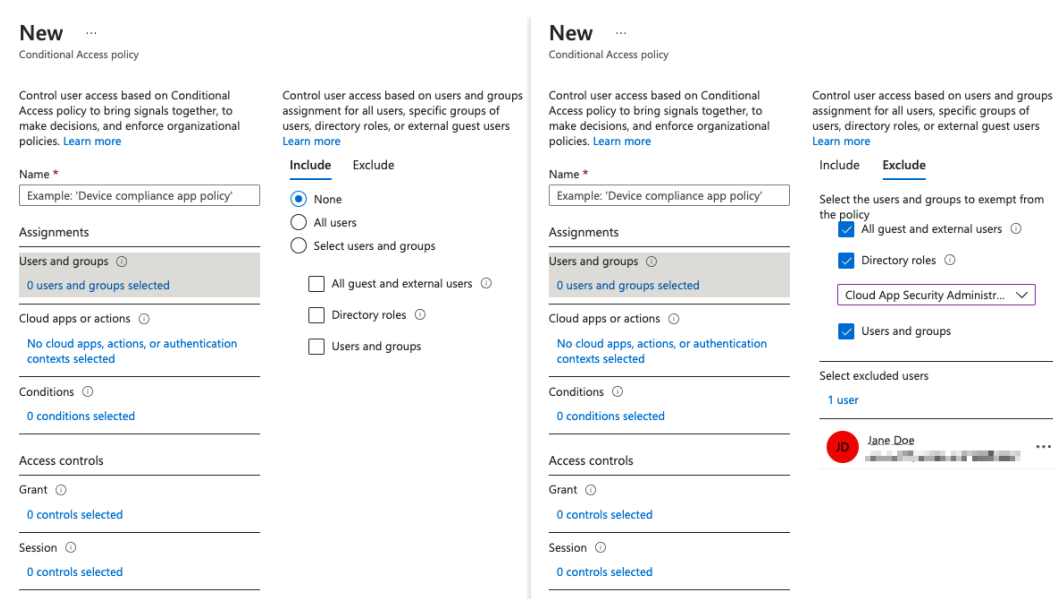


Figure 3.17 – Users and groups – Selecting which users and groups are to be included or excluded in a Conditional Access policy

You can choose **Include** or **Exclude** | **None** | **All users**, or a defined subset of users and groups in a Conditional Access policy. You can also select to exclude all guest and external users, directory roles (such as Global Administrators), or a defined subset of users and groups in your Conditional Access policy.

Important Note

When selecting all users and then blocking access with this policy, this will also affect all your administrator accounts. That said, you could lock yourself out when activating this policy. It is recommended to apply a policy to a small set of users first to make sure it behaves as expected!

You can also include or exclude cloud applications or user actions in the Conditional Access policy. Again, you can select **None** or **All apps**, or selected apps to be included, or define selected apps to be exempted from the policy.

Important Note

When selecting all cloud apps to be included in your policy, this will also affect the Azure portal. Make sure not to lock yourself out by only activating the policy on a small set of users first so you can ensure that it behaves as expected!

In the **User actions** section, you can select whether this policy affects the registration of security information or device registration/join for your users. This means that the policy is active for users when proceeding through the MFA registration process or when registering or joining a device to your Azure AD.

New ...
Conditional Access policy

Control user access based on Conditional Access policy to bring signals together, to make decisions, and enforce organizational policies. [Learn more](#)

Name *
Example: 'Device compliance app policy'

Assignments

Users and groups 0
0 users and groups selected

Cloud apps or actions 0
No cloud apps, actions, or authentication contexts selected

Conditions 0
0 conditions selected

Access controls

Grant 0
0 controls selected

Session 0
0 controls selected

Control user access based on all or specific cloud apps or actions. [Learn more](#)

Select what this policy applies to
Cloud apps

Include Exclude

☒ None
☐ All cloud apps
☐ Select apps

New ...
Conditional Access policy

Control user access based on Conditional Access policy to bring signals together, to make decisions, and enforce organizational policies. [Learn more](#)

Name *
Example: 'Device compliance app policy'

Assignments

Users and groups 0
0 users and groups selected

Cloud apps or actions 0
1 user action included

Conditions 0
0 conditions selected

Access controls

Grant 0
0 controls selected

Session 0
0 controls selected

Control user access based on all or specific cloud apps or actions. [Learn more](#)

Select what this policy applies to
User actions

Select the action this policy will apply to

☒ Register security information
☐ Register or join devices

Figure 3.18 – Configuring cloud apps to be included/excluded and user actions in the policy

Conditions to be included in your assignments are sign-in risk, device platforms, locations, client apps, and device state. Sign-in risk is calculated by Azure AD Identity Protection. You can select the sign-in risk level this policy will apply to from **No risk/Low/Medium/High**. Device platforms can be included or excluded. You can select to include any device platform in your policy, or to include or exclude one or several of the following:

- Android
- iOS
- Windows Phone
- Windows
- macOS

You can include any location or select to include or exclude all trusted locations, or selected locations you have already defined in one of the preceding steps.

Client apps to include can either be a browser or mobile apps and desktop clients, including modern authentication clients, Exchange ActiveSync clients, or other clients (older office clients or other mail protocols).

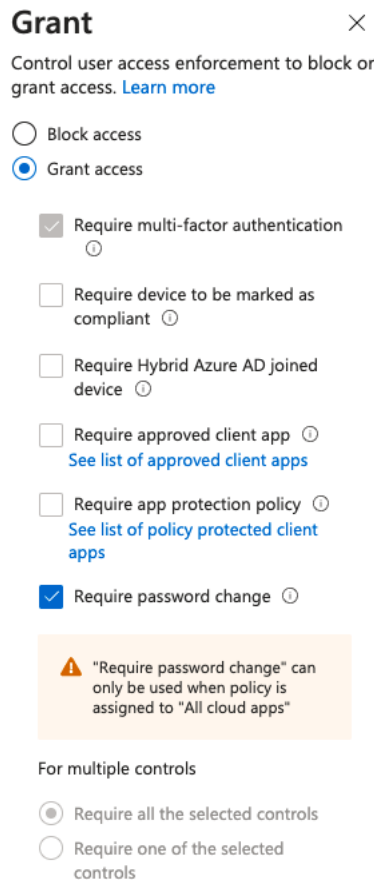
You can either include all device states in a policy or exclude devices that are hybrid Azure AD joined or marked as compliant in Intune.

Access controls

In the **Access controls** section, you define whether you want to block or grant access to your environment depending on the selection you made before. If you want to grant access, you can either grant it without any condition or you can require one or all of the following controls to be enforced:

- **Require multi-factor authentication:** When the conditions you defined in the policy are met, user access is granted when the user responds to the MFA challenge.
- **Require device to be marked as compliant:** In Intune, you can define device compliance policies. For example, you can mark a device as compliant only when it is corporate-owned and currently patched.
- **Require Hybrid Azure AD joined device:** Devices that are hybrid Azure AD joined are owned by an organization. They exist in the cloud and on-premises, which means that they are joined to both Windows Server AD and Azure AD.

- **Require approved client app:** Access is only granted if one of the approved client apps is used to access corporate information. Today, an approved client app can be one out of more than 25 apps related to Microsoft 365, including, but not limited to, Microsoft Azure Information Protection, Microsoft Excel, Microsoft Edge, Microsoft Intune Managed Browser, Microsoft Teams, and Microsoft Outlook.
- **Require app protection policy:** You can use Intune to define an app protection policy for Microsoft Cortana, Microsoft Outlook, Microsoft OneDrive, and Microsoft Planner. With this setting, you can enforce that an app protection policy is present before granting access to corporate information.
- **Require password change:** You can require a user to change their password to lower the sign-in risk. This option requires MFA and cannot be used together with other controls. Also, the Conditional Access policy needs to be assigned to *All cloud apps*, as shown in *Figure 3.19*:



Grant ×

Control user access enforcement to block or grant access. [Learn more](#)

☐ Block access

☒ Grant access

☒ Require multi-factor authentication ⓘ

☐ Require device to be marked as compliant ⓘ

☐ Require Hybrid Azure AD joined device ⓘ

☐ Require approved client app ⓘ
[See list of approved client apps](#)

☐ Require app protection policy ⓘ
[See list of policy protected client apps](#)

☒ Require password change ⓘ

⚠ "Require password change" can only be used when policy is assigned to "All cloud apps"

For multiple controls

☒ Require all the selected controls

☐ Require one of the selected controls

Figure 3.19 – Granting access based on conditions

Now that you know how to configure Conditional Access policies in theory, let's go a step further and try to define settings for some behaviors.

For example, you could create a policy that blocks access for all devices that are not hybrid Azure AD joined. However, this policy should not affect users who are members of a particular group of administrators. A policy that would fit your needs could include the following:

- Include all users and exclude a particular group of administrators.
- Exclude devices that are hybrid Azure AD joined.
- Block access.

You might also want to enforce MFA when authentication attempts are not coming from a trusted location or if users are not using corporate-owned devices. This policy would do the following:

- Include all users.
- Include all locations and exclude all trusted locations.
- Exclude devices that are marked as compliant.
- Grant access and require MFA.

Important Note

It is best practice not to include all users without exception in your Conditional Access policies. This is because you would risk locking yourself out of your cloud environment.

Your organization might enforce your users to proceed through the MFA enrollment from their office. That way, your organization can make sure that even if a user's credentials have been leaked, an attacker would need to be within your network perimeter to configure MFA. A Conditional Access policy to enforce that might be configured like this:

- Include all users and exclude privileged roles or a particular user group containing administrative accounts.
- Activate the **Register security information** user action.
- Include a selected location that defines your office location.
- Block access.

With Conditional Access, you can granularly define when and how access to your cloud environment is granted or declined. During the course of this topic, we have already touched on Azure AD Identity Protection. Now, let's move on to the next section, in which you will learn what Azure AD Identity Protection is and how you can use it.

Introducing Azure AD Identity Protection

As already mentioned, an important goal in terms of security is to make attack attempts as expensive as possible for an attacker. That said, it is important to raise the bar and have several complementing technologies in place that add extra value to your security posture. On the other hand, you want to make sure not to block legitimate user authentications, so you do not negatively affect your users' login experience. Microsoft has been protecting cloud identities for more than 10 years now, and with Azure AD Identity Protection, they let you protect identities using their protection systems.

Stealing the credentials of user accounts, even if they are not highly privileged, has always been a good way of gaining access to corporate resources. Over the years, attackers have learned to leverage third-party breaches or highly sophisticated phishing attacks to get hold of corporate credentials. Even if attackers gain access to low-privileged user accounts, as soon as they do, it is relatively easy for them to elevate their level of privilege or gain access to important corporate resources through lateral movement.

So, what we need to do is the following:

- Protect all identities at all levels of privilege
- Prevent compromised identities from accessing corporate resources

Now, the second step is the more difficult one because you first need to find out whether an identity is compromised. This is when Azure AD Identity Protection comes into play.

Azure AD Identity Protection at a glance

Adaptive machine learning algorithms and heuristics are what Azure AD utilizes when determining anomalies and suspicious behavior during the authentication of user accounts. With this data, Azure AD Identity Protection can generate reports and alerts that enable customers to evaluate recent logons. Furthermore, Azure AD Identity Protection can calculate user and sign-in risk levels that can be used in risk-based Conditional Access policies. For example, you can decide to enforce the usage of MFA when a login attempt's risk level is calculated as medium or above, or to block a sign-in attempt based on the risk calculation.

In *Figure 3.20*, you see that a sign-in was blocked because the account was used within a suspicious sign-in event. Before John can sign in again, he needs to contact an admin to get his account unlocked.

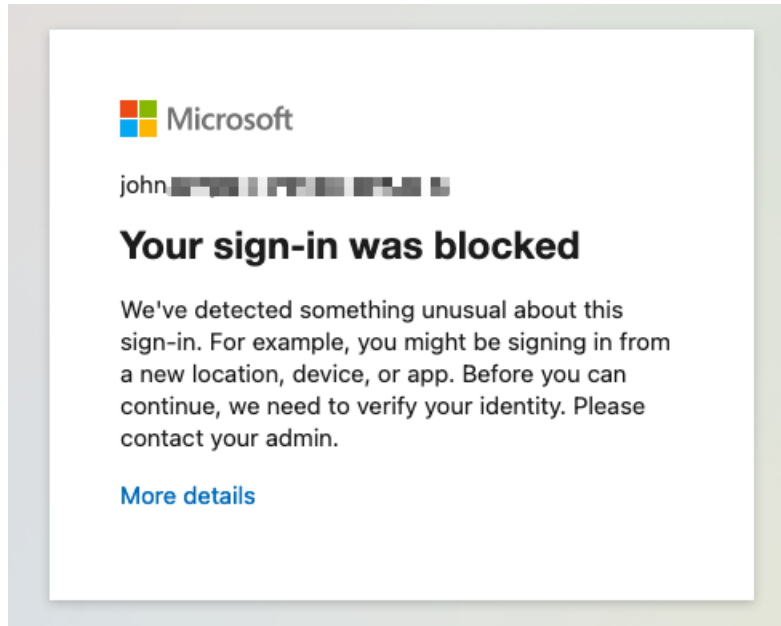


Figure 3.20 – John's sign-in was blocked by Azure AD Identity Protection

Azure AD uses several types of risk detection in real time (shown in the reporting after 5–10 minutes) and offline (2–4 hours). As soon as there has been a risk detection related to an account sign-in, this detection is added to a logical concept called **risky sign-ins**. A risky sign-in indicates a sign-in attempt that might have been performed by an attacker and not the legitimate account owner. Based on those detections, the probability that the sign-in has been performed by a bad actor is calculated and is reflected in the *sign-in risk level*. You can use the sign-in risk level (**Low**, **Medium**, **High**) to define a *sign-in risk policy*. The sign-in risk policy is an automated response depending on the specified sign-in risk level that is used to either block access to your resources or to require a user to pass an MFA challenge before being granted access to your corporate environment.

Besides the sign-in risk, Azure AD also detects the probability that a user account has been compromised. This behavior is called *user risk detection*. All risk detections that have been detected for a user and that have not been resolved are known as *active risk detections*. All active risk detections that are associated with a user define the risk to the user. Based on the risk to the user, Azure AD calculates the probability (**Low, Medium, High**) that the user account has been compromised. This probability is known as the user risk level. For example, if a risky sign-in is detected and the MFA challenge is passed, there is no active risk detection for this user. Thus, in this case, the user risk level will stay low. You can define a *user risk policy* as an automated response for a specific user risk level. With this policy, you can either block access to corporate resources or enforce a password change to get the user account back to a clean state.

Risk detection

In Azure AD, there are a bunch of different types of risk detection, six of which we want to highlight:

- **Users with leaked credentials:** Leaked credentials occur due to mass credential theft by cybercriminals followed by public posting of these credentials, particularly on the dark web. Microsoft actively monitors such channels and compares the leaked username and password pairings against existing users and identifies accounts that have leaked credentials.
- **Sign-ins from anonymous IP addresses:** Anonymous IP addresses are used by people who want to mask their device's real IP address. There can be a number of reasons that lead to the use of such IP addresses, some of them malicious. Microsoft checks incoming sign-ins from IP addresses that have previously been identified as anonymous IP addresses.
- **Impossible travel to atypical locations:** These are sign-ins from geographically distant locations. At least one location needs to be an atypical location based on the user's sign-in behavior. For example, the user is on their first business trip to the US and logs in from there. In addition to that, the underlying machine learning algorithm considers the time between the sign-ins and the time it would have taken the user to travel from one location to the other. Microsoft tries to ignore obvious false positives, such as **virtual private networks (VPNs)** between locations, or locations that have been known as familiar for other user accounts within the same organization. This risk detection type needs 14 days of analyzing user behavior before being able to calculate the risk for a user account.

- **Sign-ins from infected devices:** This method of risk detection depends on Microsoft keeping track of IP addresses that are known to communicate with bot servers. Microsoft matches all IP addresses connecting to the service against these known rogue IPs to determine whether there are sign-ins from infected devices. This type of risk detection also targets simple attack methods such as password spray attacks.
- **Sign-ins from IP addresses with suspicious activity:** In this method of detection, Microsoft examines IPs for signs of suspicious activity. The main indicator is when an IP address has a large number of failed sign-ins in a relatively short period of time. This can be an indication of a bad actor who is trying to guess passwords using simple attack methods such as brute-force or password spray attacks.
- **Sign-ins from unfamiliar locations:** This method of risk detection involves Microsoft maintaining a historical record of the IP addresses used by users regularly. The system detects whether a request is coming from an IP address that has not been used before. The system needs at least 30 days to create a useable history of familiar and unfamiliar locations.

Note

Microsoft is regularly updating existing and adding new risk types and detections. Please refer to <https://aka.ms/AADIPRiSkTypes> for the current list.

Creating a sign-in risk or user risk policy

Policy creation is the easiest part of Azure AD Conditional Access. To create a sign-in risk policy, you just need to navigate to **Azure Active Directory | Security | Identity Protection | Sign-in risk policy** in the Azure portal. There, you define **conditions** (**Sign-in risk level** – **Low and above**, **Medium and above**, or **High**), **access controls** (block access or challenge the user with MFA), and select for which user or group you want to enable the policy. That's it.

Home > Mastering Azure Security > Security > Identity Protection

Identity Protection | Sign-in risk policy

Search (Cmd+/) «

- Overview
- Diagnose and solve problems
- Protect**
 - User risk policy
 - Sign-in risk policy**
 - MFA registration policy
- Report**
 - Risky users
 - Risky sign-ins
 - Risk detections
- Notify**
 - Users at risk detected alerts
 - Weekly digest
- Troubleshooting + Support**
 - Virtual assistant (Preview)
 - Troubleshoot
 - New support request

Policy Name
Sign-in risk remediation policy

Assignments

- Users**
 - All users
- Sign-in risk** ⓘ
 - Low and above

Controls

- Access** ⓘ
 - Require multi-factor authentication

Enforce policy

On Off

Save

Figure 3.21 – Creating a sign-in risk policy

For a new user risk policy, you navigate to **Azure Active Directory | Security | Identity Protection | User risk policy** and define the respective parameters. The difference from a sign-in risk policy is that you can either block access or enforce a password change for a risky user.

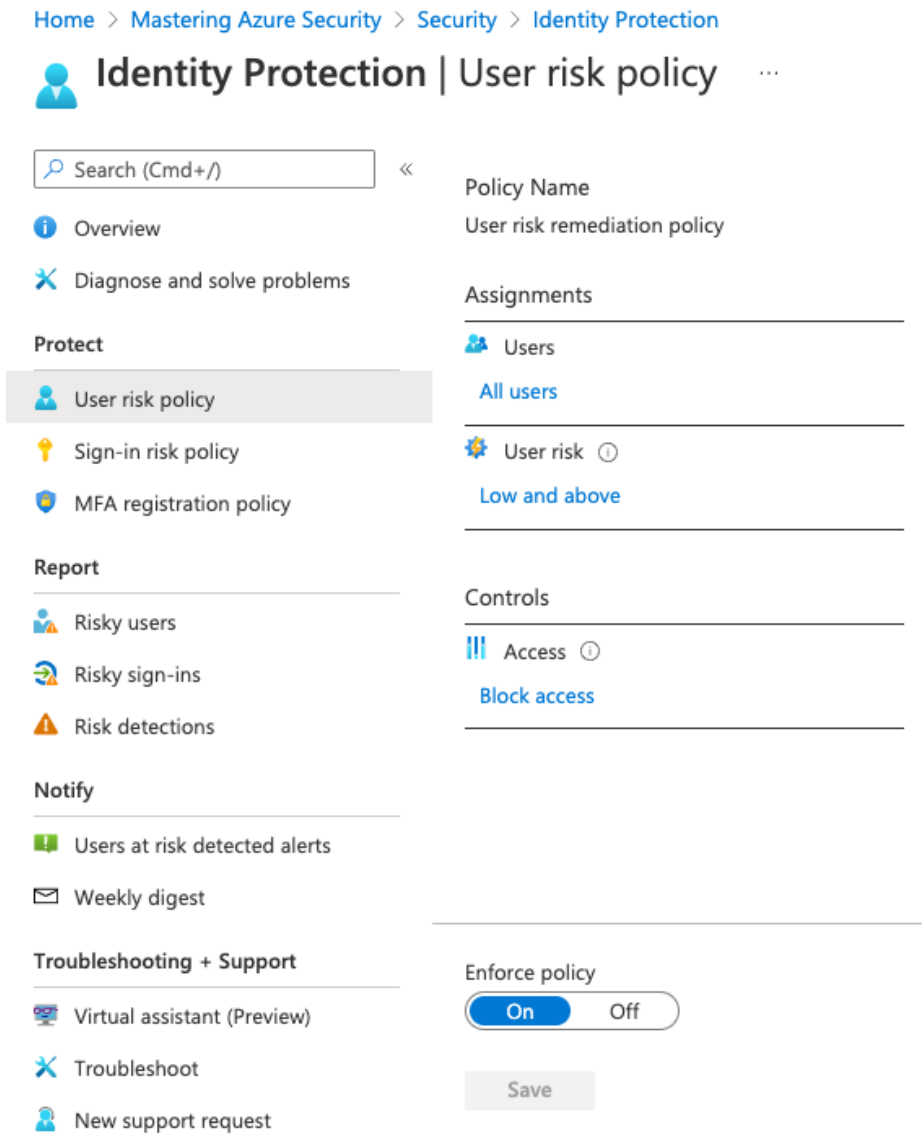


Figure 3.22 – Creating a user risk policy

Now you know how Azure AD Identity Protection works and how you can use calculated user and sign-in risk levels to protect your accounts. Now, we will move forward and focus on privileged user accounts.

Understanding role-based access control

In *Chapter 2, Governance and Security*, we mentioned security principles such as segregation of duties and the principle of least privilege. With **role-based access control (RBAC)**, we have the technical feature to implement these principles in Azure AD. RBAC is the way to manage access to all Azure resources, but also to Azure AD and Office 365.

As already discussed earlier, Azure offers several levels of scope that can be used to grant access rights to accounts. Furthermore, there are several built-in roles, such as Owner, Security Admin, or Reader. To create a *role assignment*, you need to bring together three different entities:

- A security principal, which can be either one of users, groups, or service principals.
- A role, which describes a set of management rights. For example, the contributor role contains all actions, except authorization-related rights. So, a contributor may manage all aspects within a given scope without being able to grant access to resources.
- A scope to set the access rights, starting from the management group level down to single resources.

The following diagram shows a schematic view of a role assignment and the entities it needs:

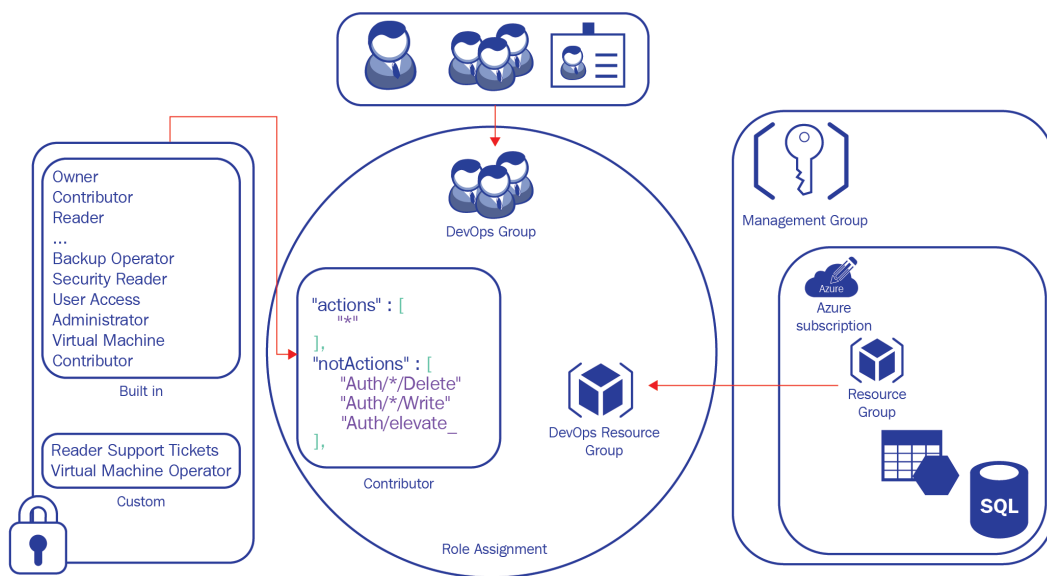


Figure 3.23 – RBAC role assignment in Microsoft Azure

Important Note

You need to be granted `Microsoft.Authorization/roleAssignments/write` and `Microsoft.Authorization/roleAssignments/delete` rights in order to be able to create or delete role assignments. These rights are, by default, granted to the RBAC roles *User Access Administrator* and *Owner*.

You can edit role assignments at all management scopes of Azure. It is important to know that role assignments are inherited top to bottom. This means that if you configure a role assignment at a resource group level, the access rights are also valid on all resources within this particular resource group; and if you configure a role assignment at a management group level, it is inherited by all subscriptions, resource groups, and resources within that scope.

In the Azure portal, there is the **Access Control (IAM)** blade (*Figure 3.24*) that is used to manage all aspects of RBAC. You can check access for particular accounts, find out explicit and inherited roles and deny assignments, and find out which built-in or custom roles exist for the given scope.

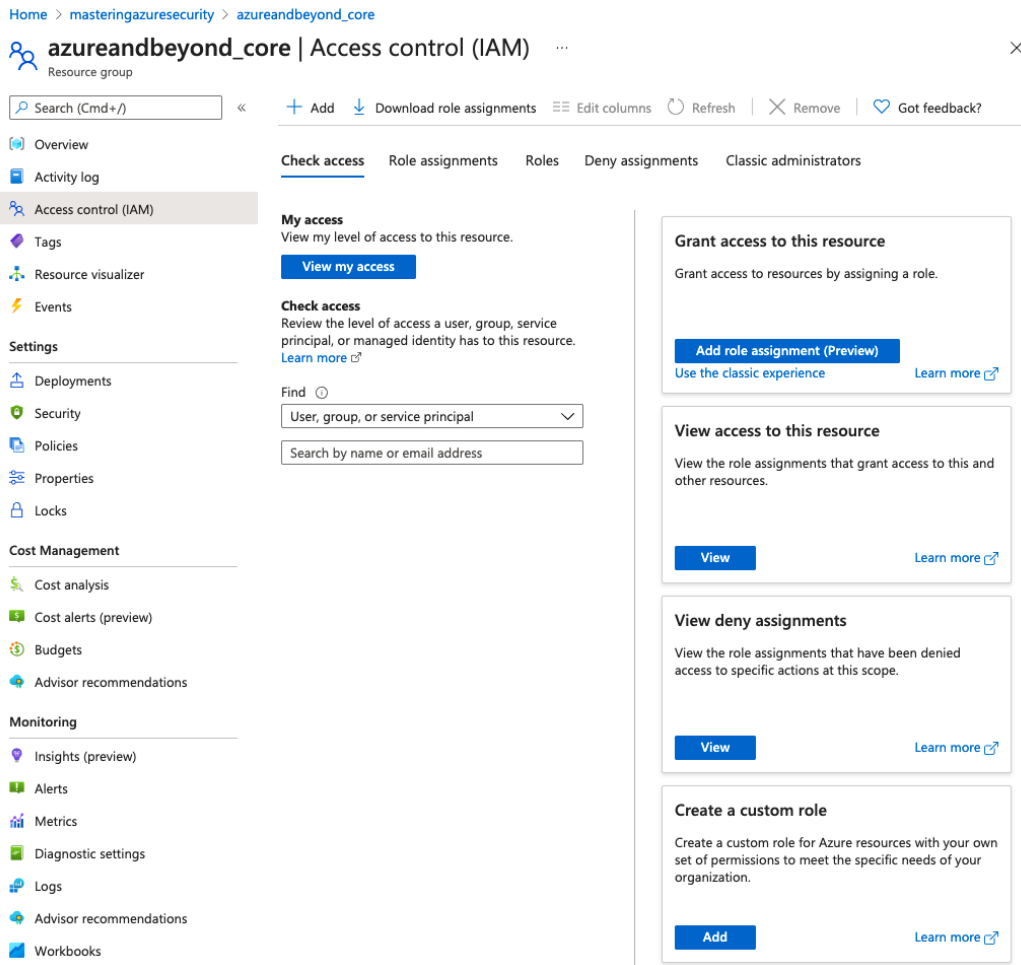


Figure 3.24 – RBAC management blade in the Azure portal

To create a new role assignment, you need to click + **Add** and then select **Add role assignment (Preview)**:

Add role assignment ×

Role ⓘ
Security Admin ⓘ

Assign access to ⓘ
User, group, or service principal

Select ⓘ
john

No users, groups, or service principals found.

Selected members:

JD

John Doe
john

[Remove](#)

Save

Discard

Figure 3.25 – Creating a role assignment

You then select the role and security principal you want to grant access to and then save your selection. And we are done!

RBAC is straightforward to implement but can be way more challenging when planning for access rights in your environment. This is why you should already plan for RBAC roles when defining your governance concept.

Important Note

Define the RBAC roles you need in your environment when creating your governance concept. By doing so, you will have a good overview of which access rights (roles) you need and who (which security principal) you need to grant access to. That said, it will be much easier to create your role assignments later.

In the following section, we will show you how to create custom RBAC roles so you can use roles more granularly according to your needs.

Creating custom RBAC roles

Custom RBAC roles are used whenever built-in roles do not entirely fit your company's needs regarding access rights to your cloud environments. As mentioned before, a role is basically a set of management rights that are then applied to users or groups in a role assignment. There is a difference between *custom Azure AD RBAC roles* and *custom RBAC roles for Azure resources*. If you want to create a custom Azure AD RBAC role, you need to perform the following steps:

1. Navigate to **Azure Active Directory | Roles and administrators** and then click the **+ New custom role** button in the Azure portal, as shown in *Figure 3.26*:

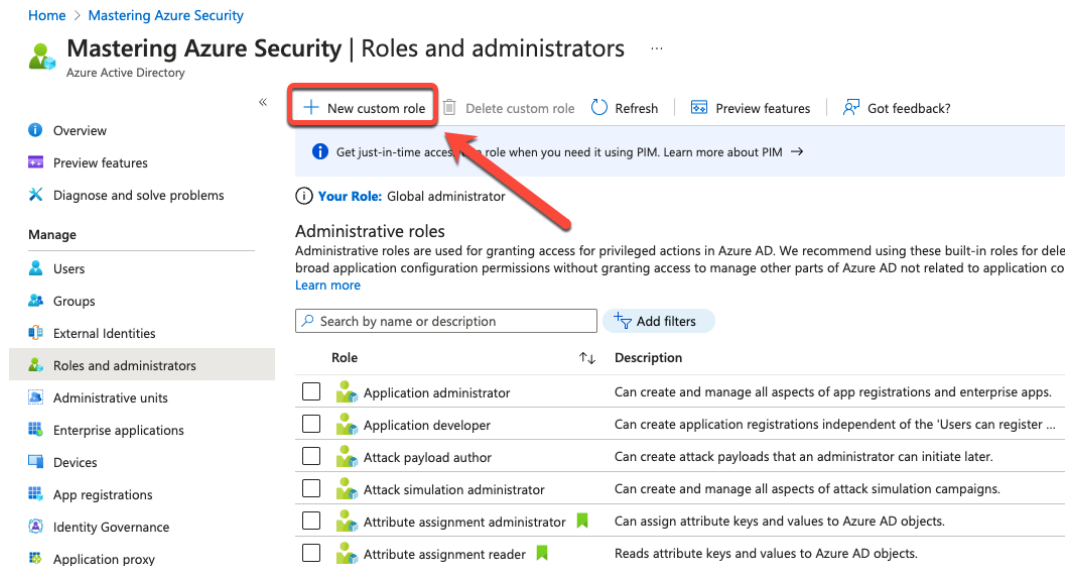


Figure 3.26 – Creating a new custom Azure AD RBAC role

2. Enter a name and, optionally, a description for your custom role. You can decide either to **Start from scratch** or to **Clone from a custom role** to continue with your custom role creation. In the following example, we choose the **Start from scratch** option:

Home > Mastering Azure Security >

New custom role

All roles

Got feedback?

Basics Permissions Review + create

Roles created here will be available for assignment on other resources as well. [Learn more](#)

Name * Mastering Azure Security 2nd Edition - Custom role ✓

Description

Baseline permissions ☒ Start from scratch ☐ Clone from a custom role

Next

Figure 3.27 – Defining basic information

3. On the **Permissions** tab, you select all permissions that you later want to grant to your identity.
4. On the **Review + create** tab, you can finally confirm your settings, and by clicking the **Create** button, your custom Azure AD RBAC role is created, as illustrated in *Figure 3.28*:

[Home](#) > [Mastering Azure Security](#) >

New custom role ...

All roles ×

 Got feedback?

Basics Permissions **Review + create**

Below is the definition of your role. Click Create to save it and then you can assign members to it.

Name Mastering Azure Security 2nd Edition - Custom role

Description

Permission	↑↓	Description	↑↓
microsoft.directory/applicationPolicies/allPrope...		Read all properties of application policies.	
microsoft.directory/applicationPolicies/allPrope...		Update all properties of application policies.	

Figure 3.28 – Reviewing and creating a custom Azure AD RBAC role

Similar to the creation of custom Azure AD RBAC roles, you can also create custom RBAC roles for Azure resources in the Azure portal. You can create these roles either on a management group or a subscription, using the highest scope as a best practice. The process itself is similar to creating a custom role for Azure AD, explained previously. To start the process, either navigate to the management group or subscription you want to create the role in and select **Access control (IAM)** in the left navigation pane. In the **Create a custom role** tile, click **Add** to start the creation process. You can then follow the tabs as explained in the preceding steps.

Besides using the portal, you can also use **PowerShell**, the Azure **command-line interface (CLI)**, or the Azure **representational state transfer (REST)** API. In this example, we will create a custom RBAC role with PowerShell.

To create a custom RBAC role, you need to define which resource operations per resource provider have to be allowed to meet your goals. Therefore, you should initially find out which resource provider operations exist. You can do so by using the `Get-AzProviderOperation` cmdlet, as shown in the following code block:

```
Get-AzProviderOperation <operation> | FT OperationName,
Operation, Description -autosize
```

For example, in order to obtain a list of all provider operations for **virtual machines** (VMs), the command would be as follows:

```
Get-AzProviderOperation Microsoft.Compute/virtualMachines/* |
FT OperationName, Operation, Description -autosize
```

The result of this command is shown in *Figure 3.29*:

```
PS /Users/tom> Get-AzProviderOperation Microsoft.Compute/virtualMachines/* | FT OperationName, Operation, Description -autosize
```

OperationName	Operation
Get Virtual Machine	Microsoft.Compute/virtualMachines/read
Create or Update Virtual Machine	Microsoft.Compute/virtualMachines/write
Delete Virtual Machine	Microsoft.Compute/virtualMachines/delete
Start Virtual Machine	Microsoft.Compute/virtualMachines/start/action
Power Off Virtual Machine	Microsoft.Compute/virtualMachines/powerOff/action
Reapply a virtual machine's current model	Microsoft.Compute/virtualMachines/reapply/action
Redeploy Virtual Machine	Microsoft.Compute/virtualMachines/redeploy/action
Restart Virtual Machine	Microsoft.Compute/virtualMachines/restart/action
Retrieve boot diagnostic logs blob URIs	Microsoft.Compute/virtualMachines/retrieveBootDiagnosticsData/action
Deallocate Virtual Machine	Microsoft.Compute/virtualMachines/deallocate/action
Generalize Virtual Machine	Microsoft.Compute/virtualMachines/generalize/action
Capture Virtual Machine	Microsoft.Compute/virtualMachines/capture/action
Run Command on Virtual Machine	Microsoft.Compute/virtualMachines/runCommand/action
Convert Virtual Machine disks to Managed Disks	Microsoft.Compute/virtualMachines/convertToManagedDisks/action
Perform Maintenance Redeploy	Microsoft.Compute/virtualMachines/performMaintenance/action
Reimage Virtual Machine	Microsoft.Compute/virtualMachines/reimage/action
Log in to Virtual Machine	Microsoft.Compute/virtualMachines/login/action
Log in to Virtual Machine as administrator	Microsoft.Compute/virtualMachines/loginAsAdmin/action
Install OS update patches on virtual machine	Microsoft.Compute/virtualMachines/installPatches/action
Assess virtual machine for available OS update patches	Microsoft.Compute/virtualMachines/assessPatches/action
Cancel install OS update patch operation on virtual machine	Microsoft.Compute/virtualMachines/cancelPatchInstallation/action
Simulate Eviction of spot Virtual Machine	Microsoft.Compute/virtualMachines/simulateEviction/action
Get Virtual Machine Instance View	Microsoft.Compute/virtualMachines/instanceView/read
Lists Available Virtual Machine Sizes	Microsoft.Compute/virtualMachines/vmSizes/read
Get Virtual Machine Extension	Microsoft.Compute/virtualMachines/extensions/read
Create or Update Virtual Machine Extension	Microsoft.Compute/virtualMachines/extensions/write
Delete Virtual Machine Extension	Microsoft.Compute/virtualMachines/extensions/delete
Get Virtual Machine run command	Microsoft.Compute/virtualMachines/runCommands/read
Create or Update Virtual Machine run command	Microsoft.Compute/virtualMachines/runCommands/write
Delete Virtual Machine run command	Microsoft.Compute/virtualMachines/runCommands/delete
Summarizes latest patch installation operation results	Microsoft.Compute/virtualMachines/patchInstallationResults/read
Lists all patches considered in patch installation operation	Microsoft.Compute/virtualMachines/patchInstallationResults/softwarePatches/read
Summarizes latest patch assessment operation results	Microsoft.Compute/virtualMachines/patchAssessmentResults/latest/read
Lists all patches assessed in patch assessment operation	Microsoft.Compute/virtualMachines/patchAssessmentResults/latest/softwarePatches/read

```
PS /Users/tom> █
```

Figure 3.29 – Resource provider operations for Azure VMs

The resource provider operations that are listed in *Figure 3.29* are all actions related to VMs to which you can grant access to users, groups, or service principals. If you want to know which resource provider operations are tied to a specific RBAC role, you can find this out with the following command:

```
(Get-AzRoleDefinition <role name>).actions
```

For example, if you want to know which actions are allowed for the VM Contributor RBAC role, the command would be as follows:

```
(Get-AzRoleDefinition "Virtual Machine Contributor").actions
```

The result of this command is shown in *Figure 3.30*:

```
PS /Users/tom> (Get-AzRoleDefinition "Virtual Machine Contributor").actions
Microsoft.Authorization/*/read
Microsoft.Compute/availabilitySets/*
Microsoft.Compute/locations/*
Microsoft.Compute/virtualMachines/*
Microsoft.Compute/virtualMachineScaleSets/*
Microsoft.Compute/cloudServices/*
Microsoft.Compute/disks/write
Microsoft.Compute/disks/read
Microsoft.Compute/disks/delete
Microsoft.DevTestLab/schedules/*
Microsoft.Insights/alertRules/*
Microsoft.Network/applicationGateways/backendAddressPools/join/action
Microsoft.Network/loadBalancers/backendAddressPools/join/action
Microsoft.Network/loadBalancers/inboundNatPools/join/action
Microsoft.Network/loadBalancers/inboundNatRules/join/action
Microsoft.Network/loadBalancers/probes/join/action
Microsoft.Network/loadBalancers/read
Microsoft.Network/locations/*
Microsoft.Network/networkInterfaces/*
Microsoft.Network/networkSecurityGroups/join/action
Microsoft.Network/networkSecurityGroups/read
Microsoft.Network/publicIPAddresses/join/action
Microsoft.Network/publicIPAddresses/read
Microsoft.Network/virtualNetworks/read
Microsoft.Network/virtualNetworks/subnets/join/action
Microsoft.RecoveryServices/locations/*
Microsoft.RecoveryServices/Vaults/backupFabrics/backupProtectionIntent/write
Microsoft.RecoveryServices/Vaults/backupFabrics/protectionContainers/protectedItems/*/read
Microsoft.RecoveryServices/Vaults/backupFabrics/protectionContainers/protectedItems/read
Microsoft.RecoveryServices/Vaults/backupFabrics/protectionContainers/protectedItems/write
Microsoft.RecoveryServices/Vaults/backupPolicies/read
Microsoft.RecoveryServices/Vaults/backupPolicies/write
Microsoft.RecoveryServices/Vaults/read
Microsoft.RecoveryServices/Vaults/usages/read
Microsoft.RecoveryServices/Vaults/write
Microsoft.ResourceHealth/availabilityStatuses/read
Microsoft.Resources/deployments/*
Microsoft.Resources/subscriptions/resourceGroups/read
Microsoft.SerialConsole/serialPorts/connect/action
Microsoft.SqlVirtualMachine/*
Microsoft.Storage/storageAccounts/listKeys/action
Microsoft.Storage/storageAccounts/read
Microsoft.Support/*
PS /Users/tom> █
```

Figure 3.30 – Virtual Machine Contributor provider operations

When you know which resource provider operations exist, you can start creating your custom RBAC role. Again, you can start by creating a new role from scratch or by using an existing role as a template to alter according to your needs.

Now that you have learned what RBAC is and how you can leverage it, we will give you an introduction to **Azure AD Privileged Identity Management (Azure AD PIM)**, another service focused on protecting privileged user accounts. Let's move on!

Protecting admin accounts with Azure AD PIM

In *Chapter 2, Governance and Security*, we discussed the fact that we often see customer environments in which administrators have a huge set of privileges they do not necessarily need. We also discussed **Separation of Duties (SoD)** and the fact that no one should have more privileges than are required for doing a particular job. Now, what if you could even reduce that set of access rights to a particular point in time or time range? Or what if you need an approval process for granting privileged access? Maybe you want to monitor the usage of privileged roles or want to decide whether a person still needs privileged access rights?

Azure AD PIM is a service that offers several features that help you further protect privileged accounts, some of which are the following:

- **Just-in-time** and **time-bound** privileged access
- **Approval** to activate privileged roles
- **Access reviews** to ensure that roles are still needed
- Enforcing **MFA** for role activation

With Azure AD PIM, you can—but do not need to—indefinitely grant access rights to privileged users; you make them *eligible* for requesting access rights. So, users are only granted access rights when they really need them. Access can be either approved automatically after a user requests access or by manual approval. The nice thing is, you can cover both Azure AD and Azure resource roles with Azure AD PIM.

Tip

Make sure you have enforced the principle of least privilege in your organization before starting with PIM.

Managing Azure AD roles in PIM

To start with PIM, sign in to the Azure portal as a Global administrator of your directory and then navigate to **Azure AD Privileged Identity Management**. In the **Manage** section of the PIM dashboard, you can select whether you want to manage privileged identity management for **Azure AD roles**, **Azure resources**, or **Privileged access groups**, as shown in *Figure 3.31*:

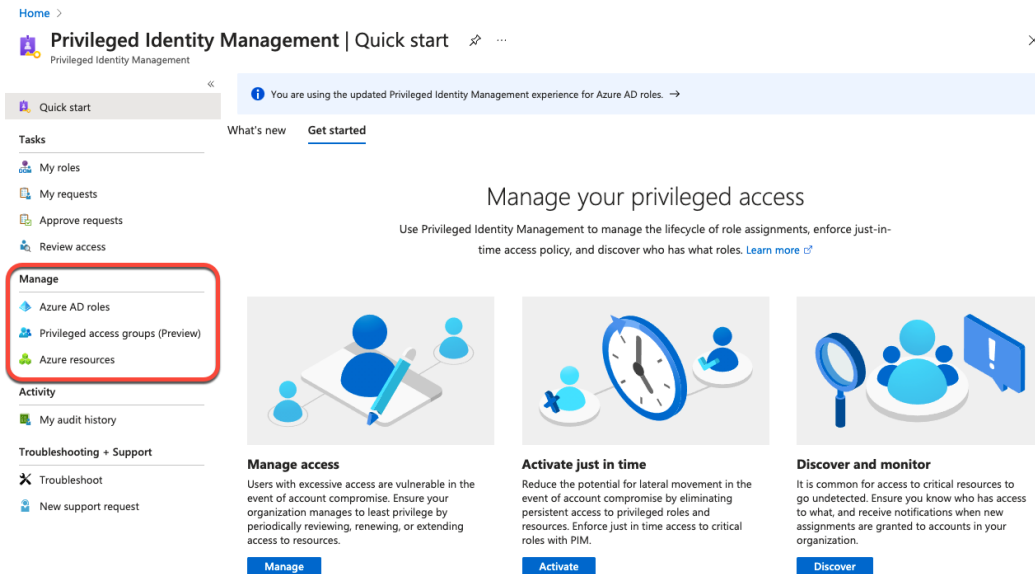


Figure 3.31 – Selecting the scope for managing PIM

In the **Azure AD roles** view, you can manage PIM behavior for all Azure AD roles and also have access to Azure AD role-related tasks, such as access reviews, approvals, or your own roles and access requests, as shown in *Figure 3.32*:

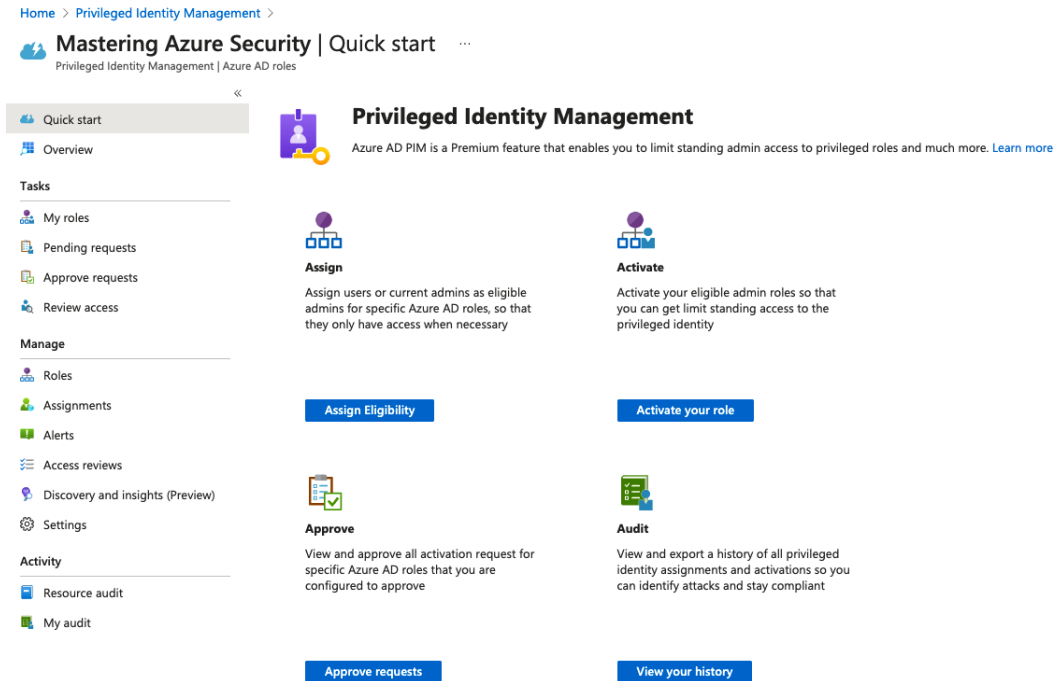


Figure 3.32 – Azure AD role management in PIM

The **Manage** section offers all the Azure AD PIM configuration options for your Azure AD directory. **Roles** gives you an overview of all Azure AD roles and enables you to add members to a particular role. In the **Assignments** section, you get an overview of all accounts that are either eligible or permanent members of privileged roles. It is an at-a-glance view of all users who have elevated rights in your directory. The **Alerts** section shows you issues to review. For example, you will get an overview of administrators who are not using their privileged roles (and therefore can be unassigned). You can also create regular **Access reviews** triggers to determine whether particular accounts still need to be role members. In the respective section, you find a configuration wizard to create these recurring reviews. The wizard in the respective section leads you through a process to convert users with existing permanent role assignments into eligible users. Finally, in **Settings**, you can configure global settings for Azure AD roles, alerts, and access reviews.

Tip

You can require approval for a role activation. This requirement is a setting that comes with the role, not with the user account. That said, when you configure this requirement, it is valid for all users who are eligible to request access for this particular role.

Once an account is made eligible, its owner needs to request access before conducting a task. Let's walk through an example: John works in the helpdesk team and this team is responsible for managing user accounts. Therefore, we want to make John's account eligible for the *User Administrator* role. When John signs in to the Azure portal and wants to manage user settings in Azure AD, he will only be able to see what standard user accounts can see; he won't be able to manage any accounts. So, John navigates to the Azure AD PIM dashboard and sees that he needs to activate his role first.

The screenshot shows the 'Mastering Azure Security | My roles' page in the Azure portal. The page title is 'Privileged Identity Management | Azure AD roles'. On the left, there is a sidebar with 'Quick start' and 'Tasks' sections. The 'Tasks' section includes 'My roles' (selected), 'Pending requests', 'Approve requests', and 'Review access'. The 'Manage' section includes 'Roles', 'Assignments', 'Alerts', and 'Access reviews'. The main content area has tabs for 'Eligible assignments', 'Active assignments', and 'Expired assignments'. Under 'Eligible assignments', there is a search bar and a table with columns: Role, Scope, Membership, End time, and Action. The table lists one role: 'User Administrator' with Scope 'Directory', Membership 'Direct', and End time 'Permanent'. The 'Action' column for this role has a blue 'Activate' link. A red arrow points to this 'Activate' link.

Role	Scope	Membership	End time	Action
User Administrator	Directory	Direct	Permanent	Activate

Figure 3.33 – Eligible roles for a user with the option to activate a role

John can activate the role for the time that you configured before in the **Settings** pane for this particular role (all as the default for all roles). In the following example, the maximum activation duration is 8 hours. In addition to that, John needs to enter a justification that can later be reviewed in the audit history.

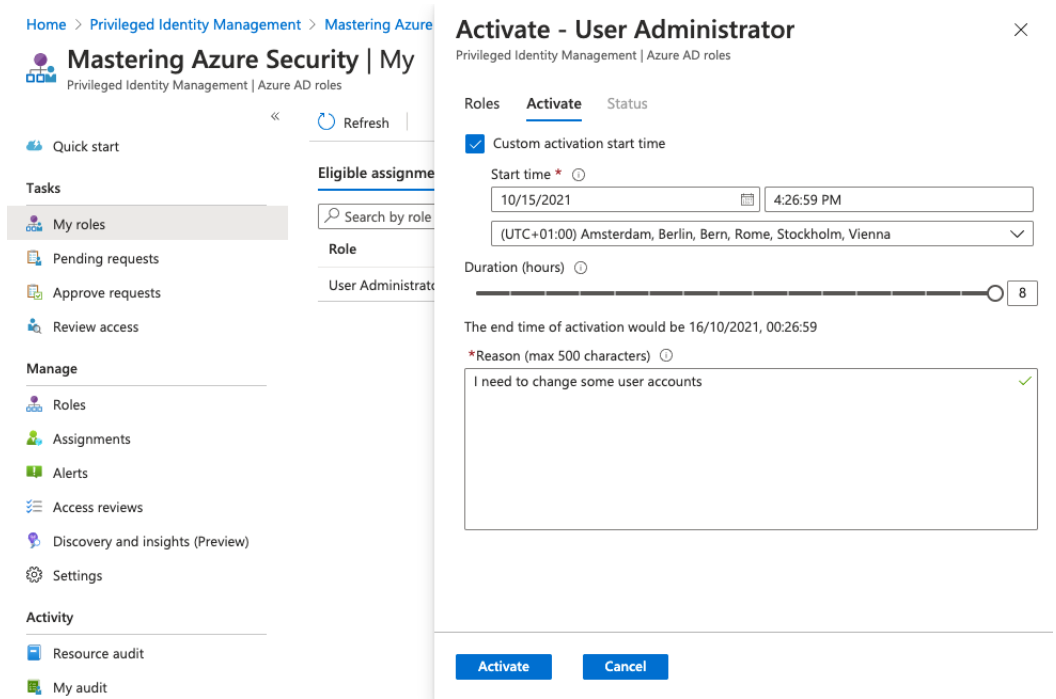


Figure 3.34 – Activating a role

As shown in *Figure 3.34*, John can also set a custom activation start time. For example, when John knows that he will need access later that day, he could manually set an activation time in the future. By clicking on **Activate**, an activation request is submitted and validated once the activation is done. Then, after signing out and logging back in, John will have role membership in **User Administrators** for 8 hours.

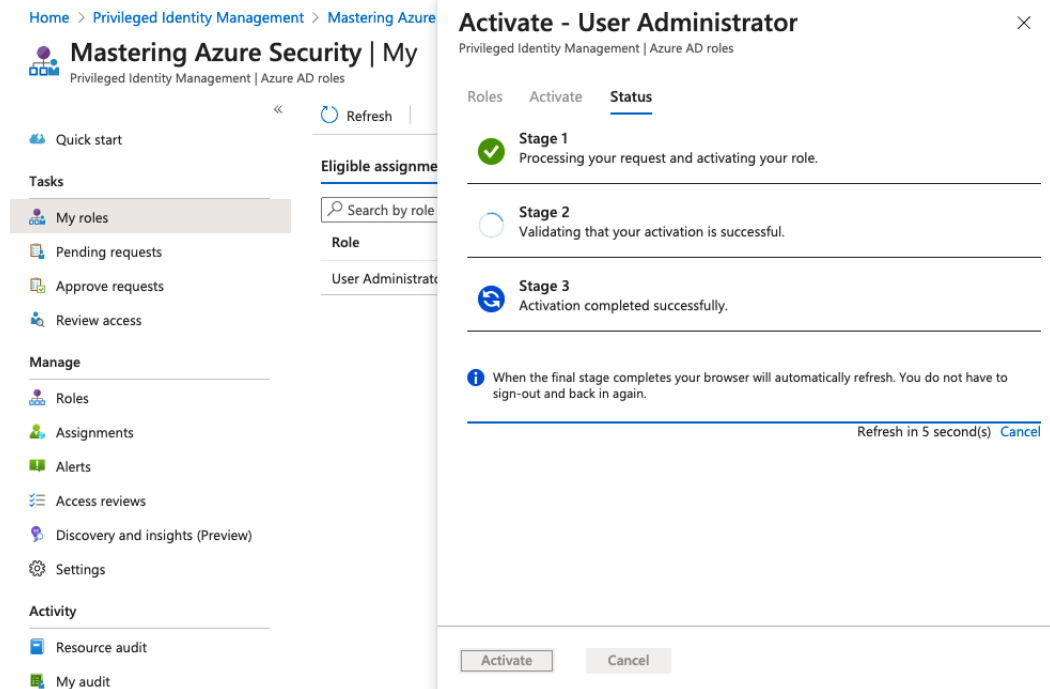


Figure 3.35 – Role activation in progress

So, the process for the user is straightforward. Request access; log out and back in again; and then do the job. Of course, every activation request is logged, as well as every configuration change a user does when having elevated rights. As already mentioned, monitoring is essential, and there's no way around it.

Managing Azure resources with PIM

You can not only use PIM for managing Azure AD roles but also for Azure resource management based on RBAC. You first need to onboard Azure subscriptions to PIM so that you can use it to manage resource-based access rights. To do so, you navigate to **Azure resources** on the Azure AD PIM dashboard and select **Discover resources**. You then filter for unmanaged resources and select **Manage resource**, as illustrated in *Figure 3.36*:

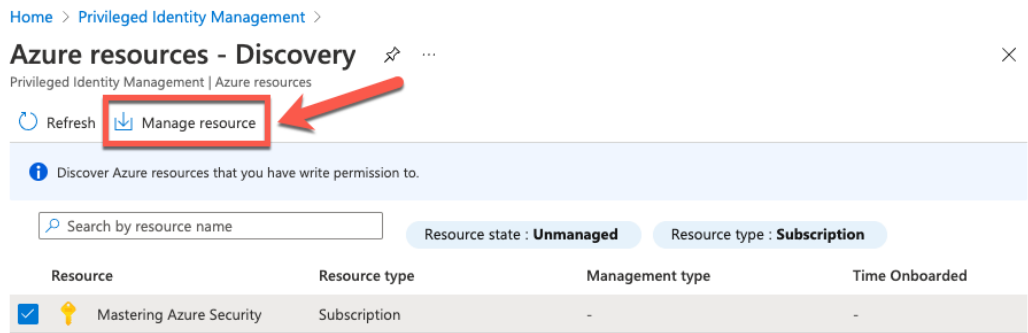


Figure 3.36 – Onboarding an Azure subscription to PIM

You can also select a management group to onboard to PIM. PIM will automatically manage role assignments for all child resources within the given scope after onboarding. The configuration options are similar to the ones for Azure AD management with PIM. One difference is that you need to enter a start and an end date for the role assignment. After the end date, the user will no longer be eligible for requesting access. The maximum is a time frame of 1 year.

Let's take an example. In his role as project owner for the *Mastering Azure Security* project, John needs to manage all resources within a particular resource group that has been created for this project. However, he should not be able to give other users or groups access to that resource group, so access to the Contributor role should be the right one for him.

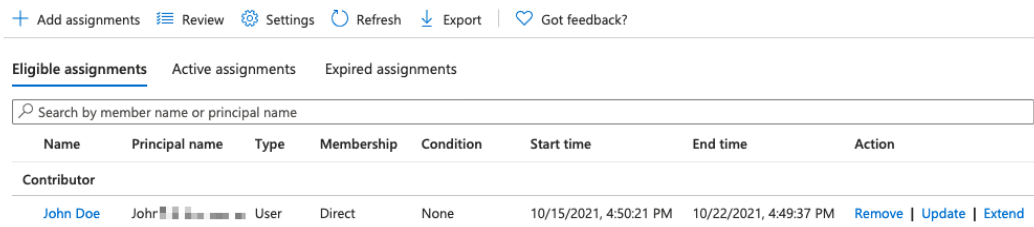


Figure 3.37 – John is a member of the Contributor role via PIM

If John logs in to the Azure portal, he won't be able to see any resources or resource groups because he has no permanent access rights to Azure resources. Again, he navigates to the **Azure AD PIM** dashboard, selects **Azure resources**, and then activates his role. After logging out and signing in again, John is able to manage his resources in the **MasteringAzSec** resource group.

Tip

PIM always activates roles for 1 or several hours, even if access is only needed for 10 minutes. Therefore, users can find an option to deactivate roles after they have finished their tasks by navigating to **My roles** and then **Active roles** in both the **Azure AD roles** and the **Azure resources** sections.

You have now learned how to further reduce your privileged accounts' attack surfaces by using Azure AD PIM.

Hybrid authentication and Single Sign-On

Organizations have come a long way from an on-premises IT infrastructure since they started to roll out cloud-based **Software as a Service (SaaS)** applications such as Microsoft Office 365. Users usually do not want to keep track of several accounts and, as they already have a corporate account based on Windows Server AD, they usually also want to use that account for cloud-based authentications. Now, this is when Azure AD Connect comes into play.

Azure AD Connect is a directory replication service that helps you to synchronize existing user accounts from an on-premises Windows Server AD with Azure AD. This is a mandatory step because you can only grant access rights and licenses to cloud-based identities in Azure AD, not to on-premises accounts.

With Azure AD Connect, you have several options to provide a single sign-on experience to your users. In the following diagram, you can see a decision tree that helps you to decide which of them is the best for your organization:

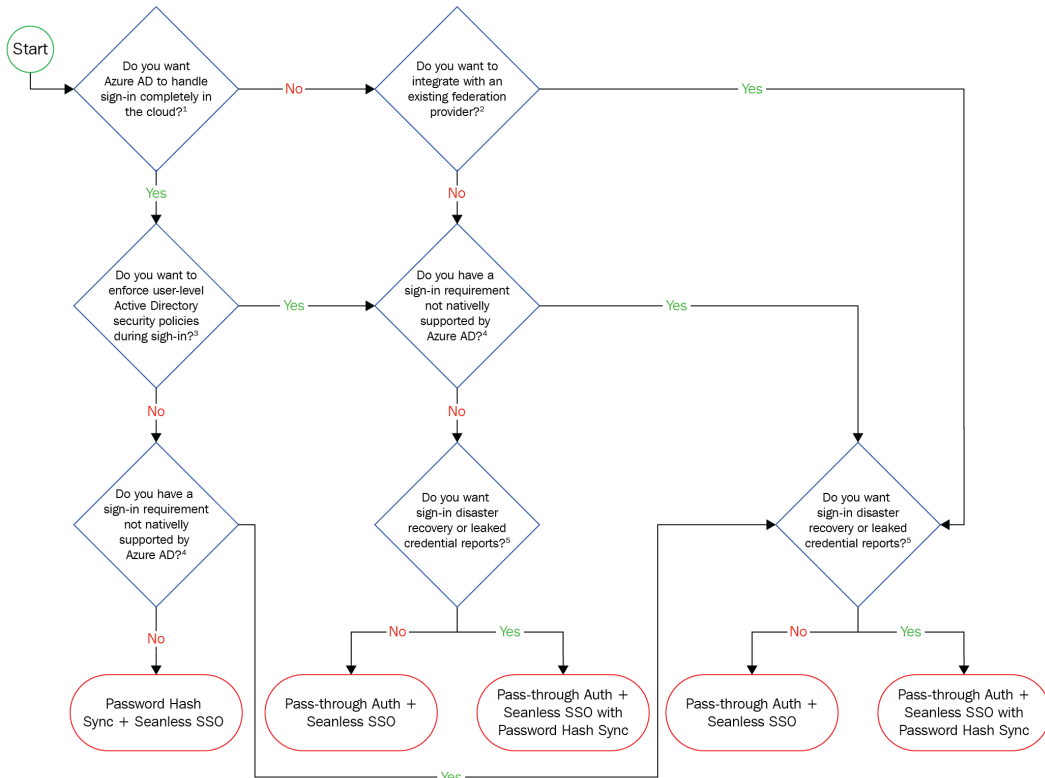


Figure 3.38 – Azure AD Connect authentication decision tree

Password Hash Sync + Seamless SSO is the easiest way to go. With this, your on-premises password hashes are replicated to the corresponding cloud identities, so users can use the same usernames and passwords across all login environments. Seamless SSO eliminates unnecessary login prompts when users are signed in.

If you have stronger requirements, such as user-level AD security policies that should be enforced during sign-in, you can go with *Pass-through Auth + Seamless SSO*. In this scenario, you install a software agent on one or more on-premises servers. These servers can validate your user credentials directly against your on-premises domain controllers by building up a secure tunnel through which cloud-based logons can be validated.

Federation with **Active Directory Federation Services (AD FS)** or another federation service is the most complex authentication method. It should be used in scenarios where sign-in features are not natively supported by Azure AD. These features include the following:

- Sign-in using smartcards or certificates
- Sign-in using an on-premises MFA server
- Sign-in using a third-party authentication solution
- Multi-site on-premises authentication solutions

Tip

We recommend always enabling password hash synchronization in Azure AD Connect. Doing so, you have a fallback scenario in case your on-premises authentication platform is not working properly, and you can use Azure AD Identity Protection to get leaked credential reports. Furthermore, you can enable **self-service password reset (SSPR)** when password hash synchronization is enabled.

With SSPR, you can effectively reduce both users' non-effective time and support efforts for your helpdesk team. SSPR provides users with an effective means to unblock their corporate accounts if they forget their passwords. In order to use SSPR, users must provide a secondary authentication method. When they lose access to their corporate password, users need to verify their previously registered alternate authentication method to prove their identity. After that, the user can enter a new password. If it is a hybrid user account, meaning an account that is replicated by Azure AD Connect, the new password is written back to the on-premises Active Directory. The feature you need to enable in Azure AD Connect in this case is called **password writeback**.

You can enable self-service password reset for your Azure AD tenant by navigating to **Azure Active Directory | Password reset** and then, in the **Properties** tab, selecting the **Self service password reset enabled** setting according to your needs, as shown in *Figure 3.39*:

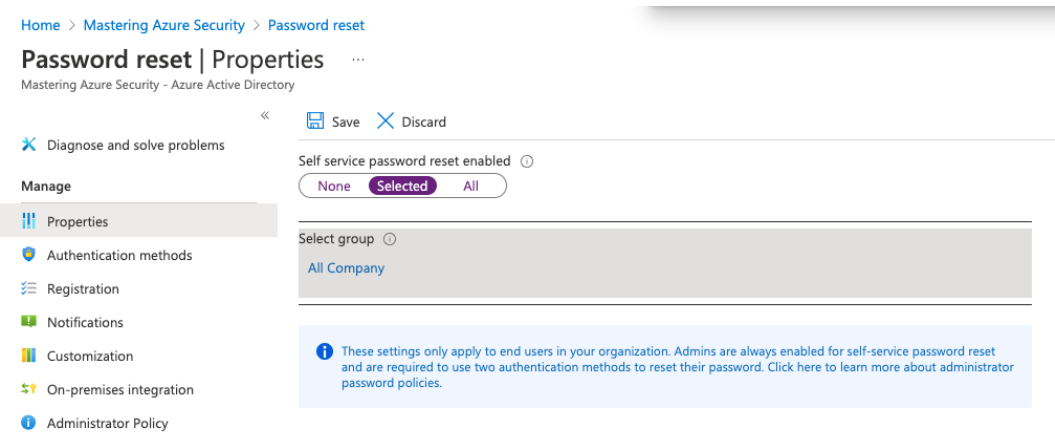


Figure 3.39 – Enabling SSPR for your Azure Active Directory tenant

After enabling SSPR, all eligible accounts will be able to set up security info that will be used in case they want to reset their passwords. This page can be found at <https://mysignins.microsoft.com/security-info>. An example is shown in *Figure 3.40*:

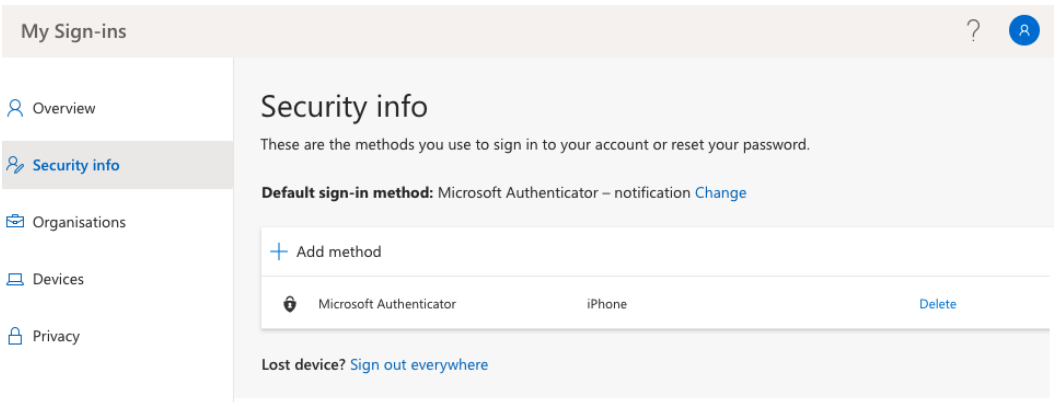


Figure 3.40 – Configuring security info to reset passwords

Integrating on-premises identities into Azure AD is a great way to enhance the sign-in experience across your hybrid organization, and you just learned about the considerations to bear in mind before starting your hybrid journey. In the *Exploring passwords and passphrases* section, you learned that passwords are not good enough to protect your accounts. So, let's move on now to passwordless sign-ins in Azure AD.

Understanding passwordless authentication

In a world in which passwords are not enough to protect identities, we need another, more secure, approach to further protect identities. Passwords can be guessed or phished without physically having access to a user's device or storage. With passwordless authentication, Azure AD offers two different methods to sign in to a cloud-based user account without needing passwords anymore.

You can use the **Microsoft Authenticator app**, the app you already know from the *Understanding multi-factor authentication* section. With this method, you are prompted to approve your sign-in by tapping or entering a number in the Authenticator app, as shown in *Figure 3.41*:

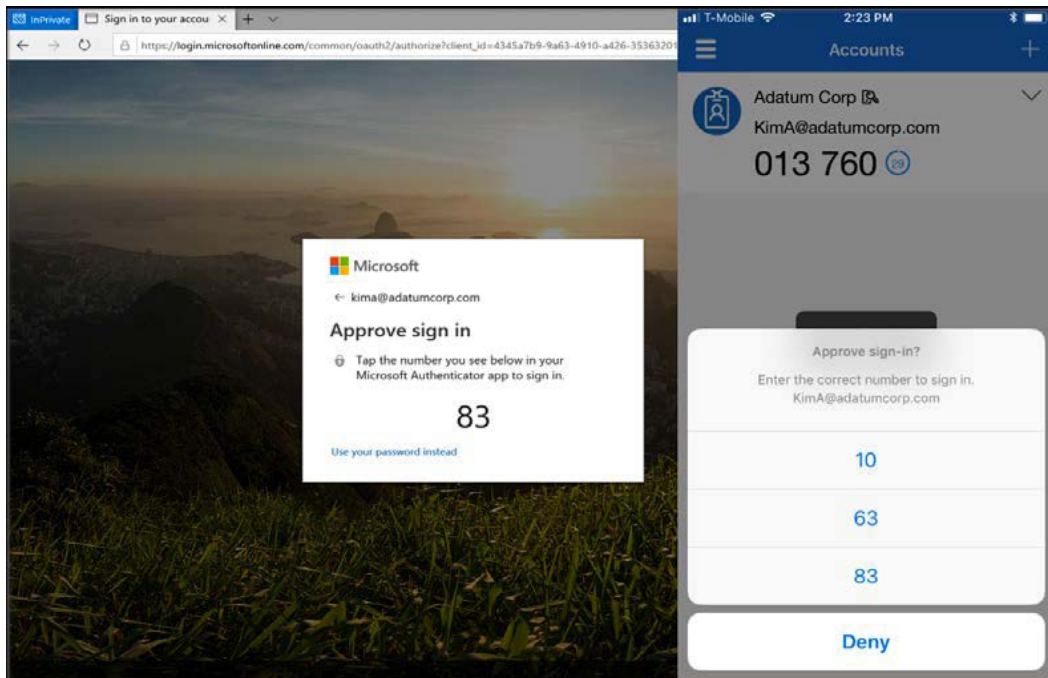


Figure 3.41 – Phone-based passwordless authentication

All the user has to do is register the app as an MFA option and then, in the smartphone app, choose **Enable phone sign-in** from the drop-down menu. After following the instructions in the app, the user can sign in to the app without entering a password.

The second option is to use a **FIDO2 security key** to sign in against Azure AD. FIDO2-based passwordless authentication is currently only supported on Windows 10/11 and in the Microsoft Edge browser.

The following image shows some FIDO2 security keys for reference:



Figure 3.42 – A variety of USB-A and USB-C FIDO2 security keys from different vendors

Using this method, the user needs to have physical access to the security key that is needed during the authentication process.

Global settings

In order to allow users to leverage FIDO2 security keys, a Global administrator needs to enable this authentication method in Azure AD. To enable authentication methods, navigate to **Azure Active Directory | Security | Authentication methods** and enable the **FIDO2 Security Key** setting, as shown in *Figure 3.43*:

Home > Mastering Azure Security > Security >

Authentication methods | Policies

Mastering Azure Security - Azure AD Security

Search (Cmd+/) Got feedback?

Manage

- Policies
- Password protection

Monitoring

- Activity
- User registration details
- Registration and reset events
- Bulk operation results

Configure your users in the authentication methods policy to enable passwordless authentication. Once configured, you will need to enable your users for the enhanced registration preview so they can register these authentication methods and use them to sign in.

Method	Target	Enabled
FIDO2 Security Key	All users	Yes
Microsoft Authenticator	All users	Yes
Text message (preview)		No
Temporary Access Pass (preview)		No

Figure 3.43 – Enabling FIDO2 Security Key authentication in Azure AD

After this, users can select **Security Key** as the authentication method in `https://mysignins.microsoft.com`, as shown in *Figure 3.44*:

My Sign-ins

Security info

These are the methods you use to sign in to your account or reset your password.

Default sign-in method: Microsoft Authenticator – notification [Change](#)

+ Add method

Method	Device	Action
Microsoft Authenticator	iPhone	Delete

Lost device? [Sign out everywhere](#)

Add a method

Which method would you like to add?

Choose a method

- Authenticator app
- Phone
- Email
- App password
- Security key

Figure 3.44 – Security key as a new authentication method

You have now learned how to enable passwordless authentication for your users. In the next section, you will learn about the licensing of security features in Azure AD.

Licensing considerations

Azure AD licensing can become a bit complex when it comes to security features. This is why we decided to compare different licenses and to explain which license you need for which feature. Azure AD is always licensed on a per-user basis, so you need one license for each user. For all security features that have been discussed in this chapter, you will either need an Azure AD Premium P1 or an Azure AD Premium P2 license. These licenses are also part of Microsoft 365 E3 and E5 license packages.

For the following features, you will need an Azure AD Premium P2 (or Microsoft 365 E5 license):

- Azure AD PIM
- Azure AD Identity Protection
- Risk-based Conditional Access policies

For the following features, an Azure AD Premium P1 (or Microsoft 365 E3) license is sufficient:

- Conditional Access based on group, location, and device status
- Self-service password reset

Please be aware that these licensing models might be subject to change, so in order to always be up to date, make sure to check Microsoft's Azure AD pricing documentation at <https://azure.microsoft.com/pricing/details/active-directory>.

Summary

Identity is the new perimeter, and so it's essential to properly protect user and administrator accounts in the cloud. In this chapter, you have learned why passwords are not enough to protect your accounts and how to protect them with MFA, Azure AD Identity Protection, and passwordless authentication. You can reduce your privileged accounts' attack surface as of now, to integrate Azure AD into your existing environments.

In the next chapter, you will learn how to protect network infrastructure resources on Azure and which platform services you need in order to achieve a better security posture.

Questions

As we conclude, here is a list of questions for you to test your knowledge regarding this chapter's material. You will find the answers in the *Assessments* section of the *Appendix*:

1. In the cloud in general, identities are...
 - A. More exposed to attacks
 - B. Less exposed to attacks
 - C. Equally exposed to attacks
2. We can control password settings with...
 - A. Azure AD protection
 - B. Account protection
 - C. Password protection
3. The best way to increase account security is to use...
 - A. More complex passwords
 - B. More frequent password changes
 - C. **Multi-Factor authentication (MFA)**
 - D. Passwordless sign-in methods
4. You can control actions that require MFA.
 - A. Yes
 - B. No
 - C. Only for some actions
5. Which report is not available in risk detection?
 - A. Users with leaked credentials
 - B. Impossible travel
 - C. Infected users
 - D. Infected devices

6. With roles in **Privileged Identity Management (PIM)**, the user has...
 - A. More privileges
 - B. Fewer privileges
 - C. To activate privileges
7. The most secure authentication method is...
 - A. A password that is easy to remember
 - B. A complex password
 - C. MFA
 - D. Passwordless authentication

Section 2: Cloud Infrastructure Security

This section explains how to deploy infrastructure in a secure way and how to manage secrets and certificates in Microsoft Azure. It also covers how to protect network resources and data in the cloud.

This part of the book comprises the following chapters:

- *Chapter 4, Azure Network Security*
- *Chapter 5, Azure Key Vault*
- *Chapter 6, Data Security*

4

Azure Network Security

In *Chapter 1, An Introduction to Azure Security*, we briefly touched on network security in Azure, but only discussed how network security is handled by Microsoft inside Azure data centers. As the network also falls under the shared responsibility model, in this chapter, we will discuss network security from a user aspect and how to handle the security we are responsible for.

We will cover the following topics in this chapter:

- Understanding Azure Virtual Network
- Considering other virtual network security options
- Azure DDoS protection
- Azure Bastion
- Hub-and-spoke network topology
- Understanding Azure Application Gateway
- Understanding Azure Front Door

Understanding Azure Virtual Network

The first step in the transition from an on-premises environment to the cloud is **Infrastructure as a Service (IaaS)**. One of the key elements of IaaS is **Virtual Networks (VNets)**. VNets are a virtual representation of our local network with IP address ranges, subnets, and all other network components that we would find in local infrastructure. Recently, we have seen a lot of cloud network components introduced to on-premises networks as well, with the introduction of **Software-Defined Networking (SDN)** in Windows Server 2016.

Before we start looking at VNet security, let's remember that naming standards should be applied to all Azure resources, and networking is no exception. As environments grow, this will help you have better control over your environment, easier management, and more insight into your security posture.

Each VNet that we create is a completely isolated piece of a network in Azure. We can create multiple VNets inside one subscription, or even multiple VNets inside one region. There is no direct communication between any VNets, even those created inside a single subscription or region, unless configured otherwise. The first thing that needs to be configured for a VNet is the IP address range. The next thing we need is a subnet with its own range. One VNet can have multiple subnets. Each subnet must have its own IP address range within the VNet's IP address range and cannot overlap with other subnets in the same VNet.

One thing we need to consider when defining the IP address range is that it should not overlap with other VNets we use. Even when there is no initial requirement to create a connection between different VNets, this may become a requirement in the future.

Important Note

VNets that have overlapping IP ranges will not be compatible for connection.

VNets are used for communication between Azure resources over private IP addresses. Primarily, they're used for communication between Azure **Virtual Machines (VMs)**, but other resources can be configured to use private IP addresses for communication as well.

Communication between Azure VMs occurs over a **Network Interface Card (NIC)**. Each VM can be assigned one or more NICs, depending on the VM size. A bigger size allows more NICs to be associated with a VM. Each NIC can be assigned a private and public IP address. A private IP address is required, and a public IP address is optional. As an NIC must have a private IP address, it must be associated with VNet and a subnet on the same VNet.

As a first line of defense, we can use a **Network Security Group (NSG)** to control traffic for Azure VMs. NSGs can be used to control inbound and outbound traffic. Default inbound and outbound rules are created during the NSG's creation, but we can change (or remove) these rules and create additional rules based on our requirements. The default inbound rules are shown in the following screenshot:

Priority	Name	Port	Protocol	Source	Destination	Action	
65000	AllowVnetInBound	Any	Any	VirtualNetwork	VirtualNetwork	✓ Allow	...
65001	AllowAzureLoadBalancerInBound	Any	Any	AzureLoadBalancer	Any	✓ Allow	...
65500	DenyAllInBound	Any	Any	Any	Any	✗ Deny	...

Figure 4.1 – Inbound security rules

The default inbound rules will allow any traffic coming from within the VNet and any traffic forwarded from Azure Load Balancer. All other traffic will be blocked.

Conversely, the default outbound rule will allow almost any outbound traffic. The default rules will allow any outgoing traffic to a VNet or the internet, as in the following screenshot:

Priority	Name	Port	Protocol	Source	Destination	Action	
65000	AllowVnetOutBound	Any	Any	VirtualNetwork	VirtualNetwork	✓ Allow	...
65001	AllowInternetOutBound	Any	Any	Any	Internet	✓ Allow	...
65500	DenyAllOutBound	Any	Any	Any	Any	✗ Deny	...

Figure 4.2 – Outbound security rules

To add a new inbound rule, we need to define the source, source port range, destination, destination port range, protocol, action, priority, and name. Optionally, we can add a description that will help us understand why this rule was created. An example of how to create a rule to allow traffic over port 443 (HTTPS) is shown in the following screenshot:

Add inbound security rule

NSG1

Basic

* Source 

Any

▼

* Source port ranges 

*

* Destination 

Any

▼

* Destination port ranges 

443

✓

* Protocol

Any

TCP

UDP

* Action

Allow

Deny

* Priority 

100

* Name

Port_443

✓

Description

Figure 4.3 – Adding new inbound security rules

Alternatively, we can create the same rule with Azure PowerShell:

1. First, we need to create a resource group where resources will be deployed:

```
New-AzResourceGroup -Name "Packt-Security" `
-Location "westeurope"
```

2. Next, we need to deploy our VNet:

```
New-AzVirtualNetwork -Name "Packt-VNet" `
-ResourceGroupName "Packt-Security" `
-Location "westeurope" `
-AddressPrefix 10.11.0.0/16
```

3. And finally, we deploy an NSG and create a rule:

```
New-AzNetworkSecurityGroup -Name "nsg1" `
-ResourceGroupName "Packt-Security" `
-Location "westeurope"
$nsg=Get-AzNetworkSecurityGroup -Name 'nsg1' `
-ResourceGroupName 'Packt-Security'
$nsg | Add-AzNetworkSecurityRuleConfig `
-Name 'Allow_HTTPS' `
-Description 'Allow_HTTPS' `
-Access Allow -Protocol Tcp `
-Direction Inbound `
-Priority 100 `
-SourceAddressPrefix Internet `
-SourcePortRange * `
-DestinationAddressPrefix * `
-DestinationPortRange 443 `
| Set-AzNetworkSecurityGroup
```


In order to add a new outbound rule, we need to define the same option as the inbound rule. An example of how to create a rule to deny traffic over port 22 is shown in the following screenshot:

Figure 4.4 – Adding new outbound security rules

Note that priority plays a very important role when it comes to NSGs. A lower number means higher priority and a higher number means lower priority. If we have two rules that are in contradiction, the rule with the lower number will take precedence. For example, if we create a rule to allow traffic over port 443 with a priority of 100 and create a rule to deny traffic over port 443 with a priority of 400, traffic will be allowed, as the Allow rule has a greater priority.

Again, we can use Azure PowerShell to create the rule:

```
$nsg | Add-AzNetworkSecurityRuleConfig -Name 'Allow_SSH' `
    -Description 'Allow_SSH' `
    -Access Allow -Protocol Tcp `
    -Direction Outbound -Priority 100 `
    -SourceAddressPrefix VirtualNetwork -SourcePortRange * `
    -DestinationAddressPrefix * -DestinationPortRange 22 `
| Set-AzNetworkSecurityGroup
```

An NSG can be associated with subnets and NICs. An NSG associated with a subnet level will have all the rules applied to all the devices associated with that subnet. When an NSG is associated with a NIC, rules will be applied only to that NIC. It's recommended to associate NSGs with subnets instead of NICs for more simple management. It's easy to manage traffic on a NIC level when we have fewer VMs. But when the number of VMs is in the dozens, hundreds, or even thousands, it becomes very hard to manage traffic on a NIC level. It's much better to have VMs that have similar requirements associated with a subnet level.

In order to associate an NSG with a subnet, follow these steps:

1. Go to the **Subnet** section under **NSG1** and select **Associate**, as in the following screenshot:

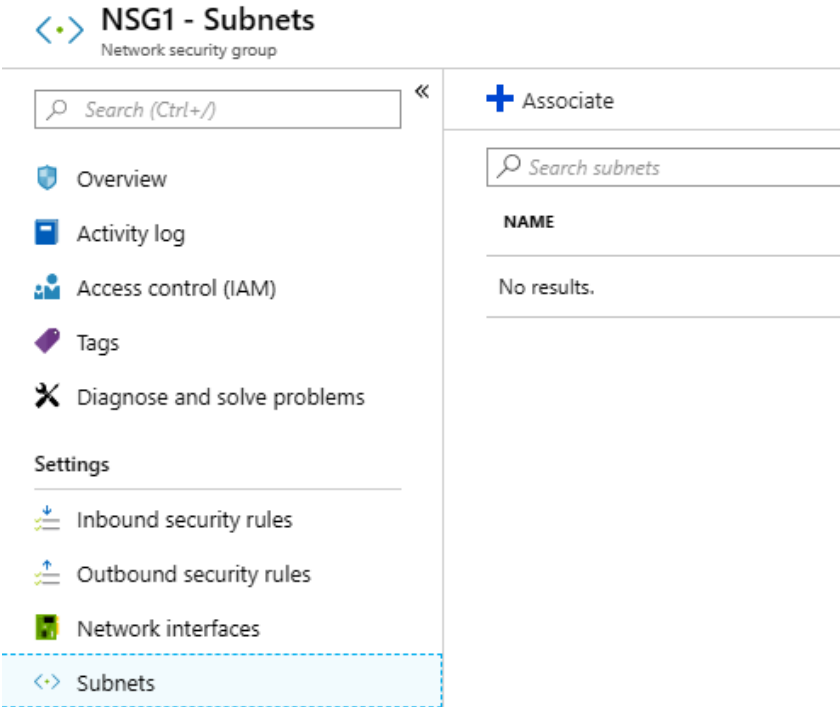


Figure 4.5 – NSG - Subnets blade

2. Next, we select the VNet, as in the following screenshot:

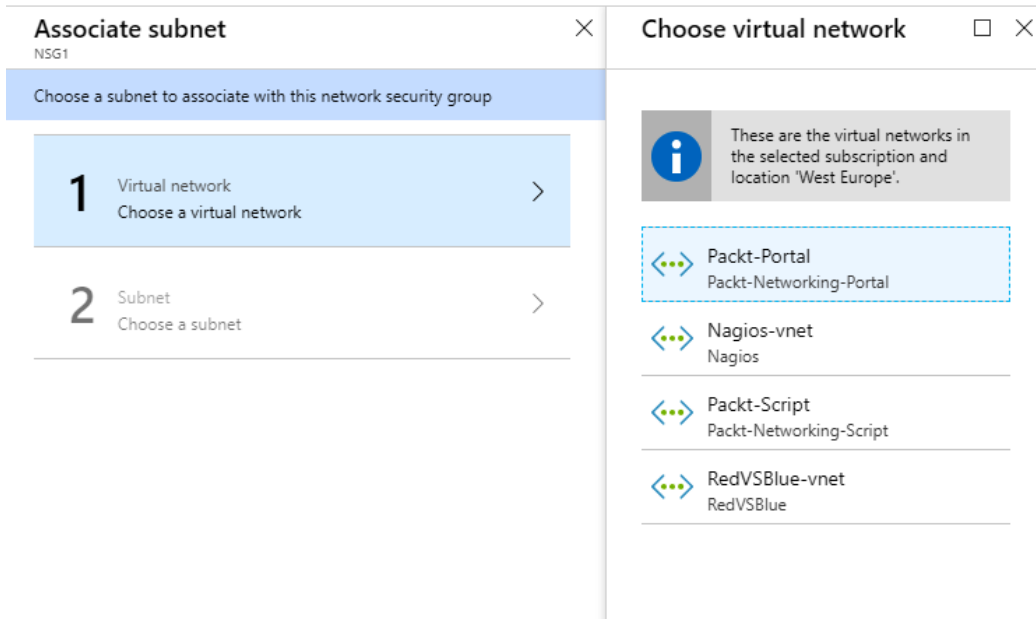


Figure 4.6 – VNet association with NSG

3. Finally, we select the subnet and confirm.

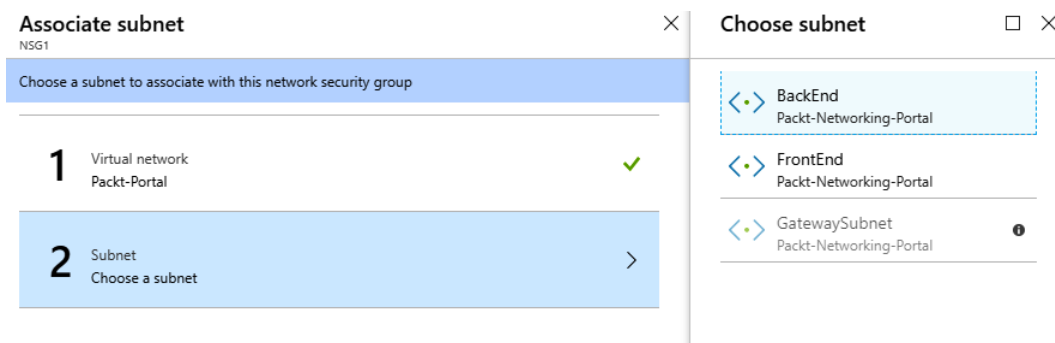


Figure 4.7 – Subnet association with NSG

Now, all of the Azure VMs added to this subnet will have all the NSG rules applied immediately.

The Azure PowerShell script to associate an NSG with a subnet is the following:

```
$vnet = Get-AzVirtualNetwork -Name 'Packt-VNet' '
-ResourceGroupName 'Packt-Security'
Add-AzVirtualNetworkSubnetConfig -Name FrontEnd '
-AddressPrefix 10.11.0.0/24 -VirtualNetwork $vnet '
$subnet = Get-AzVirtualNetworkSubnetConfig '
-VirtualNetwork $vnet -Name FrontEnd '
$nsg = Get-AzNetworkSecurityGroup '
-ResourceGroupName 'Packt-Security' -Name 'nsg1'
$subnet.NetworkSecurityGroup = $nsg
Set-AzVirtualNetwork -VirtualNetwork $vnet
```

Let's take a simple three-tier application architecture as an example. Here, we would have VMs accessible from outside (over the internet), and these VMs should be placed in a DMZ subnet, which would be associated with an NSG that would allow such traffic. Next, we would have an application tier, which would allow traffic inside a VNet but no direct access over the internet.

The application tier would be associated with an appropriate subnet, which would (with the NSG on a subnet level) deny traffic over the internet but allow any traffic coming from the DMZ. Lastly, we would have a database tier, which would allow only traffic coming from the application tier, using the NSG associated with a subnet level. This way, any request would be able to reach the DMZ tier. Once a request is validated, it can pass to the application tier and, from there, it can reach the database tier. No direct communication is allowed between the DMZ and database tiers, and a direct request is not allowed to go from the internet to the application or database tiers.

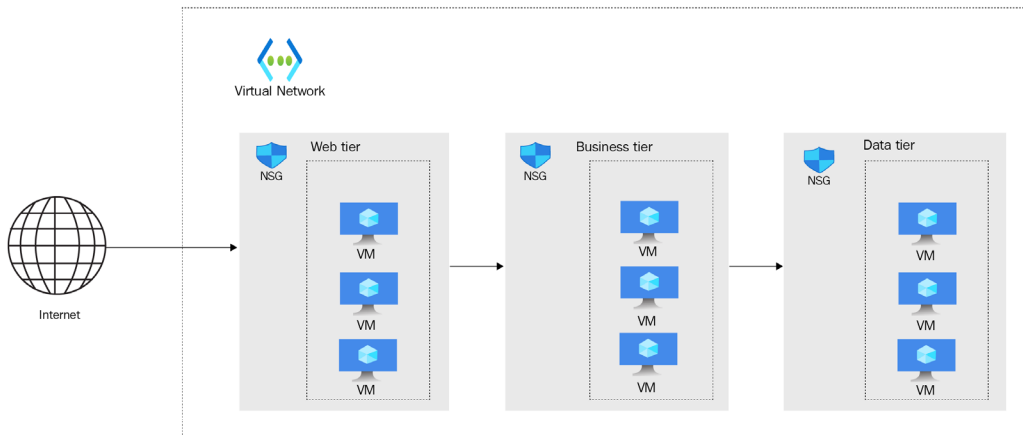


Figure 4.8 – 3-tier network setup

For more security, resources can be associated with application groups. Using NSGs and application groups, we can create additional security rules with more network filtering options. Resources are associated with **application security groups (ASGs)**, and then we can link this group to the NSG rule and allow traffic to access a specific group of resources inside VNet or the subnet. Association is usually made based on workload type, so traffic can be filtered on an additional level. Combining NSG and ASG provides better control, so traffic is not only controlled based on network segmentation but at the application level as well, which provides a way to allow specific traffic only for some resources inside the same network.

We will now be looking at connecting on-premises networks with Azure.

Connecting on-premises networks with Azure

In most cases, we already have some sort of local infrastructure and want to use the cloud as a hybrid where we combine cloud and on-premises resources. In such cases, we need to think about how we are going to access VNet from our local network.

There are three options available:

- **Point-to-Site connection (P2S)** is usually used for management and/or end use connections. It enables you to create a connection from a single on-premises computer to Azure VNet. It has a secure connection, but not the most persistent one, and shouldn't be used for production purposes, only to perform management and maintenance tasks, or to access applications.
- **Site-to-Site connection (S2S)** is a persistent connection that enables a network-to-network connection. In this case, that would be from an on-premises network to a VNet, where all on-premises devices can connect to Azure resources and vice versa. Using S2S enables you to expend local infrastructure to Azure, use a hybrid cloud, and take advantage of the best things both on-premises and cloud networks can offer.

- **ExpressRoute** is a direct connection from a local data center to Azure. It doesn't go over the internet and offers a much better connection. Compared to an S2S connection, ExpressRoute offers more reliability and speed with lower network latency.

Next, we will be checking out how to create an S2S connection.

Creating an S2S connection

In order to create an S2S connection, several resources must be created. First, we need to create a **Virtual Network Gateway (VNG)**. During the process of creating a VNG, we need to define a subscription, a name for the VNG, the region where it will be created, and a VNet must be selected.

The VNet that we can select is limited to the region where the VNG will be created. A separate gateway subnet must be defined, so we can either select an existing one or it will be created automatically if it doesn't exist on the selected VNet.

In the same section, we need to define the public IP address (create a new one or select an existing one) and select to enable (or disable) active-active mode or BGP. An example is shown in the screenshot that follows.

The following details need to be filled in:

- **Subscription**
- **Instance details**
- **Public IP address**

You can see an example of this in the following screenshot:

Create virtual network gateway

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription *

Resource group ⓘ

Instance details

Name *

Region *

Gateway type * ⓘ ☒ VPN ☐ ExpressRoute

VPN type * ⓘ ☒ Route-based ☐ Policy-based

SKU * ⓘ

VIRTUAL NETWORK

Virtual network * ⓘ

Gateway subnet address range * ⓘ
10.11.1.0 - 10.11.1.255 (256 addresses)

ⓘ Only virtual networks in the currently selected subscription and region are listed.

Public IP address

Public IP address * ⓘ ☒ Create new ☐ Use existing

Public IP address name *

Public IP address SKU

Assignment * ☒ Dynamic ☐ Static

Enable active-active mode * ⓘ ☐ Enabled ☒ Disabled

Configure BGP ASN * ⓘ ☐ Enabled ☒ Disabled

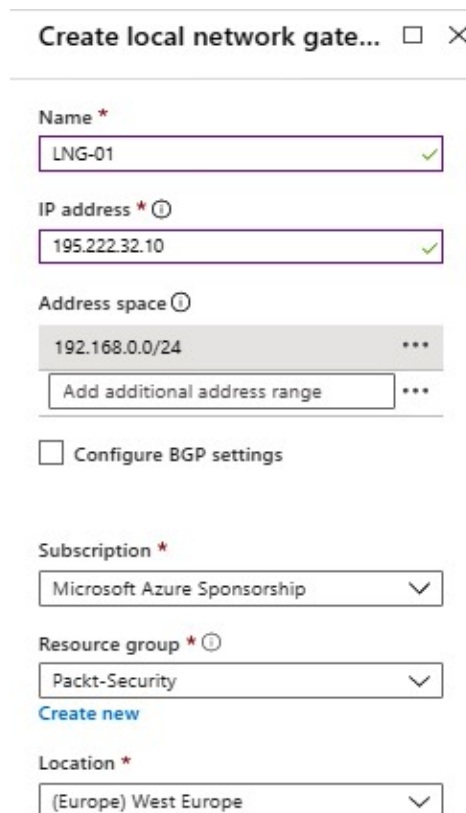
Azure recommends using a validated VPN device with your virtual network gateway. To view a list of validated devices and instructions for configuration, refer to Azure's [documentation](#) regarding validated VPN devices.

Figure 4.9 – Creating a VNG

Another resource we need to create is a **Local Network Gateway (LNG)**. To create an LNG, we need to define the following:

- **Name**
- **IP address**
- **Address space**
- **Subscription**
- **Resource group**
- **Location**
- **BGP settings**

The BGP settings are optional. The IP address we need to define is the public IP address of our VPN device, and the address range is the address range of our local network. An example is shown in the following screenshot:



The screenshot shows the 'Create local network gateway' form in the Azure portal. The form is titled 'Create local network gate...' with a close button (X) in the top right corner. The form contains the following fields and options:

- Name ***: A text input field containing 'LNG-01' with a green checkmark icon on the right.
- IP address * ⓘ**: A text input field containing '195.222.32.10' with a green checkmark icon on the right.
- Address space ⓘ**: A dropdown menu showing '192.168.0.0/24' with a three-dot menu icon on the right. Below it is a button labeled 'Add additional address range' with a three-dot menu icon on the right.
- Configure BGP settings**: A checkbox that is currently unchecked.
- Subscription ***: A dropdown menu showing 'Microsoft Azure Sponsorship' with a downward arrow icon on the right.
- Resource group * ⓘ**: A dropdown menu showing 'Packt-Security' with a downward arrow icon on the right. Below it is a link labeled 'Create new' in blue text.
- Location ***: A dropdown menu showing '(Europe) West Europe' with a downward arrow icon on the right.

Figure 4.10 – Creating an LNG

After a VNG and an LNG are created, we need to create a connection in VNet:

1. Under **Connection settings** in the **VNet** blade, add a new connection.
2. The following parameters need to be defined: **Name**, **Connection type**, **Virtual network gateway**, **Local network gateway**, **Shared key (PSK)**, and **IKE Protocol**.
3. **Subscription**, **Resource group**, and **Location** will be locked and will use the same options as the ones assigned to the selected VNet:

Add connection VNG-01

Name *
On-prem ✓

Connection type ①
Site-to-site (IPsec) ✓

***Virtual network gateway ①**
VNG-01 🔒

***Local network gateway ①**
LNG-01 >

Shared key (PSK) * ①
verysecretpassword! ✓

IKE Protocol ①
☐ IKEv1
 ☒ IKEv2

Subscription ①
Microsoft Azure Sponsorship ✓

Resource group ①
Packt-Security 🔒
Create new

Location ①
West Europe ✓

Figure 4.11 – Creating a VPN connection

After a connection is created in Azure, we still need to create a connection on our local VPN device. It's highly recommended to only use supported devices (most industry leaders are supported, such as Cisco, Palo Alto, and Juniper, to name a few). When configuring a connection on a local VPN device, we need to take into account all the parameters used on the Azure side.

Once a connection is configured on both sides, the tunnel is up, and we should be able to access Azure resources from an on-premises network and from Azure to a local network. Of course, we can control how traffic flows and what can access what, how, and under which conditions.

We have now seen how to create connections between Azure VNets and local networks, but we often need to connect one VNet with another VNet. Of course, it's still important to keep the same level of security, even if everything is inside Azure. In the next section, we'll discuss how to connect networks in such a situation.

Connecting a VNet to another VNet

In a case where we have multiple VNets in Azure, we may need to create a connection between them in order to allow services to communicate between networks. There are two ways in which we can achieve this goal:

- The first one would be to create an S2S between the VNets. In this case, the process is very similar to creating an S2S between a VNet and a local network. We need to create a VNG for both VNets, but we don't need an LNG. When creating a connection, we need to select **VNet-to-VNet** in **Connection types** and select appropriate VNGs.
- Another option would be to create VNet peering. An S2S connection is secure and encrypted, but it passes over the internet. Peering uses an Azure backbone network to route traffic, and it never leaves Azure. This makes peering even safer.

To create peering between VNets, we need to carry out the following steps:

1. Go to the **Peerings** section in the VNet blade and add a new peering.
2. We need to define the name and the VNet we want to create a connection to.
3. Other settings are also present, such as whether we want to allow connections to go both ways, or whether we want to allow forwarded traffic.

A peering example is shown in the following screenshot:

Add peering
Packt-VNet

Name of the peering from Packt-VNet to RedVSBlue-vnet *

VNet-Peering ✓

Peer details

Virtual network deployment model ⓘ

☒ Resource manager ☐ Classic

☐ I know my resource ID ⓘ

Subscription * ⓘ

Microsoft Azure Sponsorship ▼

Virtual network *

RedVSBlue-vnet (RedVSBlue) ▼

Name of the peering from RedVSBlue-vnet to Packt-VNet *

VNet-Peering ✓

Configuration

Configure virtual network access settings

Allow virtual network access from Packt-VNet to RedVSBlue-vnet ⓘ

Disabled **Enabled**

Allow virtual network access from RedVSBlue-vnet to Packt-VNet ⓘ

Disabled **Enabled**

Configure forwarded traffic settings

Allow forwarded traffic from RedVSBlue-vnet to Packt-VNet ⓘ

Disabled Enabled

Allow forwarded traffic from Packt-VNet to RedVSBlue-vnet ⓘ

Disabled Enabled

Configure gateway transit settings

☒ Allow gateway transit ⓘ

Figure 4.12 – Creating VNet-to-VNet peering

It's very important to understand additional security settings in VNet peering and how they affect network traffic.

The network access settings will define traffic access from one VNet to another. For example, we may want to enable access from **VNet A** to **VNet B**. But, because of security settings, we want to block access from **VNet B** to **VNet A**. This way, resources in **VNet A** will be able to access resources in **VNet B**, but resources in **VNet B** will not be able to access resources in **VNet A**.

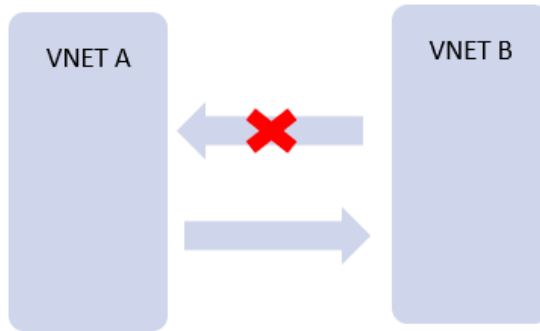


Figure 4.13 – VNet peering

In the next section, we will define how we handle forwarded traffic. Let's say that **VNet A** is connected to **VNet B** and **VNet C**. There is no connection between **VNet B** and **VNet C**. With these settings, we define whether we want to allow traffic from **VNet B** to reach **VNet C** via **VNet A**. The same thing can be defined the other way around.

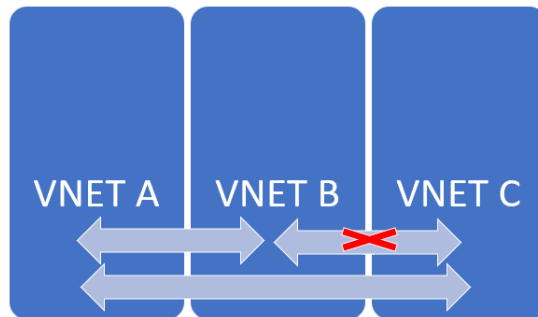


Figure 4.14 – VNet peering with multiple VNets

The **gateway transit** settings allow us to control whether we want a current connection to be able to use other connections with another network. For example, **VNet A** is connected to **VNet B**, and **VNet B** is connected to an on-premises network (or another VNet). This setting will define whether traffic from **VNet A** will be able to reach the on-premises network. In this case, one of the VNets would be replaced with the on-premises network. If there is a connection between the on-premises network and **VNet A**, and a connection between **VNet A** and **VNet B**, the gateway transit would decide whether traffic from **VNet B** can reach the on-premises network.

In the next section, we will be discussing another important security option, which is service endpoints in VNets.

VNet service endpoints

VNet service endpoints enable us to extend some **Platform as a Service (PaaS)** services to use private address spaces. With service endpoints, we connect services (that don't have this option by default) to our VNet enabling services to communicate over a private network. This way, traffic is never exposed publicly, and data exchange is carried out over the Microsoft Azure backbone network.

Only some Azure services are supported when it comes to service endpoints. The list of services is subject to change and new services can be added over time. For more details, check the Microsoft documentation for service endpoints.

The first security benefit from using service endpoints is definitely that data never leaves the private space. Let's say that we have Azure App Service and Azure SQL Database connected to the VNet with service endpoints. This way, all communication between the web application on App Service and the database on Azure SQL Database would be done securely over the Azure backbone network. No data would be exposed publicly, as is the case when using the same services without endpoints.

Without this feature, both services would only have public IP addresses and communication between them going over the internet. Even though there are ways of doing this securely, with communication being sent encrypted over HTTPS, using service endpoints partly removes the security risk in this communication.

But the security benefits of using service endpoints don't stop there. As services connected to VNet with service endpoints are assigned to a specific subnet, all security rules associated with this subnet are applied to our services as well. If an NSG blocks specific traffic on our subnet, the same traffic will be blocked for PaaS services as well.

We can enable service endpoints on VNet either during the creation of a VNet or at a later time. Service endpoints are enabled on a subnet level, and this can be done either on a VNet or subnet configuration. Follow these steps to enable service endpoints in VNet:

1. Go to the VNet blade and select **Service endpoints**. Click **Add** and select the subnet and services you want to use, as in the following screenshot:

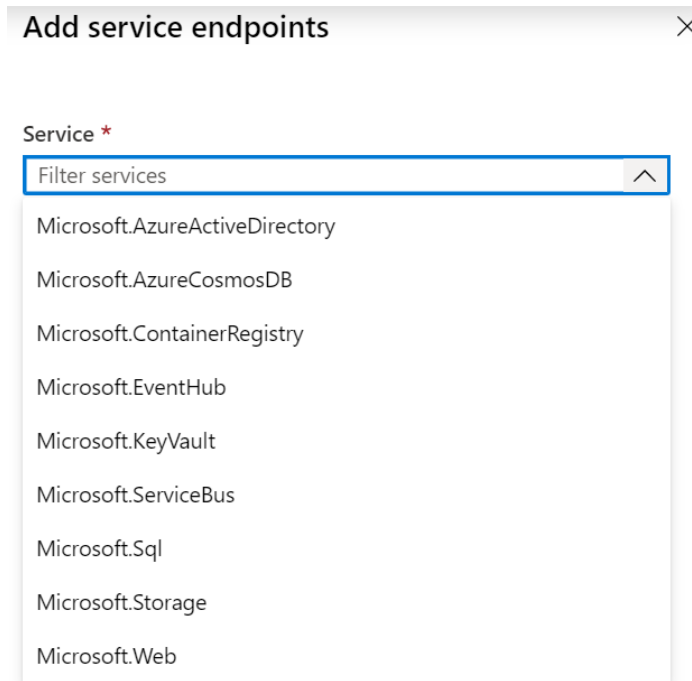


Figure 4.15 – Adding PaaS service endpoints

2. Go to the **subnet** configuration and select the services, as in the following screenshot:

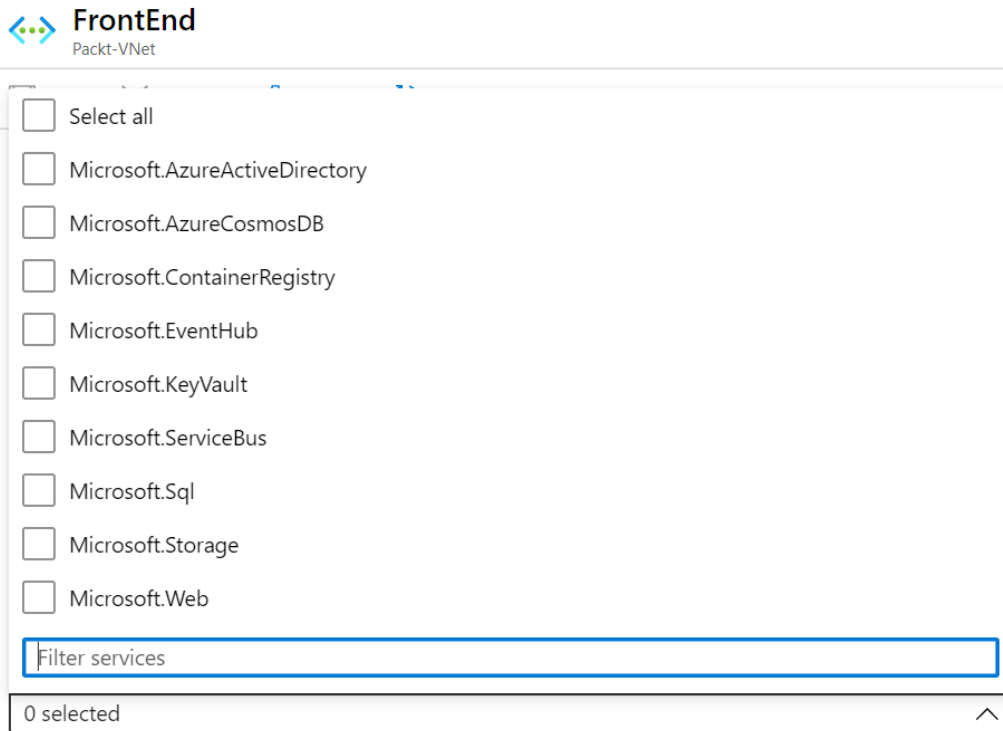


Figure 4.16 – Enabling service endpoints on a subnet

Enabling service endpoints on a VNet and subnet is only half the job. We need to enable settings on a PaaS service for the service endpoint to take effect. When enabling the service endpoint in the service settings, only subnets with enabled service endpoints will show up.

Service endpoints allow us to combine PaaS with IaaS. However, there are additional options when it comes to integrating PaaS with a private VNet using private endpoints. Let's take a look at this security enhancement as well.

Private endpoints

Private endpoints enable further integration between Azure PaaS services and VNet. Whereas service endpoints allow secure communication between PaaS and IaaS, private endpoints fully integrate PaaS to VNet. Service endpoints allow communication over the Microsoft backbone network, but PaaS services are still available over the internet. Private endpoint integrates service to VNet, after which a service is assigned a private IP address and all communications are done over a private network (VNet).

Using private endpoints, PaaS workloads can be accessed exclusively over a private network and never exposed to access over the internet. This provides an additional network security layer and mitigates the risk of publicly exposing services (even through a firewall). Services configured to use private endpoints can be accessed from the same VNet, a peered VNet, and on-premises using S2S or ExpressRoute, if other security rules (such as NSGs for example) so allow. Not all PaaS services support private endpoints, new services are added all the time, and it's recommended to check the Microsoft documentation to verify the currently supported list.

With private endpoints, we complete the network section that is available directly on Azure VNet settings. However, there are other things we need to consider when it comes to network security. Let's see what else is available to increase network security in Azure.

Considering other VNet security options

For additional security and traffic control, a **Network Virtual Appliance (NVA)** can be used. An NVA can be deployed from Azure Marketplace. Once deployed, you will realize that an NVA is, in fact, an Azure VM with a third-party firewall installed. Most industry leaders are present in Azure Marketplace and we can deploy firewall solutions that we are used to in an on-premises environment. It's important to mention that we don't have to decide between NSGs or NVAs; these can be combined for additional security.

Additional network security can be achieved with Azure Firewall as well. Azure Firewall is a firewall as a service. It allows better network control than an NSG and can be compared to an NVA solution in many aspects. But Azure Firewall also has a few advantages compared to an NVA, such as built-in high availability, the option to deploy to multiple availability zones, and cloud scalability. This means that no load balancers are needed. We can span Azure Firewall across multiple Availability Zones (and achieve an SLA of 99.99%), and scaling is configured to automatically accommodate any change in network traffic. Some options that are supported include application filtering, network traffic filtering, FQDN tags, service tags, outbound SNAT support, inbound DNAT support, and multiple public IP addresses. With these options, we can have complete control of network traffic in our VNet. It's important to mention that Azure Firewall is compliant with many security standards, including SOC 1 Type 2, SOC 2 Type 2, SOC 3, PCI DSS, and ISO 27001, 27018, 20000-1, 22301, 9001, and 27017.

Next, we will be looking at how to deploy and configure Azure Firewall with PowerShell.

Azure Firewall deployment and configuration

This example – to deploy and configure Azure Firewall – requires Azure PowerShell. However, Azure Firewall can be configured and deployed through the Azure portal as well.

Azure Firewall deployment

In order to deploy Azure Firewall, we need to set up the required network and infrastructure:

1. First, we need to create subnets, create a VNet, and associate the subnets with the VNet:

```
$FWsub = New-AzVirtualNetworkSubnetConfig -Name '
AzureFirewallSubnet -AddressPrefix 10.0.1.0/26
$Worksub = New-AzVirtualNetworkSubnetConfig `
-Name Workload-SN `
-AddressPrefix 10.0.2.0/24
$Jumpsub = New-AzVirtualNetworkSubnetConfig `
-Name Jump-SN `
-AddressPrefix 10.0.3.0/24
$testVnet = New-AzVirtualNetwork -Name Packt-VNet `
-ResourceGroupName Packt-Security `
-Location "westeurope" `
```

```
-AddressPrefix 10.0.0.0/16 `
-Subnet $FWsub, $Worksub, $Jumpsub
```

2. Next, we need to deploy Azure VM, which will be used as a jump box (the VM we connect to in order to perform admin tasks on other VMs in the network; we don't connect to other VMs directly, but only through a jump box):

```
New-AzVm -ResourceGroupName Packt-Security `
-Name "Srv-Jump" `
-Location "westeurope" `
-VirtualNetworkName Packt-VNet `
-SubnetName Jump-SN `
-OpenPorts 3389 `
-Size "Standard_DS2"
```

3. After the jump box, we create a test VM:

```
$NIC = New-AzNetworkInterface `
-Name Srv-work `
-ResourceGroupName Packt-Security `
-Location "westeurope" `
-Subnetid $testVnet.Subnets[1].Id
$VirtualMachine = New-AzVMConfig `
-VMName Srv-Work `
-VMSize "Standard_DS2"
$VirtualMachine = Set-AzVMOperatingSystem `
-VM $VirtualMachine `
-Windows -ComputerName Srv-Work `
-ProvisionVMAgent -EnableAutoUpdate
$VirtualMachine = Add-AzVMNetworkInterface `
-VM $VirtualMachine `
-Id $NIC.Id
$VirtualMachine = Set-AzVMSourceImage `
-VM $VirtualMachine `
-PublisherName 'MicrosoftWindowsServer' `
-Offer 'WindowsServer' `
-Skus '2016-Datacenter' `
```

```
-Version latest
New-AzVM -ResourceGroupName Packt-Security `
-Location "westeurope" `
-VM $VirtualMachine -Verbose
```

4. Finally, we deploy Azure Firewall:

```
$FWpip = New-AzPublicIpAddress `
-Name "fw-pip" `
-ResourceGroupName Packt-Security `
-Location "westeurope" `
-AllocationMethod Static `
-Sku Standard
$Azfw = New-AzFirewall -Name Test-FW01 `
-ResourceGroupName Packt-Security `
-Location "westeurope" `
-VirtualNetworkName Packt-VNet `
-PublicIpName fw-pip
$AzfwPrivateIP = ` $Azfw.IpConfigurations.
privateipaddress
```

Next, we will look at the Azure Firewall configuration.

The Azure Firewall configuration

After Azure Firewall is deployed, it doesn't actually do anything. We need to create a configuration and rules in order for Azure Firewall to be effective:

1. First, we will create a new route table with the BGP propagation disabled:

```
$routeTableDG = New-AzRouteTable `
-Name Firewall-rt-table `
-ResourceGroupName Packt-Security `
-location "westeurope" `
-DisableBgpRoutePropagation
Add-AzRouteConfig -Name "DG-Route" `
-RouteTable $routeTableDG `
-AddressPrefix 0.0.0.0/0 `
-NextHopType "VirtualAppliance" `
-NextHopIpAddress $AzfwPrivateIP `
```

```
| Set-AzRouteTable
Set-AzVirtualNetworkSubnetConfig `
-VirtualNetwork $testVnet `
-Name Workload-SN `
-AddressPrefix 10.0.2.0/24 `
-RouteTable $routeTableDG `
| Set-AzVirtualNetwork
```

2. Next, we create an application rule that allows outbound access to `www.google.com`:

```
$AppRule1 = New-AzFirewallApplicationRule `
-Name Allow-Google `
-SourceAddress 10.0.2.0/24 `
-Protocol http, https `
-TargetFqdn www.google.com
$AppRuleCollection = `
New-AzFirewallApplicationRuleCollection `
-Name App-Coll01 -Priority 200 `
-ActionType Allow -Rule $AppRule1
$Azfw.ApplicationRuleCollections = $AppRuleCollection
Set-AzFirewall -AzureFirewall $Azfw
```

3. We then create a rule to allow a DNS on port 53:

```
$NetRule1 = New-AzFirewallNetworkRule `
-Name "Allow-DNS" `
-Protocol UDP -SourceAddress 10.0.2.0/24 `
-DestinationAddress 209.244.0.3,209.244.0.4 `
-DestinationPort 53
$NetRuleCollection = `
New-AzFirewallNetworkRuleCollection `
-Name RCNet01 -Priority 200 `
-Rule $NetRule1 -ActionType "Allow"
$Azfw.NetworkRuleCollections = $NetRuleCollection
Set-AzFirewall -AzureFirewall $Azfw
```

4. And then we need to assign a DNS to an NIC:

```
$NIC.DnsSettings.DnsServers.Add("209.244.0.3")
$NIC.DnsSettings.DnsServers.Add("209.244.0.4")
$NIC | Set-AzNetworkInterface
```

Try connecting to a jump box, and then through a jump box to test the VM. From the test VM, try resolving multiple URLs. Only `www.google.com` should succeed, as all outbound traffic is denied except for the explicit allow rule we created.

It's important to remember that Azure Firewall offers Premium SKU, which provides additional features including TLS Inspection, IDPS, URL filtering, and web categories. TLS Inspection is used to analyze encrypted outbound data by decrypting outbound traffic, processing the data, and encrypting it again before forwarding it to its final destination. **Intrusion Detection and Prevention System (IDPS)** monitors all network activities, provides analyses, and detects potential malicious activities. URL filtering extends the standard capability of FQDN filtering (`www.packt.com`) to consider the entire URL (`https://www.packtpub.com/product/mastering-azure-security/9781839218996`). Web categories provide the option to allow or deny access to certain website categories, including gambling, social media, and other websites. Azure Firewall helps us to control and inspect traffic, but there are many other threats that can disrupt and endanger our network communication. Let's take a look at a solution that can help us mitigate DDoS attacks.

Azure DDoS protection

Distributed Denial of Service (DDoS) is one of the most common cyber attacks. A DDoS attack attempts to overload system resources and make a system unavailable to legitimate users. An attack can target any endpoint that is publicly reachable through the internet.

Azure DDoS protection comes in two different flavors: **Basic** and **Standard**.

Every property in Azure is protected by DDoS Basic protection at no additional cost. To protect customers and prevent impacts on other customers, Basic protection provides defense against network layer attacks with always-on traffic monitoring and real-time mitigation. It requires no additional configuration or any user action; it is a built-in service protecting all Azure services, both IaaS and PaaS.

The standard plan provides additional functionalities, including the following:

- Guaranteed availability
- Cost protection
- Custom mitigation policies
- Metrics and alerts
- Mitigation reports and flow logs
- DDoS rapid response support

Azure DDoS Standard protection is a tenant-wide service protecting up to 100 public IP addresses by default, with an additional charge for each public IP address over 100. There is no need to deploy an instance in each subscription; one instance can protect all endpoints across tenants in multiple subscriptions.

However, the Standard plan comes in a bundle of 100 IP addresses by default and should be used only when multiple endpoints require protection. For resource-specific attacks on the application layer, take a look at the Web Application Firewall with Application Gateway and Front Door (later in this chapter).

Azure Bastion

When running IaaS, exposing management ports such as RDP (port 3389) or SSH (port 22) is not a good idea. Bad actors are constantly scanning public networks in the search for exposed endpoints. If they detect such a port open, they will trigger a brute-force attack in the hope of gaining access to a service. This is usually mitigated by creating a jump box, a VM that enables us to securely connect to it before connecting to other VMs on the network.

Azure Bastion is a service that provides the ability to connect to our VMs using the browser and Azure portal. Similar to a jump box, it provides a secure way to connect to our virtual network. But unlike a jump box (which we need to maintain and update), Azure Bastion is a fully managed service. With Azure Bastion, we are able to securely access VMs over RDP/SSH from the Azure portal over TLS.

Hub-and-spoke network topology

For large and enterprise organizations, a hybrid cloud can become complex, and hard to manage and secure. With multiple VNets and hybrid cloud implementation, it can become difficult to monitor network traffic or even know the exact traffic flow. For complex network topologies, it is recommended to implement the hub-and-spoke model. In this model, we have a central point (hub) to which all on-premises connections and VNets (spokes) are connected. This way, traffic is easy to monitor, inspect, and manage.

There are two possible implementations for the hub-and-spoke topology in Azure.

Hub VNet

Hub VNet implementation has a single VNet and a central network where everything else is connected:

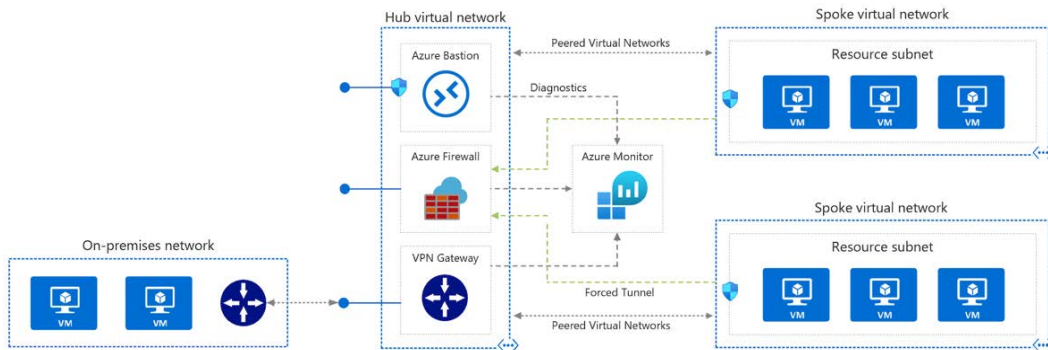


Figure 4.17 – Hub virtual network

All other networks (spokes) are connected to the hub VNet. On-premises networks are connected over VPN Gateway (or ExpressRoute) and VNets are connected with peering. A hub network can come with other network resources, such as Azure Firewall, Azure Bastion, and Azure DDoS Protection. All traffic needs to go through the hub network and it enables us to easily manage network traffic and monitor it in a central location.

Let's say we need to connect from an on-premises network to one of the VNets in Azure. The on-premises network is connected to the hub over VPN, and the VNet is connected to the hub over peering. If traffic needs to go from one network to another, it needs to go through the hub network. Using the hub network, we can define what types of traffic are allowed as well as monitor and inspect network packages.

A similar process can be applied when only VNets are in place. All VNets are only connected to hub networks over peering. If traffic needs to go from one network to another, it needs to go through the hub network.

Azure Virtual WAN is an alternative to the previous design, replacing the hub VNet with a managed service. All on-premises and VNets are still connected to the hub, but instead of managing hub networks ourselves, we have a managed service in place. Besides not managing hub networks, another benefit of this design is easier connectivity of networks across regions. Under Azure Virtual WAN, we can have multiple hubs in different regions for connecting VNets in the corresponding region. Communication between regions is done over a connection between hubs (over the Azure backbone network). All hubs are still managed in a central location, in Azure Virtual WAN.

Let's move on to networking in PaaS and see what else is available, besides securing PaaS with service endpoints. We can have better network control and prevent unwanted traffic even with publicly available endpoints.

Understanding Azure Application Gateway

The next Azure service that can help increase security is Application Gateway. Application Gateway is a web-traffic load balancer that enables traffic management for web applications. It operates as **layer 7 (L-7, or application layer)** load balancing. This means that it supports URL-based routing and can route requests based on the URI path or host header.

Application Gateway supports **Secure Socket Layer (SSL/TSL)** termination at the gateway. After the gateway, traffic flows unencrypted to backend servers, which are unburdened from encryption and decryption overheads. However, if this is not an option on account of security, compliance, or any other requirements, full end-to-end encryption is supported as well.

Application Gateway also supports scalability and zone redundancy. Scalability allows autoscaling depending on the traffic load, and zone redundancy allows the service to be deployed to multiple availability zones in order to provide better fault resiliency and remove the need to deploy the service to multiple zones manually.

Overall, Azure Application Gateway is an L-7 load balancer and we could question the security aspects of it (if we exclude SSL/TSL termination), as it's more a question of reliability and availability. However, Application Gateway has an amazing security feature called Azure **Web Application Firewall (WAF)**. WAF protects web applications against common exploits and vulnerabilities.

WAF is based on the **Open Web Application Security Project (OWASP)** and is updated to address the latest vulnerabilities. As it's PaaS, all updates are done automatically without any user configuration. From a policy perspective, we can create multiple custom policies and apply different sets of policies to different web applications.

WAF can operate in two modes – **detection** and **prevention**. In detection mode, WAF will detect all suspicious requests but will not stop them, only log them. It's important to mention that WAF can be integrated with different logging tools, so logs can be stored for auditing purposes. When in protection mode, WAF will also block any malicious requests, return a 403 `unauthorized access exception`, and close the connection. Prevention mode also logs all attacks.

Attacks are categorized by four severity levels:

- Critical (5)
- Error (4)
- Warning (3)
- Notice (2)

Each level has a severity value and the threshold for blocking is 5. So, a single critical issue is enough to block a session with the value 5, but at least two error issues are needed to block a session, as one error with a value of 4 is below the threshold.

WAF works as a filter before Application Gateway – it will process a request, decide whether it's valid, and, based on this decision, it will allow the request to proceed to Application Gateway or reject the request. Once the request is allowed by WAF, Application Gateway acts as a normal L-7 load balancer, as if WAF was turned off. You can see that in the following screenshot:

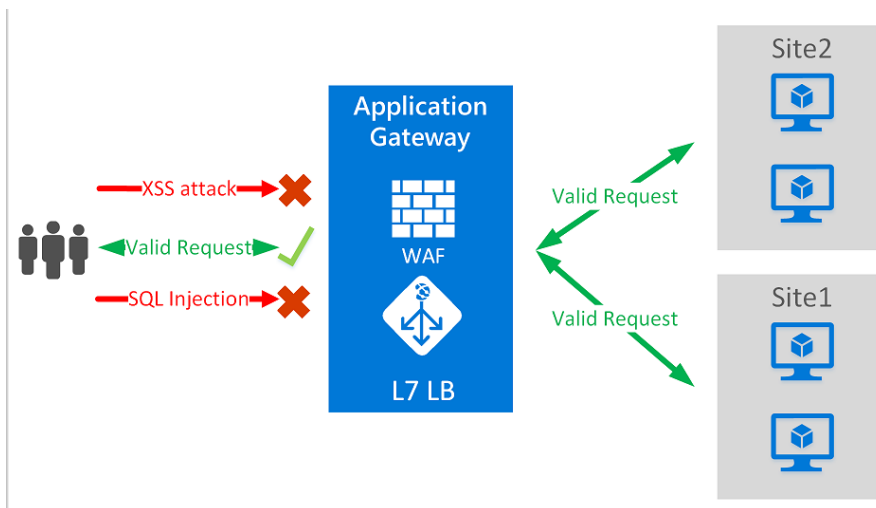


Figure 4.18 – Application Gateway traffic flow

Some of the attacks that can be detected and prevented with WAF are listed here:

- SQL injection
- Cross-site scripting
- Command injection
- Request smuggling
- Response splitting
- HTTP protocol violations and anomalies
- Protection against crawlers and scanners
- Geo-filter traffic

WAF on Application Gateway supports logging options to Azure Monitor, diagnostic logs to storage accounts, and integration with security tools such as Azure Security Center or Azure Sentinel.

Understanding Azure Front Door

Azure Front Door works very similarly to Application Gateway but on a different level. Like Application Gateway, it's an L-7 load balancer with an SSL offload. The difference is that Application Gateway works with services in a single region, whereas Azure Front Door allows us to define, manage, and monitor routing on a global level. With Azure Front Door, we can ensure the highest availability using global distribution. A similar thing can be achieved with Azure Traffic Manager (in terms of global distribution), but this service lacks L-7 load balancing and SSL offloading.

What Azure Front Door provides actually combines Application Gateway and Traffic Manager to enable an L-7 load balancer with global distribution. It's also important to mention that a WAF is also available on Azure Front Door. Using a WAF on Azure Front Door, we can provide web application protection for globally distributed applications.

Summary

In this chapter, we addressed network security, and prior to that, we saw how to manage cloud identities. We need to remember that network security doesn't stop with IaaS and VNets. Network security basics are usually associated with VNets and NSGs. But even with IaaS, it does not stop there, and we have options to extend with an NVA or Azure Firewall. With PaaS, we can leverage VNet's service endpoints, but extend security with services such as Application Gateway or Azure Front Door.

However, with all the preclusions limiting who, how, when, and from where we can access our resources, we still need to handle sensitive information and data. The next chapter will address how we can manage certificates, secrets, passwords, and connection strings using Azure Key Vault.

Questions

As we conclude, here is a list of questions for you to test your knowledge regarding this chapter's material. You will find the answers in the *Assessments* section of the *Appendix*:

1. We can control traffic in virtual networks with...
 - A. A network interface
 - B. A Network Security Group (NSG)
 - C. An Access Control List (ACL)

2. What type of connection is available with on-premises networks?
 - A. Point-to-Site
 - B. Site-to-Site
 - C. VNet-to-VNet
3. A connection between VNets can be made with...
 - A. VNet-to-VNet
 - B. VNet peering
 - C. Both of the above
 - D. None of the above
4. Which feature allows us to connect PaaS services to a VNet?
 - A. Service connection
 - B. Service endpoints
 - C. ExpressRoute
5. When multiple networks are involved...
 - A. We can define a route
 - B. Traffic is blocked by default
 - C. Traffic is allowed by default
 - D. A and B are correct
 - E. A and C are correct
6. What type of attack cannot be blocked with Application Gateway?
 - A. **SQL injection (SQLi)**
 - B. **Cross-Site Scripting (XSS)**
 - C. **Distributed Denial of Service (DDoS)**
 - D. HTTP protocol violations

5

Azure Key Vault

When talking about cloud computing, discussions are often directed to data protection, encryption, compliance, data loss (and data loss prevention), trust, and other buzzwords that center around the same group of topics. What they all have in common is the need for a trusted service that helps them to secure cloud data without giving a cloud vendor access to both your data and the corresponding encryption keys. Let's imagine that you want to create an Azure resource, such as a VM, that you will need admin credentials for. In this case, you don't want to hardcode usernames and passwords in your deployment script or template, do you? This is a scenario where Azure Key Vault comes into play. In this chapter, we will cover the following topics:

- Understanding Azure Key Vault
- Understanding service-to-service authentication
- Using Azure Key Vault in deployment scenarios

Understanding Azure Key Vault

Azure Key Vault is a secure, cloud-based storage solution for keys, secrets, and certificates. Tokens, passwords, certificates, API keys, and other secrets can be securely stored and access to them can be granularly controlled using Azure Key Vault. The service can also be used as a key management solution. Azure Key Vault makes it easy to create and control the encryption keys that are used to encrypt your data. Another usage scenario is **Secure Sockets Layer/Transport Layer Security (SSL/TLS)** certificate enrollment and management. You can use Azure Key Vault to address certificate life cycle management for both Azure and internally connected resources. Secrets and keys that are stored in Azure Key Vault can be protected either by software or **hardware security modules (HSMs)** that are FIPS 140-2 Level 2 validated.

As you have already learned, you can use Azure Key Vault to manage keys, secrets, and certificates. The following list explains what each of these artifacts can be used for:

- A cryptographic key is used for data encryption. Azure Key Vault represents keys as **JSON Web Key (JWK)** objects, which are declared as soft or hard keys. A hard key is processed in a **hardware security module (HSM)**, whereas a soft key is processed in the software by Azure Key Vault. A soft key is still encrypted at rest using a hard key, which is stored in an HSM. Clients can either request Azure Key Vault to generate a key or import an existing RSA or **elliptic curve (EC)** key. RSA and EC are the algorithms that are supported by Azure Key Vault.
- A secret is basically a string that is encrypted and stored in Azure Key Vault. A secret can be used to securely store passwords, storage account keys, and other highly valuable strings.
- A certificate in Azure Key Vault is an x509 certificate that is issued by a **public key infrastructure (PKI)**. You can either let Azure Key Vault request a certificate from a supported public **certification authority (CA)**, which today are *DigiCert* and *GlobalSign*, or you can create a **certificate signing request (CSR)** within Azure Key Vault and manually let this CSR be signed by any public CA of your choice.

Azure Key Vaults are offered in two different service tiers, further referred to as **SKUs**:

- The **standard** service tier offers encryption based on software-protected keys.
- The **Premium** service tier offers HSM-protected keys for encryption.

Besides vaults, the service also offers **managed HSMs** that provide single-tenant, zone-redundant, and highly resilient HSMs to store and manage cryptographic keys. Every managed HSM will only be used for one single customer, helping you to meet compliance and regulatory requirements.

In this chapter, you will learn how to work with key vault entities. But first, let's look at service-to-service authentication in Azure Key Vault, which is needed to enable other Azure services to leverage Azure Key Vault during deployment or resource-management operations.

Access to an Azure key vault is granted by RBAC. That said, you need to have an Azure AD account to get access to the service, which means that you can use all the protective options for interactive authentications that were discussed in *Chapter 3, Managing Cloud Identities*. Furthermore, access to items protected by Azure Key Vault can be restricted to only single aspects of Azure Key Vault. For example, an account could be granted access just to secrets, but not to keys or certificates, or you could grant an account just a subset of permissions, but for all entities stored in a key vault. This granular rights management, in addition to RBAC, which will only grant access to an Azure key vault (being an Azure resource), is implemented by access policies. Let's look at these policies in a little more detail in the next section.

Understanding access policies

With access policies, you can granularly define who will get what level of access rights to a single Azure Key Vault instance and its artifacts.

MasteringAzSec | Access policies

Key vault

Search (Cmd+/)

Save Discard Refresh

Overview
Activity log
Access control (IAM)
Tags
Diagnose and solve problems
Events
Settings
Keys
Secrets
Certificates
Access policies
Networking
Security
Properties
Locks
Monitoring
Alerts
Metrics
Diagnostic settings

Quickly protect your Key vault from accidental deletion by turning on soft-delete. Please enable soft-delete in 'Properties' page. Click here to learn more. →

You have enabled the network access control. Only allowed networks will have access to this key vault.

Enable Access to:

- ☒ Azure Virtual Machines for deployment
- ☒ Azure Resource Manager for template deployment
- ☒ Azure Disk Encryption for volume encryption

Permission model

☒ Vault access policy
☐ Azure role-based access control

+ Add Access Policy

Current Access Policies

Name	Email	Key Permissions	Secret Permissions	Certificate Permissions	Action
USER					
Tom Janetscheck		0 selected	7 selected	0 selected	Delete
Tom Janetscheck		9 selected	7 selected	15 selected	Delete

Figure 5.1 – Azure Key Vault access policies

As you can see in *Figure 5.1*, there are two different user accounts called Tom Janetscheck that have been granted several permissions to access keys, secrets, and certificates in the **Access policies** settings section of Azure Key Vault called **MasteringAzSec**. Besides that, you can enable access to keys and secrets for Azure VMs, ARM, and Azure Disk Encryption. These options are necessary if you want to grant Azure VMs in your tenant read access to secrets so that they can be retrieved during VM deployments or if you want to enable Azure Resource Manager to retrieve secrets so they can be used in a template deployment. The third option specifies whether Azure Disk Encryption—a service that encrypts Azure VMs' disks using BitLocker or dm-crypt, depending on the operating system used in the Azure VM—is allowed to retrieve secrets from Azure Key Vault and unwrap values from stored keys.

Understanding service-to-service authentication

As we mentioned before, access to an Azure key vault and its entities is usually granted on a per-user basis. So, to enable service-to-service authentication, you could create an Azure AD application with associated credentials and use this service principal to get an access token for your application. It's quite an easy process:

1. Navigate to **Azure Active Directory** | **App registrations** in the Azure portal and select **New registration** to start the wizard.

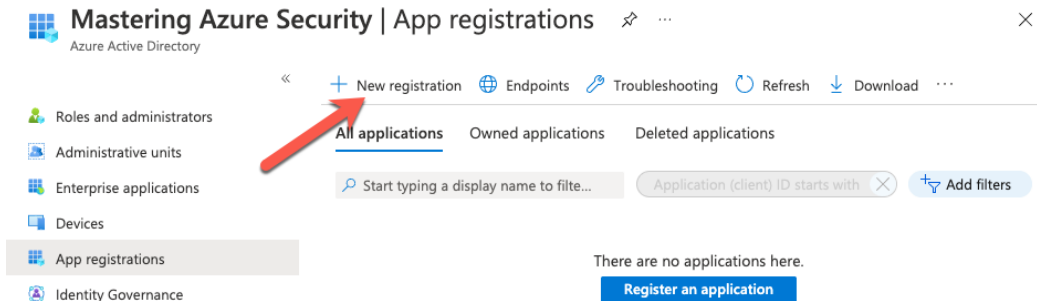


Figure 5.2 – Creating a new app registration

2. Enter a name and confirm your choice.

3. Create a client secret by navigating to the **Certificates & secrets** option in app registration and then select **New client secret**.

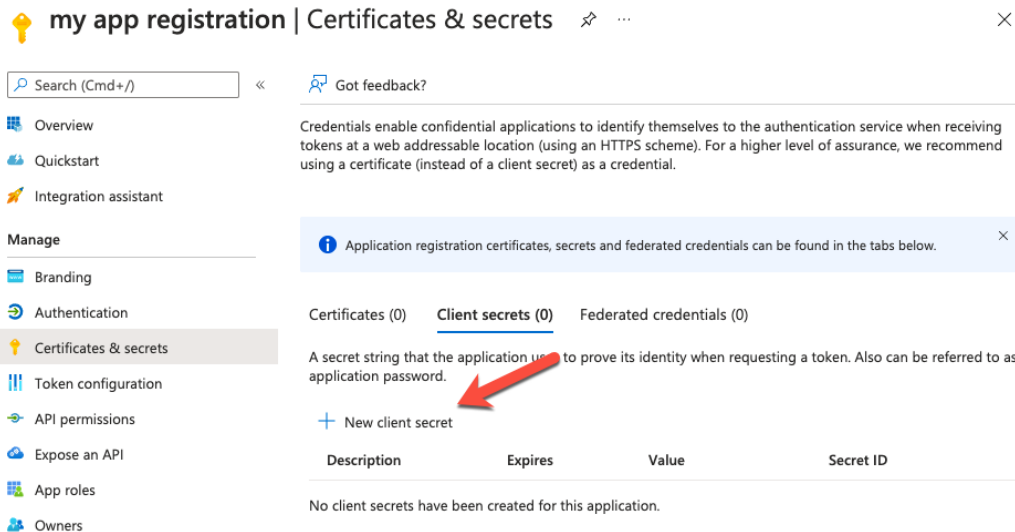


Figure 5.3 – Creating a new client secret

4. Enter a description and decide whether the secret will expire in 3, 6, 12, 18, or 24 months, or enter a custom period. After confirming your choices and leaving the wizard, you are presented with the new client secret and its value, which you can copy and then use for authentication.

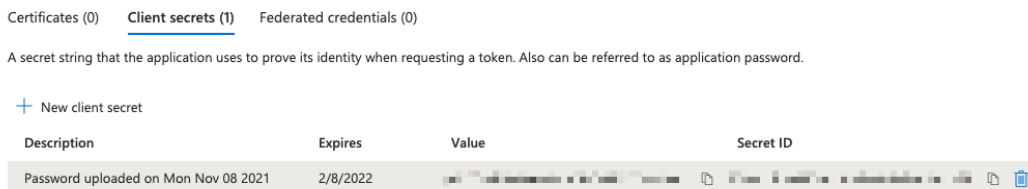


Figure 5.4 – Your client secret

An easier and quicker method is to use the Azure CLI. With the following command, you can simply create a new service principal with the name `MasteringAzSecSP`:

```
az ad sp create-for-rbac --name MasteringAzSecSP
```

The engine will use default settings for the account creation, and once the process is finished, you will find the username (**appId**) and the client secret (**password**) in the CLI window, as shown in the following screenshot:

```
tom@Toms-MacBook-Pro ~ % Az ad sp create-for-rbac --name MasteringAzSecSP
In a future release, this command will NOT create a 'Contributor' role assignment by default. If needed, use the --role argument to explicitly create a role assignment.
Creating 'Contributor' role assignment under scope '/subscriptions/12345678-9012-3456-7890-123456789012'
The output includes credentials that you must protect. Be sure that you do not include these credentials in your code or check the credentials into your source control. For more information, see https://aka.ms/azadsp-cli
'name' property in the output is deprecated and will be removed in the future. Use 'appId' instead.
{
  "appId": "12345678-9012-3456-7890-123456789012",
  "displayName": "MasteringAzSecSP",
  "name": "12345678-9012-3456-7890-123456789012",
  "password": "12345678-9012-3456-7890-123456789012",
  "tenant": "12345678-9012-3456-7890-123456789012"
}
```

Figure 5.5 – Using the Azure CLI to create a new service principal

The service principal behaves similarly to a user account in terms of access management, which means that you can use the username (the application or client ID) as a principal when granting access to the key vault and its entities.

While this approach works great, two caveats come with it:

- When creating the application credentials, you will get the app ID and the client secret, which are usually hardcoded in your source code. It's a dilemma because you cannot store these credentials in Azure Key Vault as they are needed to authenticate before being granted access to the key vault.
- Application credentials expire and the renewal process may cause application downtime. You don't want to use a client secret that will never expire and that is hardcoded in your source code.

So, for automated deployments, we need another approach, which is where the **Managed identities for Azure resources** service comes into play. Let's move one step further and learn how this service can address the dilemma.

Understanding managed identities for Azure resources

Needing credentials to get access to services is a common problem that you will often encounter. Azure Key Vault is an important part of your application design because you can use it to securely store and manage credentials for other services. But Azure Key Vault itself is a service that requires authentication before you are granted access. With managed identities for Azure resources, a free feature of Azure Active Directory, you can solve this dilemma. The service provides other Azure services with an automatically managed identity in Azure AD.

There are two different types of managed identities within the service:

- A *system-assigned managed identity* is directly enabled on an instance of an Azure service. When the managed identity is enabled, Azure AD automatically creates an identity for the particular service in Azure AD that is automatically trusted by the Azure subscription that the service instance is created in. Credentials are automatically provided to the service instance after the identity is created. The identity's life cycle is directly tied to the service's life cycle, which means that a system-assigned managed identity is automatically removed from Azure AD when the service is deleted.
- A *user-assigned managed identity* is a manually created Azure resource. When creating a user-assigned managed identity, Azure AD will create a service principal in the Azure AD tenant that is trusted by the Azure subscription you are currently using. After creating the identity, you can assign it in one or several Azure service instances. The user-assigned managed identity's life cycle is organized separately from the services' life cycles that the identity is assigned to. In other words, when an Azure resource with a user-assigned managed identity is deleted, the managed identity is not automatically removed from Azure AD.

The relationship between a system-assigned managed identity and an Azure resource is *1:1*, which means that an Azure resource can only have one system-assigned managed identity and this identity is only usable by the particular service it was created for.

The relationship between the user-assigned managed identity and the Azure resource is *n:n*, which means that you can use several user-assigned managed identities with one Azure resource at the same time and that a single user-assigned managed identity can be used by several different Azure resources.

Important Note

Microsoft provides a list of Azure services that currently support system-assigned, user-assigned, or both types of managed identities at <https://docs.microsoft.com/azure/active-directory/managed-identities-azure-resources/services-support-managed-identities>.

The creation process of a system-assigned managed identity in the Azure portal is very easy. All Azure resources that currently support managed identities have an **Identity** option in the **Settings** section of the resource.

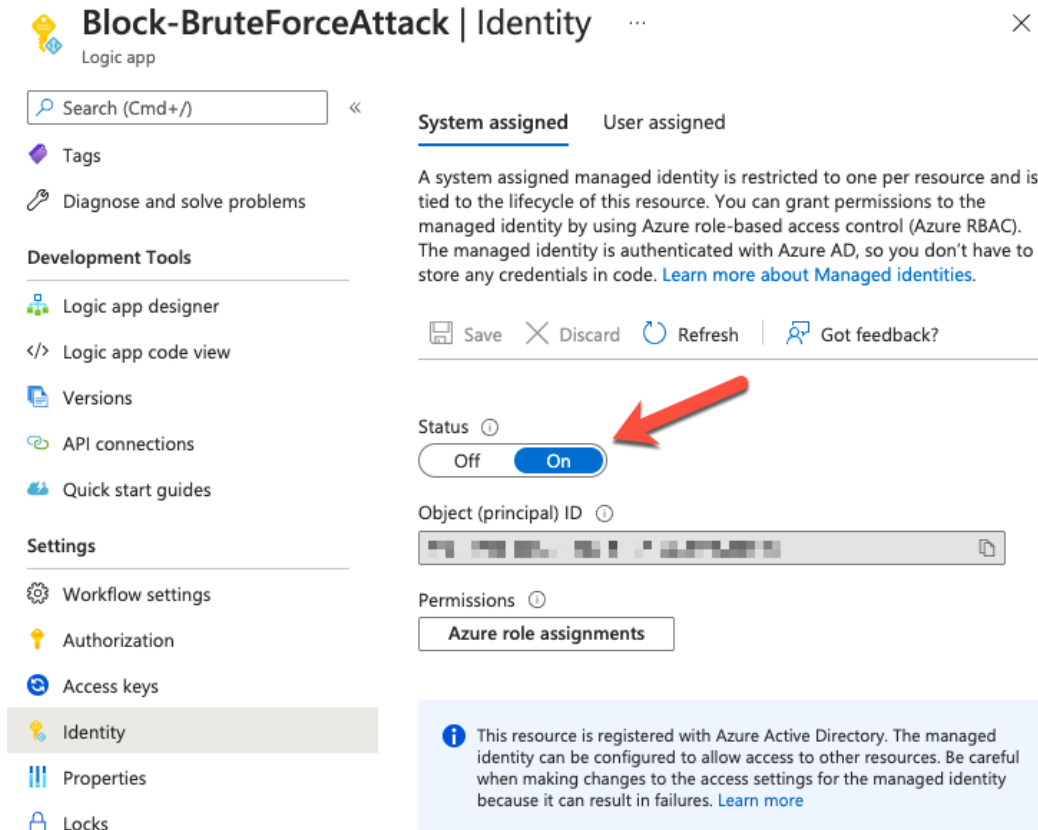


Figure 5.6 – Activating a system-assigned managed identity for a Logic app

The following steps explain how to activate a system-assigned managed identity for a Logic app:

1. On the Logic app's **Identity** page, you can choose whether you want to activate a system-assigned managed identity or whether you want to assign a user-assigned managed identity. To activate a system-assigned managed identity, you just have to set the **Status** toggle switch to **On** and then save your selection, as shown in *Figure 5.6*.

2. You will then be informed that once you confirm this configuration, a managed identity for your resource will be registered in Azure AD and that once the process has finished, you can grant permissions to that particular managed ID. In the preceding example, I have enabled a system-assigned managed identity for a Logic app called **Block-BruteForceAttack**.
3. When creating a new Key Vault access policy, you can now select the identity with the same name to grant access to the Key Vault's entities.

If you want to assign a *user-assigned managed identity* to your resource, you first have to navigate to the managed identity service and create a new one in the Azure portal:

1. In the portal's top search bar, enter **Managed Identities** and select the service.
2. Click on **Create**.
3. As mentioned before, a user-assigned managed identity is a new Azure resource and therefore needs to be created in an Azure subscription and stored in an Azure resource group, as shown in *Figure 5.7*:

Create User Assigned Managed Identity ...

Basics

Tags

Review + create

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * ⓘ

Contoso

Resource group * ⓘ

masteringazuresecurity

Create new

Instance details

Region * ⓘ

West Europe

Name * ⓘ

myManagedID

Review + create

< Previous

Next : Tags >

Figure 5.7 – Creating a user-assigned managed identity

4. Click on **Review + create**, and after the final validation is passed, click **Create**. Once the user-assigned managed identity is created, you can assign it to your Azure resource, like the Logic app's system-assigned managed identity we used in the preceding scenario.

Managed identities can also be used for Azure DevOps or Terraform authentication against your Azure environment.

Note

You can use a managed identity for Terraform authentication against Azure AD; however, in this case, the managed identity is created for an Azure VM and Terraform needs to be started from within the VM so that it can make use of the ID.

Let's assume that you only want to allow Azure resource creation via a DevOps pipeline with all related processes, such as pull requests and authoring. From a technical point of view, Azure DevOps is nothing but an application that needs to be granted access to an Azure subscription (or management group). Therefore, Azure DevOps needs a service principal that is either manually managed as an application registration with all its downsides, or that can be automatically managed using a managed identity. The same applies to Terraform, which is also just an application that needs rights to be granted in an Azure environment.

Note

You can use the Azure CLI, PowerShell, ARM templates, and Terraform to create managed identities in Azure. You can find examples of these methods in the GitHub repository that has been created for this book at <https://github.com/PacktPublishing/Mastering-Azure-Security-Second-Edition>.

Now that you know how managed identities work and what options you have for service-to-service authentication, let's move one step forward and see how you can use Azure Key Vault in your deployment scenarios.

Using Azure Key Vault in deployment scenarios

Azure Key Vault is a nice service when it comes to securely storing and retrieving credentials that are needed during resource creation. It also helps you to encrypt Azure resources, such as Azure storage accounts or VM disks, with your own encryption key. In this section, we will cover several options for how to use Azure Key Vault in deployment scenarios. You will find examples for PowerShell, ARM templates, and Terraform, as these are the most common deployment tools when it comes to creating Azure resources.

Important Note

The first step you will always have to go through is to authenticate with Azure AD using a principal that has been assigned the appropriate set of access rights in the Azure environment that you want to deploy resources to, depending on the task you want to perform and the resource that is affected by it.

Are you ready? Then let's start by creating a new Azure key vault and a secret that can later be used in a VM deployment scenario in the following section.

Creating an Azure Key Vault and secret

As with all Azure resources, you can use the Azure portal to create and manage an Azure key vault. Although it might be convenient to click through the portal, it is a better idea to use a scripting or template language for this. Azure Key Vault is a critical resource when it comes to automated deployments. Today, there is no way to granularly grant access to single items of the same type within the same key vault. You can manage levels of access to keys, secrets, and certificates, but only on a key-vault level, not on an item level. This is why you might want to create several key vaults in the same Azure subscription. Using deployment automation, you can make sure that all key vaults in your environment adhere to the rules and policies you have defined.

Key Vault creation in PowerShell

With PowerShell being an imperative scripting language, you need to define all the steps that are necessary in the correct order:

1. The first thing you need to do is to log in with an account that has the appropriate set of access rights to create a new resource group and Azure key vault instance in your Azure subscription:

```
# Login to your Azure subscription
Connect-AzAccount
```

2. You will then be prompted to enter your Azure login credentials, which are used by PowerShell to go through the next steps. After logging in, you either create a new resource group or refer to an existing one.
3. We will now assume that a new resource group will be created using the following code snippet. Before we do so, it makes sense to define all values for variables that are then used in the following script sections:

```
# Define variable values
$rgName = "myResourceGroup"
$azRegion= "WestEurope"
$kvName = "myAzKeyVault"
$secretName = "localAdmin"
$localAdminUsername = "myLocalAdmin"
# Create a new resource group in your Azure subscription
$resourceGroup = New-AzResourceGroup `
    -Name $rgName `
    -Location $azRegion
```

4. Now, you can move on and create a new Azure key vault:

```
# Create a new Azure Key Vault
New-AzKeyVault `
    -VaultName $kvName `
    -ResourceGroupName $rgName `
    -Location $azRegion `
    -EnabledForDeployment `
    -EnabledForTemplateDeployment `
    -EnabledForDiskEncryption `
    -Sku standard
```

In the preceding script, there are some mandatory and optional parameters listed. It is mandatory to use the following parameters when creating a new Azure key vault:

- `VaultName` defines the name of the Azure key vault. We are using the `$kvName` variable, which is defined at the beginning of the script.
- `ResourceGroupName` defines the Azure resource group that the key vault will be created in.
- `Location` defines the Azure region where the key vault will be created.

Some optional parameters are defined in the preceding section:

- `EnabledForDeployment` enables the `Microsoft.Compute` resource provider to retrieve secrets from the Azure key vault during resource creation—for example, when deploying a new VM.
- `EnabledForTemplateDeployment` enables the **Azure Resource Manager (ARM)** to get secrets for an Azure key vault when it is referenced in a template deployment, such as when you are using ARM or Terraform.
- `EnabledForDiskEncryption` enables the **Azure Disk Encryption** service to get secrets and unwrap keys from an Azure key vault to use them in the disk encryption process.
- `SKU` defines the Azure key vault's SKU (standard or premium), with standard being the default.

5. After the Azure key vault has been created, you need to create an access policy. In the following example, we grant access rights to secrets in the new Azure key vault for the currently logged-in user account:

```
# Grant your user account access rights to Azure Key
Vault secrets
Set-AzKeyVaultAccessPolicy '
    -VaultName $kvName '
    -ResourceGroupName $rgName '
    -UserPrincipalName (Get-AzContext).account.id '
    -PermissionsToSecrets get, set
```

6. You can then create a new key vault secret. In the following snippet, you enter the secret as a secure string in the PowerShell session:

```
# Create a new Azure Key Vault secret
$password = read-host -assecurestring
Set-AzKeyVaultSecret '
```

```
-VaultName $kvName '  
-Name $secretName '  
-SecretValue $password
```

Congratulations! You have just created your first Azure key vault and a secret using PowerShell. Now, that you know how to create an Azure key vault and a key vault secret for your deployment scenario, we can move on to the next section, *Azure VM deployment*, in which you will learn how to use the resources that you have just created in a more complex scenario.

Azure VM deployment

When deploying an Azure VM, you always need to pass local admin credentials to it during the deployment process. The downside of deploying VMs using the Azure portal is that you need to manually enter the respective local admin credentials instead of using a secret that is stored in an Azure key vault. This is only one of the reasons why infrastructure-as-code deployments definitely make sense in an enterprise environment. In this section, you will learn how to reference credentials that are stored in an Azure key vault instead of hardcoding the information in the deployment script or template.

We will start by referencing a key vault secret for VM deployments using PowerShell.

VM deployments with PowerShell

You can easily access secrets in an Azure key vault with PowerShell, but also with ARM templates and Terraform. Let's see how we can do this by performing the following steps:

1. After you have retrieved a secret, you need to create a new `PSCredential` object that can be used in the VM deployment, as follows:

```
# retrieve an Azure Key Vault secret  
$secret = Get-AzKeyVaultSecret '  
-VaultName $kvName '  
-Name $secretName  
  
# Create a new PSCredential object  
$cred = [PSCredential]::new($localAdminUsername,$secret.  
SecretValue)
```

2. Later, you can use this `PSCredential` object in your deployment in the respective position. This would look similar to the following code snippet:

```
$myVM = Set-AzVMOperatingSystem '
    -VM $myVM '
    -Windows '
    -ComputerName $vmName '
    -Credential $cred '
[...]
```

It is always a good idea to work with variables in a PowerShell script. By doing so, you can have a variable section at the beginning of the script where you can define values that change depending on your needs and the environments that the script is used in.

Note

Since the complete VM deployment script in PowerShell consists of almost 200 lines, we have not printed it in the book, but have published it in the book's GitHub repository.

PowerShell is a good way to deploy Azure resources, but being an imperative scripting language, it is not the best fit for usage in DevOps/CI/CD scenarios. This is why we will explain how to reference a key vault secret in Terraform in the next section.

Referencing a key vault secret in Terraform

In Terraform, you can refer to an existing Azure object with data sources. For a key vault secret, the data source is called `azurerm_key_vault_secret`:

```
# Azure Key Vault data source to access local admin password
data "azurerm_key_vault_secret" "mySecret" {
    name = "secretName"

    key_vault_id = "/subscriptions/GUID/resourceGroups/
RGName/
providers/Microsoft.KeyVault/vaults/VaultName"
}
```

This object can then be referenced in the `os_profile` section of a Terraform deployment template, as shown in the following code snippet:

```
os_profile {  
  computer_name = "myVM"  
  admin_username = "myLocalAdminUserName"  
  admin_password = "$(data.azurerm_key_vault_secret.  
    mySecret.value) "  
}
```

Terraform is quite an easy way of deploying and referencing Azure resources. As you can see from the preceding examples, you simply need to define a data source and then reference it in the respective resource section of your deployment template.

Tip

We have published a complete example for a VM deployment with Terraform in this book's GitHub repository: <https://github.com/PacktPublishing/Mastering-Azure-Security-Second-Edition>.

ARM templates are Microsoft's way of using automatic Azure resource deployments in DevOps pipelines. This example is described in detail in the following section.

Referencing a key vault secret in ARM templates

ARM templates might be the most complex way to refer to key vault secrets during template deployment. This is because you need to use linked templates in this scenario. That said, you need to have two different template files that are used for different purposes.

The main template is used as a reference to existing Azure resources, such as the Azure key vault and its secrets. In it, there is a parameters section that contains values that are either defined directly in the main template or passed to the template by an external call that is an Azure CLI or a PowerShell call and then passed directly to the linked template.

If the parameters section is filled by pipeline input, it will only contain the parameters' definitions:

```
"parameters": {  
  "vaultName": {  
    "type": "string"  
  },  
}
```

```

    "vaultResourceGroup": {
      "type": "string"
    },
    "secretName": {
      "type": "string"
    }
  }
}

```

If the parameters' values are defined within the main template, then this section will look like this:

```

"parameters": {
  "vaultName": {
    "type": "string",
    "defaultValue": "<default-value-of-parameter>"
  },
  "vaultResourceGroup": {
    "type": "string",
    "defaultValue": "<default-value-of-parameter>"
  },
  "secretName": {
    "type": "string",
    "defaultValue": "<default-value-of-parameter>"
  }
}

```

Behind the parameters section, there is a resource section in which the key vault reference is defined:

```

"parameters": {
  "adminPassword": {
    "reference": {
      "keyVault": {
        "id": "[resourceId(subscription().subscriptionId,
parameters('VaultResourceGroup'), 'Microsoft.KeyVault/vaults',
parameters('vaultName'))]"
      },
      "secretName": "[parameters('secretName')]"
    }
  }
}

```

```

    }
  }
}

```

The *linked template* is used for the actual resource deployment. In this file, the local admin username is defined, but the password value is passed from the main template, as follows:

```

"resources": {
  "parameters": {
    "adminUsername": {
      "type": "string",
      "defaultValue": "localAdminUsername",
      "metadata": {
        "description": ""
      }
    },
    "adminPassword": {
      "type": "securestring"
    }
  }
}

```

Using ARM templates to refer to a key vault secret is a bit more complex, but also a great fit for DevOps pipelines.

Tip

We have published a complete example for a VM deployment with Terraform in this book's GitHub repository: <https://github.com/PacktPublishing/Mastering-Azure-Security-Second-Edition>.

You have now learned how to use Azure key vaults and key vault secrets during automated Azure resource deployments with PowerShell, Terraform, and ARM templates. Please make sure that you take a look at this book's GitHub repository, as you will find examples of the steps that we have outlined in this chapter for your reference.

Summary

Azure Key Vault is one of the many services that are underrated but very valuable when it comes to security in Azure. In this chapter, you have learned how to create Azure key vaults and their entities, not only with the Azure portal, but also with scripting and deployment languages. You now know how to grant access to an Azure key vault for both individual users and Azure resources and how to reference items that have been securely stored in a key vault.

In the next chapter, we will address data security and encryption, two topics that are heavily dependent on Azure Key Vault, so make sure that you have read and understood this chapter before moving on.

Questions

As we conclude, here is a list of questions for you to test your knowledge regarding this chapter's material. You will find the answers in the *Assessments* section of the *Appendix*:

1. Azure Key Vault is used to secure____
 - A. Keys
 - B. Secrets
 - C. Certificates
 - D. None of the above
 - E. All of the above
2. How do we control who can access Azure Key Vault information?
 - A. Key Vault permissions
 - B. Access policies
 - C. Conditional Access
3. Service-to-service authentication is done via____
 - A. Service principals
 - B. Certificates
 - C. Direct link

4. To use Azure Key Vault in a VM deployment, which option do we need to enable?
 - A. EnabledForDeployment
 - B. EnabledForTemplateDeployment
 - C. EnabledForDiskEncryption
5. To use Azure Key Vault for **Azure Resource Manager (ARM)** deployment, which option do we need to enable?
 - A. EnabledForDeployment
 - B. EnabledForTemplateDeployment
 - C. EnabledForDiskEncryption
6. To use Azure Key Vault for VM encryption, which option do we need to enable?
 - A. EnabledForDeployment
 - B. EnabledForTemplateDeployment
 - C. EnabledForDiskEncryption
7. To secure secrets during deployment, we need to____
 - A. Provide a password
 - B. Encrypt a password
 - C. Reference an Azure key vault

6

Data Security

Like everything else, data in the cloud must be treated differently from data in on-premises environments. As data is leaving our local environment and is usually accessible over the internet, we need to be extra careful. We have already mentioned that all data is encrypted at rest, and most communication goes over **Hypertext Transfer Protocol over Secure Socket Layer (HTTPS)** and is encrypted on the move as well. However, there are multiple steps that we can take to ensure additional security and satisfy compliance and different security requirements.

In this chapter, we will be using Azure Key Vault extensively. We've seen how Azure Key Vault can be used for secrets and password management, but we will also see how it can be used to increase data security as well.

We will cover the following topics in this chapter:

- Understanding Azure Storage
- Understanding Azure virtual machine disks
- Working on Azure SQL Database

Technical requirements

For this chapter, the following is required:

- PowerShell 5.1, or higher, on Windows (or PowerShell Core 6.x and later on any other platform, including Windows)
- Azure PowerShell modules
- Visual Studio Code
- An Azure subscription

Understanding Azure Storage

Azure Storage is Microsoft's cloud storage solution and is highly available, secure, and scalable, plus, it supports a variety of programming languages. It is usually the first data solution users encounter when they start using Azure and it contains several services, including Disk, Blob, Table, File, and Queue.

All communication inside Azure data centers is over HTTPS, but what happens when we access data from outside? As per the shared responsibility model (explained in *Chapter 1, An Introduction to Azure Security*), this falls under the user's responsibility. For newly deployed storage accounts, traffic over HTTPS is enabled by default. If required, it can be disabled, but obviously, it's not recommended. Keep in mind that HTTPS is the default option only for storage accounts created after August 2021 and previously created accounts should be revisited.

Under **Configuration**, there is an option called **Secure transfer required**. To ensure that all traffic is encrypted and over HTTPS, we need to enable this option, as in the following screenshot:

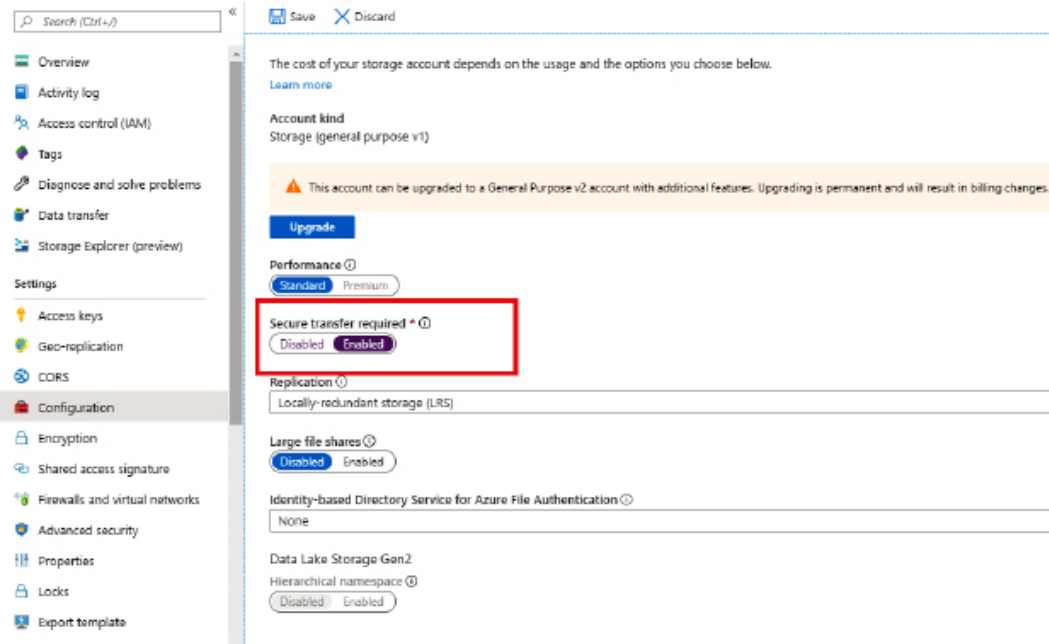


Figure 6.1 – Azure Storage secure transfer

When **Secure transfer required** is enabled, all requests that are not over HTTPS will be rejected, even if they have valid access parameters (such as an access signature or a token). This enhances transfer security by allowing the request to Azure Storage only when coming by a secure connection.

Important Note

It's important to remember that Azure Storage does not support HTTPS for custom domain names, so this option does not apply when a custom domain name is used. It is possible to use HTTPS and custom domains when combined with Azure CDN.

Another thing we need to consider is what happens when files are accidentally deleted. This can occur for a number of reasons, such as a user deleting a file by mistake, numerous applications using the same Azure Storage, and one application deleting a file that is required by another application. Of course, such situations can be caused by malicious attacks and with the intent to cause damage as well. To avoid such situations, we can enable the **Blob soft delete** option under the **Data Protection** setting.

An example is shown in the following screenshot:

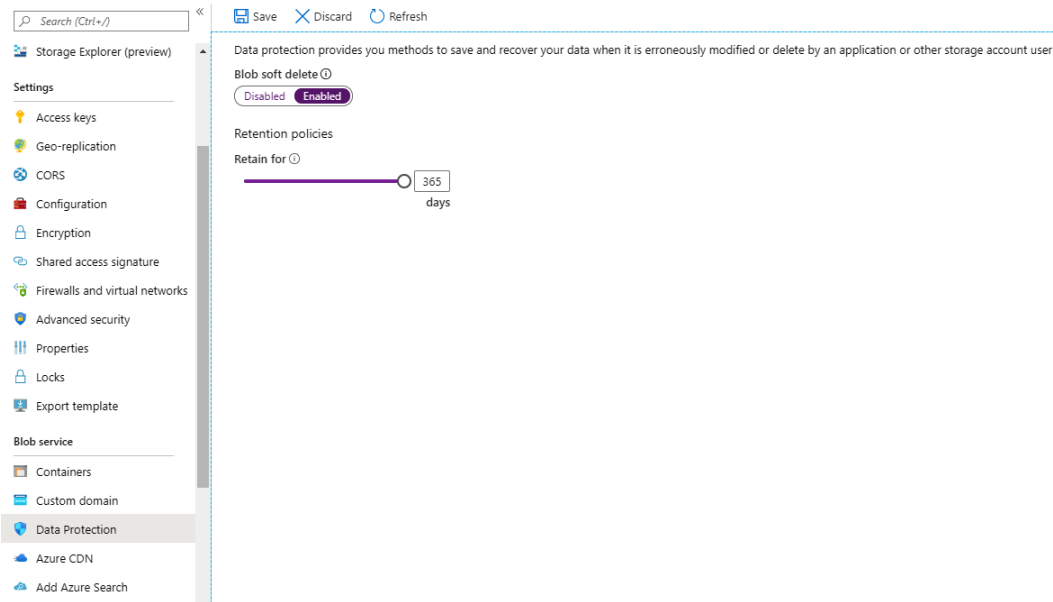


Figure 6.2 – Blob soft delete

Blob soft delete also has retention period settings, which will determine how long files will be saved after they are deleted. These settings can be adjusted to any value between 1 and 365 days (7 days by default). Besides recovering deleted files, **Blob soft delete** enables us to recover previous versions of files, too. If a file is accidentally modified, or we need a previous version of a file for some other reason, **Blob soft delete** can help us to restore the file to any point in time, as long as it's within the timeframe of the effective retention period.

Important Note

Blob soft delete and the retention period are effective from the moment they are set and cannot be used retroactively. For example, if we discover that we lost a file and then enable this option, it will not help us recover the lost file.

To be able to recover files, we need to have the option in place before the incident happens.

Data in Azure is encrypted at rest, and that's no different with Azure Storage. But data is encrypted with Microsoft-managed keys, and this is not always acceptable. In certain situations, caused by compliance and security requirements, we must be in control of keys that are used for encryption. To address such needs, Microsoft has enabled encryption of data with the **Bring Your Own Key (BYOK)** option. The option to use your own key can be enabled under the **Encryption** option, as in the following screenshot:

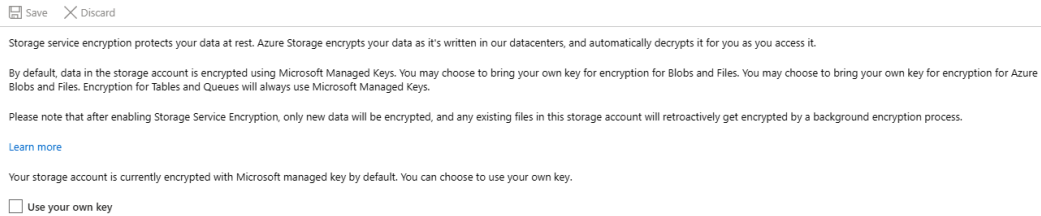


Figure 6.3 – Azure Storage encryption options

BYOK is enabled by the use of Azure Key Vault, where keys used for storage encryption are stored.

Once we enable **Use your own key**, new settings will appear where we need to provide key vault information. An example is shown in the following screenshot:



Figure 6.4 – Storage encryption with a custom key

We can either provide a key **Uniform Resource Identifier (URI)**, which will contain information about the key vault and key that will be used for encryption, or we can use a second option, which will allow us to select the available key vault and key from a list. Both options will have identical outcomes and the storage will be encrypted. Any new files added to Azure Storage will be encrypted automatically. Any existing files will be retroactively encrypted by a background encryption process, but it will not happen instantly, and the process to encrypt existing data will take time.

Keys in Azure Key Vault can be easily created, but we can also import existing keys from a **Hardware Security Module (HSM)**. This is for further security and compliance requirements and enables ultimate key management where crypto keys are safeguarded by a dedicated crypto processor.

Encryption in Azure Storage is not only available through the Azure portal, but also through using Azure PowerShell. The following script will create a new Azure Storage account, key vault, and key, and then use all the resources provided to encrypt the Azure Storage account that was just created:

```
New-AzResourceGroup -Name 'Packt-Encrypt' `
-Location 'EastUS'

New-AzStorageAccount -ResourceGroupName 'Packt-Encrypt' `
-Name 'packtstorageencryption' `
-Location 'EastUS' `
-SkuName Standard_GRS

$storageAccount = Set-AzStorageAccount `
-ResourceGroupName 'Packt-Encrypt' `
-Name 'packtstorageencryption' `
-AssignIdentity

New-AzKeyvault -name 'Pact-KV-01' `
-ResourceGroupName 'Packt-Encrypt' `
-Location 'EastUS' `
-EnabledForDiskEncryption `
-SoftDeleteRetentionInDays 7 `
-EnablePurgeProtection

$KeyVault = Get-AzKeyVault -VaultName 'Pact-KV-01' `
```

```

-ResourceGroupName 'Packt-Encrypt'

Set-AzKeyVaultAccessPolicy `
  -VaultName $keyVault.VaultName `
  -ObjectId $storageAccount.Identity.PrincipalId `
  -PermissionsToKeys wrapkey,unwrapkey,get,recover

$key = Add-AzKeyVaultKey `
  -VaultName $keyVault.VaultName `
  -Name 'MyKey' `
  -Destination 'Software'

Set-AzStorageAccount `
  -ResourceGroupName $storageAccount.ResourceGroupName `
  -AccountName $storageAccount.StorageAccountName `
  -KeyvaultEncryption `
  -KeyName $key.Name `
  -KeyVersion $key.Version `
  -KeyVaultUri $keyVault.VaultUri

```

Another thing we need to consider with Azure Storage is **Microsoft Defender for Storage**. This option is enabled under **Security** and it takes our security one step further. It uses security intelligence to detect any threat to our data and provides recommendations in order to increase security.

An example of the **Microsoft Defender for Storage** blade under the storage account is shown in the following screenshot:

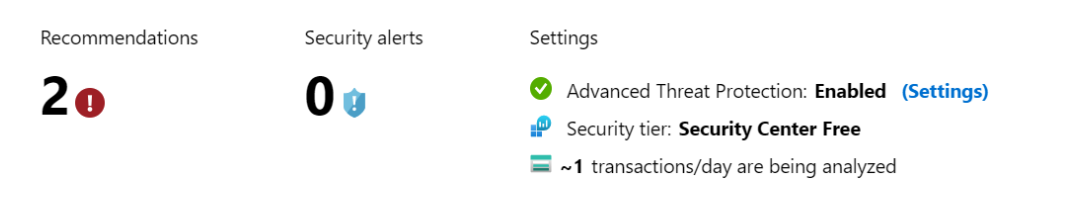


Figure 6.5 – Microsoft Defender for Storage

ATP compares our security settings to the recommended baseline and provides us with additional security options that can be implemented. The second part is to detect unusual and potentially harmful attempts to access or exploit Azure Storage. Microsoft Defender for Storage is closely connected to Microsoft Defender for Cloud, which will be covered in *Chapter 7, Microsoft Defender for Cloud*. However, the storage account is not the only Azure service related to storage. Almost every Azure service uses some kind of storage. Among these services, we have Azure **virtual machine** (VM) disks, so let's see how we can make these more secure.

Understanding Azure virtual machine disks

Azure VMs are a part of Microsoft's **Infrastructure as a Service (IaaS)** offering and another service that a lot of users encounter early in the cloud journey. Azure VMs are usually selected when the user requires more control over the environment than other services can offer. But with more control also comes more responsibility.

Besides network management, which was covered in *Chapter 4, Azure Network Security*, we need to address how we handle data in Azure VMs, and by data, we mainly mean disks. Just like all other data, disks for Azure VM are encrypted at rest, and VM uses Azure Fabric Controller to securely access content stored on these disks.

But what happens if we decide to download or export a disk used by these machines? Once a disk leaves Azure, it is in an unencrypted state and can be used in any way. This opens certain vulnerabilities that need to be addressed. What if someone gains access to Azure (but does not have access to a VM) and downloads a disk? What if someone decides to back up all disks, but to an unsecure location? These are just a couple of the scenarios that can create serious problems with unauthorized and malicious access to Azure VM disks.

Fortunately, we have the option to use Azure Key Vault and enable the further encryption of disks. Disks encrypted in this way will stay encrypted even after they are exported, downloaded, or leave an Azure data center in any way.

If we go to Azure VMs and take a look at our disks, we can see that, by default, disk encryption is not enabled. An example is shown in the following screenshot:

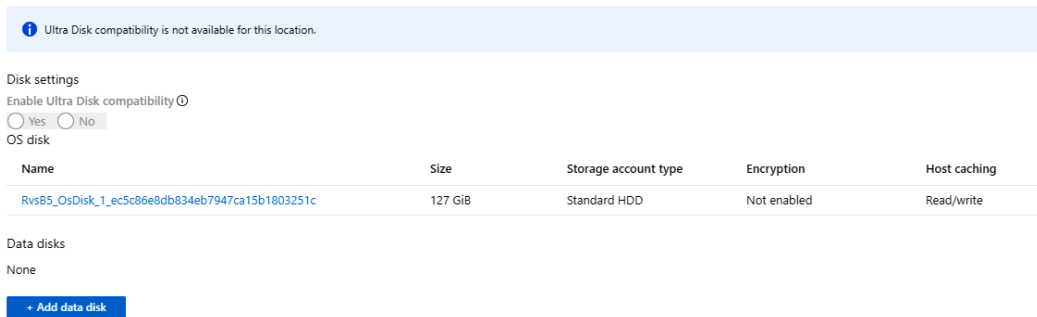


Figure 6.6 – Azure VM disk options to encrypt disks

We can use Azure PowerShell to enable disk encryption for Azure VM disks. A sample script is provided here:

```
New-AzResourceGroup -Name "Packt-Encrypt" -Location "EastUS"

$cred = Get-Credential

New-AzVM -Name 'Packt-VM-01' `
  -Credential $cred `
  -ResourceGroupName 'Packt-Encrypt' `
  -Image win2016datacenter `
  -Size Standard_D2S_V3

New-AzKeyvault -name 'Pact-KV-01' `
  -ResourceGroupName 'Packt-Encrypt' `
  -Location EastUS `
  -EnabledForDiskEncryption `
  -EnableSoftDelete `
  -EnablePurgeProtection

$KeyVault = Get-AzKeyVault -VaultName 'Pact-KV-01' `
  -ResourceGroupName 'Packt-Encrypt'
```

```
Set-AzVMDiskEncryptionExtension `
  -ResourceGroupName 'Packt-Encrypt' `
  -VMName 'Packt-VM-01' `
  -DiskEncryptionKeyVaultUrl $KeyVault.VaultUri `
  -DiskEncryptionKeyVaultId $KeyVault.ResourceId

Get-AzVmDiskEncryptionStatus -VMName Packt-VM-01 `
  -ResourceGroupName Packt-Encrypt
```

This script creates a new Azure VM, a new key vault, and a new key. The created resources are then used to enable disk encryption for the just-created Azure VM. Finally, we can check the status of the disk and verify whether it was successfully encrypted. We can verify that the disk is encrypted in the Azure portal, as shown in the following screenshot:

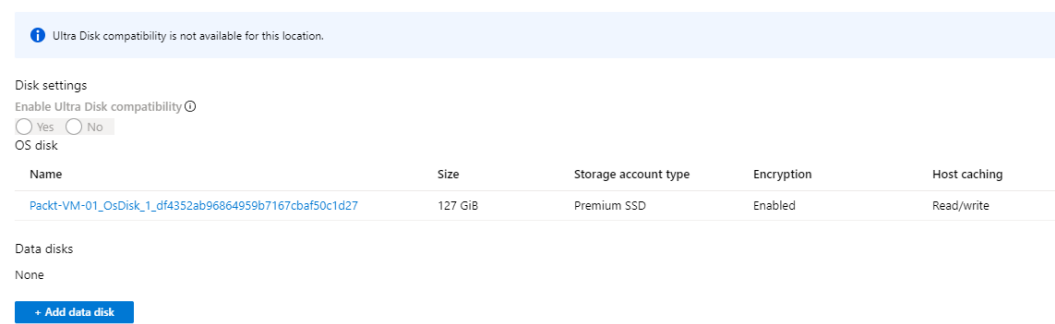


Figure 6.7 – Azure VM portal

The Azure VM disk is not encrypted: it can be accessed, used to create a new VM (Azure or a local Hyper-V), or attached to an existing VM. Encrypting the disk will prevent such access unless the user has access to the key vault that was used during the encryption process.

There are other possibilities for encrypting disks at an operating system level, for example, BitLocker (for Windows Server) or DMCCrypt (for Linux). However, these types of encryption are not Azure-related as they can be applied in on-premises environments (both physical and virtual) or any cloud.

Working on Azure SQL Database

Azure SQL Database is Microsoft's relational database cloud offering, which falls under the platform as a service model, often referred to as **Database as a Service (DBaaS)**. It's highly available and provides a high-performance data storage layer. Because of everything mentioned above, it's often the first choice when it comes to selecting a database in the cloud.

There is a whole section of security-related features in Azure SQL Database's settings. There are four different options:

- **MICROSOFT DEFENDER FOR SQL**
- **AUDITING**
- **DYNAMIC DATA MASKING**
- **TRANSPARENT DATA ENCRYPTION**

If we enable **ADVANCED DATA SECURITY**, we get a few advantages:

- **VULNERABILITY ASSESSMENT SETTINGS**
- **ADVANCED THREAT PROTECTION SETTINGS**
- **DATA DISCOVERY & CLASSIFICATION**

The options to enable **ADVANCED DATA SECURITY** are shown in the following screenshot:

ADVANCED DATA SECURITY

ON

OFF



Advanced Data Security costs 12.6495 EUR/server/month. It includes Data Discovery & Classification, Vulnerability Assessment and Advanced Threat Protection. We invite you to a trial period for the first 30 days, without charge.

VULNERABILITY ASSESSMENT SETTINGS

Subscription

Microsoft Azure Sponsorship

>

Storage account

sqlvanlsmjb32qfuio

>

Periodic recurring scans ⓘ

ON

OFF

Send scan reports to ⓘ

Email addresses

✓

☒ Also send email notification to admins and subscription owners ⓘ

ADVANCED THREAT PROTECTION SETTINGS

Send alerts to ⓘ

Email addresses

☒ Also send email notification to admins and subscription owners ⓘ

Advanced Threat Protection types

All

>

ⓘ [Enable Auditing for better threats investigation experience](#)

Figure 6.8 – Microsoft Defender for SQL settings

Each of these options has additional configurations, which we need to set before using **ADVANCED DATA SECURITY**:

1. **VULNERABILITY ASSESSMENT SETTINGS** requires Azure Storage, which is where data will be kept and will provide security recommendations based on the current status. The assessment will be performed manually, and we can select whether the report will be sent to specific users or to all admins and subscription owners. The assessment report has the following format:

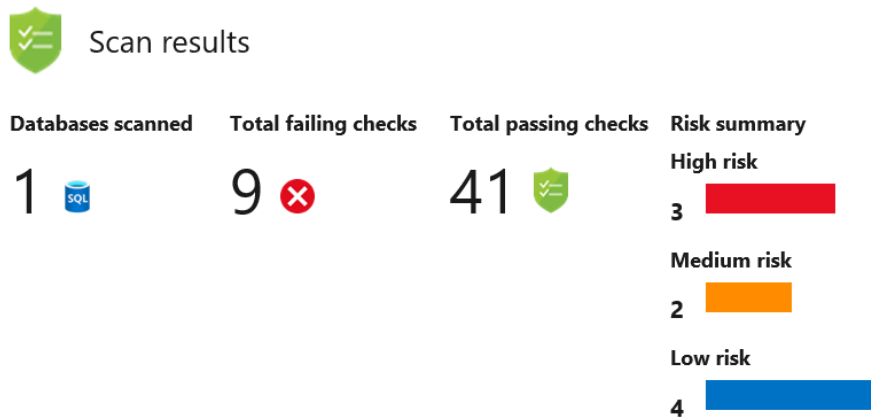


Figure 6.9 – Azure SQL Database assessment results

The vulnerability assessment compares our current settings to the baseline and provides a list of checks that are not satisfied. It also provides a classification of checks based on risk level: **High risk**, **Medium risk**, and **Low risk**.

2. **ADVANCED THREAT PROTECTION SETTINGS** is very similar to this option in Azure Storage: it detects anything unusual and potentially harmful. We have a couple of additional settings compared to Azure Storage: we can select what types of threats we want to detect and define who gets notifications if a threat is detected.

3. A list of threats we can monitor is shown in the following diagram:

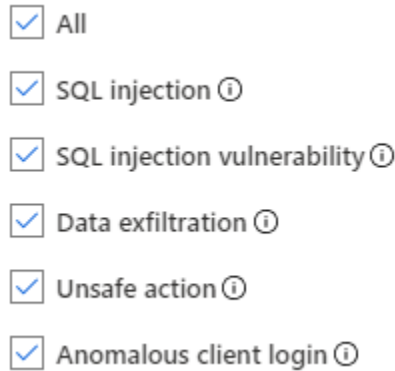


Figure 6.10 – Azure SQL Database advanced threat protection settings

4. Similar to **VULNERABILITY ASSESSMENT SETTINGS**, notifications about threats can be sent to specific users or all admins and subscription owners.
5. The final option is **DATA DISCOVERY & CLASSIFICATION**. This option carries out data assessment and provides a classification for any data that should be confidential. Assessments are carried out based on various compliance and security requirements, and data is classified as confidential based on one or more criteria. For example, we may have some user information that should be classified based on the **General Data Protection Regulation (GDPR)**. But among user information, we may have social security or credit card numbers that are classified by many other compliance requirements. **DATA DISCOVERY & CLASSIFICATION** only provides information about classified data types, but it's up to the user to decide what to proceed with and how to do so.

This brings us to the next security setting under Azure SQL Database – **Dynamic data masking**. In the past, we could provide the user with access to different layers, such as database, schema, table, or row. However, we could not provide access based on the column. Dynamic data masking changes this and enables us to provide user access to a table but mask certain columns that the user should not be able to access.

For example, let's say that we want to provide access to a customer table, but the user should not be able to access the customer's email address. We can create masking on the email column, and the user will be able to see all the information except the masked column, which will be displayed as masked (there are default masks for common data types, but custom masks can be created). For masked columns, we can exclude certain users. All administrators are excluded by default and cannot be excluded. An example of dynamic data masking is shown in the following screenshot:

Masking rules

Mask name	Mask Function
SalesLT_Customer_EmailAddress	Email (aXXX@XXXX.com)

SQL users excluded from masking (administrators are always excluded) ⓘ

✓

Recommended fields to mask

Schema	Table	Column	
bpst_aal	ServiceHealthData	serviceHealthId	Add mask
SalesLT	Customer	FirstName	Add mask
SalesLT	Customer	LastName	Add mask
SalesLT	Customer	Phone	Add mask
SalesLT	Customer	PasswordHash	Add mask

Figure 6.11 – Dynamic data masking

Dynamic data masking is connected to **DATA DISCOVERY & CLASSIFICATION**. The user will receive data masking recommendations based on information provided by **DATA DISCOVERY & CLASSIFICATION**.

The **Auditing** option enables us to define where various logs will be stored. There are three different options available:

- **Storage**
- **Log Analytics**
- **Event Hubs**

As many regulatory and security compliance standards require access and event logs to be stored, this option enables us to not only keep logs for auditing purposes but also to help us to understand and trace events whenever needed. Auditing can be enabled for a single database or on a server level. In the case that server auditing is enabled, logs will be kept for all databases located on that server.

The last security setting under Azure SQL Database is **Transparent Database Encryption (TDE)**. TDE is a method of encrypting data at rest and performs real-time encryption and decryption of data and log files at the page level. It uses a **Database Encryption Key (DEK)** to encrypt data, associated backups, and transactional logs.

For older databases, this setting is not enabled, the TDE status is **OFF**, and **Encryption status** is **Unencrypted**. An example of this is shown in the following screenshot:

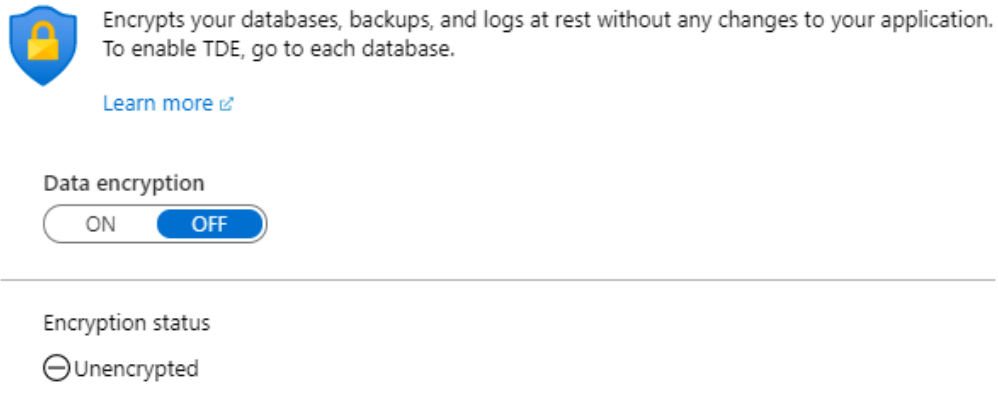


Figure 6.12 – Azure SQL Database TDE settings

For newer databases, TDE is already on, and the encryption status is encrypted, as shown in the following screenshot:

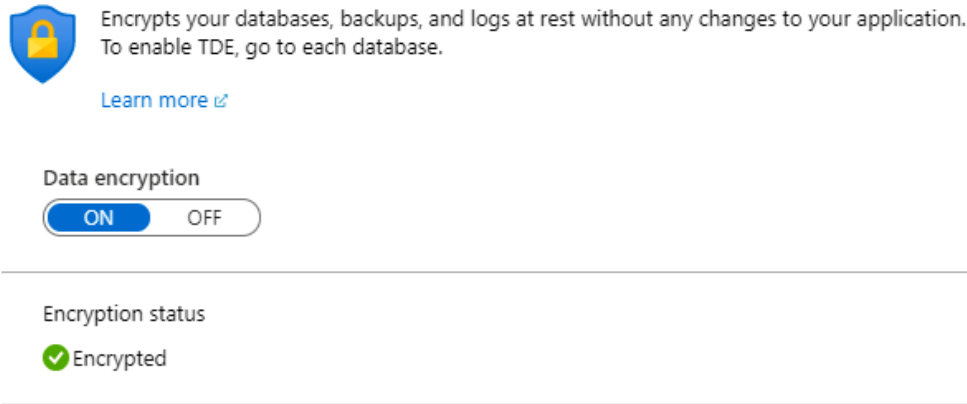


Figure 6.13 – Azure SQL Database encrypted with TDE

We will get the same effect by switching TDE on for older databases. However, this method uses Microsoft-provided keys for encryption. If we need to follow regulatory and security compliance standards and manage keys ourselves, we must turn to Azure **Key Vault** once again. To enable TDE with our own key from Azure **Key Vault**, we must enable this setting at the server level. Under TDE settings, under **Server**, we can turn **Use your own key** on, as in the following screenshot:

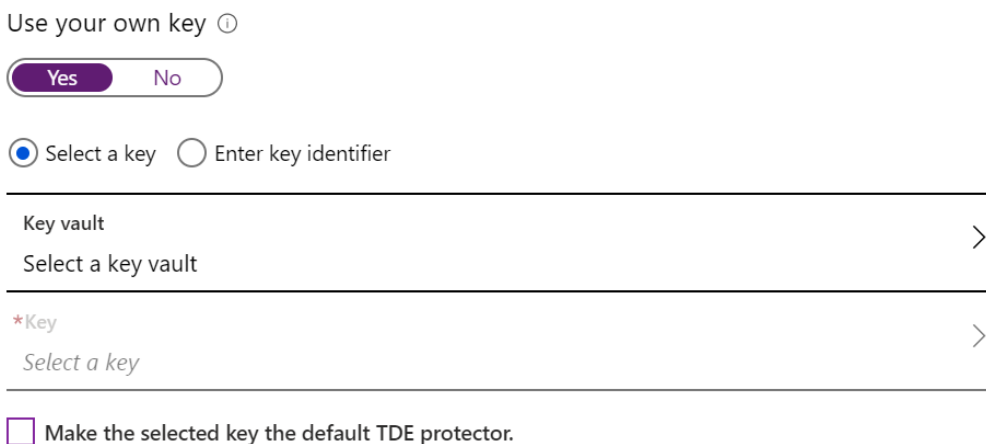


Figure 6.14 – Using a custom key for TDE

Once we turn this option on, we need to provide the key vault and key that will be used. Optionally, we can set the selected key to be the default TDE protector.

Enabling the option to use the key from the key vault for TDE will not enable TDE on every database located on the selected server. TDE must be enabled for each database separately. If this option is not set on the server, enabling TDE on the database will use Microsoft-managed keys. If the option is enabled on the server level, enabling TDE on the database will use the key from the key vault that is defined.

Encryption can be set using Azure PowerShell, as in the following sample script:

1. We need to define parameters for execution. These will be used across the entire script, basically in each and every command we execute:

```
$RGName = 'Packt-Encrypt'
$servername = 'packtSQL'
$DBName = 'test'
```

2. Next, we need to log in to our Azure subscription:

```
$cred = Get-Credential
```

3. We will create a resource group, Azure SQL server, and Azure SQL database:

```
$RG = New-AzResourceGroup -Name $RGName `
-Location 'EastUS'

$server = New-AzSqlServer `
-ResourceGroupName $RG.ResourceGroupName `
-Location 'EastUS' `
-ServerName $servername `
-ServerVersion "12.0" `
-SqlAdministratorCredentials $cred `
-AssignIdentity

$server = Set-AzSqlServer `
-ResourceGroupName $RG.ResourceGroupName `
-ServerName $servername `
-AssignIdentity
```

```
$database = New-AzSqlDatabase `
-ResourceGroupName $RG.ResourceGroupName `
-ServerName $server.ServerName `
-DatabaseName $DBName `
-RequestedServiceObjectiveName "S0" `
-SampleName "AdventureWorksLT"
```

4. Next, we need to create the Azure key vault, which will be used in the data encryption process:

```
New-AzKeyvault -name 'Pact-KV-01' `
-ResourceGroupName $RG.ResourceGroupName `
-Location 'EastUS' `
-EnabledForDiskEncryption `
-SoftDeleteRetentionInDays 7 `
-EnablePurgeProtection

$KeyVault = Get-AzKeyVault -VaultName 'Pact-KV-01' `
-ResourceGroupName $RG.ResourceGroupName
```

5. We now need to set the key vault access policy to enable encryption and add a key that will be used to encrypt the data:

```
Set-AzKeyVaultAccessPolicy `
-VaultName $keyVault.VaultName `
-ObjectId $server.Identity.PrincipalId `
-PermissionsToKeys wrapkey,unwrapkey,get,recover

$key = Add-AzKeyVaultKey `
-VaultName $keyVault.VaultName `
-Name 'MyKey' `
-Destination 'Software'
```

6. Finally, we assign a key to Azure SQL Server, enable TDE using a custom key from Key Vault, and encrypt the database using that key:

```
Add-AzSqlServerKeyVaultKey `
-ResourceGroupName $RG.ResourceGroupName `
-ServerName $server.ServerName `
```

```
-KeyId $key.Id

Set-AzSqlServerTransparentDataEncryptionProtector `
-ResourceGroupName $RG.ResourceGroupName `
-ServerName $server.ServerName `
-Type AzureKeyVault `
-KeyId $key.Id

Get-AzSqlServerTransparentDataEncryptionProtector `
-ResourceGroupName $RG.ResourceGroupName `
-ServerName $server.ServerName

Set-AzSqlDatabaseTransparentDataEncryption `
-ResourceGroupName $RG.ResourceGroupName `
-ServerName $server.ServerName `
-DatabaseName $database.DatabaseName `
-State "Enabled"
```

If the database has TDE enabled, with Microsoft-managed keys, enabling our own key from the key vault on a server level will not automatically transfer the encryption from the managed key to our own key. We must perform decryption and enable encryption again to use the key from the key vault.

Azure aims to offer a consistent experience and most services offer similar options. When it comes to databases, we took Azure SQL as an example, but similar options can be found in other databases in Azure, such as Azure SQL Database for MySQL, Azure SQL Databases for Postgres, or Cosmos DB.

Summary

With all this work on the network and identity levels, we still need to do more and encrypt our data in the cloud. But is that enough? Often it is not, and then we find ourselves needing to do even more. Threats are becoming more and more sophisticated, and we need to find ways of increasing our security. In this chapter, we briefly mentioned Azure Security Center. It may be the one thing that can give us an advantage in creating a secure cloud environment and stopping threats and attacks before they even happen.

In the next chapter, we will discuss how Azure Security Center uses security intelligence to be a central security safeguard in Azure.

Questions

As we conclude, here is a list of questions for you to test your knowledge regarding this chapter's material. You will find the answers in the *Assessments* section of the *Appendix*:

1. What will the **Secure transfer required** option do?
 - A. Enforce data encryption
 - B. Enforce HTTPS
 - C. Enforce FTPS
2. To protect data from accidental deletion, what needs to be enabled?
 - A. Data protection
 - B. Recycle Bin
 - C. Blob soft delete
3. Is data in Azure Storage encrypted by default?
 - A. Yes
 - B. No
4. Are Azure VM disks encrypted by default?
 - A. Yes
 - B. No
5. Is Azure SQL Database encrypted by default?
 - A. Yes
 - B. No
6. Data in Azure can be encrypted with...
 - A. Microsoft-provided keys
 - B. User-provided keys
 - C. Both of the above
7. Access to sensitive data in Azure SQL Database can be restricted using...
 - A. Data classification
 - B. Dynamic data masking
 - C. **Transparent Database Encryption (TDE)**

Section 3: Security Management

This section deals with monitoring security in Microsoft Azure, applying recommendations, and detecting threats before they happen.

This part of the book comprises the following chapters:

- *Chapter 7, Microsoft Defender for Cloud*
- *Chapter 8, Microsoft Sentinel*
- *Chapter 9, Security Best Practices*

7

Microsoft Defender for Cloud

In *Chapter 2, Governance and Security*, you have learned that *monitoring* is essential for an organization to know what happens in their cloud environments. With Microsoft Defender for Cloud, this aspect is taken to the next level as it is meant to be the tool to give organizations a unified view into their hybrid and multi-cloud security configuration as well as inform them about current threats that are occurring in their environments.

In this chapter, you will learn about the following topics:

- Introducing Microsoft Defender for Cloud
- Cloud Security Posture Management with Defender for Cloud
- Custom policies and (regulatory) compliance
- Cloud workload protection and multi-cloud capabilities
- Automating security

Let's get started!

Introducing Microsoft Defender for Cloud

With cloud computing being the main paradigm in the modern IT world, many benefits are associated with this new way of working. IT is no longer an end in itself and employees are way more productive than they were back in the day. But there are also new challenges when it comes to protecting modern IT environments.

In *Chapter 3, Managing Cloud Identities*, we covered advanced identity protection and that it is no longer enough to protect network boundaries; however, some other key security challenges come with cloud computing. How can you make sure you protect your ever-changing cloud services and applications? This is one of the value propositions of cloud computing and, in fact, probably the main benefit is that you can easily change and adapt in cloud environments. Be it **continuous integration/continuous delivery (CI/CD)**, **Virtual Machine (VM)** upscaling, or service decommissioning, cloud environments are dynamic. But, at the same time, one of the main challenges is to keep track of these changes and to make sure that a company's services always adhere to its security baseline.

The threat landscape is evolving, and attacks are becoming increasingly sophisticated. Bad actors are using attack automation and evasion techniques, and at the same time, they are leveraging tools that help them to conduct attacks across the cyber kill chain. So, they no longer need to be highly trained technology experts, which results in an increasing number of sophisticated attacks, some of which are spearphishing and credential theft attacks. Also, attackers are using hijacked computers, tied to bot networks, to conduct widely spread password spray attacks, which can be hard to recognize.

We need human expertise, creativity, and adaptability to combat human threat actors. The downside is that security skills are in short supply. Currently (in 2021), there are 3.5 million open positions in the cybersecurity sector out there worldwide, with a prediction that this number will stay the same for the next 5 years. This includes not only cyber threat hunters but also security engineers and administrators with a focus on managing internal IT systems.

Tip

To learn more about the number of open positions in the cybersecurity sector, visit <https://cybersecurityventures.com/jobs/>.

Microsoft Defender for Cloud is a service that helps organizations fill the gap by helping them focus on two main pillars of protecting cloud environments:

1. As a **Cloud Security Posture Management (CSPM)** solution, Microsoft Defender for Cloud constantly provides information about the current configuration status of all cloud resources in an organization to avoid misconfiguration with regard to security. Defender for Cloud's CSPM capabilities include secure score, recommendations, auto-remediation, and more.
2. As a **Cloud Workload Protection Platform (CWPP)**, Microsoft Defender for Cloud provides protection against cyber threats aimed at a company's infrastructure, no matter whether it is running in Microsoft Azure, on-premises, or in another cloud platform.

Microsoft Defender for Cloud's main dashboard provides a unified view of the most important aspects of security coverage, as shown in *Figure 7.1*:

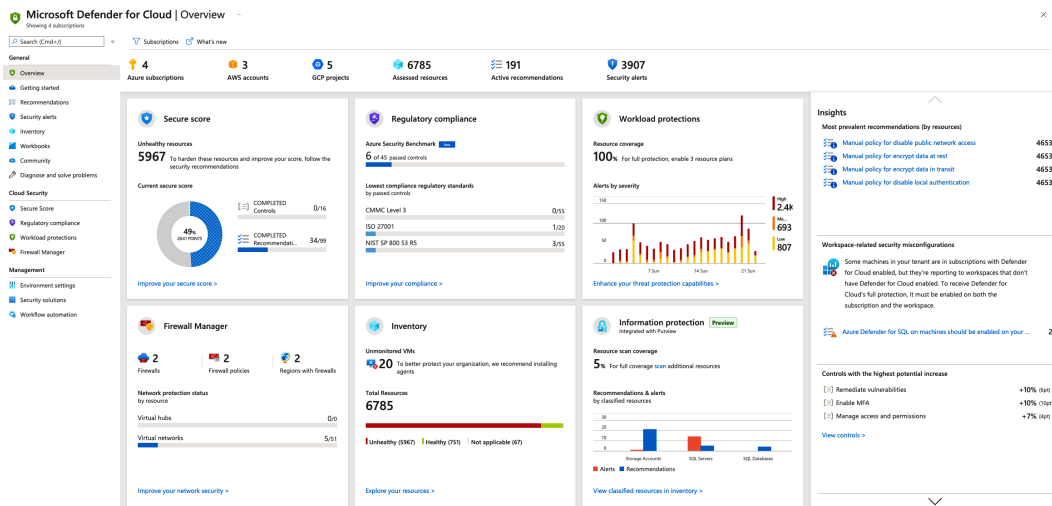


Figure 7.1 – Microsoft Defender for Cloud overview dashboard

On top of the screen, Defender for Cloud provides a high-level overview of Azure subscriptions, AWS accounts, and GCP projects that are connected. It also shows the number of assessed resources, active recommendations, and active security alerts that have been raised during the last 30 days:



Figure 7.2 – High-level resource overview

The tiles in the section underneath provide a more detailed representation of several aspects, including overviews of the secure score and regulatory compliance, but also an alert graph, an inventory view, and information about the integration with Azure Firewall Manager and Azure Purview.

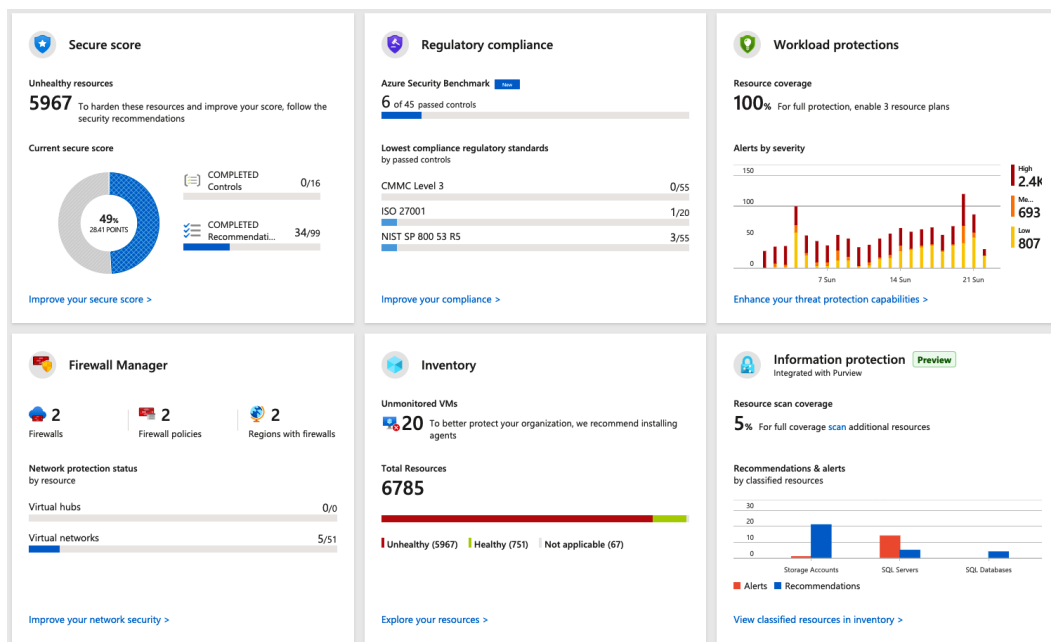


Figure 7.3 – Main tiles in Microsoft Defender for Cloud

Here's a brief description of each of these tiles:

- The **Secure score** tile is an overall representation of all subscriptions in your organization, providing an indication of how good (or badly) your resources are protected. Secure score is calculated based on the Azure Security Benchmark, which we will cover later in this chapter.
- In the **Regulatory compliance** tile, a view on regulatory standards, such as **ISO 27001**, **NIST SP 800 53 R5**, or **Azure Security Benchmark**, is presented for all resources and subscriptions in an organization.
- The **Workload protections** tile shows an alerts graph, sorted by severity.
- **Firewall Manager** is an integrated tile that links to Azure Firewall Manager, a service that is covered in *Chapter 4, Azure Network Security*.
- **Inventory** is a resource-focused representation on open recommendations, based on *Azure Resource Graph*.

- Last but not least, the **Information protection** tile is an integration with **Azure Purview**, Microsoft's data classification service. The tile shows the number of recommendations and alerts for classified resources. It's a predefined inventory view (based on Azure Resource Graph) that shows all resources with open recommendations and an information protection label, according to Purview.

The right side of the main dashboard provides different views of misconfiguration or critical alerts to catch an organization's interest, as well as further links to the Defender for Cloud GitHub community and TechCommunity blogs. On the left-hand side, the navigation menu is divided into three categories, as shown in *Figure 7.4*:

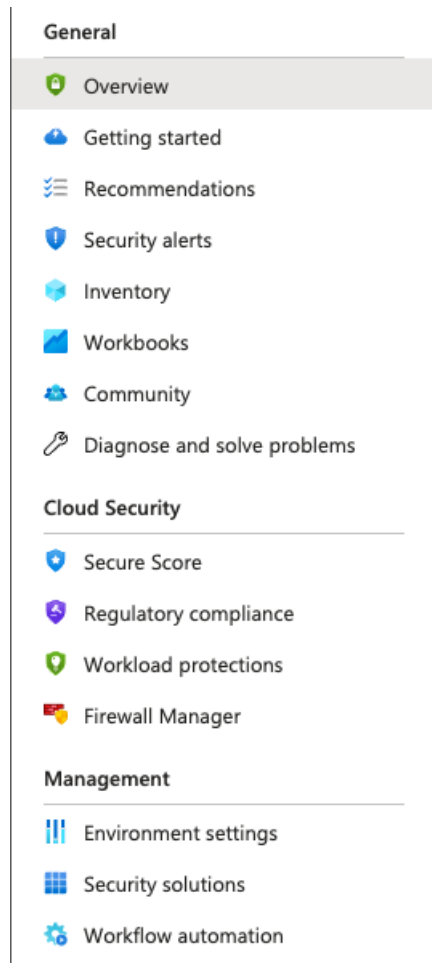


Figure 7.4 – Navigation menu

- The **General** section provides access to the main capabilities of Microsoft Defender for Cloud. In this section, recommendations and security alerts, the inventory, and integration with Workbooks are the main areas of interest.
- The **Cloud Security** section provides access to the **Secure score** and **Regulatory compliance** sections, and lets you navigate to **Workload protections**, an area that gives you access to advanced cloud defense capabilities, as shown in *Figure 7.5*:

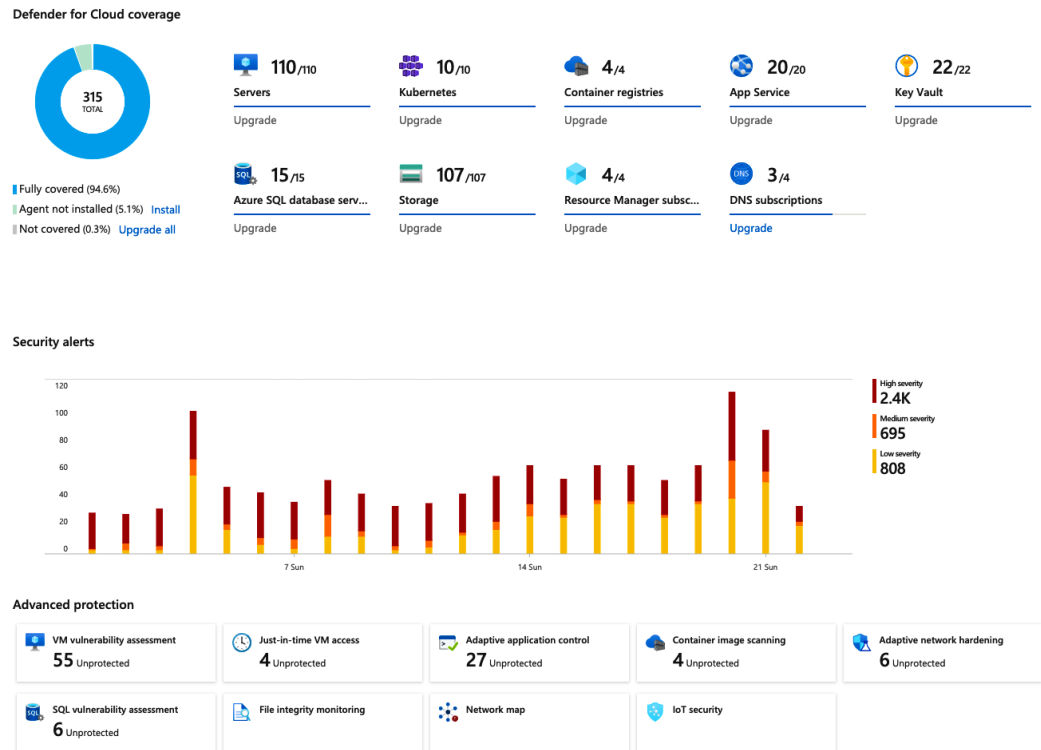


Figure 7.5 – Workload protections view in Microsoft Defender for Cloud

- The **Management** section provides access to environment settings, security solutions, and workflow automation. It is the area you use to configure Microsoft Defender for Cloud.

Now that you have gotten a quick overview of Microsoft Defender for Cloud and its Overview dashboard, let's dig a bit deeper and see how it works in technical terms.

Enabling Microsoft Defender for Cloud

Microsoft Defender for Cloud is not automatically enabled by default, so as a first step, we need to enable this service and provide initial configuration parameters. There are several key components we need to consider before starting.

Microsoft Defender for Cloud relies on some main Azure services that add benefit to the solution:

- A **Log Analytics Workspace** is used for storing all kinds of log information, such as Windows Server security event logs, or Linux server syslog entries, but also for storing security alerts and other information. It can also be used within the scope of **Continuous Export**, a feature we will discuss later in this chapter. Defender for Cloud will create a default workspace in case you're not using an existing one.

Important Note

If you want to connect Defender for Cloud with Microsoft Sentinel, you need to use a custom workspace, not the default one.

- A **Log Analytics Agent** is mandatory for Defender for Cloud to be able to monitor and analyze a machine's operating system. It is used for Defender for Cloud's Threat Intelligence to create security alerts, but also for some recommendations to be able to retrieve information about operating system configuration. When writing this book, integration with the **Azure Monitor Agent (AMA)** has not yet been available.
- **Azure Policy**, a service we have already covered in *Chapter 2, Governance and Security*, is what recommendations in Defender for Cloud rely on. This service is also used to provide insights into regulatory compliance standards, and it can be used for remediation and deployment at scale.
- **Azure Arc** is used as the vehicle to onboard non-Azure machines to Microsoft Defender for Cloud. By connecting a machine through Azure Arc, Microsoft Defender for Cloud can treat it as an Azure resource, including the capability to use auto-provisioning capabilities based on Azure Policy and Azure Arc extensions.

For Microsoft Defender for Cloud to be enabled on a subscription, you have several options. In general, you need to make sure that the `Microsoft.Security` resource provider is registered on a subscription. While this happens automatically whenever you open Defender for Cloud's portal in a particular subscription, or whenever you navigate through an Azure resource's security settings, this is not feasible for larger environments with several subscriptions. To enable Defender for Cloud at scale, you can follow the steps outlined in the next subsection

Enabling Microsoft Defender for Cloud via PowerShell

You can register the `Microsoft.Security` resource provider with the following PowerShell command:

```
Set-AzContext -Subscription "yourSubscriptionID"
Register-AzResourceProvider -ProviderNamespace 'Microsoft.Security'
```

With this command, you can enable the resource provider on all existing subscriptions, but in case there is a new subscription created, you would have to run the command for the new subscription, again. That's when a **deploy-if-not-exists (DINE)** policy comes into play.

Enabling Microsoft Defender for Cloud via Azure Policy

Azure Policy comes with a variety of built-in policy definitions, one of which is used to enable Microsoft Defender for Cloud on the scope you chose. In order to enable the service on all existing and future subscriptions, it's enough to simply assign the **Enable Azure Security Center on your subscription** policy to your root management group. To onboard a management group and all its subscriptions to Defender for Cloud, follow these steps:

1. Make sure to log in with an account that has Security Admin permissions, open **Azure Policy**, and search for the **Enable Azure Security Center on your subscription** policy definition.

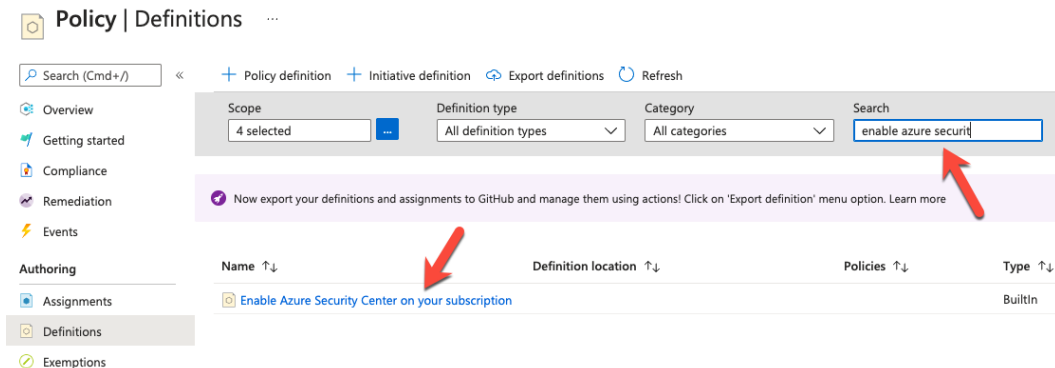


Figure 7.6 – Policy definition to enable Defender for Cloud

2. Select the definition, and then click **Assign**:



Figure 7.7 – Assigning the policy definition

3. Select **Tenant Root Group** as the assignment scope. There are no other parameters you need to change.

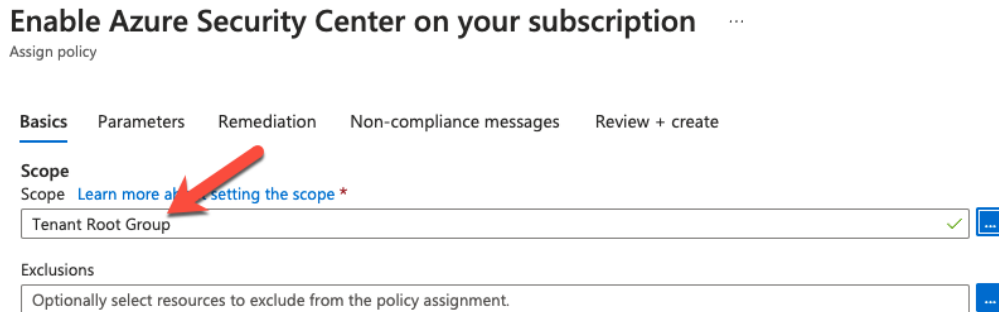


Figure 7.8 – Assigning the policy definition to your Tenant Root Group

4. Click **Review + create** and then **Create**.
5. After the definition has been assigned, navigate to the **Assignments** blade in Azure Policy and select the assignment you have just created. In the assignment, click on **Create Remediation Task**. This remediation task is necessary to make sure Defender for Cloud will not only be enabled for future subscriptions, but also for all subscriptions that already exist.

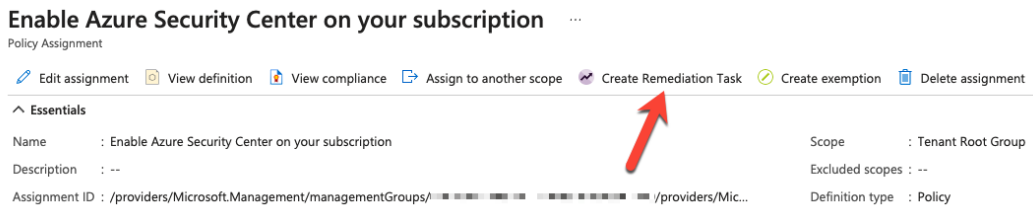


Figure 7.9 – Create Remediation Task

6. Select all applicable resources (if any) and click on **Remediate**.

Now that you have enabled Microsoft Defender for Cloud on all of your subscriptions, let's move on and deploy mandatory agents and extensions to your IaaS.

Using auto-provisioning to deploy extensions

Microsoft Defender for Cloud brings a set of extensions that are used to manage security for Azure VMs and non-Azure machines connected through Azure Arc. Azure Arc can be regarded as the *vehicle* being used to create a representative Azure resource of the non-Azure machine. That way, Azure governance and management capabilities, such as RBAC, policies, extensions, and more, can be used to manage and secure these on-premises or third-party cloud machines. The following extensions can be deployed to all machines in an Azure subscription by enabling auto-provisioning:

- **Log Analytics agent for Azure VMs** is used by Defender for Cloud to gather telemetry and threat evidence so it can create security alerts based on behavioral analytics and threat intelligence.
- On Azure Arc machines, **Log Analytics agent for Azure Arc machines** is used for the same purpose. It is a VM extension that is deployed to the Azure Arc machine, and which will then install the agent in the machine's operating system.
- When enabling the Defender for Servers plan, the **Vulnerability assessment for machines** extension will either deploy the integrated Qualys VA extension or enable **Microsoft Defender for Endpoint (MDE) Threat and Vulnerability Management (TVM)** on machines that are onboarded to MDE.
- **Guest Configuration agent** is a new capability in Azure Policy that allows operating system configurations to be audited. It can be deployed to both Azure VMs and Azure Arc machines.
- **Microsoft Dependency agent** is used to collect and store network traffic data for Azure VMs, which is then shown in Defender for Cloud's Network Map feature. Outside of Defender for Cloud, the dependency agent will collect relevant statistics about correlated applications to show communication in Azure Monitor's Service Map and VM Insights features. The latest addition is **Microsoft Defender for Containers components**, which include the policy add-on for Kubernetes (used to leverage Gatekeeper V3 for central rule enforcements), Azure Kubernetes Service Profile (a Daemon Set agent to collect security-related data from inside the cluster, similar to the Log Analytics agent on machines), and the Azure Arc-enabled Kubernetes extension (Daemon Set for Arc-connected Kubernetes clusters).

You can enable any of these extensions in Defender for Cloud when navigating to **Environment Settings** by selecting the management group, selecting the subscription, and then clicking the **Auto provisioning** button in the left navigation pane, as shown in *Figure 7.10*:

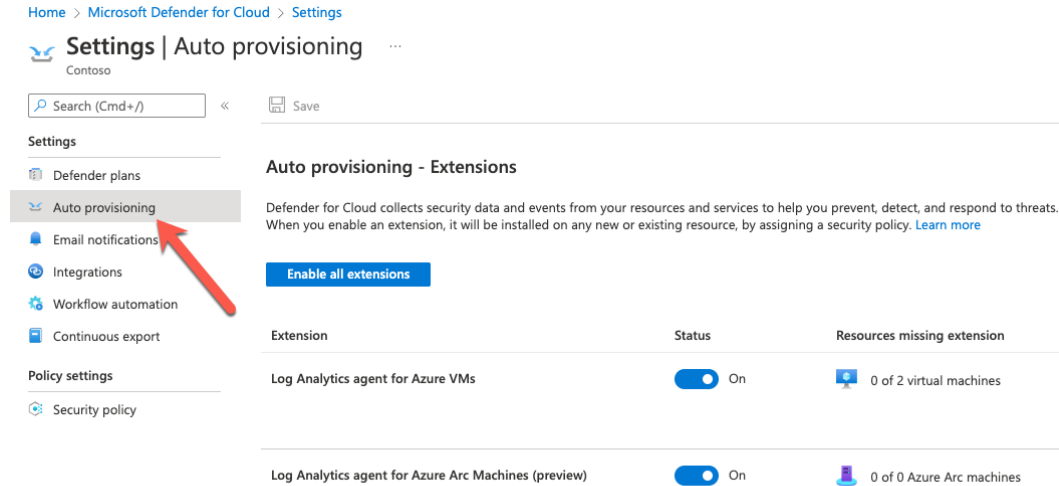


Figure 7.10 – Enabling auto-provisioning in Defender for Cloud

Once you enable one of the extensions to be deployed, you might need to add some configuration. For the Log Analytics workspace, you can either use the default workspace(s) created by Defender for Cloud or connect your machines to a custom workspace, as shown in *Figure 7.11*. In general, it is always a great idea to use a custom workspace because that way, you can make sure to connect all machines to the same workspace and use that workspace in Microsoft Sentinel.

Extension deployment configuration

×

Log Analytics agent for virtual machines

i If a VM already has either SCOM or OMS agent installed locally, the Log Analytics agent extension will still be installed and connected to the configured workspace.

i Any other solutions enabled on the selected workspace will be applied to Azure VMs that are connected to it. For paid solutions, this could result in additional charges. For data privacy considerations, please make sure your selected workspace is in your desired region.

Workspace configuration

Data collected by Defender for Cloud is stored in Log Analytics workspace(s). You can select to have data collected from Azure VMs stored in workspace(s) created by Defender for Cloud or in an existing workspace you created. [Learn more >](#)

☐ **Connect Azure VMs to the default workspace(s) created by Defender for Cloud**

☒ **Connect Azure VMs to a different workspace**

azureandbeyond-loganalytics-workspace
▼

Figure 7.11 – Auto-provisioning settings for the Log Analytics workspace

In addition to the workspace setting, you can also configure raw event data collection for the Windows Server Security Event Log. You can decide to transfer none (default), minimum, common, or all security events to your Log Analytics workspace.

Important Note

Windows Security Event Data Collection is not used by Defender for Cloud. Even if you set the collection tier to *none*, Defender for Servers will create all security alerts that apply to a machine. The collection of raw events is used by SIEM solutions, such as Microsoft Sentinel. Also, data collection for security events is only available in case you enable the Defender for Servers enhanced protection plan, which is covered later in this chapter.

By enabling one of the auto-provisioning settings, a built-in DINE policy is assigned to the subscription and a remediation task is created to auto-remediate all applicable resources in that subscription. Since all settings apply to the whole subscription, it is not possible to use different settings for different machines in the same subscription when using auto-provisioning. However, you can use policy assignments on a resource group to make sure all extensions are automatically deployed and still adhere to the configuration that best suits your environment.

Enabling Microsoft Defender for Cloud's enhanced security

Microsoft Defender for Cloud offers free capabilities and advanced capabilities that are included in Defender for Cloud plans, as shown in *Figure 7.12*:

 [Enable the enhanced security features of Microsoft Defender for Cloud. Learn more >](#)

Enhanced security off

- ✓ Continuous assessment and security recommendations
- ✓ Secure score
- ✗ Just in time VM Access
- ✗ Adaptive application controls and network hardening
- ✗ Regulatory compliance dashboard and reports
- ✗ Threat protection for Azure VMs and non-Azure servers (including Server EDR)
- ✗ Threat protection for supported PaaS services

Enable all Microsoft Defender for Cloud plans

- ✓ Continuous assessment and security recommendations
- ✓ Secure score
- ✓ Just in time VM Access
- ✓ Adaptive application controls and network hardening
- ✓ Regulatory compliance dashboard and reports
- ✓ Threat protection for Azure VMs and non-Azure servers (including Server EDR)
- ✓ Threat protection for supported PaaS services

 **Defender for Cloud plans will be enabled on 5 resources in this subscription**

^ Select Defender plan by resource type Enable all

Microsoft Defender for	Resources	Pricing	Configuration	Plan
 Servers	4 servers	\$15/Server/Month ⓘ		On Off
 App Service	0 instances	\$15/Instance/Month ⓘ		On Off
 Azure SQL Databases	0 servers	\$15/Server/Month ⓘ		On Off
 SQL servers on machines	0 servers	\$15/Server/Month ⓘ \$0.015/Core/Hour		On Off
 Open-source relational databases	0 servers	\$15/Server/Month ⓘ		On Off
 Storage	1 storage accounts	\$0.02/10k transactions ⓘ		On Off
 Containers	0 container registries; 0 kubernetes cores	\$7/VM core/Month ⓘ		On Off
 Kubernetes (deprecated)	0 kubernetes cores	\$2/VM core/Month ⓘ	 Update available ⓘ	On Off
 Container registries (deprecated)	0 container registries	\$0.29/Image	 Update available ⓘ	On Off
 Key Vault	0 key vaults	\$0.02/10k transactions		On Off
 Resource Manager		\$4/1M resource management operations ⓘ		On Off
 DNS		\$0.7/1M DNS queries ⓘ		On Off

Figure 7.12 – Enabling Defender for Cloud enhanced security

With enhanced security switched off, Defender for Cloud will continuously assess your Azure resources' current configuration and provide security recommendations for all of them. In addition to that, a secure score is calculated for each subscription, giving you an indication of your security posture's current state (the higher the score, the better your resources are protected). If you want to also protect on-premises servers or VMs running in another public cloud platform, or if you want to leverage other features, such as just-in-time VM access, adaptive application controls, adaptive network hardening, the regulatory compliance dashboards, and threat protection, you need to enable enhanced security by switching on the relevant Defender for Cloud plan.

Defender for Cloud is a subscription-based service, which is one of the reasons that Defender for Cloud plans can be enabled on a subscription-only basis when using the portal view shown in *Figure 7.12*. However, certain plans allow you to either be enabled on a subscription, or on a particular resource. This applies to the following plans:

- Microsoft Defender for Azure SQL databases
- Microsoft Defender for open source relational databases
- Microsoft Defender for Storage

There is a common misunderstanding that all Defender for Cloud plans can be enabled on a particular resource, only. Also, all Defender for Cloud plans can be enabled on a subscription, but not on a management group. However, there is a solution to this.

Enabling Microsoft Defender for Cloud plans on a subscription will basically change a setting on the subscription's `Microsoft.Security/pricings` API endpoint. In addition, there might be some additional configuration needed, but all of these settings can be enforced through Azure Resource Manager or an API. As one of the recent changes, Microsoft has added built-in policy definitions that can be used to enable Microsoft Defender for Cloud enhanced security plans on subscription. *Figure 7.13* shows the current list of built-in definitions. Please note that the display names still refer to Azure Defender when writing this chapter, while the product has been renamed to Microsoft Defender for Cloud at Microsoft Ignite in November 2021.

Name ↑↓	Definition location ↑↓	Policies ↑↓	Type ↑↓	Definition type ↑↓	Category ↑↓
 Configure Azure Defender to be enabled on SQL Servers and SQL Managed Instances		2	BuiltIn	Initiative	Security Center
 Configure Azure Defender for Key Vaults to be enabled			BuiltIn	Policy	Security Center
 Configure Azure Defender for DNS to be enabled			BuiltIn	Policy	Security Center
 [Preview]: Configure Azure Defender for SQL agent on virtual machine			BuiltIn	Policy	Security Center
 Configure Azure Defender for open-source relational databases to be enabled			BuiltIn	Policy	Security Center
 Configure Azure Defender for SQL servers on machines to be enabled			BuiltIn	Policy	Security Center
 Configure Azure Defender for Storage to be enabled			BuiltIn	Policy	Security Center
 Configure Azure Defender for servers to be enabled			BuiltIn	Policy	Security Center
 Configure Azure Defender for App Service to be enabled			BuiltIn	Policy	Security Center
 Configure Azure Defender for Resource Manager to be enabled			BuiltIn	Policy	Security Center
 Configure Azure Defender for Azure SQL database to be enabled			BuiltIn	Policy	Security Center

Figure 7.13 – Built-in policy definitions to enable Defender for Cloud's enhanced security

As with all policy definitions, you can create an assignment on top of your root management group and, that way, make sure that the plans that are relevant for your environment are enabled on all existing and future Azure subscriptions within this scope.

Important Note

Microsoft Defender for Servers and Defender for SQL on machines need to be enabled on both the Azure subscription your machines are connected to and the Log Analytics workspace their agent is reporting to!

All policy definitions will leverage the `Microsoft.Security/pricings` API endpoint to set or change the `pricingTier` property from `free` to `Standard`. The following code shows the reference for enabling Microsoft Defender for Servers in the **Configure Azure Defender for servers to be enabled** policy:

```
{
  "type": "Microsoft.Security/pricings",
  "apiVersion": "2018-06-01",
  "name": "VirtualMachines",
  "properties": {
    "pricingTier": "Standard"
  }
}
```


The names for the different Defender for Cloud plans are as follows:

- Microsoft Defender for Resource Manager = `Arm`
- Microsoft Defender for DNS = `Dns`
- Microsoft Defender for Servers = `VirtualMachines`
- Microsoft Defender for SQL = `SqlServers`
- Microsoft Defender for App Service = `AppServices`
- Microsoft Defender for Storage = `StorageAccounts`
- Microsoft Defender for SQL on machines = `SqlServerVirtualMachines`
- Microsoft Defender for Key Vaults = `KeyVaults`
- Microsoft Defender for open source relational databases = `OpenSourceRelationalDatabases`
- Microsoft Defender for Containers = `Containers`
- Microsoft Defender for Kubernetes = `KubernetesService` (deprecated)
- Microsoft Defender for Container Registries = `ContainerRegistry` (deprecated)

You can either use the built-in policy definitions or create a custom definition and select the prices you want to enable.

Tip

You can find a list of all built-in policy definitions for Microsoft Defender for Cloud at <https://aka.ms/DefenderForCloudPolicies>.

After enabling Microsoft Defender for Cloud, it will start assessing all connected resources for configuration issues and provide recommendations for closing these gaps. In the next section, you will learn about how to work with secure score and recommendations and how to improve your resources' security posture.

Cloud Security Posture Management with Defender for Cloud

As you learned in the previous section, **Cloud Security Posture Management (CSPM)** is one of the two main pillars in Microsoft Defender for Cloud. CSPM is all about hardening your cloud resources and that is why Defender for Cloud will provide you with a large list of security recommendations to help you understand what is good and what can be improved in your resources' configuration. **Secure score** is the main **Key Performance Indicator (KPI)** when it comes to understanding how good (or bad) you have configured your resources. The idea of secure score is to show a percentage value based on fixed points that are given for remediating **recommendations** that are grouped in **security controls**, as shown in *Figure 7.14*:

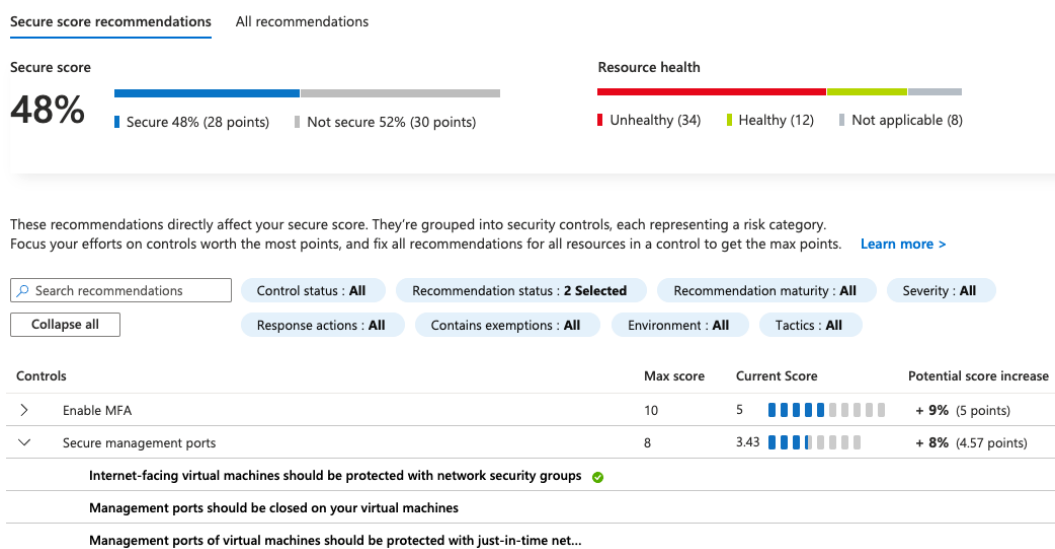


Figure 7.14 – Secure score, security controls, and recommendations

Figure 7.14 shows an environment with a secure score of 48%. The higher this percentage value is, the better protected your resources are. Secure score is calculated based on the following formula:

$$\text{Secure Score [\%]} = \frac{\text{current score}}{\text{maximum score}} * 100$$

In the preceding example, the calculation is as follows:

$$\text{Secure Score [\%]} = \frac{28}{(28 + 30)} * 100 = 48\%$$

The maximum score is a fixed number, depending on the security controls that apply to your environment. In Figure 7.14, you see two security controls (**Enable MFA** and **Secure management ports**), which have a maximum score of 10 and 8. While Defender for Cloud has more than 200 recommendations, not all of them might apply to your environment. For example, in case you don't have any VMs in your Azure subscription, you won't see the **Secure management ports** control, and therefore might have a maximum secure score of only 50 instead of 58.

Tip

The maximum secure score when writing this book is 58 based on the maximum score of all 15 security controls. This number might change slightly when Microsoft is adjusting the maximum score per control, or when adding new controls.

In order to increase your secure score, you need to make sure to remediate all recommendations in a particular security control that apply to a single resource. Let's take a closer look at the **Secure management ports** control in Figure 7.15:

Controls	Max score	Current Score	Potential score increase	Unhealthy resources
> Enable MFA	10	5	+ 9% (5 points)	2 of 4 resources
▼ Secure management ports	8	3.43	+ 8% (4.57 points)	4 of 7 resources
Internet-facing virtual machines should be protected with netw...				None
Management ports should be closed on your virtual machines				2 of 7 virtual machines
Management ports of virtual machines should be protected wit...				4 of 7 virtual machines
> Remediate vulnerabilities	6	1.5	+ 8% (4.5 points)	6 of 8 resources

Figure 7.15 – Secure score calculation per resource

The total number of resources in this example is seven VMs, two of which need to remediate the **Management ports should be closed on your virtual machines** recommendation, and four of which need to remediate the last recommendation. For the whole control, you see that four out of seven resources are unhealthy, which means that three out of seven VMs in this control's scope are already completely remediated (and therefore count toward this environment's secure score). The maximum score per resource is the maximum score per control divided by the number of resources within a control, using this formula:

$$\text{Score per resource} = \frac{\text{max score}}{\text{number of resources}}$$

In our scenario, the per-resource score is as follows:

$$\text{Score per resource} = \frac{8}{7} = \mathbf{1.143}$$

The *current score* is calculated as follows:

$$\text{Current score} = \text{score per resource} * \text{number of healthy resources}$$

Using the numbers from our scenario, the current score is as follows:

$$\text{Current score} = 1.143 * 3 = \mathbf{3.429}$$

That's why *Figure 7.15* shows a current score of **3.43**.

Now that you know how secure score is calculated, let's move on and focus on remediating recommendations.

Working with recommendations

As you have already learned, recommendations in Microsoft Defender for Cloud are grouped into security controls. This change compared to the initial concept of secure score was made to reflect the need to remediate all recommendations that belong to a particular protection idea. Those of you who have been working with Azure Security Center a few years back will remember that back then, secure score impact was calculated based on points for each recommendation. For example, for enabling a vulnerability assessment solution, you got 30 points, and an additional 30 points for remediating vulnerability findings in your machines. You will agree that by only enabling a solution but not taking care of remediating findings, your security posture is not really enhanced and, therefore, the change was made in favor of a secure score calculation based on controls.

Nowadays, you need to focus on remediating all applicable recommendations for a particular resource so your secure score will increase and, that way, indicate that your security posture is enhanced. Secure score recommendations are based on the **Azure Security Benchmark**, an initiative created by Microsoft to use industry standards and security best practices to highlight misconfigurations in an Azure environment. In the end, this benchmark is supposed to be the security baseline for securely deploying resources to Azure.

To get information about secure score and recommendations, the Azure Security Benchmark policy initiative needs to be assigned to either all Azure subscriptions or to one or several management groups in your environment. By default, once a subscription is onboarded to Defender for Cloud, the Azure Security Benchmark is automatically assigned as the default security policy. This is great for environments in which you have to work with different recommendations and want to disable some of them in some subscriptions. However, this might cause a lot of management overhead. As it's always a great idea to enable Defender for Cloud for all (existing and future) subscriptions, and to also get full recommendation and secure score coverage across the whole cloud estate, we recommend assigning the Azure Security Benchmark on your Tenant Root Group, similar to enabling Defender for Cloud, as described earlier in this chapter. In order to assign the benchmark to your root management group, simply follow these steps:

1. Log in with an account that has Security Admin privileges and, in Defender for Cloud, navigate to **Environment settings** and select **Tenant Root Group**:

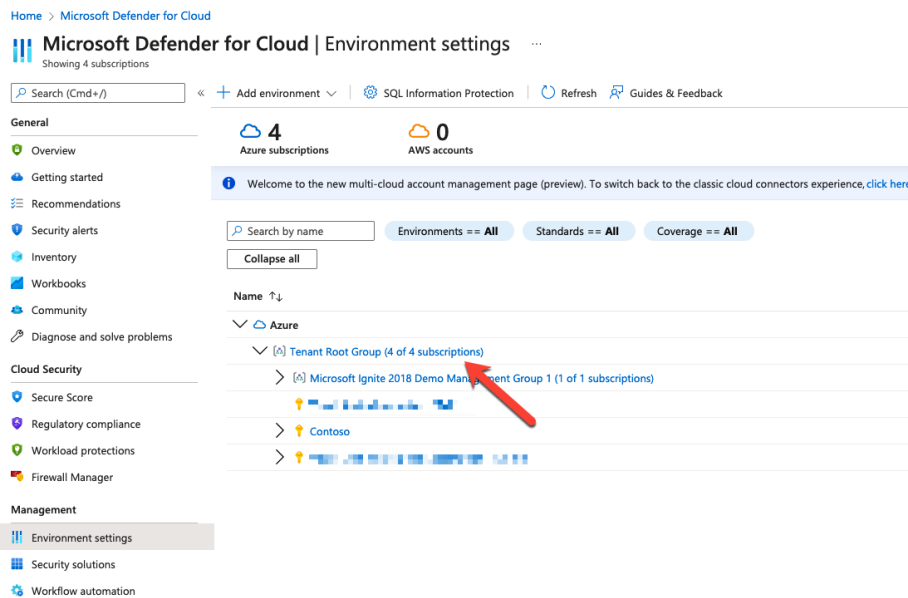


Figure 7.16 – Selecting your Tenant Root Group in Defender for Cloud

2. Select **Assign policy**:

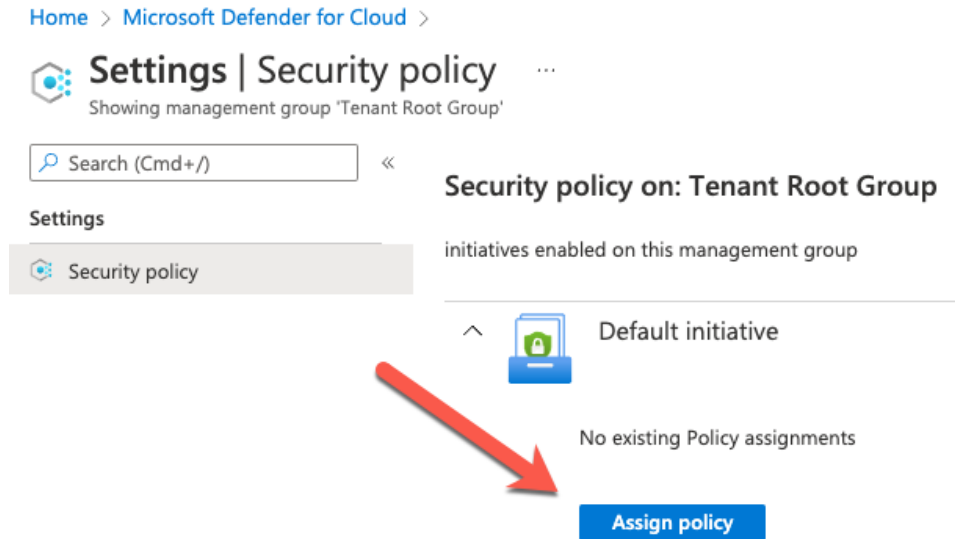


Figure 7.17 – Assign a new security policy

3. Click **Review + create** and then click **Create**.

Once the Azure Security Benchmark initiative has been assigned, Microsoft Defender for Cloud will start assessing all resources in an Azure subscription and provide the results in the recommendations view. Microsoft Defender for Cloud comes with three different assessment status codes:

- Resources are **healthy** when there was a positive assessment result. In other words, the resource is configured in a secure way.
- Resources are **unhealthy** whenever the resource is insecurely configured, or in case the resource has been securely configured, but Defender for Cloud is not able to assess the actual setting. This can happen, for example, when you are using a third-party solution or process to secure your resource.
- Resources are **not applicable** whenever an assessment does not apply to a resource. For example, in case you deploy a third-party virtual firewall appliance from the marketplace, it technically might be a Linux VM. VMs should have a **Network Security Group (NSG)** applied to their network interfaces or subnets, but in case it's a firewall appliance, the software itself is supposed to handle traffic and open/close network ports. Defender for Cloud will recognize this and mark this particular VM as not applicable for this recommendation.

Although Microsoft Defender for Cloud's recommendations are based on a policy initiative (the Azure Security Benchmark), the assessments are not – at least not all of them. Defender for Cloud has its own capability to run assessments (configuration checks) in its backend. An assessment in this context will always be one check for a particular resource toward a particular recommendation. Once there is a result, the assessment status code (healthy, unhealthy, not applicable) is being sent to Azure Policy. Azure Policy, however, only knows two different *health states* for resources:

- Resources are **compliant** when their configurations adhere to the policy intend.
- They are **non-compliant** if they do not.

Therefore, all policy definitions used in the Azure Security Benchmark expect the assessment status code and flag resources as being *compliant*, in case the status code is *healthy* or *not applicable*, and as *non-compliant* if the assessment status code is *unhealthy*. That way, the recommendations view in Defender for Cloud and the Policy compliance dashboard will have similar results when taking a look at the corresponding policy setting/recommendation.

The following section is taken from the **Management ports should be closed on your virtual machines** policy definition:

```
"details": {
  "type": "Microsoft.Security/assessments",
  "name": "bc303248-3d14-44c2-96a0-55f5c326b5fe",
  "existenceCondition": {
    "field": "Microsoft.Security/assessments/status.
code",
    "in": [
      "NotApplicable",
      "Healthy"
    ]
  }
}
```

As you can see, the `existenceCondition` value is not that assessment itself, which means that this policy definition is not being used to assess (audit) the VM configuration; it will audit a resource of the `Microsoft.Security/assessments` type with the name `bc303248-3d14-44c2-96a0-55f5c326b5fe`. That's the underlying assessment for the *Management ports should be closed on your virtual machines* recommendation. The policy will audit the `status.code` field and expect `NotApplicable` or `Healthy` as values. In these cases, the assessed resource will be flagged as being compliant.

How to prioritize remediation

As the Azure Security Benchmark comes with more than 200 recommendations (with more recommendations constantly being added), it might become challenging to prioritize remediation. Each recommendation comes with a lot of details that help you with this challenge. *Figure 7.18* shows the details view of the **Management ports should be closed on your virtual machines** recommendation. Every recommendation contains a **Remediation steps** section that explains what needs to be done to make an unhealthy resource healthy. **Freshness interval** is an indicator of how long it can take after the remediation until the resource appears as being healthy. By clicking on the **Open query** button, you are redirected to the Azure Resource Graph explorer, where you can see (and run) a KQL query that will give you an ARG presentation of this recommendation and all connected resources.

Home > Microsoft Defender for Cloud >

Management ports should be closed on your virtual machines ...

Exempt View policy definition Open query

Severity
Medium

Freshness interval
24 Hours

Exempted resources
1 View all exemptions

Tactics and techniques
Initial Access

^ **Description**

Open remote management ports are exposing your VM to a high level of risk from internet-based attacks. These attacks attempt to brute force credentials to gain admin access to the machine.

^ **Remediation steps**

Manual remediation:

We recommend that you edit the inbound rules of some of your virtual machines, to restrict access to specific source ranges.

To restrict access to your virtual machines:

1. Select a VM to restrict access to.
2. In the 'Networking' blade, click on each of the rules that allow management ports (for example, RDP-3389, WINRM-5985, SSH-22).
3. Either change the 'Action' property to 'Deny', or, improve the rule by applying a less permissive range of source IP ranges.
4. Click 'Save'.

Use Defender for Cloud's Just-in-time (JIT) virtual machine (VM) access to lock down inbound traffic to your Azure VMs by demand. Learn more in [Understanding just-in-time \(JIT\) VM access](#).

^ **Affected resources**

Unhealthy resources (3) Healthy resources (3) Not applicable resources (1)

Search virtual machines

Name	Subscription
windows-victim2	Visual Studio Enterprise Subscription - MS FTE
windows-victim1	Visual Studio Enterprise Subscription - MS FTE
linux-victim	Visual Studio Enterprise Subscription - MS FTE

Trigger logic app Exempt

Figure 7.18 – Recommendation details in Defender for Cloud

The following list is a good starting point for prioritizing remediation in your environment:

- Take a look at the maximum secure score per security control. The higher the maximum secure score is, the more important it is to remediate all recommendations in this particular control. For example, the **Enable MFA** control counts 10 points toward your secure score, **Secure management ports** an additional 8 points, and so on.
- Use **Severity**. Every recommendation has a severity assigned to it, as shown in *Figure 7.18*. There are three different values (*low*, *medium*, *high*). The higher the severity, the more important it is to remediate a recommendation.
- **Tactics and techniques** have recently been added to recommendations. This is a mapping to the MITRE ATT&CK Framework, giving you an indication of what step on the attack kill chain a recommendation fits into. Based on this information, you can make your own plan for prioritizing remediation
- Use data sensitivity classifications. By integrating Azure Purview and using the Defender for Cloud inventory, you can easily find all resources in your environment that contain classified data (databases, storage accounts, and so on). These resources should be prioritized when it comes to remediating recommendations. *Figure 7.19* shows a SQL server that contains classified data and all of its recommendations:

Home > Microsoft Defender for Cloud > Inventory >

Resource health (Preview) ...

Is this new resource health page helpful to you? ☐ Yes ☐ No

6 Active recommendations

22 Active alerts

Resource information

Subscription
CyberSecSOC

Resource Group
soc-purview

Environment
Azure

Location
eastus

Status
Ready

Security value

Microsoft Defender for Azure SQL database servers
On

Data classifications

Person's Name (4)

Canada Social Insurance Number (2)

Credit Card Number (1)

See more (2)

Purview account

Recommendations Alerts

Search

Status == All Severity == All

Severity	Description
High	SQL servers should have an Azure Active Directory administrator provisioned
High	SQL servers should have vulnerability assessment configured
High	Microsoft Defender for SQL should be enabled for unprotected Azure SQL servers
Medium	Private endpoint connections on Azure SQL Database should be enabled Preview
Medium	Public network access on Azure SQL Database should be disabled Preview
Low	Auditing on SQL server should be enabled
Low	[Enable if required] SQL servers should use customer-managed keys to encrypt data at rest Preview
Low	Audit retention for SQL servers should be set to at least 90 days Preview
Low	Vulnerability Assessment settings for SQL server should contain an email address to receive scan reports
Low	SQL server should use customer-managed keys to encrypt data at rest

< Previous Page 1 of 1 Next >

Figure 7.19 – A resource containing classified data

Some recommendations come with a Quick Fix feature that will let you select resources and auto-remediate them by using built-in automation capabilities. *Figure 7.20* shows one of these recommendations:

Home > Microsoft Defender for Cloud >

Management ports of virtual machines should be protected with just-in-time network access control ... X

Exempt View policy definition Open query

Severity **High** Freshness interval **24 Hours** Tactics and techniques **Initial Access +1**

Description
Defender for Cloud has identified some overly-permissive inbound rules for management ports in your Network Security Group. Enable just-in-time access control to protect your VM from internet-based brute-force attacks. Learn more in [Understanding just-in-time \(JIT\) VM access](#).

Remediation steps

Affected resources
Unhealthy resources (4) Healthy resources (3) Not applicable resources (0)

Search virtual machines

Name	Subscription
☐ windows-victim2	Visual Studio Enterprise Subscription - MS FTE
☒ windows-victim1	Visual Studio Enterprise Subscription - MS FTE
☐ linux-victim	Visual Studio Enterprise Subscription - MS FTE
☐ contoso-linux-vm	Contoso

Fix Trigger logic app Exempt

Figure 7.20 – Auto-remediation capability in some recommendations

With a click on the **Fix** button, a dialog will appear that lets you configure the corresponding settings. Once you save, the selected resource(s) will be remediated.

Working with resource exemptions

There are cases in which some recommendations might not be applicable to your environment. You already know that you can disable one or several recommendations directly in the security policy assigned to your environment, but that has some disadvantages:

- You can only disable a recommendation for the entire scope. If the Azure Security Benchmark is assigned to your management group, disabling a definition will remove that recommendation for all subordinate resources and subscriptions, too.
- Disabling a recommendation is a static setting.
- After disabling a recommendation, it's next to impossible to track who disabled which recommendation for what reason.

This is where **Resource exemptions** come into play. Resource exemptions are a great way to exempt a resource, or a whole scope from being assessed while still being able to react dynamically and keep track of these exemptions for auditing purposes.

From a technical perspective, the resource exemption capability in Microsoft Defender for Cloud leverages the policy exemptions feature in Azure Policy. Whenever you create a resource exemption, Defender for Cloud will trigger Azure Policy to create policy exemptions for all assignments that a particular recommendation is part of. *Figure 7.21* shows a recommendation with the **Exempt** button on top:

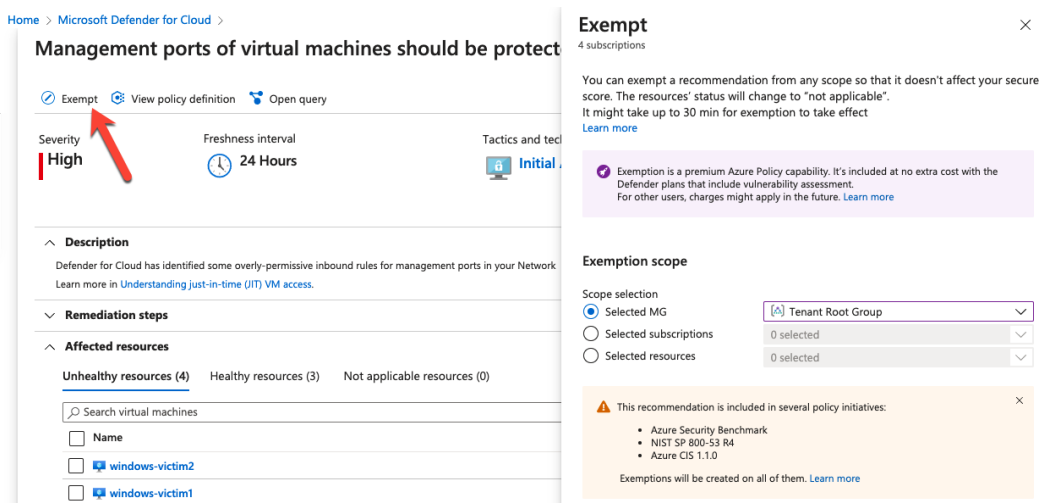


Figure 7.21 – Exempting a resource from being assessed

To create a policy exemption, you need to follow these steps:

1. Click the **Exempt** button. Once you do so, the **Exempt** dialog on the right side appears.
2. On the exempt dialog, you first select **Exemption scope**. You can exempt one or several resources, subscriptions, or management groups. You will see an indication regarding what policy initiatives include the recommendation you are about to create an exemption for. Azure Policy will create an exemption for each corresponding policy assignment.

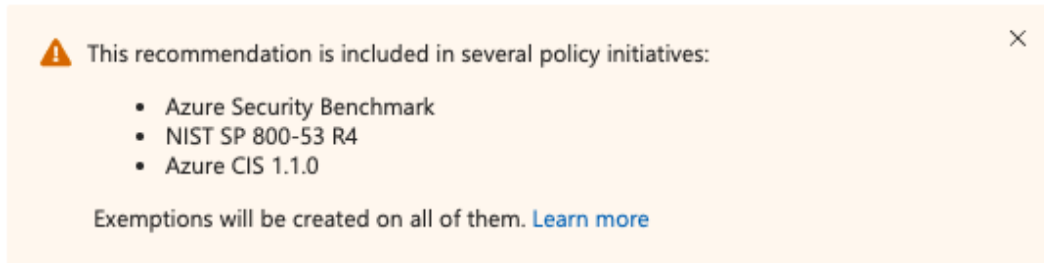


Figure 7.22 – List of policy initiatives that include the recommendation to be exempted

3. You can set an expiration date (up to 6 months from the time you create the exemption). As you can see in *Figure 7.23*, **Edited By** is an immutable field. It will contain the account name of the person who is just creating this exemption and store it in the corresponding policy exemption. That way, you will always know who created an exemption.

Exemption details

Exemption name *

ASC-Management ports of virtual machines should be protected with just-in-time netw...

☐ Set an expiration date

Edited By

XXXXXXXXXX

Exemption category * ⓘ

- ☐ Mitigated (resolved through a third-party service)
- ☒ Waiver (risk accepted)

Exemption description * ⓘ

demo exemption



Create

Cancel

Figure 7.23 – Exemption details

4. You now need to select an exemption category. **Mitigated** is used in case the recommendation's intent has been met by a process or solution that cannot be tracked by Defender for Cloud. **Waiver** is an indicator for you to accept the risk that goes along with not taking care of a particular recommendation.
5. Last but not least, you have to enter a description. This field can (and should) be used to be more specific about the *why*.
6. Finally, by clicking on the **Create** button, the exemption is created. All resources within the exemption's scope will change their health state to not applicable.

With all the information you have to add to a resource exemption, you will always be able to determine who created which exemption for what reason. Also, by selecting a particular scope from a single resource up to an entire management group, you can react dynamically to your organization's needs. Also, from an auditing perspective, it's a great idea to leverage resource exemptions versus entirely switching off a particular recommendation.

Note

Since resource exemptions rely on policy exemptions, which is a premium capability in Azure Policy, charges may apply in the future for customers that are not using Defender for Cloud's enhanced security plans. It is included at no additional cost with the Defender for Cloud plans, which include vulnerability assessments (Defender for Servers, SQL, Containers).

Now, that you know about Defender for Cloud's CSPM capabilities, let's take a look at custom recommendations and regulatory compliance.

Custom policies and (regulatory) compliance

Besides Azure Security Benchmark as the default security policy in Defender for Cloud, you can add custom policies for getting custom recommendations in addition to the built-in recommendations. Custom recommendations will appear in the **Custom recommendations** security control, which does not count toward the secure score.

Controls	Max score	Current Score
> Enable MFA	10	5
> Secure management ports	8	3.43
> Remediate vulnerabilities	6	0
> Apply system updates	6	6
> Enable encryption at rest	4	0
> Manage access and permissions	4	0
> Restrict unauthorized network access	4	1.5
> Encrypt data in transit	4	2.67
> Remediate security configurations	4	3
> Apply adaptive application control	3	3
> Protect applications against DDoS attacks	2	0
> Enable endpoint protection	2	0.57
> Enable auditing and logging	1	0.33
> Enable enhanced security features	Not scored	Not scored
> Implement security best practices	Not scored	Not scored
▼ Custom recommendations	Not scored	Not scored
[Custom] Audit Key Vault secrets without expiry date Custom		

Figure 7.24 – Custom recommendations in Defender for Cloud

To add custom recommendations to Defender for Cloud, you need to create a custom policy initiative that contains all custom policy definitions:

1. In Azure Policy, navigate to **Definitions** and select **+ Initiative definition**.
2. Select the highest scope for creating your initiative so you can later assign it to whatever scope you need. Then, enter a name and select a category.

[Home](#) > [Policy](#) >

Initiative definition ...

New Initiative definition

Basics Policies Groups Initiative parameters Policy parameters Review + create

An initiative definition is a collection of policy definitions that are tailored towards achieving a singular overarching goal. Initiative definitions simplify managing and assigning policy definitions by grouping them as a single assignable object.

Initiative location * ⓘ

Tenant Root Group

Name * ⓘ

Mastering Azure Security

Description ⓘ

Category ⓘ

☐ Create new ☒ Use existing

Security Center

Figure 7.25 – Creating a custom initiative

3. In the **Policies** tab, click **Add policy definition(s)** to add your custom policies.
4. On the **Groups** tab, create one or several groups to group your recommendations. Then, on the **Policies** tab, select your definitions and move them to the corresponding group.
5. In case your policies require parameters, you can now configure them; click **Review + create**, and then **Create**.
6. Navigate to **Defender for Cloud** -> **Environment settings**. Select the scope you want to assign your custom initiative to.
7. On the **Security policy** tab, you can add a custom initiative by clicking the corresponding button at the bottom of the page.
8. Find the initiative you have just created and click the **Add** button next to it.
9. Follow the steps on this page and then create the assignment by clicking **Review + create** and then **Create**.

After some time, your custom initiative will appear in the **Regulatory compliance** dashboard within Defender for Cloud. Also, custom policies that apply to your resources will appear in the **Custom recommendations** control within the **Recommendations** view.

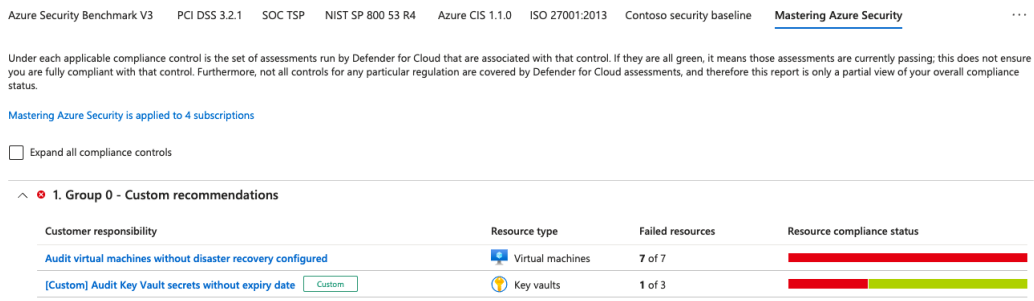


Figure 7.26 – Your custom initiative in the regulatory compliance dashboard

Now, after you have learned how to build and add custom recommendations to Defender for Cloud, let's move on to learn more about the regulatory compliance dashboard.

Using the regulatory compliance dashboard

The regulatory compliance dashboard within Defender for Cloud provides a different perspective to resources and their configuration. While secure score and recommendations help you understand the impact of remediating insecure configurations, regulatory compliance will provide an auditing perspective based on regulatory compliance standards.

Note

The regulatory compliance dashboard is a premium offering within Defender for Cloud. In order to use it, you need to enable Defender for Cloud's enhanced security features.

Taking Azure Security Benchmark as one of these standards, it's easy to understand: While the same initiative is being used in both views within Defender for Cloud, the secure score and recommendations view are aimed at helping you focus on security misconfiguration. With every resource you remediate, your secure score will increase. At the same time, the number of open recommendations will decrease. The regulatory compliance view is more restrictive. It doesn't consider if you only have a few unhealthy resources, whereas the rest of your environment is secure. As long as *all* resources are not flagged as healthy, your environment does not comply with a regulatory standard. In other words: your whole environment is non-compliant.

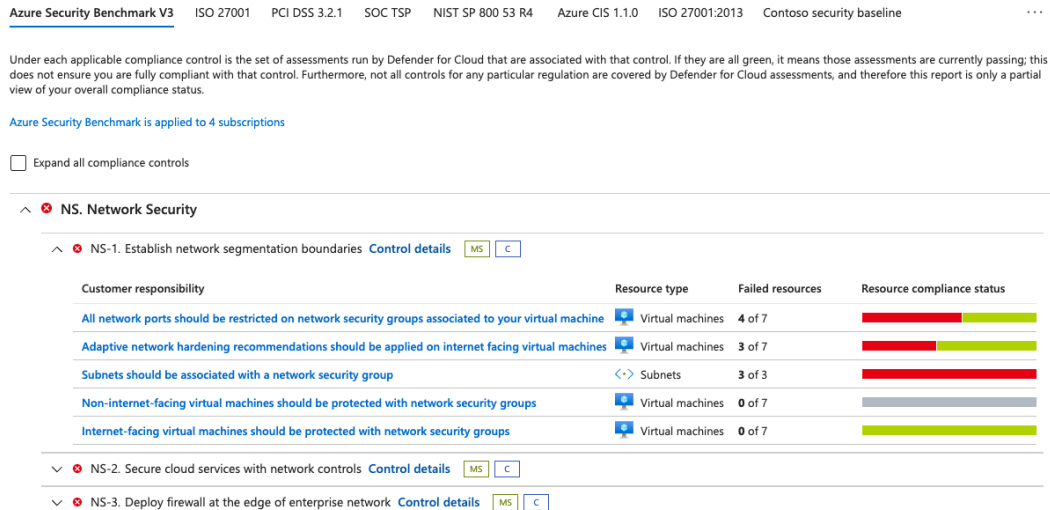


Figure 7.27 – Regulatory compliance representation of the Azure Security Benchmark

Figure 7.27 shows the regulatory compliance representation of the Azure Security Benchmark **Network Security (NS)** control. As you can see, the control is shown as unhealthy, marked by a cross in a red circle. NS-1 is a sub-control that is also reflected as being unhealthy, although some of its recommendations do not contain any failed resources. The idea of the regulatory compliance dashboard is to give you a robust compliance view that you can use as proof of an audit. In case a (sub-)control is reflected as being healthy, you are good to go. As long as a control is unhealthy, you are not yet fully compliant and need to put more effort into your environment's configuration.

It's important to understand that there is no *resource exemption* capability for the regulatory compliance view – and that's for a good reason! As mentioned previously, the dashboard is supposed to give you resource configuration insights from a compliance perspective. In case you were to exempt a resource, recommendation, or control, your environment would still not be compliant in relation to the particular compliance standard, you would simply no longer see the assessment. Therefore, you cannot *switch off* settings you don't want to focus on in this view.

Working with regulatory compliance standards

By default, Defender for Cloud shows a representation of the following regulatory compliance standards:

- Azure Security Benchmark V3
- PCI DSS 3.2.1
- ISO 27001
- SOC TSP

You can disable one or several of these standards or add more standards in Defender for Cloud's **Environment settings**. To disable ISO 27001 and add another standard, follow these steps:

1. Navigate to **Defender for Cloud | Environment settings** and select the scope of your choice.
2. In the **Security policy** view, look for the ISO 27001 standard and click the **Disable** button next to it. Confirm your selection.
3. Click the **Add more standards** button underneath and select one or several other standards you want to add. For each selected standard, click the **Add** button next to it. *Figure 7.28* shows a list of all standards that are built into Defender for Cloud when writing this chapter:

Add regulatory compliance standards ...



Click **Add** on the standards that you want to add to the regulatory compliance dashboard and then assign it to the subscription. After completing the assignment, the custom policies will be available in the **Regulatory compliance** dashboard.

Search to filter items...				
Name	↑↓	Description	↑↓	↑↓
NIST SP 800-53 R4		Track NIST SP 800-53 R4 controls in the Compliance Dashboard, based ...		<button>Add</button>
NIST SP 800 171 R2		Track NIST SP 800 171 R2 controls in the Compliance Dashboard, based...		<button>Add</button>
UKO and UK NHS		Track UK OFFICIAL and UK NHS controls in the Compliance Dashboard, ...		<button>Add</button>
Canada Federal PBMM		Track Canada Federal PBMM controls in the Compliance Dashboard, bas...		<button>Add</button>
Azure CIS 1.1.0		Track Azure CIS 1.1.0 controls in the Compliance Dashboard, based on a...		<button>Add</button>
HIPAA HITRUST		Track HIPAA/HITRUST controls in the Compliance Dashboard, based on ...		<button>Add</button>
SWIFT CSP CSCF v2020		Track SWIFT CSP CSCF v2020 controls in the Compliance Dashboard, ba...		<button>Add</button>
ISO 27001:2013		Track ISO 27001:2013 controls in the Compliance Dashboard, based on ...		<button>Add</button>
New Zealand ISM Restricted		Track New Zealand ISM Restricted controls in the Compliance Dashboard...		<button>Add</button>
CMMC Level 3		Track CMMC Level 3 controls in the Compliance Dashboard, based on a ...		<button>Add</button>
Azure CIS 1.3.0		Track Azure CIS 1.3.0 controls in the Compliance Dashboard, based on a...		<button>Add</button>
NIST SP 800-53 R5		Track NIST SP 800-53 R5 controls in the Compliance Dashboard, based ...		<button>Add</button>
FedRAMP H		Track FedRAMP H controls in the Compliance Dashboard, based on a re...		<button>Add</button>
FedRAMP M		Track FedRAMP H controls in the Compliance Dashboard, based on a re...		<button>Add</button>

Figure 7.28 – Selecting one or several regulatory compliance standards to be added

- For each standard, you might have to add some policy parameters. Once done, confirm your selection by clicking on **Review + create** and then **Create** to assign the corresponding policy initiative.

Once you are done, the new compliance standard will appear in the regulatory compliance view after some time.

You have now learned how to add custom and additional built-in policies to Defender for Cloud and know about the differences between secure score and regulatory compliance perspectives. In the next section, you will learn about enhanced security capabilities within Microsoft Defender for Cloud.

Cloud workload protection and multi-cloud capabilities

The second main pillar of Microsoft Defender for Cloud is **Cloud Workload Protection**, which is all about enhanced security features and threat intelligence. These enhanced capabilities are activated once you enable one or several of the Defender for Cloud enhanced security plans, as highlighted earlier in this chapter. As Microsoft is constantly working on improving existing and adding future plans to Defender for Cloud, we can only cover some of them in this chapter to give you an idea of what additional capabilities they provide. In general, when enabling Defender for Cloud plans, this will enable threat detection capabilities for the workload that is being protected by a plan, but also other advanced cloud defense capabilities, such as vulnerability assessments.

Microsoft Defender for Servers

One of the most popular Defender for Cloud plans is Microsoft Defender for Servers. Defender for Servers leverages the machine's Log Analytics agent to collect threat signals from inside the operating system and create security alerts based on these signals, whenever enough evidence has been collected for malicious behavior on a machine. *Figure 7.29* shows alert details for a **Successful SSH brute force attack** alert that has been generated by Defender for Cloud:

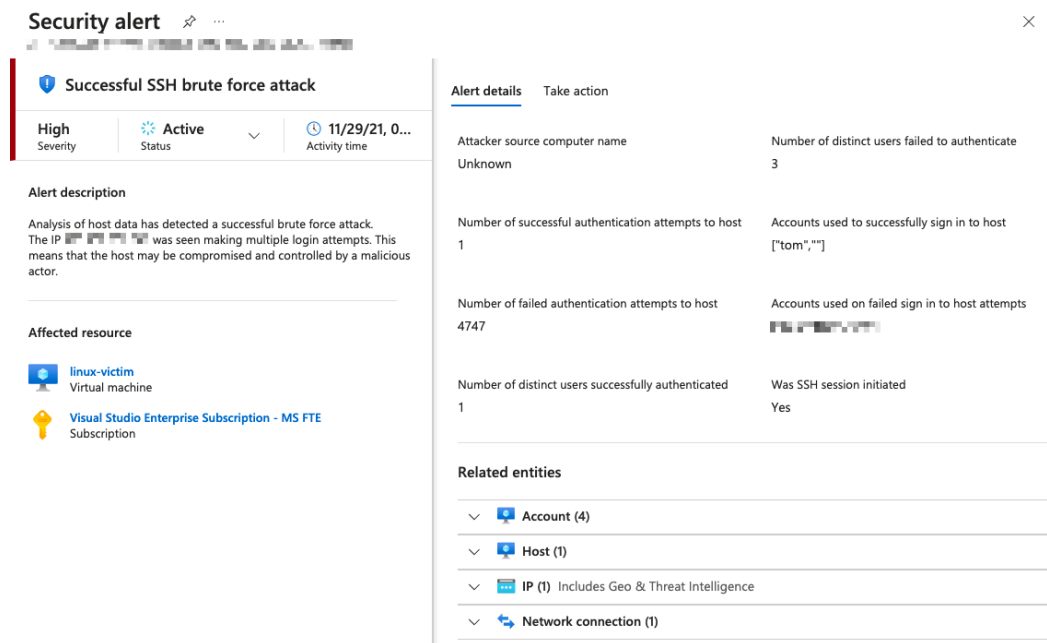


Figure 7.29 – Successful SSH brute force attack alert

The alert details might vary depending on the alert type, but in general, this section contains additional context. In case of a brute force attack, the alert will contain IP address and regional information about the attacker, the accounts that have been attacked, the host (the attacked machine), and more.

The second tab in each alert contains information about what to do next.

Alert details

Take action

^

 Mitigate the threat

1. Escalate the alert to the information security team
2. In case this is an Azure virtual machine, add the source IP to NSG block list for 24 hours (see <https://azure.microsoft.com/en-us/documentation/articles/virtual-networks-nsg/>)
3. Enforce the use of strong passwords and do not re-use them across multiple resources and services (see <http://windows.microsoft.com/en-us/Windows7/Tips-for-creating-strong-passwords-and-passphrases>)
4. In case this is an Azure virtual machine, Create an allow list for SSH access in NSG (see <https://azure.microsoft.com/en-us/documentation/articles/virtual-networks-nsg/>)

You have 43 more alerts on the affected resource. [View all >>](#)

^

 Prevent future attacks

Your top 3 active security recommendations on  [linux-victim](#):

High		Management ports of virtual machines should be protected with just-in-time network access control
High		Adaptive network hardening recommendations should be applied on internet facing virtual machines
High		Virtual machines should encrypt temp disks, caches, and data flows between Compute and Storage resources

Solving security recommendations can prevent future attacks by reducing attack surface.
[View all 9 recommendations >>](#)

^

 Trigger automated response

Trigger logic app as an automated response to this security alert. You can get suggested responses in [the ASC Community GitHub Repo](#).

Trigger logic app

^

 Suppress similar alerts

You can suppress similar alerts by creating suppression rule with pre-defined conditions.

Create suppression rule

Figure 7.30 – Next steps when remediating an alert

The first section explains how to mitigate the threat by conducting immediate steps, such as escalating the alert to your organization's **Security Operations Center (SOC)**, blocking the attacking IP addresses, and others. It also contains a reference to additional alerts on the affected resource and, to provide future attacks, a direct connection to open recommendations that apply to this resource. You can also trigger an automated response by manually sending the alert details to a Logic app that will then take action or create an alert suppression rule to suppress similar alerts in the future. This, however, should only be used with caution as you don't want to leverage Defender for Servers but not have it create alerts whenever something suspicious is happening on your machines.

In addition, most security alerts contain information about the MITRE ATT&CK tactics a particular alert belongs to, as shown in *Figure 7.31*:

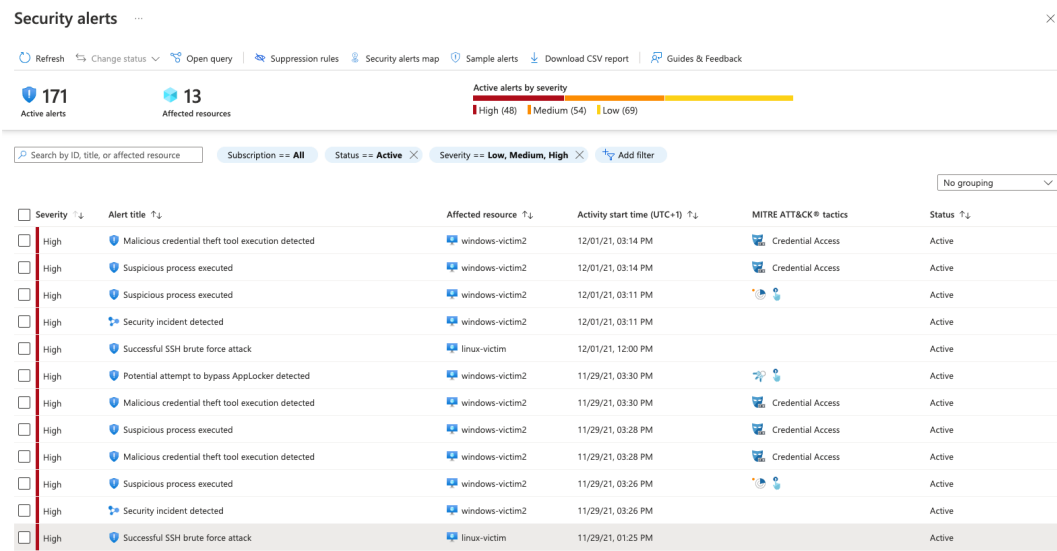


Figure 7.31 – Security alerts dashboard

The security alerts dashboard is built on top of Azure Resource Graph. By clicking on **Open query**, you are redirected to Azure Resource Graph Explorer with a query that will present the same result as being pre-filtered in the dashboard. The **Sample alerts** button lets you create sample alerts for most Defender plans to get an idea of what alerts will look like and which information they contain.

MDE integration

Some alerts in *Figure 7.31* are not generated by Defender for Servers, but by MDE. MDE is, by default, integrated with Microsoft Defender for Servers. With that being said, once you enable Defender for Servers on your subscription, Defender for Cloud will take care of onboarding all protected servers to MDE at no additional cost. MDE is an **Endpoint Detection and Response (EDR)** solution that helps you protect your operating systems. By integrating this solution into Defender for Cloud, you can make sure to have a first-class EDR solution on your machines and, at the same time, leverage a unified cloud security view by seeing all multi- and hybrid cloud resources in a unified dashboard.

Security alert ✕

Malicious credential theft tool execution detected

High Severity **Active** Status 12/01/21, 0... Activity time

Alert description

A known credential theft tool execution command line was detected. Either the process itself or its command line indicated an intent to dump users' credentials, keys, plain-text passwords and more.

Affected resource

- windows-victim2 Virtual machine
- Visual Studio Enterprise Subscription - MS FTE Subscription

MITRE ATT&CK® tactics

- Credential Access

Alert details Take action

Microsoft Defender for Endpoint link
[Investigate in M365 Defender portal](#)

File Sha256
e46ba4bdd4168a399ee5bc2161a8c918095fa30eb20a...
[See more](#)

Detected by
Microsoft

WdatpTenantId

Machine Name
windows-victim2

File Name
mimikatz.exe

User Name
tom

File Path
C:\tools\mimikatz\x64

User Domain
windows-victim2

Related entities

- Account (1)
- File (1)
- File hash (3)
- Host (1)

Figure 7.32 – Security alert created by MDE

You can easily see alerts that have been created by MDE as they will contain a link that takes you to the M365 Defender portal for further investigation, as shown in *Figure 7.32*.

Vulnerability assessments

Microsoft Defender for Cloud comes with two **vulnerability assessment (VA)** solutions – Qualys, and MDE's **Threat and Vulnerability Management (TVM)**. Once one of these solutions has been enabled, for example, by using auto-provisioning as explained earlier in this chapter, it will start assessing a machine for installed software and configurations and provide a list of vulnerability findings within the scope of a recommendation in Defender for Cloud's recommendations view. In addition to that, Defender for Cloud offers filtering capabilities in the inventory view that let you easily find resources that are affected by a certain vulnerability; for example, the recently announced Log4j vulnerability. *Figure 7.33* shows how to filter for resources that are affected by the Log4j vulnerability (CVE-2021-44228).

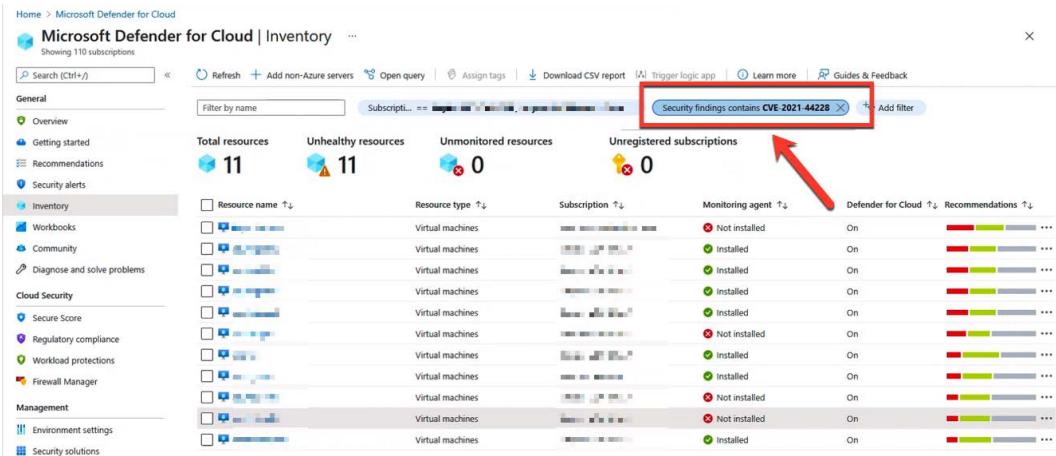


Figure 7.33 – Resources that are affected by a log4j vulnerability

Within this scope, it doesn't make any difference if you use the Qualys or TVM integration. Qualys, however, is great for environments in which another EDR solution is being used. Since TVM relies on MDE, it can only be used when MDE integration has been enabled. On the other hand, TVM brings a unique capability to Defender for Cloud: the **software inventory**. Once enabled, TVM will provide information about installed software and software versions to the Defender for Cloud inventory, which lets you filter the view for installed applications and installed application versions. In addition to that, it will add software information to every machine's resource health blade.

Resource health (Preview) ...

Is this new resource health page helpful to you? ☐ Yes ☐ No

linux-victim
virtual machine

Installed Monitoring agent

9 Active recommendations

44 Active alerts

Resource information

Subscription: Visual Studio Enterprise S...
Resource Group: asdemo

Environment: Azure
Location: westeurope

Operating System: Linux
Status: VM deallocated

Security value

Microsoft Defender for Servers
On

Recommendations Alerts **Installed applications**

Search

Powered by Microsoft Threat and Vulnerability Management

Vendor ↑↓	Software name ↑↓	Version ↑↓	First seen at	Number of known vulnerabilities
Ubuntu	Samba-common For Linux	2:4.7.6+dfsg-ubuntu...	Fri, 24 Sep 2021 02:21:19 GMT	9
Ubuntu	Libwbclient0 For Linux	2:4.7.6+dfsg-ubuntu...	Fri, 24 Sep 2021 02:21:19 GMT	9
Ubuntu	Samba-libs For Linux	2:4.7.6+dfsg-ubuntu...	Fri, 24 Sep 2021 02:21:19 GMT	9
Ubuntu	Python-samba For Linux	2:4.7.6+dfsg-ubuntu...	Fri, 24 Sep 2021 02:21:19 GMT	9
Ubuntu	Samba-common-bin For Linux	2:4.7.6+dfsg-ubuntu...	Fri, 24 Sep 2021 02:21:19 GMT	9
Ubuntu	Linux-image-azure For Linux	5.4.0.1063.43	Fri, 24 Sep 2021 02:21:19 GMT	67

Figure 7.34 – Installed applications with a number of known vulnerabilities on a machine

By clicking on the number of known vulnerabilities, you are presented with a list and details of vulnerabilities that have been found on this particular resource and with corresponding remediation steps.

Home > Microsoft Defender for Cloud > Resource health (Preview) > linux-victim ...

Resource: linux-victim

Total vulnerabilities: 125

Vulnerabilities by severity:

- High: 79
- Medium: 42
- Low: 4

Findings

Search to filter items...

ID	Security check	Category
6YEVJR	Update Ubuntu Libkrb5support0 For Linux	Update
77CT3E	Update Ubuntu Cifs-utils For Linux	Update
ZAMGPS	Update Ubuntu Libavahi-common-data For Linux	Update
GNECP1	Update Ubuntu Libtinfo5 For Linux	Update
AUZOW7	Update Ubuntu Gcc-8-base For Linux	Update
0GZMA6	Update Ubuntu Grub2-common For Linux	Update
6NQWIU	Update Ubuntu Busybox-initramfs For Linux	Update
CCSQ6D	Update Ubuntu Libwbclient0 For Linux	Update
FF3VUM	Update Ubuntu Gdisk For Linux	Update
DC7GO1	Update Ubuntu Dirmngr For Linux	Update

Update Ubuntu Libkrb5support0 For Linux

General information

ID: 6YEVJR
Category: Update
Solution: Microsoft threat and vulnerability management

Remediation

Update Libkrb5support0 For Linux (from Ubuntu) to the latest version.

Weaknesses

Highest severity: High
Highest CVSS Score: 7.5
CVSS Version: 3

CVE ID	Severity
CVE-2018-5710	High
CVE-2021-36222	High
CVE-2018-5709	High
CVE-2021-37750	Medium
CVE-2018-20217	Medium

Figure 7.35 – Vulnerabilities found on a machine

In addition to the integration with MDE and threat detection capabilities, Microsoft Defender for Servers offers additional advanced cloud defense capabilities.

Additional advanced cloud defense in Defender for Servers

Advanced cloud defense presents additional tooling for enhanced threat mitigation. Four additional tools are available:

- Adaptive application controls
- Just-in-time VM access
- Adaptive network hardening
- File integrity monitoring

Let's look at them in detail.

Adaptive application controls

Adaptive application controls enable you to have control over which applications can run on servers protected by Defender for Cloud. This applies both to Azure and non-Azure VMs and servers. We can control what can run on servers by allowing only specific applications or types of applications to run and prevent any malicious or unauthorized software. We can create different groups that will track the file type protection based on location, operating system, and environment type. Servers can be added to multiple groups to track different protection modes. An example of a group to protect a Windows VM in Azure, located in the Europe location, with the audit option for any file, is shown in the following screenshot:

Group settings ✕

New machine group

Create a new custom machine group to monitor with adaptive application control

Subscription ⓘ

Location ⓘ
☐ Global ☒ Europe

Group name * ⓘ

OS type ⓘ
☐ Windows ☒ Linux

Environment type ⓘ
☒ Azure ☐ Non-Azure

File type protection mode
 FILE ☒ Audit ☐ Unconfigured

Add machines to group

✕

☐ Configured machines	↑↓	Group name	↑↓
No results			

✕

☐ Unconfigured machines	↑↓	Group name	↑↓
☐ contoso-linux-vm		REVIEWGROUP1	

Figure 7.36 – Adaptive application control group settings

Azure VM management is another topic that needs to be taken very seriously. A best practice is to perform any management tasks only over a secure connection, using a P2S or S2S connection. An alternative is to use one VM as a jump box, and then perform management from there. But even in this situation, the connection must be secure. The recommended way today is to use Azure Bastion as a native service to connect to Azure VMs.

Just-in-time VM access

Just-in-time access for Azure VM is used to block inbound traffic to VMs until specific traffic is temporarily allowed. This reduces their exposure to attacks by narrowing down the surface and enabling access. Enabling **Just In Time (JIT)** will block inbound traffic on all ports that are usually used for management, such as RDP, SSH, or WinRM. The user must explicitly request access, which will be granted for a period of time, but only for a known IP address. This approach is the same as is used with **Privileged Identity Management (PIM)**, where having rights doesn't necessarily mean that we can use them all the time; we have to activate/request for them to be used for a period of time.

Important Note

JIT access for Azure VMs supports only VMs deployed through **Azure Resource Manager (ARM)**. It's not available for non-Azure VMs or legacy Azure VMs deployed through **Azure Service Management (ASM)**.

JIT can be configured from the Defender for Cloud blade, or from the Azure VM blade. Configuring JIT for an Azure VM requires a few parameters to be defined, such as which ports we want to use, whether we want to allow access from a specific IP address or range in the **Classless Inter-Domain Routing (CIDR)** format, and the maximum period of time for which access will be available.

An example of configuring JIT is shown in the following screenshot:

Home > Microsoft Defender for Cloud > Just-in-time VM access >

JIT VM access configuration

Contoso-Linux-VM

+ Add Save X Discard

Configure the ports for which the just-in-time VM access will be applicable

Port	Protocol	Allowed source IPs	IP range	Time range (hours)
22 (Recommended)	Any	Per request	N/A	3 hours
3389 (Recommended)	Any	Per request	N/A	3 hours
5985 (Recommended)	Any	Per request	N/A	3 hours
5986 (Recommended)	Any	Per request	N/A	3 hours

Add port configuration

Port *

Protocol: Any TCP UDP

Allowed source IPs: Per request CIDR block

IP addresses

Max request time: 3 (hours)

Figure 7.37 – Configuring JIT access

By default, you have the most common management ports available. You can edit rules for these ports, and delete or add custom rules. Besides ports, we can change protocols, the allowed source, and the maximum request time (this can be between 1 and 24 hours). Once JIT is configured, access needs to be requested each time we want to access the VM. This, again, can be done from the Defender for Cloud blade or the Azure VM blade. While you are requesting, you can ask for a specific (or more than one) port to be opened, state whether you want to enable access from your current IP address or from an IP range that's been pre-defined by an administrator, and finally, you need to define the time range. The time range depends on the configuration. By default, the time ranges from 1 to 3 hours, but can be configured for up to 24 hours.

Requesting JIT access is shown in the following screenshot:

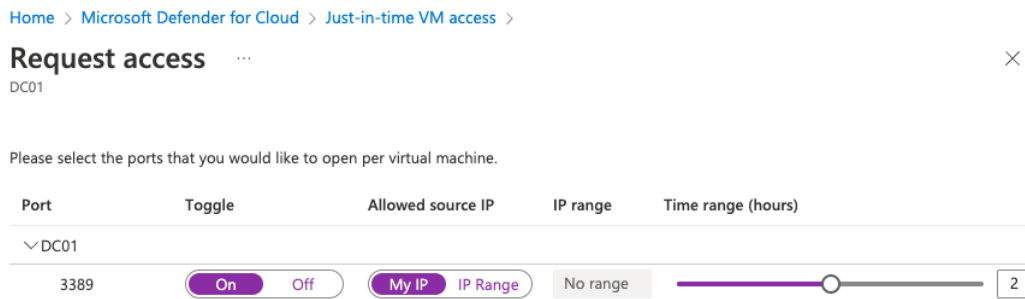


Figure 7.38 – Requesting JIT access

JIT uses NSGs to control traffic that is allowed or blocked. When JIT is configured for an Azure VM, NSG rules are created to block access over configured ports. Ports that are not configured for JIT should be automatically blocked unless configured otherwise. When JIT access is requested, another NSG is temporarily created that will allow access on the requested port. The new NSG rule will have a higher priority and override the block rule to enable access. Once the requested time period expires, the allow rule will be deleted, and access will be blocked again.

Important Note

If JIT is in use, no NSG rules for management ports should be created manually. Azure Security Center should control these ports at all times. If you create rules manually, you may override JIT rules, or you may create a rule that will allow one of the management ports at all times and use JIT for the rest of the management ports. Both situations render JIT pointless.

Advanced network hardening

Advanced network hardening analyzes our traffic communication patterns. An analysis is performed to determine whether NSG rules are overly permissive and create a threat in the form of an increased attack surface. Three potential threats are tracked: a malicious insider, data spillage, and data exfiltration. Azure Security Center reports all threats and assesses the current rules to determine whether changes to NSG rules are required and offers recommendations that will increase security.

File integrity monitoring

File integrity monitoring is used to validate files and registries of the operating system and application software. It tracks changes and compares the current checksum of the file with the last scan of the same file to see whether they are different. Information can be used to determine whether changes are valid or the result of a malicious modification.

Microsoft Defender for Containers

Microsoft Defender for Containers is the latest plan that has been added to Defender for Cloud. It consolidates two deprecated plans (Defender for Container Registries and Defender for Kubernetes) and adds additional capabilities.

Within the scope of Container Registries, Defender for Containers can scan container images for vulnerabilities. Images are scanned when they are pushed to the registry, and regularly once a week after they have recently been pulled (within the last 30 days). In addition, the plan adds threat detection for cluster- and host-level protection.

One of the most interesting additions is the Daemon Set that's being used as a replacement for the Log Analytics agent. Before that, the agent had to be installed on all worker nodes and Defender for Servers had to be enabled to provide host-level detection. With the new plan, the Daemon Set runs inside the Kubernetes cluster so it can also protect clusters that use **virtual machine scale sets (VMSS)**. It also no longer depends on Defender for Servers for host-level detection.

Threat detection summary

Defender for Cloud's threat detection provides information and reports on detected threats. It's very useful to have information on what is happening with our resources and whether anyone is trying anything against us. Information provided by the enhanced security plans can be eye-opening. You might not even be the intended target but still get attacked as a victim of opportunity or due to a moment of carelessness.

To provide an example, we created an experiment where we deployed five Azure VMs and left the RDP port accessible over a private IP address. These were five blank VMs with no data and nothing special about them. What happens is that *bad guys* scan public IP addresses in the hope of detecting open ports. After something like that is detected, a *brute-force* attack will begin. Brute-force attacks use a combination of most common usernames and passwords to try to connect. These five Azure VMs, running for a single month, got hit approximately 20,000 times each. All these attacks can be tracked and seen in Microsoft Defender for Cloud's security alerts view, as demonstrated earlier in this chapter. With that knowledge, it's always a great idea to enable Defender for Cloud's enhanced security plans so you get threat intelligence, vulnerability assessments, and additional cloud defense capabilities. In the end, these plans will provide you with useful information to stop an attack before it actually starts.

Finally, let's take a look at automation capabilities within Defender for Cloud that let you improve security coverage, automate responses, or remediate recommendations at scale in the next section.

Automating security

Microsoft Defender for Cloud comes with several automation and export capabilities that help you further customize your experience and automatically react to alerts or recommendations.

Continuous export

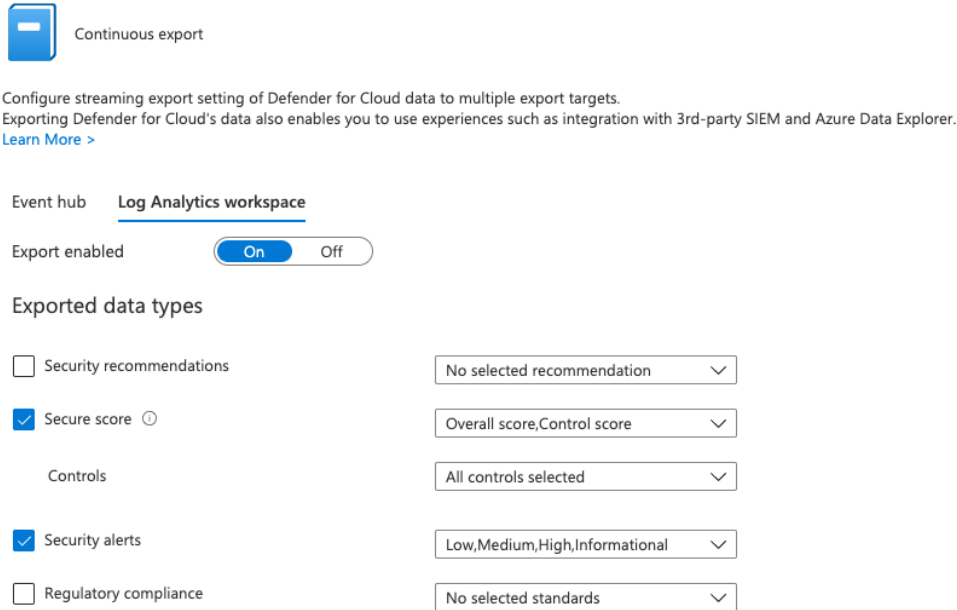
With continuous export, Defender for Cloud offers a capability to export security alerts, recommendations, secure scores, and regulatory compliance assessment results to a Log Analytics workspace, or to an Azure event hub. By exporting this set of information, you are offered a huge set of capabilities. With data exported to an event hub, you can connect Defender for Cloud to a third-party SIEM solution, such as IBM QRadar, Splunk, SumoLogic, ArcSight, and others. By exporting information to a Log Analytics workspace, you can leverage the data to build your own, custom dashboards based on Power BI, or Azure Monitor Workbooks.

Tip

The Defender for Cloud portal will always provide a just-in-time view of your environment. In case you want access to historical information, for example, in a secure score over time report, you can leverage continuous export to have access to that data in a Log Analytics workspace.

To configure continuous export, follow these steps:

1. In Defender for Cloud, select **Environment Settings** and your subscription of choice.
2. Select **Continuous export** in the left navigation pane.
3. In the **Continuous export** view, select **Event hub** or **Log Analytics workspace** as your export target, switch **Export enabled** to **On**, and define **Exported data types**, as shown in *Figure 7.39*:



Continuous export

Configure streaming export setting of Defender for Cloud data to multiple export targets.
Exporting Defender for Cloud's data also enables you to use experiences such as integration with 3rd-party SIEM and Azure Data Explorer.
[Learn More >](#)

Event hub **Log Analytics workspace**

Export enabled **On** Off

Exported data types

☐ Security recommendations	No selected recommendation
☒ Secure score ⓘ	Overall score, Control score
Controls	All controls selected
☒ Security alerts	Low, Medium, High, Informational
☐ Regulatory compliance	No selected standards

Figure 7.39 – Defining continuous export settings

4. As **Export frequency**, you can define **Streaming updates** or **Snapshots**. Streaming updates will export real-time updates whenever there is a change (for example, a new security alert is created, a recommendation's assessment status is changing, or secure score has changed). Snapshots are exported once a week, but they are not available for security alerts. You can use snapshot export to create an over-time view of secure score, recommendations, or regulatory compliance assessments.
5. The export configuration needs to be stored in a resource group within this subscription.
6. The export target can be an event hub or Log Analytics workspace in *any* subscription within your environment.
7. Once you have selected your export target, click the **Save** button on top of this page. Continuous export will now start according to your configuration.

While continuous export is the perfect solution for exporting Defender for Cloud data, either whenever a change happens or based on a weekly schedule, you can also automatically react to these changes when using Workflow automation, a technology that is covered in the next section.

Workflow automation

Workflow automation is a capability that lets you leverage Azure Logic Apps for automating responses to recommendations, regulatory compliance assessments, or security alerts.

Azure Logic Apps is a cloud service that helps you to schedule, automate, and orchestrate different tasks, processes, and workflows. It also offers the ability to integrate with different applications, datasets, systems, and services across organizations. This makes it very useful in a number of scenarios, whether you want to create a repetitive automated task, trigger an action whenever a condition happens, or connect different systems.

Logic apps have built-in trigger types to automatically react to alerts, changes in recommendations, or changes in regulatory compliance assessments, as shown in *Figure 7.40*:

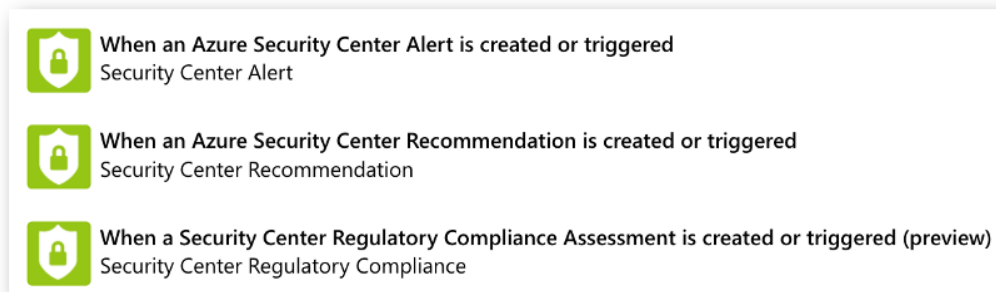


Figure 7.40 – Microsoft Defender for Cloud Logic apps trigger types

In the end, workflow automation is the technology to connect Logic apps to Defender for Cloud. In case you want to automatically react to a particular alert type, you first need to create a Logic app that uses the **When an Azure Security Center Alert is created or triggered** Logic app trigger. After connecting the Logic app to Defender for Cloud whenever a new alert is created, it is evaluated. Based on some pre-filter configuration, the alert information is then sent to the Logic app. The Logic app in the end will do whatever it has been created for.

To create a workflow automation, you need to do the following:

1. Create a Logic app with one of the supported trigger types. In this example, we are using a Logic app that reacts to security alerts.
2. In Microsoft Defender for Cloud, select **Workflow automation** in the left navigation pane.
3. Click + **Add workflow automation**.
4. In the **Add workflow automation** dialog, make sure to add all mandatory information, as shown in *Figure 7.41*:

Add workflow automation

General

Name *

MasteringAzSec-WFA

Description

This workflow automation will connect a Logic App to Defender for Cloud.

Subscription

Contoso

Resource group *

block-bruteforce

Trigger conditions

Choose the trigger conditions that will automatically trigger the configured action.

Defender for Cloud data type *

Security alert

Alert name contains

brute

Alert severity *

All severities selected

Actions

Configure the Logic App that will be triggered.
Choose an existing Logic App or [visit the Logic Apps page](#) to create a new one

Show Logic App instances from the following subscriptions *

4 selected

Logic App name

Block-BruteForceAttack (Security Center alerts connector)

[Refresh](#) [View logic app](#)

Figure 7.41 – Adding a workflow automation to Defender for Cloud

5. In the **General** section, you need to define which subscription and resource group to create the workflow automation in. This is the target for the configuration, not the Logic app you want to connect. Make sure to create one workflow automation for each subscription you want to use this automation in. The Logic app itself only needs to exist once as it will just be connected to Defender for Cloud by using one workflow automation configuration per subscription.
6. In the **Trigger conditions** section, make sure to select the correct **Defender for Cloud** data type. Depending on your selection, the other drop-down fields might change slightly. In this example, make sure to use the **Security alert** data type, and select the severity. Once you have created a workflow automation without using the **Alert name contains** filter, your automation will be triggered whenever any new alert is created. To avoid unnecessary Logic app runs, you can filter by alert name. In this example, using the filter word `brute` will make sure that the Logic app is only triggered by one of the various brute-force attack alerts within Microsoft Defender for Cloud.

Tip

You can find a list of all Microsoft Defender for Cloud security alerts in the Defender for Cloud alerts reference at <https://aka.ms/DefenderForCloudAlerts>.

7. In the **Actions** section, you can select the Logic app you want to connect. Make sure your Logic app uses the trigger that you've configured before. In this example, the Logic app uses the Security Center alerts connector.
8. Click **Create**. Once the workflow automation has been created, the Logic app will be triggered whenever a new alert is created and when it aligns with the filter conditions in your configuration.

Logic apps that are created with one of the built-in Defender for Cloud trigger types cannot only be used within the scope of workflow automation; you can also trigger them manually from alert or recommendation details. Every alert or recommendation comes with a **Trigger logic app** button. For recommendations, you need to select one or several affected resources and then click the button, as shown in *Figure 7.42*:

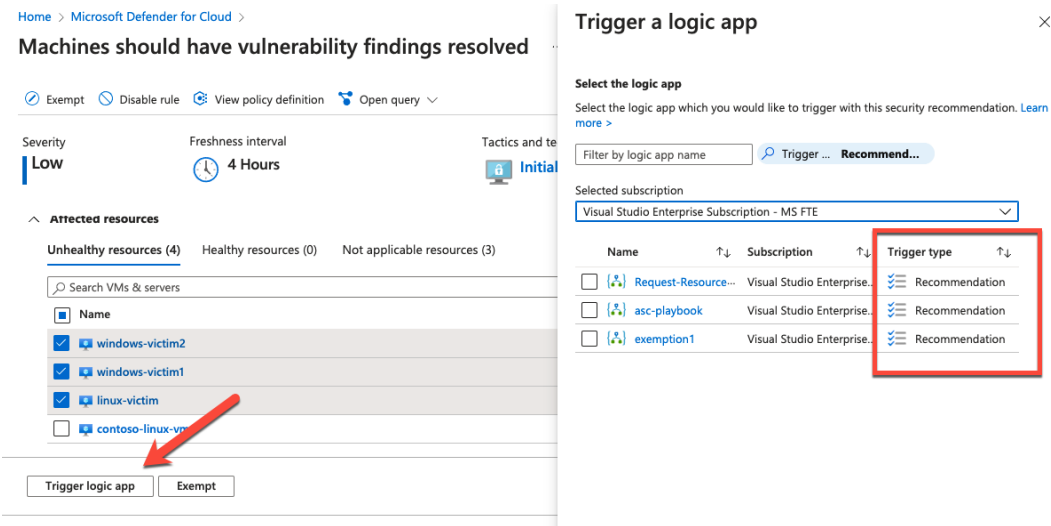


Figure 7.42 – Triggering a logic app from a recommendation's details

Once you click the **Trigger logic app** button, you can see that in the dialog that appears, you can only select Logic apps that use the **Recommendations** trigger. If you click the button from alert details, it will only show Logic apps with the alert trigger type.

Tip

Microsoft has published a sample automation to automatically block an incoming brute-force attack by creating a deny rule in a VM's NSG. You can find and deploy this Logic app at <https://aka.ms/BlockBruteForce>.

REST APIs

All Microsoft Defender for Cloud data is exposed by a REST API endpoint in the `Microsoft.Security` API. You can use all of its information in Logic apps, but also in your own applications.

Tip

You can find all Defender for Cloud REST API endpoints in the API reference at <https://aka.ms/DefenderForCloudAPIRef>.

Multi-cloud capabilities in Microsoft Defender for Cloud

As the new product name indicates, Microsoft Defender for Cloud is no longer an Azure-focused security solution. It's rather a CSPM and CWPP for both, multi- and hybrid cloud environments. At Microsoft Ignite in November 2021, Microsoft announced its new API-based multi-cloud CSPM for AWS. As of now, you no longer need to enable AWS Security Hub in order to connect your resources to Defender for Cloud. By integrating AWS in the native Defender for Cloud experience, it is now very easy to connect your resources from there and get recommendations and visibility into their security posture in the same, unified experience. In addition, you can opt in to use Microsoft Defender for Servers and/or Defender for Containers on your AWS resources, too.

The screenshot shows the 'Microsoft Defender for Cloud | Environment settings' page. The left sidebar contains navigation links for General, Overview, Getting started, Recommendations, Security alerts, Inventory, Workbooks, Community, Diagnose and solve problems, Cloud Security, Secure Score, Regulatory compliance, Workload protections, Firewall Manager, and Management. The main content area shows '4 Azure subscriptions' and '2 AWS accounts'. Below this, a table lists environments with columns for Name, Total resources, Defender coverage, and Standards. The table includes entries for Azure and AWS (preview). The AWS section is highlighted with a red box, showing two connectors: 'AWSNinjaConnector' and 'securityConnector'. The 'securityConnector' is also highlighted with a red box.

Name	Total resources	Defender coverage	Standards
Azure			
> [a] 72f988bf-86f1-41af-91ab-2d7cd011...	6735		Limited permissions
> [a] 4b2462a4-bbee-495a-a0e1-f23ae52...	904		Limited permissions
AWS (preview)			
AWSNinjaConnector	259	3/3 plans	AWS CIS 1.2.0 (preview), AWS Foundational ...
securityConnector	1694	3/3 plans	AWS CIS 1.2.0 (preview), AWS Foundational ...

Figure 7.43 – AWS connectors in Defender for Cloud's environment settings view

Once you have connected an AWS account to Defender for Cloud, assessments will start, based on the AWS CIS 1.2.0 and AWS Foundational Security Best Practices compliance standards. Recommendations for AWS resources will appear in the recommendations view, and they will also affect your secure score. You can even filter for recommendations that apply to your AWS resources by using the **Environment** filter in the recommendations view. Just make sure you select **AWS (preview)** only, as shown in Figure 7.44:

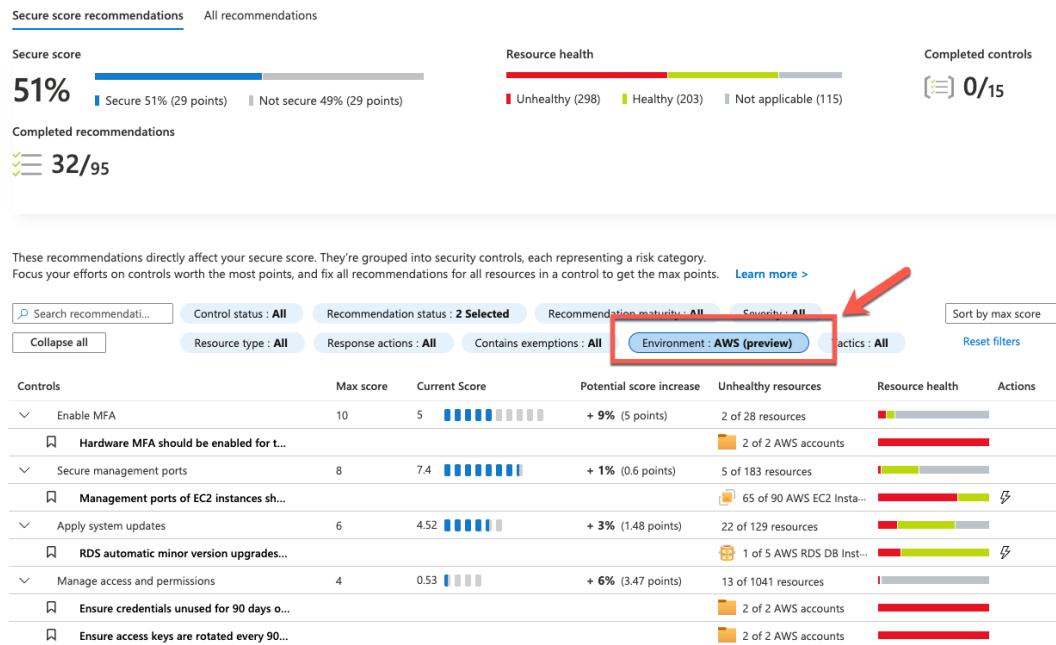


Figure 7.44 – Filter for recommendations that apply to your AWS resources

For hybrid and multi-cloud machines (VMs, on-premises servers, and so on), Defender for Cloud leverages Azure Arc for making non-Azure resources *Azure-like*. Once a machine has been onboarded to Azure Arc, Defender for Cloud can use Azure-native capabilities to install agents, manage settings, and much more besides. From a Defender for Cloud perspective, it makes no difference if the machine is natively running on Azure, or if it has just been *connected*. All agents that are used within the scope of Defender for Cloud can be installed as Arc extensions, and so Defender for Servers, Containers, and SQL on machines can treat these machines as if they were part of the Azure environment.

Summary

Microsoft Defender for Cloud helps us to keep our hybrid and multi-cloud environment safe in various ways, from offering recommendations on what needs to be improved to detecting threats as they happen, to response automation to act on possible threats. With different settings and policies, we can define our focus, track the health of our resources, and create a more secure infrastructure.

In this chapter, we discussed how important Azure Security Center as a CSPM and CWPP tool is. The next chapter will focus on Microsoft Sentinel, which is Microsoft's cloud-based **Security Information and Event Management (SIEM)** and **Security Orchestration, Automation, and Response (SOAR)** solution.

Questions

1. Where is Microsoft Defender for Cloud able to store data?
 - A. Azure Storage
 - B. Azure SQL Database
 - C. Log Analytics Workspace
2. Information about how to improve your resources' security posture is provided as?
 - A. Recommendations
 - B. Alerts
 - C. Issues
 - D. Fixes
3. How can you automate responses to Defender for Cloud alerts?
 - A. Azure Monitor alerts
 - B. Logic apps
 - C. PowerShell
 - D. Azure CLI

4. When **JIT VM Access** is enabled on a machine, the user...?
 - A. Has the same level of access rights
 - B. Has fewer access rights
 - C. Has more access rights
 - D. Has to request access
 - E. Both A and C
5. With enhanced security plans enabled, what happens when an attack occurs?
 - A. It will be automatically blocked.
 - B. The user can create an automated response to an attack.
 - C. A security alert is automatically created.
 - D. Both B and C.
6. What are the main capabilities in Defender for Cloud?
 - A. **Security Information and Event Management (SIEM)**
 - B. **Cloud Security Posture Management (CSPM)**
 - C. **Security Orchestration, Automation, and Response (SOAR)**
 - D. **Cloud Workload Protection (CWP)**
 - E. Both B and D

8

Microsoft Sentinel

Security Information and Event Management (SIEM) combines two solutions that were previously separate, **Security Information Management (SIM)** and **Security Event Management (SEM)**.

We have already mentioned that large organizations rely on SIEM solutions. And Microsoft's SIEM solution for the cloud is Microsoft Sentinel. But let's first take a step back and discuss what SIEM is and what functionalities it should have.

We will be covering the following topics in this chapter:

- Introduction to SIEM
- Getting started with Microsoft Sentinel
- Creating workbooks
- Using threat hunting and notebooks

Introduction to SIEM

Many security compliance standards require long-term storage, where security-related logs should be kept for long periods of time. This varies from one compliance standard to another and can be up to many years. This is where SIM comes into the picture: long-term storage where all security-related logs are stored for analysis and reports.

When we speak of SEM, we tend to be talking about live data streaming rather than long-term event tracking. SEM's focus is on real-time monitoring; it aims to correlate events using notifications and dashboards. When we combine these two, we have SIEM, which tries to live stream all security-related logs and keep them for the long term. With this approach, we have a real-time monitoring and reporting tool in one solution.

When discussing the functionalities required, we have a few checkboxes that SIEM must tick:

- **Data aggregation:** Logs across different systems are kept in a single place. This can include network, application, server, and database logs, to name a few.
- **Dashboards:** All aggregated data can be used to create charts. These charts can help us to visually detect patterns or anomalies.
- **Alerts:** Aggregated data is analyzed automatically, and any detected anomaly becomes an alert. Alerts are then sent to individuals or teams that must be aware of them, or act on them.
- **Correlation:** One of SIEM's responsibilities is to provide a meaningful connection between common attributes and events. This is also where data aggregation comes in because it helps to identify connected events across different log types. A single line in a database log may not mean much, but if it's combined with logs from the network and the application, it can help us prevent a disaster.
- **Retention:** As mentioned, one of the compliance requirements is to keep data for extended periods of time. But this also helps us to establish patterns over long periods and detect anomalies more easily.
- **Forensic analysis:** Once we are aware of a security issue, SIEM is used to analyze events and detect how and why the issue happened. This helps us to neutralize damage and prevent the issue from repeating.

To summarize, SIEM should have the ability to receive different data types in real time, provide meaning to received data, and store it for an extended period of time. Received data is then analyzed to find patterns, detect anomalies, and help us prevent or stop security incidents.

So, let's see how Microsoft Sentinel addresses these requirements.

Getting started with Microsoft Sentinel

Microsoft Sentinel is Microsoft's SIEM solution in the cloud. As cloud computing continues to revolutionize how we do IT, SIEM must evolve to address the new challenges that the changes to IT create. Microsoft Sentinel is a scalable cloud solution that offers intelligent security and threat analytics. On top of that, Microsoft Sentinel provides threat visibility and alerting, along with proactive threat hunting and responses.

So, if we look carefully, all the checkboxes for SIEM are ticked.

The pricing model for Microsoft Sentinel comes with two options:

- **Pay-As-You-Go:** In Pay-As-You-Go, billing is done per GB of data ingested to Microsoft Sentinel.

Important Note

At the time of writing, the price in the Pay-As-You-Go model was \$2.60 per ingested GB.

- **Capacity reservation:** Capacity reservation offers different tiers with varying amounts of data reserved. Reservation creates a commitment and billing is done per tier, even if we don't use the reserved capacity. However, reservation provides a discount on ingested data and is a good option for organizations that expect large amounts of data.

The following diagram shows the capacity reservation prices for Microsoft Sentinel at the time of writing:

Tier	Price	Effective Per GB Price ¹	Savings Over Pay-As-You-Go
100 GB per day	\$123 per day	\$1.23 per GB	50%
200 GB per day	\$222 per day	\$1.11 per GB	55%
300 GB per day	\$320 per day	\$1.07 per GB	57%
400 GB per day	\$410 per day	\$1.03 per GB	58%
500 GB per day	\$492 per day	\$0.99 per GB	60%
1,000 GB per day	\$960 per day	\$0.96 per GB	61%
2,000 GB per day	\$1,821 per day	\$0.92 per GB	63%
5,000 GB per day	\$4,305 per day	\$0.87 per GB	65%

Figure 8.1 – Microsoft Sentinel pricing

If ingested data exceeds the reservation limit, further billing is done based on the Pay-As-You-Go model. For example, if we have a reserved capacity of 100 GB and we ingest 112 GB, we will pay the tier price for the reserved capacity up to 100 GB and will also pay for the additional 12 GB for the data that exceeds the reservation.

It's interesting to reflect on pricing presented in the first edition of this book and how it changed in 2 years. The price decreased in the region of 5%, but I find capacity options more interesting. 2 years ago, the maximum capacity was 500 GB per day, while today we have a capacity of up to 5,000 GB per day. This perfectly paints the picture of constant data growth and how data is becoming bigger and bigger right before our eyes.

When enabling Microsoft Sentinel, we need to define the Log Analytics workspace that will be used for storing data. We can either create a new Log Analytics workspace or use an existing one.

Microsoft Sentinel uses Log Analytics for storing data. The price for Microsoft Sentinel does not include charges for Log Analytics.

Important Note

An additional charge for Log Analytics will be incurred for ingested data. Information about pricing can be found at <https://azure.microsoft.com/en-us/pricing/details/monitor/>.

In the following screenshot, we can see all the pricing options for Microsoft Sentinel at the time of writing:

✓ 100 GB/day	50% discount over Pay-as-you-go
✓ 200 GB/day	55% discount over Pay-as-you-go
✓ 300 GB/day	57% discount over Pay-as-you-go
✓ 400 GB/day	58% discount over Pay-as-you-go
✓ 500 GB/day	60% discount over Pay-as-you-go
✓ 1 TB/day	61% discount over Pay-as-you-go
✓ 2 TB/day	63% discount over Pay-as-you-go
✓ 5 TB/day	65% discount over Pay-as-you-go
✓ Pay-as-you-go	Per GB

Current tier

Figure 8.2 – Changing the Microsoft Sentinel pricing tier

Data retention is configured in Log Analytics Workspace. Log Analytics Workspace default retention is 30 days, but it can be increased to a period of up to 2 years. However, keep in mind that changing the retention period will create additional costs as this can increase the amount of data significantly.

Important Note

Microsoft Sentinel customers can increase data retention to 90 days for free. This only applies to a Log Analytics workspace connected to Microsoft Sentinel.

Now, let's see how Microsoft Sentinel does everything that SIEM should be able to do. We will analyze all the requirements one by one and see how they are satisfied by Microsoft Sentinel.

Let's start with data connectors and retention.

Configuring data connectors and retention

One of SIEM's requirements is data aggregation. However, data aggregation doesn't just mean collecting data, but also the ability to collect data from multiple sources. Microsoft Sentinel does this really well and has many integrated connectors. We can see how much attention Microsoft is paying to this part when comparing the number of available data connectors between the two editions of this book. In the first edition, it was mentioned that 32 data connectors were available. Today, we have 120 of them, an increase of over 350% in 2 years. A lot of the connectors are for different Microsoft sources, such as Azure Active Directory, Azure Active Directory Identity Protection, Microsoft Defender for Cloud, Microsoft Cloud App Security, or Office 365, to name but a few. However, there is also a number of connectors for data sources outside the Microsoft ecosystem, such as Amazon Web Services, Barracuda Firewalls, Cisco, Citrix, and Palo Alto Networks.

Important Note

For more information on connectors, you can refer to the following link:
<https://docs.microsoft.com/en-us/azure/sentinel/connect-data-sources>.

The data connectors page is shown in the following screenshot:

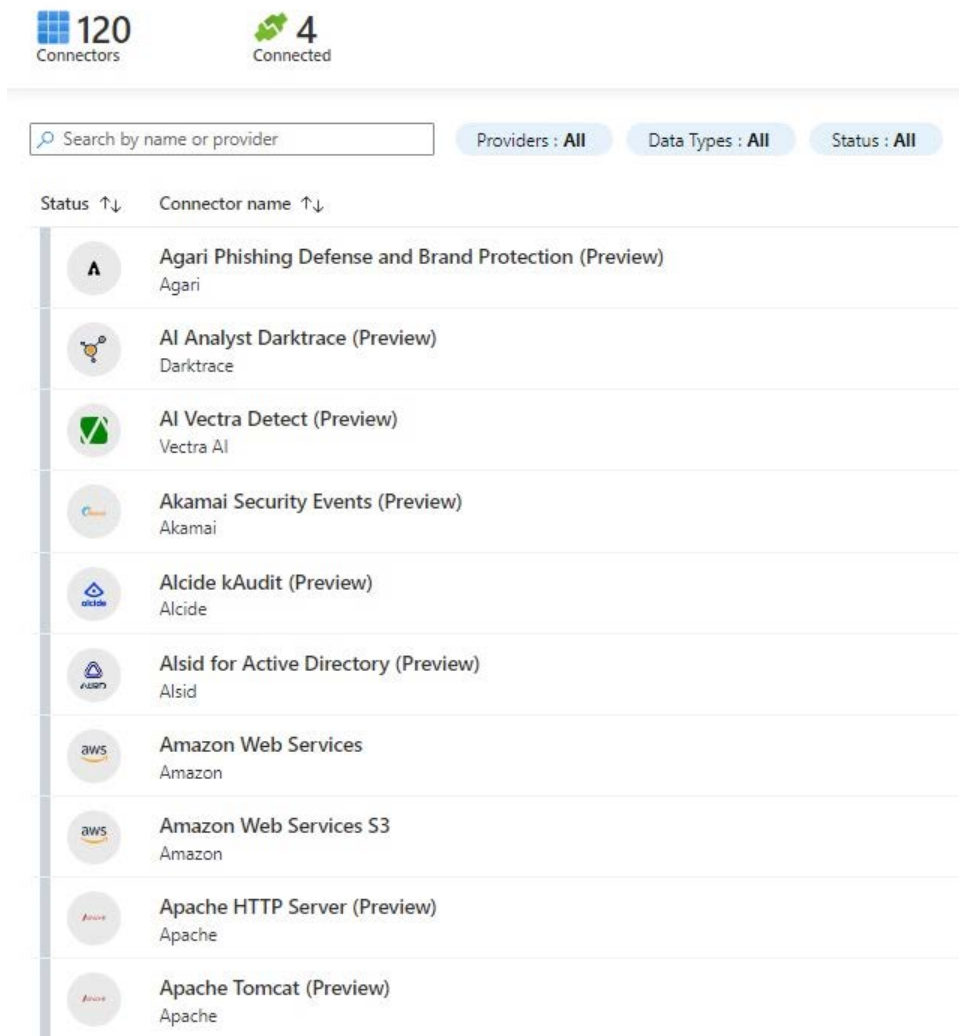


Figure 8.3 – Microsoft Sentinel data connectors

All data connectors include step-by-step instructions explaining how to configure data sources to send data. It's important to mention that all the data is stored in Log Analytics. Instructions vary from data source to data source. Most Microsoft data sources can be added by just enabling a connection from one service to another. Other data sources require the installation of an agent or editing of the endpoint's configuration.

There are many default connectors in Microsoft Sentinel. Besides the obvious Microsoft connectors with services in Azure and Office 365, we have many connectors for on-premises services as well. But it doesn't stop there, and many other connectors are available, including Amazon Web Services, Barracuda, Cisco, Palo Alto, F5, and Symantec.

Once the data is imported into Log Analytics and is ready to be used in Microsoft Sentinel, that is when the real work starts.

Working with Microsoft Sentinel dashboards

After the data is gathered, the next step is to display data using various dashboards. Dashboards visually present data using **Key Performance Indicators (KPIs)**, metrics, and key data points in order to monitor security events. Presenting data visually helps set up baseline patterns and detect anomalies.

The following screenshot shows events and alerts over time:

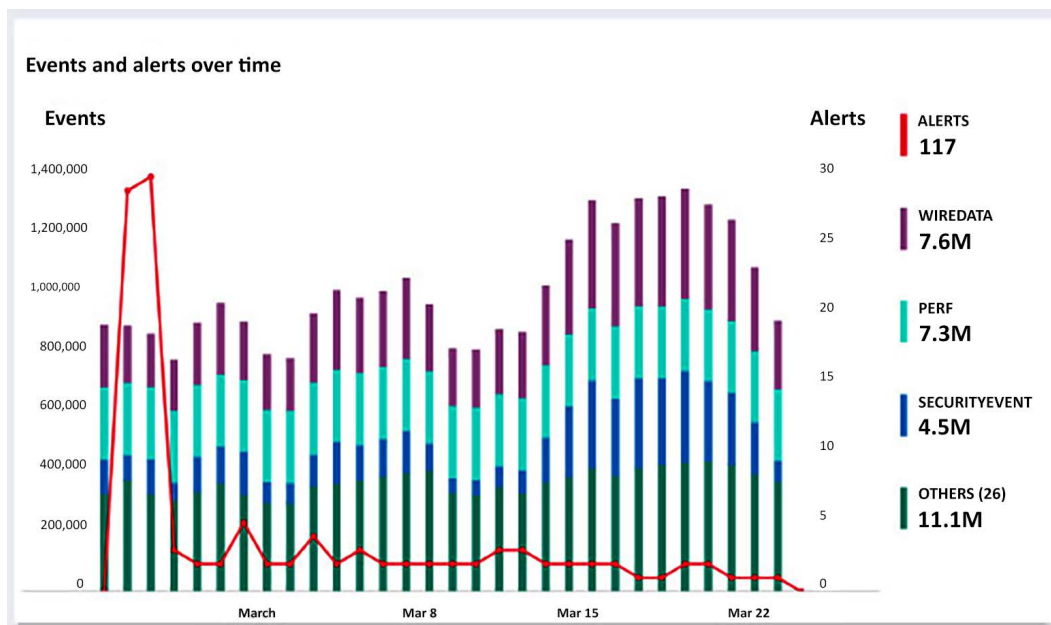


Figure 8.4 – Events and alerts dashboard

In this screenshot, we can see how the baseline is established. There is a similar number of events over time. Any sudden increase or decrease would be an anomaly that we would need to investigate.

The **Events and alerts over time** dashboard uses metrics to display data, but we can also use KPIs to create different types of dashboards. The following diagram shows anomalies in the data source:

Data source anomalies

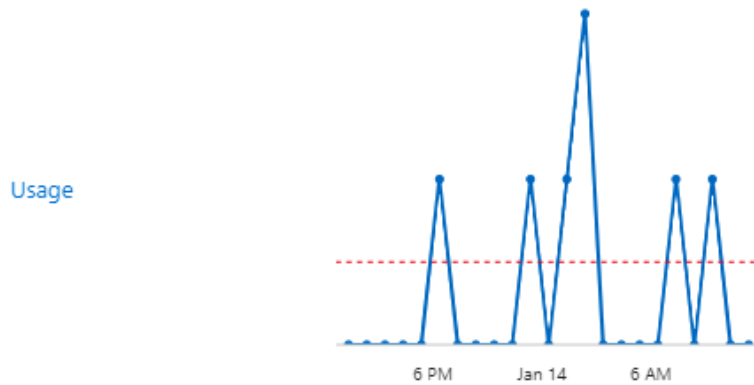


Figure 8.5 – Anomalies dashboard

These two examples represent default dashboards that are available when Microsoft Sentinel is enabled. We can also create custom dashboards, based on the requirements and the KPIs defined.

However, this is only the first step in detecting something that's out of the ordinary. We need additional steps to automate the process.

Setting up rules and alerts

The only problem with dashboards is that they are useful if someone is watching them. Data is displayed visually, and we can detect issues only if we are monitoring dashboards at all times. But what happens when no one is watching? For these situations, we define rules and alerts.

Using rules, we can define a baseline and send notifications (or even automate responses) when any type of anomaly appears. In Microsoft Sentinel, we can create custom rules on the **Analytics** blade, with three types of rules available:

- The first type of rule is the **Microsoft incident creation rule**, where we can select from a list of predefined analytic rules. The rules here are **Microsoft Cloud App Security**, **Microsoft Defender for Cloud**, **Azure Advanced Threat Protection**, **Azure Active Directory Identity Protection**, and **Microsoft Defender Advanced Threat Protection**. The only other option is to select the severity of the incident that will be tracked.

- The second type of rule is **Scheduled query rule**. We have more options here, and we can define rules that track basically anything. The only limitation we have here is our data. The more data we have, the more things we can track. Using **Kusto Query Language**, we can create custom rules and check for any type of information as long as the data is already in the Log Analytics workspace.
- The third type of rule is **NRT query rule**. NRT stands for **Near Real Time** and the main difference between NRT and Scheduled is in the time delay. Where Scheduled rules have a time delay of up to 5 minutes, the NRT delay is 2 minutes. This is the type of rule we want to use when time is of the essence, and we want to detect an anomaly as soon as possible.

To create a custom query rule, the following steps are required:

1. We need to define a name, select the tactics, and set the severity we want to detect. There are several tactics options we can choose from: Initial Access, Execution, Persistence, Privileged Escalation, Defense Evasion, Credential Access, Discovery, Lateral Movement, Collection, Exfiltration, Command and Control, and Impact.

Optionally, we can add a description. Adding a description is recommended because it can help us track down rules and detect their purpose. An example is shown in the following screenshot:

Analytics rule details

The screenshot shows the 'Analytics rule details' form with the following fields:

- Name ***: A text input field containing 'SignINLogs' with a green checkmark icon on the right.
- Description**: A text input field containing 'Respurces on which an Account was logged on during a given time period' with a green checkmark icon on the right.
- Tactics**: A dropdown menu showing '0 selected' with a downward arrow.
- Severity**: A dropdown menu showing 'Medium' with a downward arrow.
- Status**: Two buttons, 'Enabled' (highlighted in blue) and 'Disabled'.

Figure 8.6 – Creating a new analytics rule

- Next, we need to set the rule's logic. In this part, we need to set up the rule query using Kusto Query Language. The query will be executed against data in Log Analytics in order to detect any events that represent a threat or an issue. The query needs to be correct because the syntax will be checked. A failed syntax check will result in the query failing to proceed. An example of a query in a custom rule is shown in the following screenshot:

Rule query

Any time details set here will be within the scope defined below in the Query scheduling fields.

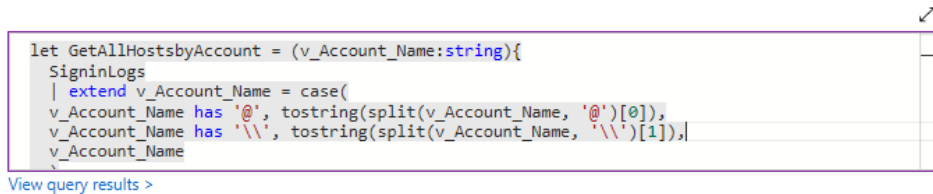


Figure 8.7 – Defining an analytic rule query

The query used in this example is tracking resources on the account that were logged in the last 24 hours, which you can see here:

```
let GetAllHostsbyAccount = (v_Account_Name:string){
    SigninLogs
    | extend v_Account_Name = case(
        v_Account_Name has '@', tostring(split(v_Account_Name,
        '@')[0]),
        v_Account_Name has '\\', tostring(split(v_Account_Name,
        '\\')[1]),
        v_Account_Name
    )
    | where UserPrincipalName contains v_Account_Name
    | extend RemoteHost =
        tolower(tostring(parsejson(DeviceDetail.
        ['displayName'])))
    | extend OS = DeviceDetail.operatingSystem, Browser =
        DeviceDetail.browser
    | extend StatusCode = tostring(Status.errorCode),
        StatusDetails = tostring(Status.additionalDetails)
    | extend State = tostring(LocationDetails.state), City
        = tostring(LocationDetails.city)
    | extend info = pack('UserDisplayName',
        UserDisplayName, 'UserPrincipalName', UserPrincipalName,
```

```

'AppDisplayName', AppDisplayName, 'ClientAppUsed',
ClientAppUsed, 'Browser', tostring(Browser), 'IPAddress',
IPAddress, 'ResultType', ResultType, 'ResultDescription',
ResultDescription, 'Location', Location, 'State', State,
'City', City, 'StatusCode', StatusCode, 'StatusDetails',
StatusDetails)

| summarize min(TimeGenerated), max(TimeGenerated),
Host_Aux_info = makeset(info) by RemoteHost, tostring(OS)

| project min_TimeGenerated, max_TimeGenerated,
RemoteHost, OS, Host_Aux_info

| top 10 by min_TimeGenerated desc nulls last

| project-rename Host_UnstructuredName=RemoteHost,
Host_OSVersion=OS

};

// change <Name> value below
GetAllHostsbyAccount('<Name>')

```

3. We can test this query either by selecting **View Query Results** (which will take us to **Log Analytics Workspace**) or by selecting **Test with current data** in the **Results simulation** part of the blade, as shown in the following screenshot:

Results simulation

This chart shows the results of the last 50 evaluations of the defined analytics rule. Click a point on the chart to display the raw events for that point in time.

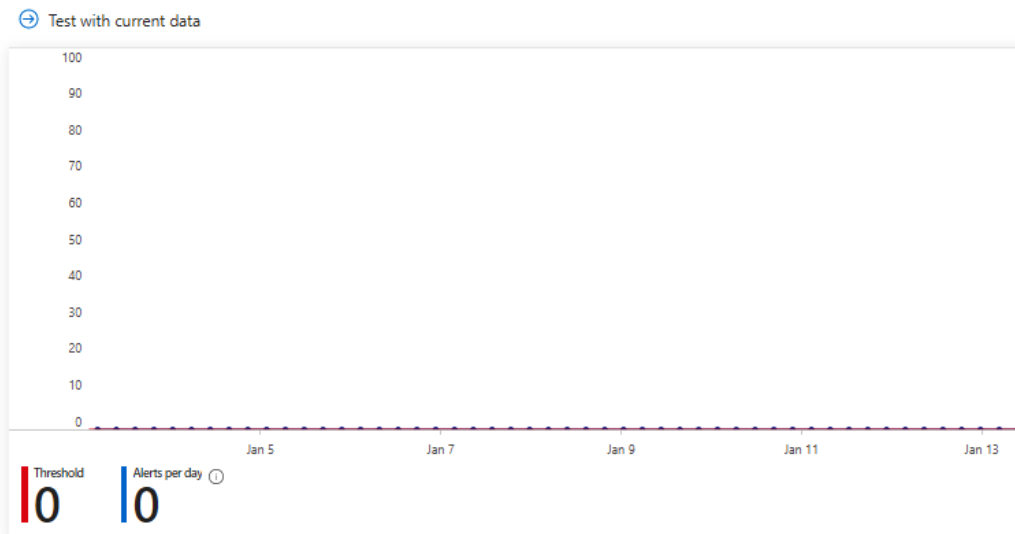


Figure 8.8 – Testing query results

4. Once the query is defined, we need to create the schedule. The schedule defines how often the query is executed and the data that it's executed against. We also define the threshold for when an event becomes an alert. For example, a failed login attempt is just an event. But if this event repeats over time, then it becomes an alert.

The following screenshot shows an example of a schedule and a threshold:

Alert enrichment

- ✓ Entity mapping
- ✓ Custom details
- ✓ Alert details

Query scheduling

Run query every *

15 ✓ Minutes ✓

Lookup data from the last * ⓘ

1 ✓ Hours ✓

Alert threshold

Generate alert when number of query results *

Is greater than ✓ 3 ✓

Event grouping

Configure how rule query results are grouped into alerts

☒ Group all events into a single alert

☐ Trigger an alert for each event (preview)

Suppression

Stop running query after alert is generated ⓘ

On Off

Figure 8.9 – Analytic rule scheduling

Several optional settings allow us to enhance alerts with entity mapping, custom details, or alert details. We can also select event grouping and choose whether we want a query to stop running if an alert is generated.

5. Next, we move to **Incident settings** (in preview at the time of writing) where we can choose to create incident form alerts and set alert grouping, as shown in the following screenshot:

Incident settings

Microsoft Sentinel alerts can be grouped together into an Incident that should be looked into. You can set whether the alerts that are triggered by this analytics rule should generate incidents.

Create incidents from alerts triggered by this analytics rule

☒ Enabled ☐ Disabled

Alert grouping

Set how the alerts that are triggered by this analytics rule, are grouped into incidents. Grouping alerts into incidents provides the context you need to respond and reduces the noise from single alerts.

Group related alerts, triggered by this analytics rule, into incidents

☒ Enabled ☐ Disabled

Limit the group to alerts created within the selected time frame

5

Hours

Group alerts triggered by this analytics rule into a single incident by

☒ Grouping alerts into a single incident if all the entities match (recommended)

☐ Grouping all alerts triggered by this rule into a single incident

☐ Grouping alerts into a single incident if the selected entity types and details match:

Select entities



Select details



Entity-based alert grouping can make use **only** of entities mapped using the new version, if any exist. Entities mapped with the old version (that appear in the query code) will be available for grouping **only** if there are no mappings defined using the new version.

Re-open closed matching incidents

☐ Enabled ☒ Disabled

Figure 8.10 – Analytic rule incident settings

6. In the last step, we can define what will happen when the alert is triggered. Similar to workflow automation in Microsoft Defender for Cloud (in *Chapter 7, Microsoft Defender for Cloud*), Logic apps are used to create automated responses. These responses can either be notifications (to users or groups of users) or automated responses that will react to stop or prevent the threat.

Alert automation

Select a playbook to run when a new alert is generated from this analytics rule. The playbook will receive the alert as its input. Only playbooks configured with the alert trigger can be selected.

0 selected ▾

Name

No playbooks selected

Incident automation (preview)

View all automation rules that will be triggered by this analytics rule and create new automation rules. The automation rule will receive the incident as its input, as will any playbooks called by the automation rule. Only playbooks configured with the incident trigger can be called by automation rules.

+ Add new

Order	Automation rule name
No automation rules	

Figure 8.11 – Automated response configuration

To have an automated response, we need to create playbooks. Let's take a look at how playbooks are created in Microsoft Sentinel.

Microsoft Sentinel automation

Under **Automation** in Microsoft Sentinel, we create playbooks that can provide an automated response if an alert is triggered. Responses can vary from blocking and preventing threats to sending notifications, depending on the severity.

We can either create a playbook from scratch or select an existing template. Let's take a look at the templates and select **Block AAD user – Alert**, as in the following example:

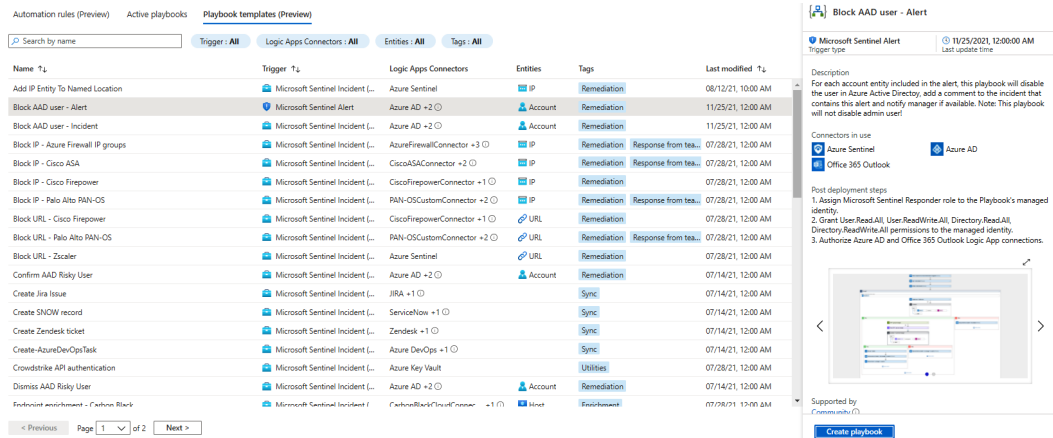


Figure 8.12 – Playbook templates

If we select a template and choose to create a playbook, it will take us to a new blade. We need to select a subscription, resource group, region, and playbook name. An example is shown in the following screenshot:

1 Basics 2 Connections 3 Review and create

Select the subscription to manage deployed resources and costs. Use resource groups like folders organize and manage all your resources.

Subscription *

Resource group * [Create new](#)

Region *

Playbook name *

☐ Enable diagnostics logs in Log Analytics

Log Analytics workspace

☐ Associate with integration service environment

Integration service environment

Figure 8.13 – Automated response

Optionally, we can select to send diagnostics logs to Log Analytics and configure different integrations.

Next, we can see connectors that will be created for the playbook in question, as shown:

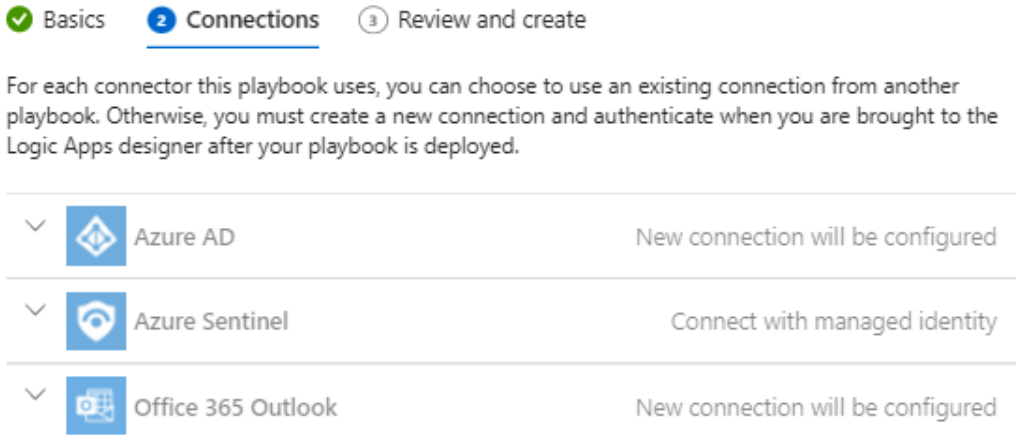


Figure 8.14 – Playbook connectors

Finally, we review the settings and create a playbook.

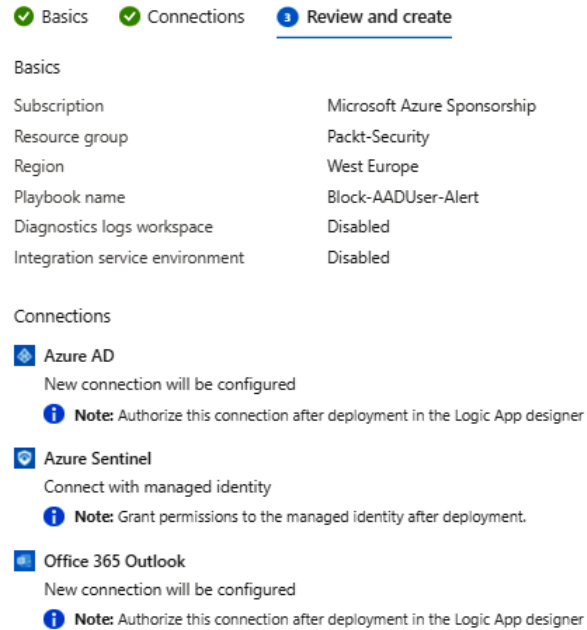


Figure 8.15 – Reviewing and creating a playbook

Note that the connection created needs to be authorized once deployment is done. And now we have a way to automatically respond to threats and incidents.

But in modern cybersecurity, this may not be enough. Let's look at other options in Microsoft Sentinel.

Creating workbooks

In Microsoft Sentinel, we can use workbooks to define what we want to monitor and how we do it. Similar to alert rules, we have the option to use predefined templates or to create custom alerts. In contrast with alert rules, with workbooks, we create dashboards to monitor data in real time.

When the first edition of this book came out, there were 39 connectors available. This is another indicator of how fast Microsoft Sentinel is developing. At this moment, there are 114 templates available, and this list is very similar to the list of data connectors. Basically, there is at least one workbook template for each data connector. We can choose any template for the list displayed in the following screenshot:

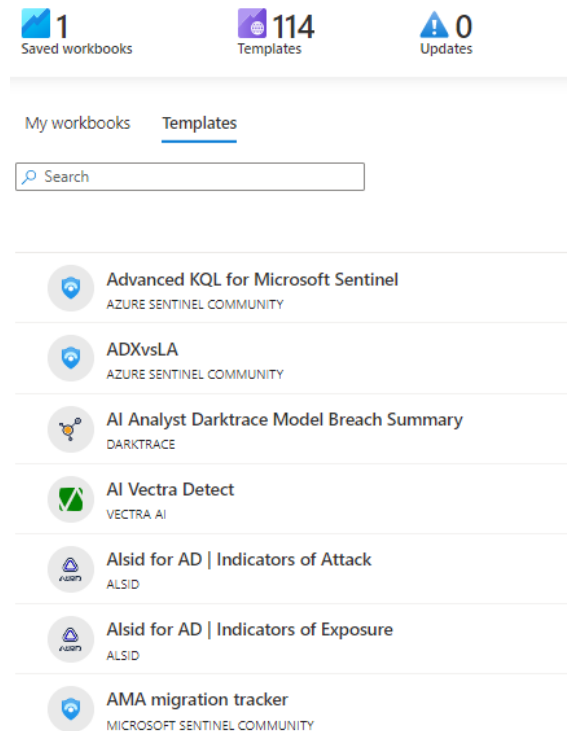


Figure 8.16 – Microsoft Sentinel workbook templates

Each template will enable an additional dashboard that is customized to monitor a certain data source. In the following screenshot, we can see the dashboard for Azure activities:

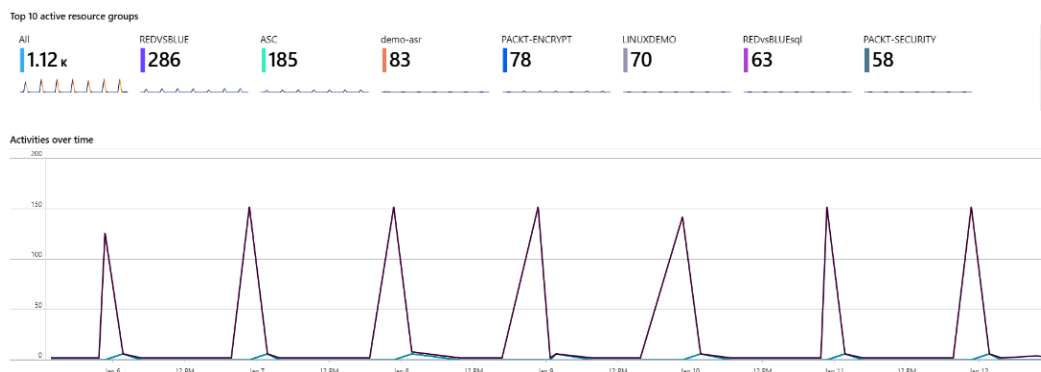


Figure 8.17 – Azure Activity template

The story doesn't end here. With Microsoft Sentinel, we can leverage machine learning and embed intelligence in our security layer.

Using threat hunting and notebooks

In Microsoft Sentinel, with dashboards and alerts, we can look for anomalies and issues, but in modern IT, we need more. Cyber threats are becoming more and more sophisticated. Traditional methods of detecting issues and threats are not enough. By the time we detect issues, it may already be too late. We need to be proactive and look for possible issues and stop threats before they occur.

For threat hunting, there is a separate section in Microsoft Sentinel. It allows us to create custom queries, but also offers an extensive list of pre-created queries to help us start. Some of the queries for proactive threat hunting are high reverse DNS count, domains linked with the WannaCry ransomware, failed login attempts, hosts with new logins, and unusual logins, to name a few. A list of some of the available queries is shown in the following screenshot:

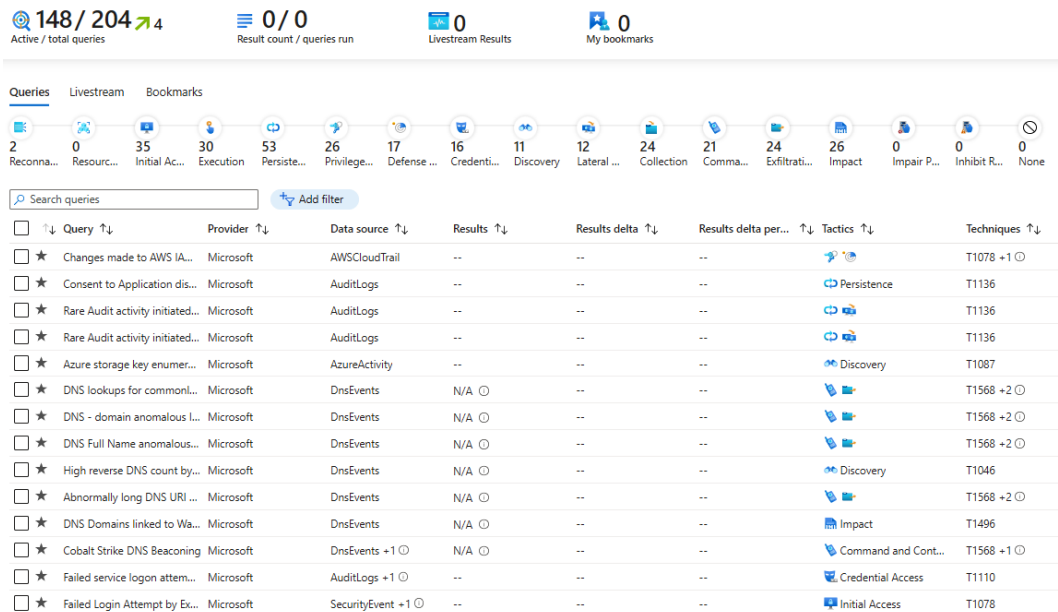


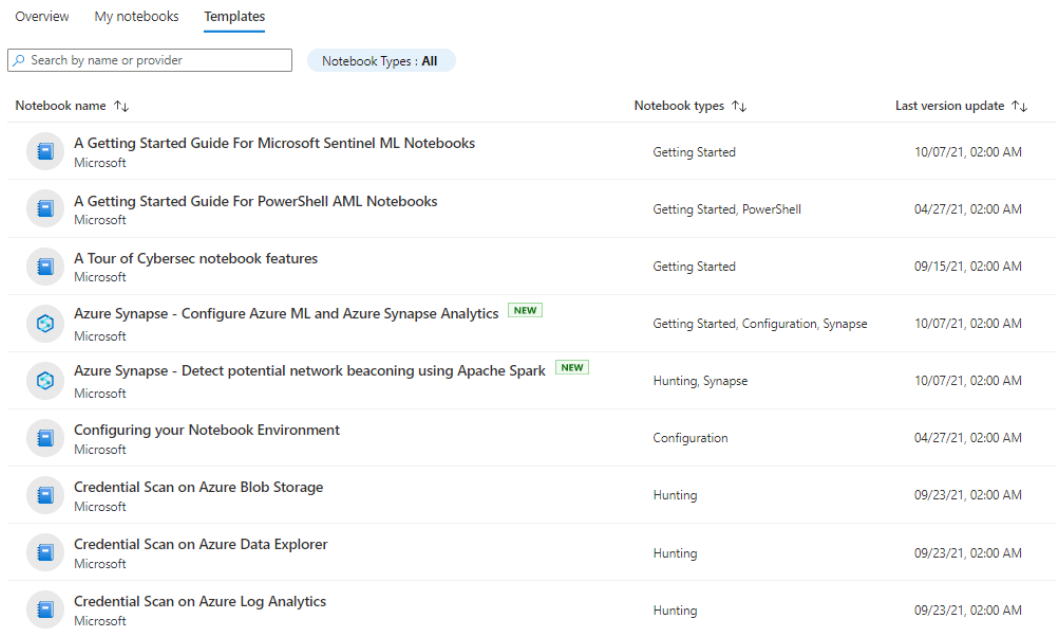
Figure 8.18 – Queries for threat hunting

We can also switch the view to live stream and use it to watch for certain events in real time. Live stream offers a graphical view of our hunting queries to help us monitor threats visually.

Another option in the hunting section is bookmarks. When exploring data and looking for threats, we may encounter events that we are not sure about. Some events may look innocent, but, in combination with other events, may prove very dangerous. Bookmarks allow us to save the results of some queries in order to revisit them later. We may want to check the same thing in the next few days and compare results, or maybe check some other logs that may give us more information.

Proactive hunting does not stop there. There is another section in Microsoft Sentinel, called notebooks. Microsoft Sentinel leverages data stores with the use of powerful queries and scaling to analyze data in massive volumes. Notebooks goes one step further and, with common APIs, allows the use of Jupyter Notebooks and Python. These tools extend what we can do with data stored in Microsoft Sentinel, allowing us to use a huge collection of libraries for machine learning, visualization, and complex analysis.

Again, we have some notebooks already available that we can start using right away. The list of available queries is shown in the following screenshot:



Overview My notebooks <u>Templates</u>		
Search by name or provider Notebook Types : All		
Notebook name ↑↓	Notebook types ↑↓	Last version update ↑↓
 A Getting Started Guide For Microsoft Sentinel ML Notebooks Microsoft	Getting Started	10/07/21, 02:00 AM
 A Getting Started Guide For PowerShell AML Notebooks Microsoft	Getting Started, PowerShell	04/27/21, 02:00 AM
 A Tour of Cybersec notebook features Microsoft	Getting Started	09/15/21, 02:00 AM
 Azure Synapse - Configure Azure ML and Azure Synapse Analytics NEW Microsoft	Getting Started, Configuration, Synapse	10/07/21, 02:00 AM
 Azure Synapse - Detect potential network beaconing using Apache Spark NEW Microsoft	Hunting, Synapse	10/07/21, 02:00 AM
 Configuring your Notebook Environment Microsoft	Configuration	04/27/21, 02:00 AM
 Credential Scan on Azure Blob Storage Microsoft	Hunting	09/23/21, 02:00 AM
 Credential Scan on Azure Data Explorer Microsoft	Hunting	09/23/21, 02:00 AM
 Credential Scan on Azure Log Analytics Microsoft	Hunting	09/23/21, 02:00 AM

Figure 8.19 – Microsoft Sentinel notebooks

But it doesn't end there, Microsoft Sentinel provides additional capabilities when it comes to threat intelligence and threat detection. Let's now take a look at some of the advanced capabilities that Microsoft Sentinel provides.

Advanced threat detection

We already mentioned some of the threat detection options in Microsoft Sentinel, including built-in threat detection and NRT analytics rules. But there are also advanced options, which include **User and entity behavior analytics (UEBA)** and **Multistage attacks (Fusion)**.

Every day, user actions generate a massive number of logs. Analyzing these logs can present an impossible task if we wanted to do them manually as we are looking at mountains of data, and it can be difficult to differentiate between malicious attempts and convert them into meaningful events. But we can't ignore them either as these can point us to threats caused by external entities (usually using compromised credentials) or even insiders (such as dissatisfied employees). UEBA provides the capability to analyze such logs and gain insight with the help of machine learning. Microsoft Sentinel can be connected to a number of data sources. Besides the obvious logs from Microsoft Azure and Azure Active Directory, other logs can be used, such as Syslog (Linux) or SecurityEvent (Windows) to name a few. Logs are aggregated and analyzed by Microsoft Sentinel in order to form a baseline (which represents normal behavior). Once a baseline is established, Microsoft Sentinel watches for any anomaly and flags it. Anomalies are also scored based on the probability of the user performing an action and the potential impact of the action. As machine learning is used, more data over more time will provide better results. With a bigger data sample to establish a baseline, results provided by UEBA will provide more accurate analyses and detect even the smallest incident.

Malicious attacks are getting more and more sophisticated and harder to discover. These attacks are often not direct or pointed at a single target, which makes them even harder to detect. We can define alerts and even actions for known attack scenarios, but what about emerging and unknown threats? Multistage attacks (Fusion) use a similar technic to UEBA and apply machine learning to this problem. Based on known attacks and data, Fusion uses algorithms to constantly learn and adapt. And not only the fact that it constantly learns and improves in order to detect new threats, but it can connect different invents that individually don't look suspicious but, when observed together, can present a significant threat.

It's important to mention that Microsoft Sentinel supports **Bring Your Own Machine Learning (BYOML)**. With the use of Azure Databricks / Apache Spark and Jupyter Notebooks, we can have our own machine learning models and algorithms, analyze data, and publish results.

Microsoft Sentinel isn't just a tool with a limited number of options; it's also very customizable. There are many ways to adjust Microsoft Sentinel to your specific needs and requirements. Many external resources can be used or further adjusted.

Using community resources

The power of Microsoft Sentinel is extended by its community. There is a GitHub repository at <https://github.com/Azure/Azure-Sentinel>. Here, we can find many additional resources that we can use. Resources developed by the community offer new dashboards, hunting queries, exploration queries, playbooks for automated responses, and much, much more.

The community repository is a very useful collection of resources that we can use to extend Microsoft Sentinel with additional capabilities and increase security even further.

Summary

Microsoft Sentinel satisfies all the requirements for SIEM. Not only that, but it also brings additional tools to the table in the form of proactive threat hunting and using machine learning and predictive algorithms. With all the other resources we have covered, most aspects of modern security have been covered, from identity and governance over network and data protection, to monitoring health, detecting issues, and preventing threats.

But security does not end here. Almost every resource in Azure has some security options enabled. These options can help us go even further and improve security. With all the cybersecurity threats around today, we need to take every precaution available.

In the final chapter, we are going to discuss security best practices and how to use every security option to our advantage.

Questions

As we conclude, here is a list of questions for you to test your knowledge regarding this chapter's material. You will find the answers in the *Assessments* section of the *Appendix*:

1. Microsoft Sentinel is...
 - A. **Security Event Management (SEM)**
 - B. **Security Information Management (SIM)**
 - C. **Security Information Event Management (SIEM)**
2. Microsoft Sentinel stores data in...
 - A. Azure Storage
 - B. Azure SQL Database
 - C. A Log Analytics workspace

3. Which data connectors are supported in Microsoft Sentinel?
 - A. Microsoft data connectors
 - B. Cloud data connectors
 - C. A variety of data connectors from different vendors
4. Which query language is used in Microsoft Sentinel?
 - A. SQL
 - B. GraphQL
 - C. Kusto
5. Dashboards in Microsoft Sentinel are used for...
 - A. The visual detection of issues
 - B. Constant monitoring
 - C. Threat prevention
6. Rules and alerts in Microsoft Sentinel are used for...
 - A. The visual detection of issues
 - B. Constant monitoring
 - C. Threat prevention
7. Threat hunting is performed by...
 - A. Monitoring dashboards
 - B. Using Kusto queries
 - C. Analyzing data with Jupyter Notebooks

9

Security Best Practices

In this book, we have already covered the most important topics regarding Azure security, from governance and monitoring to identity, network, and data protection, to specific tools such as Azure Security Center and Azure Sentinel. In this final chapter, you will find some tips and tricks the authors are using regularly in their projects.

The topics we will cover in the following sections include the following:

- Log Analytics design considerations
- Understanding Azure SQL Database security features
- Security in Azure App Service

Log Analytics design considerations

You've already learned that Microsoft Defender for Cloud and Microsoft Sentinel rely heavily on Azure Log Analytics. But there are some important considerations to make before you start your security monitoring journey in the cloud.

The three most important paradigms in this context are as follows:

- Use as few Log Analytics workspaces as possible.
- Use regional workspaces to avoid Azure bandwidth costs.
- Use different workspaces for security and performance monitoring

From a technical point of view, it's best to only use a single, central workspace so all the data resides in one place. Having a single workspace, you can easily, efficiently, and quickly correlate your data to get the respective insights. You also only need to take care of a single **role-based access control (RBAC)** model for this workspace. However, fine-grained RBAC models demand more effort.

Important Note

Remember that you cannot use the default workspace that is created by Microsoft Defender for Cloud within the context of Microsoft Sentinel. To connect both systems, make sure to create your own workspace(s) based on the recommendations within this section.

From a monetary point of view, however, you should plan for regional workspaces instead of a central workspace if you are using Azure resources in different Azure regions. For on-premises servers, it does not make a big difference what layout you are planning for because all **incoming network traffic (ingress)** to an Azure region is free. The Microsoft Monitoring Agent will send data to Azure, but there is no regular data transfer from Azure to your on-premises servers. The problem is that you must pay for **outgoing traffic (egress)** out of an Azure region. Now, if you use a single, central Log Analytics workspace and have Azure **Virtual Machines (VMs)** deployed in different Azure regions, outgoing data transfers from these VMs to your workspace would generate traffic costs.

There is another monetary decision that needs to be made. Microsoft Sentinel costs will apply to all data that is stored in a Log Analytics workspace used by Microsoft Sentinel. In general, performance data is not relevant to security analytics, so it wouldn't make sense to pay for this data even though it is not being used within the scope of Sentinel. Therefore, it can make sense to separate security-related information from performance-related information.

In order to bring these paradigms together, the idea is to create one single workspace for security-related data in every Azure region you are using, not only to avoid regular traffic costs, but also to stick to the idea of having as few workspaces as possible.

When using several workspaces, you might need to re-consider your auto-provisioning setup in Defender for Cloud. Auto-provisioning will always apply to a subscription, not an Azure region, or resource group. If you have resources in one subscription deployed to several Azure regions, you cannot use the auto-provisioning capability in the Defender for Cloud portal, but you can leverage built-in policy definitions as explained in *Chapter 7, Microsoft Defender for Cloud*.

You can also leverage **Infrastructure as Code (IaC)** with **Azure Resource Manager (ARM)** templates or Terraform to deploy your VMs and install the Log Analytics VM extension at the same time. The Log Analytics VM extension is deployed and managed through the ARM and fully supports CI/CD pipelines in your DevOps scenario. That said, when you use IaC deployments for your VM in your DevOps scenario, you can also update, manage, and configure the agent through your pipeline. If you have also configured policies, blueprints, and all the other governance features you have learned about in *Chapter 2, Governance and Security*, you can easily adhere to your corporate security policy and leverage automation at the same time.

Next, we are going to look at other Azure services' security features and how they can help us increase security.

Understanding Azure SQL Database security features

Some security features related to Azure SQL Database were mentioned in *Chapter 6, Data Security*, where we discussed data security. However, there are additional features we can use to increase security.

The first feature and line of defense when it comes to Azure SQL Database is a firewall. This built-in tool, by default, blocks access to the database from any IP address that is not preauthorized (whitelisted). It's important to mention that firewall settings are on the Azure SQL Server level and will be inherited by all databases on the server. If we need to allow a single IP address to access a single database, we may want to reconsider our resource strategy. Allowing an IP address to access a single database will enable access to all databases on the same server. Because of this, we need to consider putting only databases used by the same applications or the same group of users on a single server.

Allowing a new IP address to connect to Azure SQL Server can be done from the Azure SQL Server firewall settings. We have the option to add a single IP address (the start and end IP will be the same) or add a range of IP addresses (from the start IP to the end IP). If we want to allow our current IP address, there is a convenient **Add client IP** button. This will automatically detect the IP address we are coming from and add a new rule to allow that IP address. After any rule is added, we need to save changes in order for them to become effective.

An example of firewall settings is shown in the following screenshot:

The screenshot shows the 'Firewall settings' page for an Azure SQL Server named 'nvsh (SQL server)'. At the top, there are buttons for 'Save', 'Discard', and 'Add client IP'. Below this, there are several configuration options: 'Deny public network access' (unchecked), 'Minimum TLS Version' (set to 1.2), 'Connection Policy' (set to Default), and 'Allow Azure services and resources to access this server' (set to No). A 'Client IP address' field shows '178.77.12.152'. Below this is a table for firewall rules with columns 'Rule name', 'Start IP', and 'End IP'. The table is currently empty with the message 'No firewall rules configured.' Below the rules table is a section for 'Virtual networks' with links to '+ Add existing virtual network' and '+ Create new virtual network'. Below this is a table for virtual network rules with columns 'Rule name', 'Virtual network', 'Subnet', 'Address Range', and 'Endpoint status'. The table is currently empty with the message 'No vnet rules for this server.' At the bottom, there is a section for 'Outbound networking' with the text 'Restrict network access to a specific set of resources by supplying their fully-qualified domain names. [Learn more](#)'. Below this, it says 'Restrict outbound networking' and 'Restrictions disabled' with a link to 'Configure outbound networking restrictions'.

Firewall settings ...
nvsh (SQL server)

Save Discard + Add client IP

☐ Deny public network access

Minimum TLS Version ⓘ
1.0 1.1 **1.2**

Connection Policy ⓘ
Default Proxy Redirect

Allow Azure services and resources to access this server ⓘ
Yes **No**

Client IP address 178.77.12.152

Rule name	Start IP	End IP
		...

No firewall rules configured.

Virtual networks
[+ Add existing virtual network](#) [+ Create new virtual network](#)

Rule name	Virtual network	Subnet	Address Range	Endpoint status
No vnet rules for this server.				

Outbound networking
Restrict network access to a specific set of resources by supplying their fully-qualified domain names. [Learn more](#)

Restrict outbound networking
Restrictions disabled
[Configure outbound networking restrictions](#)

Figure 9.1 – Azure SQL Server firewall settings

Under **Firewall settings**, there are two additional options we can use:

- **Allow Azure services and resources to access this server**
- **Connections from the VNet/Subnet**

The option to allow Azure services sounds compelling, but we need to be very careful. At first look, this option looks helpful, and most people will probably think they should allow Azure services to connect. It makes our life so much easier when we don't have to worry about how our web app connects to a database. But this option does not allow only your Azure service to connect; it allows any Azure service to connect. Allowing Azure services and resources to connect will only check whether the connection is coming from an Azure data center, it will not limit connections to your subscription or tenant. Allowing this option will make Azure SQL Database more vulnerable, and I would recommend you keep this option off.

Important Note

Allow Azure services and resources to access this server is set to **OFF** by default, but it wasn't always the case. Previously, this setting was enabled by default. If you have already created an Azure SQL Database, make sure this setting is disabled.

Connection from VNET/Subnet allows us to create VNET integration by allowing certain VNETs (or limiting this only to subnets) to connect to Azure SQL Database. This approach is more secure as the database does not need to be exposed over the internet and all communication is done on a secure private network.

But Azure SQL Database is just one example of built-in security features on a service level. Almost every service has some service-specific security features. Azure App Service is another excellent example with many security options available, which we will be covering next.

Security in Azure App Service

Azure App Service is an HTTP-based service for hosting web applications and APIs with support for multiple programming languages. It's also another great example of an Azure service with multiple security features built in. We can control access, protocols, certificates, and many other things.

One of the early problems we need to solve for any application is authentication. App Service allows us to set up authentication based on a few different providers: **Azure Active Directory (Azure AD)**, Microsoft (or Live) account, Facebook, Google, and Twitter. Naturally, the best integration method is using Azure AD, as it is native and also allows you further control through Azure AD tools and features.

In order to set up Azure AD authentication for App Service, we need to do the following:

1. In the **App Service Authentication** blade, we need to select **Microsoft**, as in the following figure:

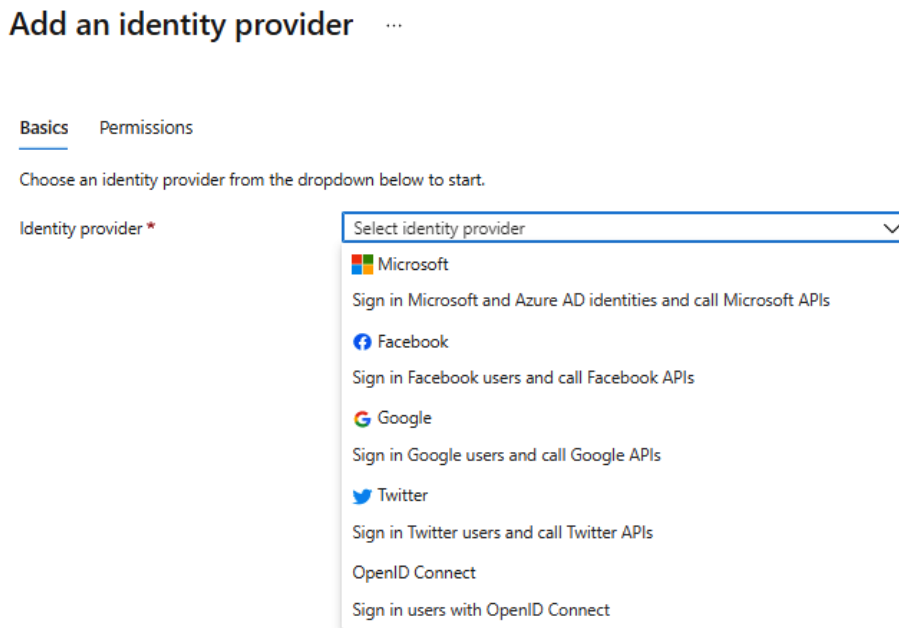


Figure 9.2 – Azure App Service authentication

2. Next, we need to select an app registration type where we can create new, select existing (select from a dropdown), or provide information for existing app registration (provide object ID). We also need to select an account type between single-tenant, multi-tenant, personal Microsoft account, or a combination of all supported types (allows logging in with any Microsoft or AAD account from any organization). An example of new app registration and a single tenant is shown in the following screenshot:

Basics Permissions

Identity provider *

Microsoft

**App registration**

An app registration associates your identity provider with your app. Enter the app registration information here, or go to your provider to create a new one. [Learn more](#)

App registration type *

- ☒ Create new app registration
- ☐ Pick an existing app registration in this directory
- ☐ Provide the details of an existing app registration

Name * ⓘ

packtdemo



Supported account types *

- ☒ Current tenant - Single tenant
- ☐ Any Azure AD directory - Multi-tenant
- ☐ Any Azure AD directory & personal Microsoft accounts
- ☐ Personal Microsoft accounts only

[Help me choose...](#)

Figure 9.3 – App Service Azure AD authentication

3. After everything is configured, we need to ensure that the user is required to log in with **Azure Active Directory** in order to access the application, as in the following screenshot:

App Service authentication settings

Requiring authentication ensures all users of your app will need to authenticate. If you allow unauthenticated requests, you'll need your own code for specific authentication requirements. [Learn more](#)

Restrict access *

- ☒ Require authentication
- ☐ Allow unauthenticated access

Unauthenticated requests *

- ☒ HTTP 302 Found redirect: recommended for websites
- ☐ HTTP 401 Unauthorized: recommended for APIs
- ☐ HTTP 403 Forbidden

Figure 9.4 – Enforcing Azure AD login

4. Optionally, we can set app permission if required. By default, only a read user profile is added.

5. When accessing the application, the user will be prompted to **Sign in** with their Azure AD account in order to proceed, as in the following figure:

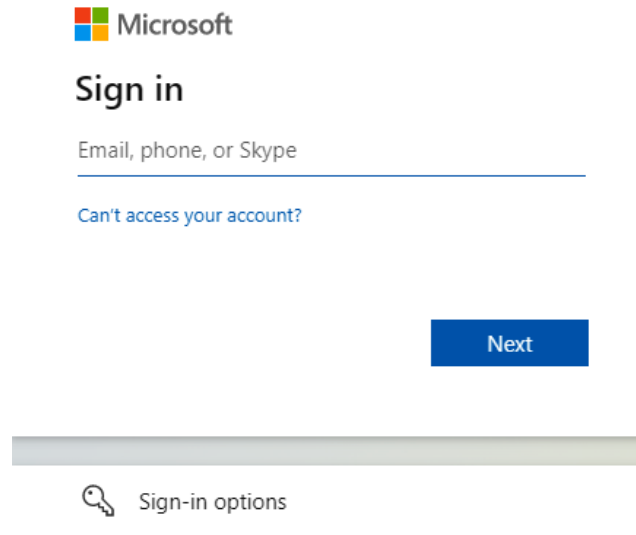


Figure 9.5 – User is required to sign in in order to access the application

Another set of security features in Azure App Service is **Protocol Settings**. Here, we can force the application to use only HTTPS, control the minimal TLS version, or configure if an incoming client certificate is required. We can also configure **TLS/SSL bindings** to specify the certificate that will be used when responding to a request to a specific hostname. Both public and private certificates can be used.

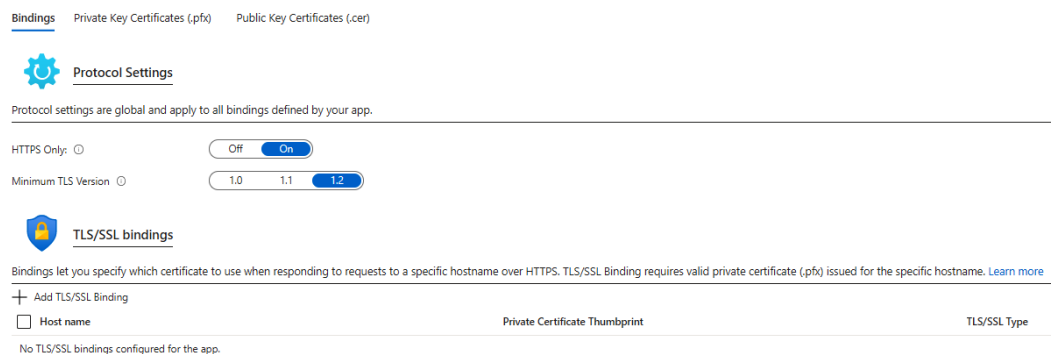


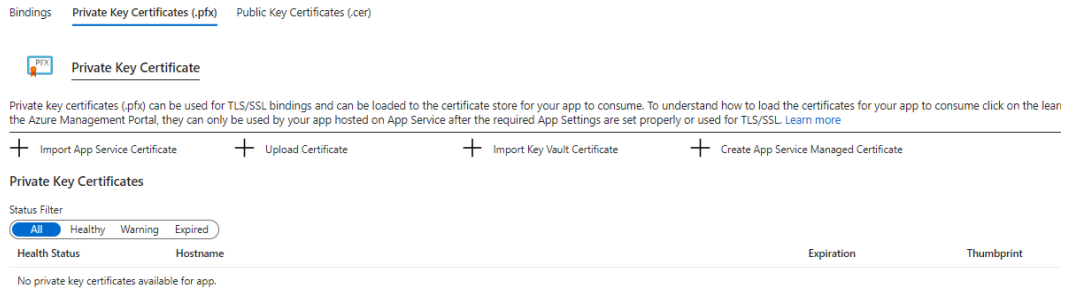
Figure 9.6 – App Service protocol settings

Private certificates can be imported from App Service (to a specific web app), uploaded, imported from Azure Key Vault, or we can create a new **App Service Managed Certificate**.

Note

App Service Managed Certificate is available to all web apps on App Service, not just on the web app where it's created.

In the following screenshot, we can see the **Private Key Certificate** settings:



Bindings Private Key Certificates (.pfx) Public Key Certificates (.cer)

Private Key Certificate

Private key certificates (.pfx) can be used for TLS/SSL bindings and can be loaded to the certificate store for your app to consume. To understand how to load the certificates for your app to consume click on the learn the Azure Management Portal, they can only be used by your app hosted on App Service after the required App Settings are set properly or used for TLS/SSL. [Learn more](#)

+ Import App Service Certificate + Upload Certificate + Import Key Vault Certificate + Create App Service Managed Certificate

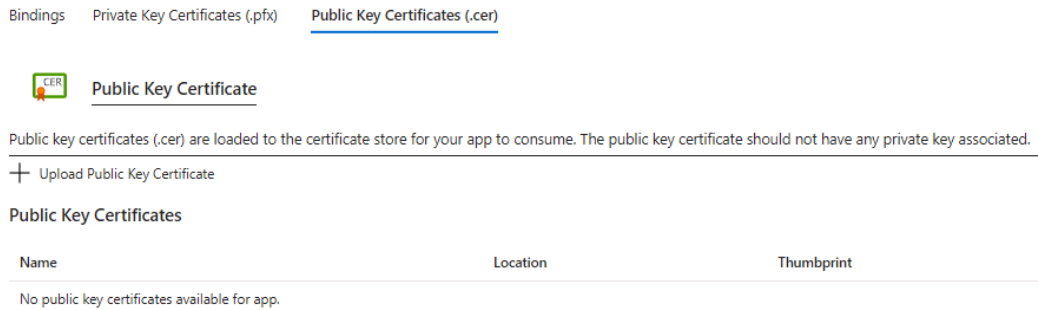
Private Key Certificates

Status Filter: All Healthy Warning Expired

Health Status	Hostname	Expiration	Thumbprint
No private key certificates available for app.			

Figure 9.7 – App Service Private Key Certificate

Public Key Certificate requires a certificate issued by a recognized **Certificate Authority (CA)**. This certificate is usually used by publicly accessible web applications, and serves as proof that the website is trustworthy.



Bindings Private Key Certificates (.pfx) Public Key Certificates (.cer)

Public Key Certificate

Public key certificates (.cer) are loaded to the certificate store for your app to consume. The public key certificate should not have any private key associated.

+ Upload Public Key Certificate

Public Key Certificates

Name	Location	Thumbprint
No public key certificates available for app.		

Figure 9.8 – Public Key Certificate

Under **Networking settings** in App Service, we have multiple options. Most of the options are related to secure connections and the integration of a web app with private networks. This was already mentioned in *Chapter 4, Azure Network Security*, when we discussed network security. We can achieve this with both VNET integration and hybrid connections. Also, this section includes options to enable services such as Azure Front Door (as the web application firewall for globally distributed applications) and **Content Delivery Network (CDN)**. The last option under **Networking settings** is access restrictions. All network settings associated with Azure App Service are shown in the following screenshot:

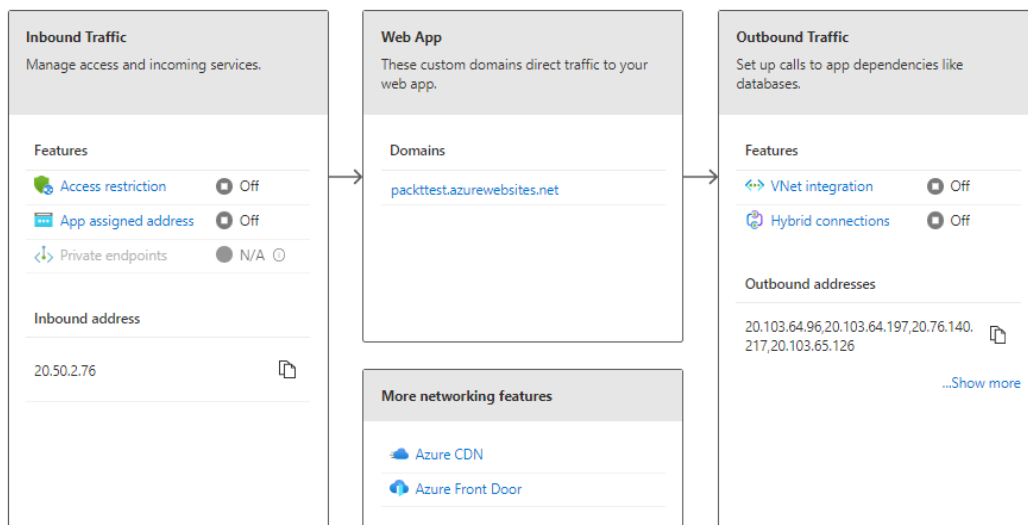


Figure 9.9 – Azure App Service network settings

Access Restrictions is a very useful feature in **Azure App Service** settings. Using **Access Restrictions**, we can block access from specific IP addresses or allow access only from whitelisted IP addresses. Essentially, **Access Restrictions** works very similarly to **Network Security Groups (NSGs)**, by creating rules and assigning priority to them. However, **Access Restrictions** focuses on public access where NSGs control broader networks with both public and private network rules. In the following figure, we can see an example of creating a block rule:

Add Restriction ×

General settings

Name ⓘ

Action

☒ Allow☐ Deny

Priority *

Description

Source settings

Type



IP Address Block *

HTTP headers filter settings

X-Forwarded-Host ⓘ

X-Forwarded-For ⓘ

X-Azure-FDID ⓘ

X-FD-HealthProbe ⓘ

Figure 9.10 – App Service access restriction

To create a block rule, we need to provide the following information:

- **Name**
- **Action** (allow or deny access)
- **Priority**
- **Description**
- **Type** (IPv4 or IPv6)
- **IP Address Block**
- **HTTP header filter settings** (optional)

In the preceding example, a rule is created to block access from IP address block `208.130.0.0/16`. If anyone tries to access the application from any IP address in the specified IP address range, the request will be blocked, and the user will be prevented from reaching the web app.

As we are able to use priorities, we can also approach things differently and allow access only from whitelisted IP addresses. In this case, we would create a block rule that would prevent anyone from accessing the application and assign **Priority 200**. Another rule would be created to allow access from a specific IP address range with **Priority 100** (this can be any value under 200). Allowing a rule for specified IP addresses would override the rule to block, because of its higher priority. But it would still prevent anyone else from accessing the application.

Another challenge we face with the cloud is the management of secrets. We need to be careful how secrets, connections strings, and other sensitive information are used and make sure that none of these are exposed. As in many cases we have already discussed in this book, Azure Key Vault comes to the rescue. Similar to passing secrets when using IaC (as discussed in *Chapter 5, Azure Key Vault*), or managing keys and certificates (discussed in *Chapter 6, Data Security*), Azure Key Vault can be used to secure any sensitive information needed by Azure App Service. Instead of storing sensitive information in web config files or in App Service Configuration (where information is hidden, but visible if the user has enough permissions), we can use a managed identity that will allow access to Azure Key Vault, where sensitive information is stored in a secure way. An example of identity settings in App Service is shown in the following figure:

System assigned User assigned

A system assigned managed identity is restricted to one per resource and in code. [Learn more about Managed identities.](#)

Save Discard Refresh | Got feedback?

Status ⓘ

Off **On**

Object (principal) ID ⓘ

9d64a7bd-d284-4a23-91ef-ae5df3d94370

Permissions ⓘ

Azure role assignments

Figure 9.11 – Managed identity in Azure App Service

As we can see, there are two types of managed identity that we can set: **System assigned** and **User assigned**. **System assigned** will generate a managed identity and set up the necessary options. In **User assigned**, we need to create a managed identity ourselves and create any necessary permissions and settings. If we require using more than one managed identity for a service, we need to use a **User assigned** managed identity, because a service can have only one system-assigned managed identity.

A **System assigned** managed identity is tied to a resource life cycle, and once a resource is removed, the managed identity is removed as well. The managed identity is generated in order to be used for that service and is not used by any other service. This is not the case with a **User assigned** identity, as more resources can use the same managed identity.

As we already mentioned, each service has some kind of security enhancements that we can use. Some storage account security options were already mentioned in *Chapter 6, Data Security*. Let's take a look at storage account access keys and how to work with them.

Storage account access keys

Access to storage accounts can be done with Azure AD (RBAC) or storage account access keys. Role-based may be the preferred way, but very often, requirements dictate that we need to use shared keys. Settings for access keys for storage accounts are shown in the following screenshot:

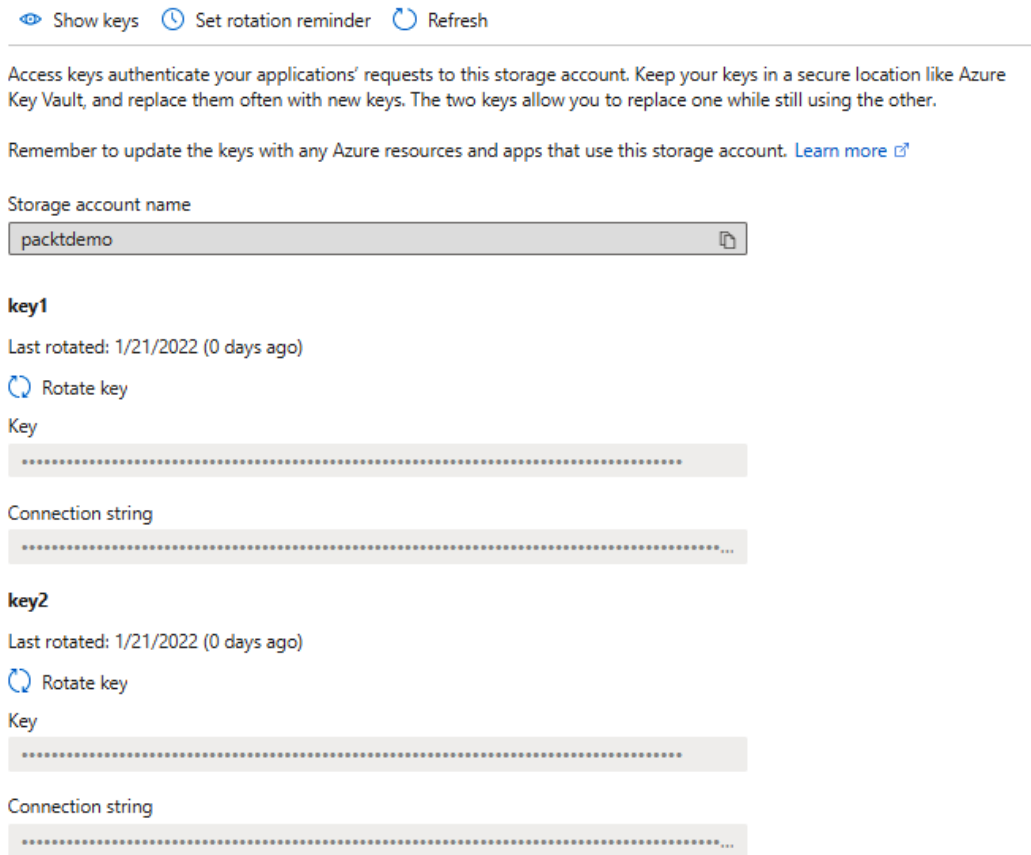


Figure 9.12 – Storage account access keys

You can notice that there are two keys present. It is recommended that keys are rotated regularly or in case keys are compromised. We can even set a reminder to rotate keys daily, weekly, monthly, yearly, or for a custom period (the default custom period is 60 days). A reminder for key rotation is shown in the following screenshot:

Set a reminder to rotate access keys ×

Set a reminder to manually rotate your access keys. This simply issues a notification banner indicating that key(s) need to be rotated based on the reminder set. This will not rotate your keys automatically. Your access key will be valid until you choose to manually rotate it.

☒ Enable key rotation reminders

Send reminders

Remind me every *

Figure 9.13 – Reminder to rotate access keys

In case we suspect our keys are compromised, we would rotate the secondary key and update our application with a new key. Then, we would rotate the primary key as well and update our application to use the new primary key. This way, any threat is eliminated, keys are updated, and application access to data was never interrupted. We would act similarly in case of regular key rotation, so application is not affected by key rotation.

Summary

At the very end of this book, we have covered all aspects of Microsoft Azure security. From the shared responsibility model to advanced security features, we have come to the end. It's important to remember that the cloud changes all the time, and more and more services are available that can help us to increase security. Let's focus on the most important takeaways from this book:

- Keep your identity and secrets secure with all the means available.
- When it comes to access, take two approaches: **Just Enough Administration (JEA)** and **Just-in-Time Administration (JIT Administration)**.
- Nothing should be exposed to public access if it's not necessary, especially management and security access.
- Data should be encrypted at all times.
- All communication and traffic should be encrypted (over HTTPS) and on a secure (private) network, if possible.

- Microsoft Defender for Cloud and Microsoft Sentinel can help you with analytics, detection, and recommendations to stay on top of cloud security and keep Azure resources safe.
- Besides services focused on security, all Azure services have security features of their own. Use these features to increase security.

Microsoft Azure changes all the time; new features are added, and existing features are updated almost daily. So, in the future, options might change, but these options are there to present an idea of how to approach cloud security. These seven key points are what we need to focus on; how we complete these tasks is insignificant.

Questions

As we conclude, here is a list of questions for you to test your knowledge regarding this chapter's material. You will find the answers in the *Assessments* section of the *Appendix*:

1. What are Log Analytics' best practices?
 - A. Use a single workspace.
 - B. Use regional workspaces.
 - C. Use multiple workspaces for each region and each service.
 - D. Based on requirements, both A and B can be correct.
2. We can control network access to Azure SQL Database with...
 - A. An access list
 - B. A firewall
 - C. Conditional access
3. Which type of certificate is supported in Azure App Service?
 - A. Private
 - B. Public
 - C. Wildcard
 - D. All of the above
 - E. Only 1 and 2
 - F. Only 2 and 3

4. We can control network access to Azure App Service with...
 - A. Access restrictions
 - B. A firewall
 - C. Conditional access
5. What is a best practice for enabling App Service communication with other Azure services?
 - A. Service Principal
 - B. Managed identity
 - C. Azure Key Vault
6. Azure App Service supports authentication with...
 - A. **Azure Active Directory (Azure AD)**
 - B. A Microsoft account
 - C. Twitter
 - D. All of the above
 - E. Only 1 and 2
 - F. Only 2 and 3
7. Security in Azure can be set up from...
 - A. Azure Security Center
 - B. Azure Sentinel
 - C. Different services and features
 - D. Azure AD

Assessments

In the following pages, we will review all the practice questions from each of the chapters in this book and provide the correct answers.

Chapter 1 – An Introduction to Azure Security

1. C. Responsibility is shared
2. B. Cloud provider
3. B. Cloud provider
4. C. Depends on the service model
5. A. User
6. C. Both, but DLA is replacing Q10
7. C. Customer is notified

Chapter 2 – Governance and Security

1. D. All of the above
2. C. We need to define both the *what* and the *how*
3. C. Delete locks
4. D. Management Group
5. E. Management Group or Subscription
6. D. All of the above
7. B. A service to gather information about deployments

Chapter 3 – Managing Cloud Identities

1. A. More exposed to attacks
2. C. Password protection
3. D. Passwordless sign-in methods
4. A. Yes
5. C. Infected users
6. C. To activate privileges
7. D. Passwordless authentication

Chapter 4 – Azure Network Security

1. B. A Network Security Group (NSG)
2. B. Site-to-Site
3. C. Both of the above
4. B. Service endpoints
5. D. A and B are correct
6. C. Distributed Denial of Service (DDoS)

Chapter 5 – Azure Key Vault

1. E. All of the above
2. B. Access policies
3. A. Service principals
4. A. EnabledForDeployment
5. B. EnabledForTemplateDeployment
6. C. EnabledForDiskEncryption
7. C. Reference an Azure Key vault

Chapter 6 – Data Security

1. B. Enforce HTTPS
2. C. Blob soft delete
3. A. Yes
4. A. Yes
5. A. Yes
6. C. Both of the above
7. B. Dynamic data masking

Chapter 7 – Microsoft Defender for Cloud

1. C. Log Analytics Workspace
2. A. Recommendations
3. B. Logic Apps
4. E. Both A and C
5. D. Both B and C
6. E. Both B and D

Chapter 8 – Microsoft Sentinel

1. C. **Security Information Event Management (SIEM)**
2. C. A Log Analytics workspace
3. C. A variety of data connectors from different vendors
4. C. Kusto
5. A. Visual detection of issues
6. B. Constant monitoring
7. C. Analyzing data with Jupyter Notebooks

Chapter 9 – Security Best Practices

1. D. Based on requirements, both A and B can be correct
2. B. A firewall
3. D. All of the above
4. A. Access restriction
5. B. Managed identity
6. D. All of the above
7. C. Different services and features

Index

A

- access policies 149, 150
- Active Directory Domain Services (AD DS) 55
- Active Directory Federation Services (AD FS) 103
- active risk detections 81
- adaptive application controls 230, 231
- admin accounts
 - protecting, with Azure AD PIM 94
- advanced network hardening 234
- advanced threat detection 264, 265
- application layer 142
- ARM templates
 - Azure key vault secret, referencing in 162-164
- assume breach 54
- AuditIfNotExists 20
- auto-provisioning
 - using, to deploy extensions 200-203
- Azure
 - governance 20-22
 - on-premises networks, connecting with 123

- Azure Active Directory (Azure AD)
 - about 274
 - licensing, considerations 108
 - multi-factor authentication (MFA), enabling 61-64
- Azure Active Directory Identity Protection
 - about 79-81, 252
 - risk detection 81
 - sign-in risk policy, creating 82-85
 - user risk policy, creating 82-85
- Azure AD pricing
 - reference link 108
- Azure AD Privileged Identity Management (Azure AD PIM)
 - Azure AD roles, managing 95-99
 - features 94
 - used, for managing Azure resources 100, 101
 - used, for protecting admin accounts 94
- Azure AD roles
 - managing, in Azure AD PIM 95-99
- Azure AD smart lockout
 - features 58
- Azure Advanced Threat Protection 252

- Azure Application Gateway
 - about 142
 - traffic flow 144
 - WAF 144
- Azure App Service
 - security 273-281
- Azure Arc 197
- Azure Bastion 140
- Azure blueprints
 - defining 38, 39
 - publishing 39-45
- Azure CLI
 - Azure Resource Graph,
 - querying with 48, 49
- Azure DDoS protection
 - about 139
 - Basic protection 139
 - Standard protection 140
- Azure Firewall
 - about 135
 - configuration 137-139
 - deployment 135-137
- Azure Front Door 145
- Azure infrastructure
 - high availability 10, 11
 - integrity 12
 - monitoring 13
- Azure Key Vault
 - about 148, 149
 - access policies 149, 150
 - artifacts 148
 - creating 157
 - in PowerShell 158, 159
 - service tiers 148
 - using, in deployment scenarios 157
- Azure key vault secret
 - creating 157
 - referencing, in ARM templates 162-164
 - referencing, in Terraform 161, 162
- Azure Log Analytics
 - design, considerations 269-271
- Azure Monitor Agent (AMA) 197
- Azure network architecture
 - about 9, 10
 - L2 switches level 9
- Azure Policy
 - about 29, 30, 197
 - assignments 34
 - best practices 37, 38
 - exemptions 35, 37
 - initiative assignments 35
 - initiative definitions 35
 - Microsoft Defender for Cloud,
 - enabling via 198, 199
 - mode 31
 - parameters 31
 - parameters, properties 31-33
 - types 29
- Azure Resource Graph
 - about 45
 - advanced queries 49, 50
 - querying, with Azure CLI 48, 49
 - querying, with PowerShell 46, 47
- Azure Resource Manager (ARM)
 - 20, 159, 232, 271
- Azure resources
 - managing, with Azure AD PIM 100, 101
- Azure Security Benchmark 210
- Azure security foundations 14, 15
- Azure Service Management (ASM) 232

Azure SQL Database
 advantages 177
 options 177
 security features 271, 273
 working 177-186
Azure Storage 168-174
Azure Virtual Network 114
Azure Virtual WAN 142
Azure VM
 deploying 160
 deploying, with PowerShell 160, 161
 disks 174-177

B

Blob soft delete 170
Bring Your Own Key (BYOK) option 171
Bring Your Own Machine
 Learning (BYOML) 265

C

Certificate Authority (CA) 148, 277
certificate signing request (CSR) 148
Classless Inter-Domain
 Routing (CIDR) 232
Cloud Security Posture
 Management (CSPM)
 about 193, 207
 with Microsoft Defender
 for Cloud 207-209
cloud service models
 about 4
 security 6
Cloud Workload Protection 224

Cloud Workload Protection
 Platform (CWPP) 193
command-line interface (CLI) 91
community resources
 using 266
compact flash (CF) 23
Conditional Access
 custom controls 72
 Named locations 71, 72
 terms of use 73
 using 70, 71
Conditional Access, policies
 about 74
 Access controls section 76-79
 Assignments section 74-76
Content Delivery Network (CDN) 278
Continuous Export
 about 197, 235, 237
 configuring 236
continuous integration/ continuous
 delivery (CI/CD) 192
costCenter 30
CSPM, with Microsoft Defender for Cloud
 remediation, prioritizing 213-215
 working, with recommendations
 209-212
 working, with resource
 exemptions 215-218
custom query rule
 creating 253-258
custom RBAC roles
 creating 89-94
custom recommendations
 about 218-220
 adding, to Defender for Cloud 219, 220

D

- Database as a Service (DBaaS) 177
- Database Encryption Key (DEK) 182
- data center network 10
- data connectors
 - configuring 249, 250
 - reference link 249
- data retention
 - configuring 249, 250
- denial of service (DoS) 55
- Deploy-if-not-exists (DINE) 30, 198
- dev/test 26
- dictionary attack 56-58
- Distributed Denial of Service (DDoS) 10, 139
- Dynamic data masking 180, 181

E

- edge network 9
- elliptic curve (EC) 148
- Endpoint Detection and Response (EDR) 227
- enterprise agreement (EA) 26
- ExpressRoute 124

F

- Fabric Controller (FC) 12
- FIDO2 security key 106
- file integrity monitoring 234

G

- General Data Protection Regulation (GDPR) 180
- governance
 - in Azure 20-22
 - management groups, using for 26-29
- governance and security
 - common sense, using to avoid mistakes 22, 23

H

- Hardware Security Module (HSM) 148, 172
- hub-and-spoke topology 141
- hub virtual network
 - implementing 141
- hybrid authentication 101-104
- Hypertext Transfer Protocol over Secure Socket Layer (HTTPS) 167

I

- incoming network traffic (ingress) 270
- Infrastructure as a Service (IaaS) 5, 114, 174
- Infrastructure as Code (IaC) 271
- Internet Service Providers (ISPs) 10
- ISO 27001 blueprint 42

J

- JSON Web Key (JWK) 148
- Just In Time (JIT) VM access 232, 233

K

Key Performance Indicator (KPI) 207, 251
Kusto Query Language (KQL) 46, 253

L

layer 7 (L-7) 142
Local Network Gateway (LNG)
 creating 126, 127
Log Analytics Agent 197
Log Analytics Workspace 197
Logic apps 240

M

managed HSMs 148
managed identities, for Azure resources
 about 152-156
 types 153
Managed Identity (MI) 44
management groups
 using, for governance 26-29
management locks
 types 23
 using 23-25
Microsoft
 security 6
Microsoft Cloud App Security 252
Microsoft Defender Advanced
 Threat Protection 252
Microsoft Defender for Cloud
 about 192-196, 235, 252
 auto-provisioning, using to
 deploy extensions 200-203
 Cloud Security Posture Management
 (CSPM) with 207-209

 enabling 197
 enabling, via Azure Policy 198, 199
 enabling, via PowerShell 198
 enhanced security, enabling 203-206
 multi-cloud capabilities 241, 242
 security, automating 235
Microsoft Defender for Cloud,
 automated security
 Continuous Export 235-237
 REST APIs 241
 workflow automation 237-240
Microsoft Defender for Cloud,
 policy definitions
 reference link 206
Microsoft Defender for Containers 234
Microsoft Defender for Endpoint
 (MDE) integration 227
Microsoft Defender for Servers
 about 205, 224-226
 additional tools 230
 advanced network hardening 234
Microsoft Defender for Servers,
 additional tools
 adaptive application controls 230, 231
 file integrity monitoring 234
 Just In Time (JIT) VM access 232, 233
Microsoft Defender for SQL 205
Microsoft Defender for Storage 173
Microsoft incident creation rule 252
Microsoft Security Response Center 13
Microsoft Sentinel
 about 247-249
 automation 258-261
 data connectors, configuring 249, 250
 data retention, configuring 249, 250
 rules and alerts, setting up 252

- Microsoft Sentinel dashboards
 - working with 251, 252
- monitoring 20
- multi-cloud capabilities 224
- multi-factor authentication (MFA)
 - about 59, 60
 - activating, from user's perspective 64-67
 - enabling, in Azure AD 61-64
 - options, in Azure AD 60
- Multistage attacks (Fusion) 264

N

- Named locations 71, 72
- Near Real Time (NRT) 253
- need plus one (N+1) 10
- Network Interface Card (NIC) 115
- Network Security Group (NSG)
 - about 115, 211, 278
 - associating, with subnet 120-122
- Network Security (NS) 221
- network topology, elements
 - data center network 10
 - edge network 9
 - regional gateway network 10
 - wide area network 9
- Network Virtual Appliance (NVA) 134
- notebooks
 - about 263
 - using 263
- NRT query rule 253

O

- one-time password (OTP) 60
- on-premises networks
 - connecting, with Azure 123
- Open Web Application Security Project (OWASP) 143
- outgoing traffic (egress) 270

P

- passphrases
 - exploring 54, 55
- passwordless authentication
 - about 105, 106
 - global settings 106-108
- password protection 56-58
- passwords
 - exploring 54, 55
- password writeback 103
- Pay-As-You-Go 26
- physical security 7, 8
- Platform as a Service (PaaS) 5, 131
- Point-to-Site connection (P2S) 123
- PowerShell
 - Azure key vault in 158, 159
 - Azure Resource Graph,
 - querying with 46, 47
 - Microsoft Defender for Cloud,
 - enabling via 198
 - used, for deploying Azure VM 160, 161
- Premium service tier 148
- pricing model, for Microsoft Sentinel
 - capacity reservation 247
 - Pay-As-You-Go 247
 - reference link 248

principle of least privilege (POLP) 21
private endpoints, VNet
 about 134
 using 134
Privileged Identity Management
 (PIM) 232
public key infrastructure (PKI) 148

R

RBAC roles 88
recommendations
 about 207
 working with 209-212
regional gateway network 10
regulatory compliance 218-220
regulatory compliance, dashboard
 about 221
 using 220-222
regulatory compliance, standards
 working with 222, 223
remediation
 prioritizing 213-215
representational state transfer
 (REST) API 91
resource exemptions
 working with 215-218
resourceOwner 30
REST API endpoints
 reference link 241
REST APIs 241
risk types and detections
 reference link 82
risky sign-in 80
role assignment
 creating 85

role-based access control
 (RBAC) 20, 85-88, 270

S

Scheduled query rule 253
secure score 207
Secure Sockets Layer (SSL) 142, 148
Secure transfer required 168
security controls 207
security defaults 67-69
Security Development Lifecycle (SDL) 12
Security Information and Event
 Management (SIEM)
 about 245, 246
 alerts 246
 correlation 246
 dashboards 246
 data aggregation 246
 forensic analysis 246
 retention 246
Security Operations Center (SOC) 226
Segregation of Duties (SoD) 21, 94
self-service password reset (SSPR) 103
service endpoints, VNet
 about 131
 enabling 131-133
Service Level Agreement (SLA) 11
service-to-service authentication 150-152
shared responsibility model
 exploring 4
 Infrastructure as a Service (IaaS) 5
 on-premises environment 5
 Platform as a Service (PaaS) 5
 security 5
 Software as a Service (SaaS) 5

- single sign-on 101-104
- Site-to-Site connection (S2S)
 - about 123
 - creating 124-126
- Software as a Service (SaaS) 5, 101
- Software-Defined Networking (SDN) 114
- software inventory 228
- standard service tier 148
- storage accounts
 - access keys 282, 283
- system-assigned managed identity 153

T

- Terraform
 - Azure key vault secret,
 - referencing in 161, 162
- Threat and Vulnerability Management (TVM) 200, 228
- threat detection
 - summary 234, 235
- threat hunting
 - using 262
- Transparent Database Encryption (TDE) 182
- Transport Layer Security (TLS) 148

U

- Uniform Resource Identifier (URI) 172
- User and entity behavior
 - analytics (UEBA) 264
- user-assigned managed identity 153
- user risk detection 81
- user risk policy 81

V

- virtual machine scale sets (VMSS) 234
- Virtual Machine (VM) 5, 92, 174, 192, 270
- Virtual Network Gateway (VNG) 124
- Virtual Networks (VNETs)
 - about 114
 - accessing, from local network 123
 - connecting, to another VNET 128
 - default inbound security rules 115
 - default outbound security rules 116
 - ExpressRoute 124
 - inbound security rules, adding 116, 117
 - inbound security rules, creating
 - with Azure PowerShell 117
 - LNG, creating 126
 - outbound security rules,
 - adding 118, 119
 - outbound security rules, creating
 - with Azure PowerShell 119
 - peering, creating 128-131
 - Point-to-Site connection (P2S) 123
 - private endpoints 134
 - private endpoints, using 134
 - service endpoints 131
 - service endpoints, enabling 132, 133
 - Site-to-Site connection (S2S) 123
 - Site-to-Site connection (S2S),
 - creating 124-126
 - three-tier application
 - architecture 122, 123
 - VPN connection, creating 127, 128

- Virtual Networks (VNETs),
 - security options
 - about 134, 135
 - Azure Application Gateway 142-144
 - Azure Bastion 140
 - Azure DDoS protection 139
 - Azure Firewall configuration 137-139
 - Azure Firewall deployment 135-137
 - Azure Front Door 145
 - hub-and-spoke network topology 141
 - hub virtual network,
 - implementing 141, 142
- virtual private networks (VPNs) 81
- vulnerability assessment (VA) 228, 229

W

- Web Application Firewall (WAF)
 - about 143
 - attacks, detecting 144
 - detection mode 143
 - prevention mode 143
 - protection mode 143
 - working 144
- wide area network 9
- workbooks
 - advanced threat detection 264, 265
 - community resources, using 266
 - creating 261, 262
 - notebooks, using 262, 263
 - threat hunting, using 262
- workflow automation
 - about 237-240
 - creating 238, 239



Packt . com

Subscribe to our online digital library for full access to over 7,000 books and videos, as well as industry leading tools to help you plan your personal development and advance your career. For more information, please visit our website.

Why subscribe?

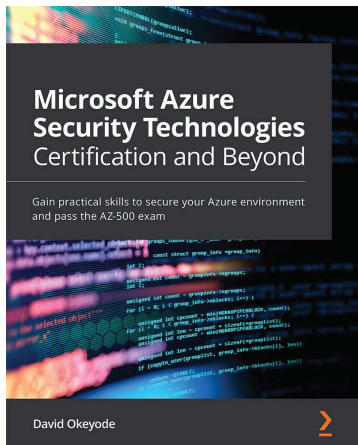
- Spend less time learning and more time coding with practical eBooks and Videos from over 4,000 industry professionals
- Improve your learning with Skill Plans built especially for you
- Get a free eBook or video every month
- Fully searchable for easy access to vital information
- Copy and paste, print, and bookmark content

Did you know that Packt offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at [packt . com](http://packt.com) and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at [customercare@packtpub . com](mailto:customercare@packtpub.com) for more details.

At [www . packt . com](http://www.packt.com), you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt books and eBooks.

Other Books You May Enjoy

If you enjoyed this book, you may be interested in these other books by Packt:

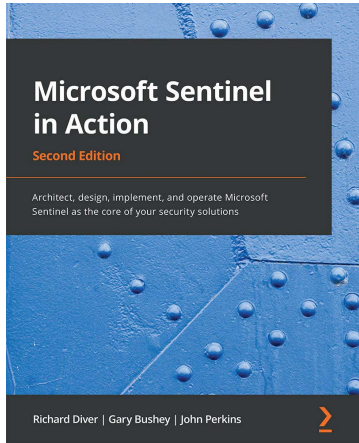


Microsoft Azure Security Technologies Certification and Beyond

David Okeyode

ISBN: 9781800562653

- Manage users, groups, service principals, and roles effectively in Azure AD
- Explore Azure AD identity security and governance capabilities
- Understand how platform perimeter protection secures Azure workloads
- Implement network security best practices for IaaS and PaaS
- Discover various options to protect against DDoS attacks
- Secure hosts and containers against evolving security threats
- Configure platform governance with cloud-native tools
- Monitor security operations with Azure Security Center and Azure Sentinel



Microsoft Sentinel in Action - Second Edition

Richard Diver, Gary Bushey, John Perkins

ISBN: 9781801815536

- Implement Log Analytics and enable Microsoft Sentinel and data ingestion from multiple sources
- Tackle Kusto Query Language (KQL) coding
- Discover how to carry out threat hunting activities in Microsoft Sentinel
- Connect Microsoft Sentinel to ServiceNow for automated ticketing
- Find out how to detect threats and create automated responses for immediate resolution
- Use triggers and actions with Microsoft Sentinel playbooks to perform automations

Packt is searching for authors like you

If you're interested in becoming an author for Packt, please visit authors.packtpub.com and apply today. We have worked with thousands of developers and tech professionals, just like you, to help them share their insight with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Share Your Thoughts

Now you've finished *Mastering Azure Security*, we'd love to hear your thoughts! If you purchased the book from Amazon, please [click here](#) to go straight to the Amazon review page for this book and share your feedback or leave a review on the site that you purchased it from.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

